Code Export

Source Code PDF Report

Generated	2026-04-28 20:50
Root directory	C:\Users\LENOVO\Documents\Java_run\Smart_EmployeeManagementSystem\SmartEmployeeManagementSys
Files found	152
Total lines	14,566

Table of Contents

1. Parent: README.md (3 lines)
2. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
3. Current: pom.xml (131 lines)
4. Current: src\main\java\com\example\EmployeeManagementSystem\code\CodeToPdf.java (347 lines)
5. Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java (52 lines)
6. Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java (165 lines)
7. Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java (32 lines)
8. Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java (38 lines)
9. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AdminController.java (178 lines)
10. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java (118 lines)
11. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java (33 lines)
12. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeviceController.java (71 lines)
13. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java (59 lines)
14. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java (132 lines)
15. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java (92 lines)
16. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java (60 lines)
17. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LoginController.java (17 lines)
18. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ManagerController.java (66 lines)
19. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OauthController.java (52 lines)
20. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java (124 lines)
21. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java (50 lines)
22. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java (67 lines)
23. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java (96 lines)
24. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java (73 lines)

25. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java (101 lines)

26. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java (59 lines)

27. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java (18 lines)

28. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ActionDTO.java (34 lines)

29. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminActionDTO.java (34 lines)

30. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AssignDeviceRequest.java (14 lines)

31. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java (28 lines)

32. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java (28 lines)

33. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java (91 lines)

34. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceDTO.java (45 lines)

35. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceResponseDTO.java (103 lines)

36. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.java (43 lines)

37. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDetails.java (41 lines)

38. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java (50 lines)

39. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeStatusDTO.java (71 lines)

40. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveDashboardResponse.java (115 lines)

41. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java (48 lines)

42. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java (73 lines)

43. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java (64 lines)

44. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\PromoteRequest.java (23 lines)

45. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RepairDTO.java (34 lines)

46. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java (42 lines)

47. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java (46 lines)

48. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestDTO.java (44 lines)

49. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java (103 lines)

50. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java (72 lines)

51. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java (26 lines)

52. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java (47 lines)

53. Current: src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication.java (16 lines)

54. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ApiKey.java (79 lines)

55. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Delivery.java (93 lines)

56. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Device.java (107 lines)

57. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\DeviceAssignment.java (68 lines)

58. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Employee.java (198 lines)

59. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveEntitlement.java (104 lines)

60. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveRequest.java (105 lines)

61. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveType.java (49 lines)

62. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveWarning.java (58 lines)

63. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java (62 lines)

64. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Restaurant.java (58 lines)

65. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ServiceRequest.java (141 lines)

66. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Subscription.java (105 lines)

67. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Vendor.java (124 lines)

68. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\YearEndCarryForwardLog.java (62 lines)

69. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java (8 lines)

70. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java (9 lines)

71. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java (10 lines)

72. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceType.java (15 lines)

73. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java (5 lines)

74. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java (9 lines)

75. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java (8 lines)

76. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java (8 lines)

77. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Permission.java (21 lines)

78. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Role.java (60 lines)

79. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java (7 lines)

80. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestStatus.java (9 lines)

81. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestType.java (10 lines)

82. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Status.java (8 lines)

83. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java (8 lines)

84. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java (8 lines)

85. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java (10 lines)

86. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java (8 lines)

87. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java (8 lines)

88. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java (8 lines)

89. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundException.java (8 lines)

90. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.java (8 lines)

91. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException.java (8 lines)

92. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java (8 lines)

93. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.java (7 lines)

94. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java (7 lines)

95. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java (45 lines)

96. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\JwtAuthFilter.java (73 lines)

97. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java (34 lines)

98. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java (23 lines)

99. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java (21 lines)

100. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceRepository.java (13 lines)

101. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java (26 lines)

102. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveEntitlementRepository.java (31 lines)

103. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveRequestRepo.java (74 lines)

104. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java (13 lines)

105. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java (21 lines)

106. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java (19 lines)

107. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java (31 lines)

108. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.java (21 lines)

109. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java (32 lines)

110. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java (13 lines)

111. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRepository.java (18 lines)

112. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java (30 lines)

113. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveAccrualScheduler.java (54 lines)

114. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java (107 lines)

115. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.java (85 lines)

116. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java (84 lines)

117. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CarryForwardWarningService.java (126 lines)

118. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java (41 lines)

119. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CustomOAuth2UserService.java (87 lines)

120. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryService.java (178 lines)

121. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java (74 lines)

122. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeviceService.java (193 lines)

123. Current: src\main\java\com\example\EmployeeManagementSystem\Service\EmployeeService.java (193 lines)

124. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveAccrualService.java (123 lines)

125. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveDashboardService.java (145 lines)

126. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveRequestService.java (273 lines)

127. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ManagerService.java (147 lines)

128. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java (22 lines)

129. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java (41

lines)

130. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RestaurantService.java (162 lines)

131. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ServiceRequestService.java (162 lines)

132. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SickLeaveResetService.java (65 lines)

133. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SubscriptionService.java (228 lines)

134. Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java (40 lines)

135. Current: src\main\java\com\example\EmployeeManagementSystem\Service\YearEndCarryForwardService.java (219 lines)

136. Current: src\main\java\com\example\EmployeeManagementSystem\Util\AuthUtil.java (21 lines)

137. Current: src\main\java\com\example\EmployeeManagementSystem\Util\JWTUtil.java (76 lines)

138. Current: src\main\java\com\example\EmployeeManagementSystem\Util\OAuth2SuccessHandler.java (85 lines)

139. Current: src\main\resources\application.yaml (69 lines)

140. Current: src\main\resources\static\dashboard.html (945 lines)

141. Current: src\main\resources\static\employee-dashboard.html (721 lines)

142. Current: src\main\resources\static\google.html (670 lines)

143. Current: src\main\resources\static\login.html (10 lines)

144. Current: src\main\resources\static\manager-dashboard.html (976 lines)

145. Current: src\main\resources\static\pending.html (76 lines)

146. Current: src\main\resources\static\tech-vendor-dashboard.html (980 lines)

147. Current: src\main\resources\static\vendor-dashboard.html (529 lines)

148. Current: src\main\resources\static\vendor-login.html (396 lines)

149. Current: src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplicationTests.java (14 lines)

150. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveAccrualServiceTest.java (112 lines)

151. Current: src\test\java\com\example\EmployeeManagementSystem\service\SickLeaveResetServiceTest.java (93 lines)

152. Current: src\test\java\com\example\EmployeeManagementSystem\service\YearEndCarryForwardServiceTest.java (137 lines)

1 Parent: README.md

3 lines

```
# SmartEmployeeManagementSystem
A secure and scalable Spring Boot backend for employee management, leave tracking, and meal subscription services with JWT, OAuth2, API key authentication, and role-based access control.

```

2 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
wrapperVersion=3.3.4
distributionType=only-script
distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.14/apache-maven-3.9.14-bin.zip

```

3 Current: pom.xml

131 lines

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.5</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.example</groupId>
    <artifactId>EmployeeLeaveManagementSystem</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name/>
    <description/>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springdoc</groupId>
            <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
            <version>3.0.2</version>
        </dependency>
        <dependency>
            <groupId>dev.samstevens.totp</groupId>
            <artifactId>totp-spring-boot-starter</artifactId>
            <version>1.7.1</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.session</groupId>
            <artifactId>spring-session-core</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>com.itextpdf</groupId>
```

```
            <artifactId>layout</artifactId>
            <version>7.2.5</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-security-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-webmvc-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-api</artifactId>
            <version>0.13.0</version>
            <scope>compile</scope>
        </dependency>
        <dependency>
            <groupId>io.jsonwebtoken</groupId>
            <artifactId>jjwt-impl</artifactId>
             <version>0.13.0</version>
             <scope>runtime</scope>
         </dependency>
         <dependency>
             <groupId>io.jsonwebtoken</groupId>
             <artifactId>jjwt-jackson</artifactId>
             <version>0.13.0</version>
             <scope>runtime</scope>
         </dependency>
         <dependency>
             <groupId>org.springframework.boot</groupId>
             <artifactId>spring-boot-starter-security-oauth2-client</artifactId>
         </dependency>

         <dependency>
             <groupId>org.springframework.boot</groupId>
             <artifactId>spring-boot-starter-security-oauth2-client-test</artifactId>
             <scope>test</scope>
         </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>

</project>

```

```java
package com.example.EmployeeManagementSystem.code;

import com.itextpdf.kernel.colors.ColorConstants;
import com.itextpdf.kernel.colors.DeviceRgb;
import com.itextpdf.kernel.font.PdfFont;
import com.itextpdf.kernel.font.PdfFontFactory;
import com.itextpdf.kernel.geom.PageSize;
import com.itextpdf.kernel.pdf.PdfDocument;
import com.itextpdf.kernel.pdf.PdfWriter;
import com.itextpdf.layout.Document;
import com.itextpdf.layout.element.AreaBreak;
import com.itextpdf.layout.element.Cell;
import com.itextpdf.layout.element.Paragraph;
import com.itextpdf.layout.element.Table;
import com.itextpdf.layout.element.Text;
import com.itextpdf.layout.properties.AreaBreakType;
import com.itextpdf.layout.properties.UnitValue;
import com.itextpdf.io.font.constants.StandardFonts;

import java.io.File;
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

/**
 * CodeToPdf - Extracts all source code files from the current, parent,
 * and immediate child directories into a formatted PDF document.
 *
 * Dependencies (add to your pom.xml or build.gradle):
 *   com.itextpdf:itext7-core:7.2.5  (or itext7 bundle)
 *
 * Compile:
 *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
 *
 * Run:
 *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
 *
 * Or with Maven:
 *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
 */
public class CodeToPdf {

    //  Configuration

    /** Output PDF filename */
    private static final String OUTPUT_FILE = "code_export.pdf";

    /** Maximum file size to include (skip very large generated files) */
    private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB

    /** Source code file extensions to include */
    private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
            // JVM languages
            ".java", ".kt", ".groovy", ".scala", ".clj",
            // Web
            ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
            // Python / Ruby / PHP
            ".py", ".rb", ".php",
            // Systems / low-level
            ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
            // Data / config
            ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
            // Shell / scripts
            ".sh", ".bash", ".zsh", ".bat", ".ps1",
            // Build
            ".gradle", ".cmake", "Makefile", "Dockerfile",
            // Other
            ".sql", ".md", ".txt"
```

```java
    ));

    /** Directories to skip entirely */
    private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
            ".git", ".svn", ".hg", "node_modules", "target", "build",
            "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
    ));

    //  Colors
    private static final DeviceRgb COLOR_HEADER_BG    = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
    private static final DeviceRgb COLOR_HEADER_TEXT  = new DeviceRgb(0xFF, 0xFF, 0xFF);
    private static final DeviceRgb COLOR_FILE_TITLE   = new DeviceRgb(0x18, 0x5F, 0xA5);
    private static final DeviceRgb COLOR_CODE_BG      = new DeviceRgb(0xF7, 0xF7, 0xF7);
    private static final DeviceRgb COLOR_LINE_NUM     = new DeviceRgb(0x99, 0x99, 0x99);
    private static final DeviceRgb COLOR_TOC_HEADER   = new DeviceRgb(0x18, 0x5F, 0xA5);

    //  Font sizes
    private static final float FONT_TITLE   = 20f;
    private static final float FONT_SECTION = 13f;
    private static final float FONT_FILE    = 11f;
    private static final float FONT_CODE    =  7.5f;
    private static final float FONT_TOC     = 10f;

    //  State
    private final List<FileEntry> fileEntries = new ArrayList<>();
    private int totalLines = 0;

    //

     public static void main(String[] args) throws Exception {
         new CodeToPdf().run();
     }

     private void run() throws Exception {
         Path currentDir = Paths.get("").toAbsolutePath();
         Path parentDir  = currentDir.getParent();

         System.out.println("=== Code to PDF Extractor ===");
         System.out.println("Current directory : " + currentDir);
         System.out.println("Parent  directory : " + (parentDir != null ? parentDir : "(none)"));
         System.out.println();

         // 1. Collect files

         // Parent directory (top-level files only, not recursive)
         if (parentDir != null && Files.exists(parentDir)) {
             collectFiles(parentDir, false, "Parent");
         }

         // Current directory (recursive — includes all child subdirectories)
         collectFiles(currentDir, true, "Current");

         System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
                 fileEntries.size(), totalLines);

         if (fileEntries.isEmpty()) {
             System.out.println("No code files found. Exiting.");
             return;
         }

         // 2. Generate PDF
         generatePdf(currentDir);

         System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
     }

     //  File Collection

     private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
         if (!Files.exists(root)) return;

         Files.walkFileTree(root, new SimpleFileVisitor<>() {
             @Override
             public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
                 String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
                 // Never recurse into skip-dirs; always allow the root itself
```

```
                if (!dir.equals(root) && SKIP_DIRS.contains(name)) {
                    return FileVisitResult.SKIP_SUBTREE;
                }
                // If not recursive, skip any subdirectory of root
                if (!recursive && !dir.equals(root)) {
                    return FileVisitResult.SKIP_SUBTREE;
                }
                return FileVisitResult.CONTINUE;
            }

            @Override
            public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
                if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
                    String content = Files.readString(file);
                    int lines = content.split("\n", -1).length;
                    String relativePath = scope + ": " + root.relativize(file);
                    fileEntries.add(new FileEntry(relativePath, file, content, lines));
                    totalLines += lines;
                    System.out.println("  + " + relativePath + "  (" + lines + " lines)");
                }
                return FileVisitResult.CONTINUE;
            }
        });
    }

    private boolean isCodeFile(Path file) {
        String name = file.getFileName().toString();
        // Check for exact filename match (e.g. "Makefile", "Dockerfile")
        if (CODE_EXTENSIONS.contains(name)) return true;
        // Check extension
        int dot = name.lastIndexOf('.');
        if (dot >= 0) {
            return CODE_EXTENSIONS.contains(name.substring(dot));
        }
        return false;
    }

    //  PDF Generation

    private void generatePdf(Path currentDir) throws IOException {
        PdfWriter writer   = new PdfWriter(OUTPUT_FILE);
        PdfDocument pdfDoc = new PdfDocument(writer);
        Document doc       = new Document(pdfDoc, PageSize.A4);
        doc.setMargins(40, 40, 40, 40);

        PdfFont sansFont  = PdfFontFactory.createFont(StandardFonts.HELVETICA);
        PdfFont boldFont  = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
        PdfFont monoFont  = PdfFontFactory.createFont(StandardFonts.COURIER);

        // Cover page
        addCoverPage(doc, sansFont, boldFont, currentDir);
        doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));

        // Table of contents
        addTableOfContents(doc, sansFont, boldFont);
        doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));

        // Code sections
        for (int i = 0; i < fileEntries.size(); i++) {
            FileEntry entry = fileEntries.get(i);
            addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
            if (i < fileEntries.size() - 1) {
                doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
            }
        }

        doc.close();
    }

    private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
            throws IOException {

        doc.add(new Paragraph("\n\n\n"));

        // Big title
        doc.add(new Paragraph("Code Export")
```

```
                .setFont(boldFont).setFontSize(FONT_TITLE + 10)
                .setFontColor(COLOR_FILE_TITLE)
                .setMarginBottom(6));

        doc.add(new Paragraph("Source Code PDF Report")
                .setFont(sansFont).setFontSize(FONT_SECTION)
                .setFontColor(ColorConstants.DARK_GRAY)
                .setMarginBottom(24));

        // Stats table
        Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
                .setWidth(UnitValue.createPercentValue(80));
        addStatRow(stats, "Generated",    LocalDateTime.now()
                .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
        addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
        addStatRow(stats, "Files found",  String.valueOf(fileEntries.size()), sansFont, boldFont);
        addStatRow(stats, "Total lines",  String.format("%,d", totalLines), sansFont, boldFont);
        doc.add(stats);
    }

    private void addStatRow(Table t, String label, String value,
                            PdfFont sans, PdfFont bold) {
        t.addCell(new Cell().add(new Paragraph(label)
                        .setFont(bold).setFontSize(10)
                        .setFontColor(ColorConstants.DARK_GRAY))
                .setBorder(null).setPaddingBottom(6));
        t.addCell(new Cell().add(new Paragraph(value)
                        .setFont(sans).setFontSize(10))
                .setBorder(null).setPaddingBottom(6));
    }

    private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
            throws IOException {
        doc.add(new Paragraph("Table of Contents")
                .setFont(boldFont).setFontSize(FONT_SECTION + 2)
                .setFontColor(COLOR_TOC_HEADER)
                .setMarginBottom(12));

        for (int i = 0; i < fileEntries.size(); i++) {
            FileEntry entry = fileEntries.get(i);
            String label = String.format("%d.  %s  (%,d lines)",
                    i + 1, entry.relativePath, entry.lineCount);
            doc.add(new Paragraph(label)
                    .setFont(sansFont).setFontSize(FONT_TOC)
                    .setMarginBottom(3));
        }
    }

    private void addCodeSection(Document doc, FileEntry entry, int index,
                                PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
            throws IOException {

        // Section header bar
        Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
                .setWidth(UnitValue.createPercentValue(100))
                .setMarginBottom(8);

        header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
                        .setFont(boldFont).setFontSize(FONT_FILE)
                        .setFontColor(COLOR_HEADER_TEXT))
                .setBackgroundColor(COLOR_HEADER_BG)
                .setBorder(null).setPadding(6));

        header.addCell(new Cell().add(new Paragraph(entry.relativePath)
                        .setFont(boldFont).setFontSize(FONT_FILE)
                        .setFontColor(COLOR_HEADER_TEXT))
                .setBackgroundColor(COLOR_HEADER_BG)
                .setBorder(null).setPadding(6));

        header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
                        .setFont(sansFont).setFontSize(FONT_FILE - 1)
                        .setFontColor(COLOR_HEADER_TEXT))
                .setBackgroundColor(COLOR_HEADER_BG)
                .setBorder(null).setPadding(6));

        doc.add(header);
```

```java

        // Code block with line numbers
        String[] lines = entry.content.split("\n", -1);
        int padWidth = String.valueOf(lines.length).length();

        for (int ln = 0; ln < lines.length; ln++) {
            String lineNum = String.format("%" + padWidth + "d", ln + 1);
            String codeLine = lines[ln]
                    .replace("\t", "    ")   // expand tabs
                    .replace("\r", "");       // strip CR

            // Truncate very long lines
            if (codeLine.length() > 200) {
                codeLine = codeLine.substring(0, 197) + "...";
            }

            Text numText  = new Text(lineNum + "  ").setFont(monoFont)
                    .setFontSize(FONT_CODE).setFontColor(COLOR_LINE_NUM);
            Text codeText = new Text(codeLine).setFont(monoFont)
                    .setFontSize(FONT_CODE).setFontColor(ColorConstants.BLACK);

            Paragraph p = new Paragraph().add(numText).add(codeText)
                    .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
                    .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
                    .setMultipliedLeading(1.3f)
                    .setFixedLeading(FONT_CODE + 2.5f);

            doc.add(p);
        }
    }

    //  Data class

    private static class FileEntry {
        final String relativePath;
        final Path   absolutePath;
        final String content;
        final int    lineCount;

        FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
            this.relativePath = relativePath;
            this.absolutePath = absolutePath;
            this.content      = content;
            this.lineCount    = lineCount;
        }
    }
}

```

5 Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java

52 lines

```
//package com.example.EmployeeManagementSystem.config;
//
//import org.springframework.beans.factory.annotation.Qualifier;
//import org.springframework.context.annotation.Bean;
//import org.springframework.context.annotation.Configuration;
//import org.springframework.core.annotation.Order;
//import org.springframework.http.HttpMethod;
//import org.springframework.security.authentication.AuthenticationManager;
//import org.springframework.security.authentication.ProviderManager;
//import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
//import org.springframework.security.config.Customizer;
//import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
//import org.springframework.security.config.annotation.web.builders.HttpSecurity;
//import org.springframework.security.core.userdetails.UserDetailsService;
//import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
//import org.springframework.security.crypto.password.PasswordEncoder;
//import org.springframework.security.web.SecurityFilterChain;
//
//
//@Configuration
//@EnableMethodSecurity
//public class BasicAuthConfig {
////    @Bean
////    @Order(3)
////   public SecurityFilterChain basicAuth(HttpSecurity security){
////      security
////               .csrf(csrf-> csrf.disable())
////               .authorizeHttpRequests(
////                       auth->auth
////
.requestMatchers("/employee/register","/employee/register/manager").permitAll()
////                                .requestMatchers(HttpMethod.POST,"/vendors").permitAll()
////                                .anyRequest().authenticated()
////               )
////               .httpBasic(Customizer.withDefaults());
////       return security.build();
////   }
////
////   @Bean
////    public PasswordEncoder passwordEncoder(){
////        return new BCryptPasswordEncoder();
////   }
//
//   @Bean
//    public AuthenticationManager authenticationManager(@Qualifier("combinedUserDetailService")
UserDetailsService userDetailsService,
//                                                        PasswordEncoder passwordEncoder){
//       DaoAuthenticationProvider employeeAuthenticationProvider=new
DaoAuthenticationProvider(userDetailsService);
//       employeeAuthenticationProvider.setPasswordEncoder(passwordEncoder);
//       return new ProviderManager(employeeAuthenticationProvider);
//   }
//
//}
```

6 Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java

165 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/config/JWTAuth/JWTAuthConfig.java
package com.example.EmployeeManagementSystem.config.JWTauth;

import com.example.EmployeeManagementSystem.Filter.ApiKeyFilter;
import com.example.EmployeeManagementSystem.Filter.JwtAuthFilter;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Service.CustomOAuth2UserService;
import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
import com.example.EmployeeManagementSystem.Util.JWTUtil;
import com.example.EmployeeManagementSystem.Util.OAuth2SuccessHandler;
import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationProvider;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Primary;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.ProviderManager;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.security.web.context.DelegatingSecurityContextRepository;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.security.web.context.RequestAttributeSecurityContextRepository;
import org.springframework.web.client.RestTemplate;

import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class JWTAuthConfig {

    private final JwtAuthFilter jwtAuthFilter;
    private final ApiKeyAuthenticationProvider apiKeyAuthProvider;
    private final EmployeeRepo employeeRepo;
    private final VendorRepo vendorRepo;
    private final RefreshTokenService refreshTokenService;
    private final RefreshTokenRepository refreshTokenRepository;
    private final JWTUtil jwtUtil;

    public JWTAuthConfig(JwtAuthFilter jwtAuthFilter,
                         ApiKeyAuthenticationProvider apiKeyAuthProvider,
                         EmployeeRepo employeeRepo,
                         VendorRepo vendorRepo,
                         RefreshTokenService refreshTokenService,
                         RefreshTokenRepository refreshTokenRepository,
                         JWTUtil jwtUtil) {
        this.jwtAuthFilter = jwtAuthFilter;
        this.apiKeyAuthProvider = apiKeyAuthProvider;
        this.employeeRepo = employeeRepo;
        this.vendorRepo = vendorRepo;
        this.refreshTokenService = refreshTokenService;
        this.refreshTokenRepository = refreshTokenRepository;
        this.jwtUtil = jwtUtil;
    }

    @Bean
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }

    @Bean
```

```
    public ApiKeyFilter apiKeyFilter() {
        return new ApiKeyFilter();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(
            @Qualifier("combinedUserDetailService") UserDetailsService userDetailsService,
            PasswordEncoder passwordEncoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
        provider.setPasswordEncoder(passwordEncoder);
        return new ProviderManager(provider);
    }

    @Bean
    public CustomOAuth2UserService customOAuth2UserService() {
        return new CustomOAuth2UserService(employeeRepo, vendorRepo, passwordEncoder());
    }

    @Bean
    public OAuth2SuccessHandler oAuth2SuccessHandler() {
        return new OAuth2SuccessHandler(jwtUtil, employeeRepo, vendorRepo,
                refreshTokenService, refreshTokenRepository);
    }

     @Bean
     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
         http
                 .csrf(csrf -> csrf.disable())
                 .securityContext(ctx -> ctx
                         .securityContextRepository(new DelegatingSecurityContextRepository(
                                 new HttpSessionSecurityContextRepository(),
                                 new RequestAttributeSecurityContextRepository()
                         ))
                 )
//                 .sessionManagement(session -> session
//                         .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
//                         .maximumSessions(1)
//                         .maxSessionsPreventsLogin(false)
//                 )
                 .authorizeHttpRequests(auth -> auth
                         // Auth endpoints
                         .requestMatchers("/Authenticate", "/Authenticate/refresh").permitAll()
                         .requestMatchers("/session/login", "/session/logout").permitAll()

                         // OAuth2 endpoints
                         .requestMatchers("/oauth2/**", "/login/oauth2/**").permitAll()
                         .requestMatchers("/auth/google/init").permitAll()

                         // API docs
                         .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()

                         // Shared login page
                         .requestMatchers("/login.html", "/google.html").permitAll()

                         // Dashboards — served as static HTML, actual data is protected by JWT
                         .requestMatchers("/login.html",
                                 "/google.html",
                                 "/vendor-login.html",
                                 "/vendor-dashboard.html",
                                 "/tech-vendor-dashboard.html",
                                 "/employee-dashboard.html",
                                 "/manager-dashboard.html",
                                 "/dashboard.html",
                                 "/pending.html").permitAll()

                         // Vendor pages
                         .requestMatchers("/vendor-login.html").permitAll()
                         .requestMatchers("/vendor-dashboard.html").permitAll()

                         // API key generation requires authentication
                         .requestMatchers("/api-keys/generate").authenticated()
```

```
                        .requestMatchers("/manager/createAdmin").permitAll()

                        // Everything else requires a valid token
                        .anyRequest().authenticated()
                )
                .oauth2Login(oauth -> oauth
                        .userInfoEndpoint(userInfo -> userInfo
                                .userService(customOAuth2UserService())
                        )
                        .successHandler(oAuth2SuccessHandler())
                )
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
                .addFilterBefore(apiKeyFilter(), UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```

7 Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java

```
//package com.example.EmployeeManagementSystem.config.Seassion;
//
//import org.springframework.context.annotation.Bean;
//import org.springframework.context.annotation.Configuration;
//import org.springframework.core.annotation.Order;
//import org.springframework.security.config.annotation.web.builders.HttpSecurity;
//import org.springframework.security.web.SecurityFilterChain;
//import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
//
 //@Configuration
 //public class SessionAuthConfig {
 //
 //    @Bean
 //    @Order(1)
 //    public SecurityFilterChain sessionFilterChain(HttpSecurity http) throws Exception {
 //        http
 //                .securityMatcher("/session/**", "/web/**")
 //                .csrf(csrf -> csrf.disable())
 //                .authorizeHttpRequests(auth -> auth
 //                        .requestMatchers("/session/login", "/session/logout").permitAll()
 //                        .anyRequest().authenticated()
 //                )
 //                .sessionManagement(session -> session
 //                        .maximumSessions(1)           // one session per user
 //                        .maxSessionsPreventsLogin(false) // new login kicks old session
 //                )
 //                .securityContext(ctx -> ctx
 //                        .securityContextRepository(new HttpSessionSecurityContextRepository())
 //                );
 //        return http.build();
 //    }
 //}
```

8 Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java

38 lines

```
package com.example.EmployeeManagementSystem.config;

import io.swagger.v3.oas.models.Components;
import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.security.SecurityRequirement;
import io.swagger.v3.oas.models.security.SecurityScheme;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

 @Configuration
 public class swaggerConfig {
     @Bean
     public OpenAPI openAPI() {
         return new OpenAPI()
                 .components(new Components()
                         .addSecuritySchemes("bearerAuth",
                                 new SecurityScheme()
                                         .type(SecurityScheme.Type.HTTP)
                                         .scheme("bearer")
                                         .bearerFormat("JWT")
                         )
                 )
                 .addSecurityItem(new SecurityRequirement().addList("bearerAuth"));
     }
 //    @Bean
 //    public OpenAPI openAPI() {
 //        return new OpenAPI()
 //                .components(new Components()
 //                        .addSecuritySchemes("basicAuth",
 //                                new SecurityScheme()
 //                                        .type(SecurityScheme.Type.HTTP)
 //                                        .scheme("basic")
 //                        )
 //                )
 //                .addSecurityItem(new SecurityRequirement().addList("basicAuth"));
 //    }
 }

```

9 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AdminController.java

178 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
import com.example.EmployeeManagementSystem.DTO.VendorDTO;
import com.example.EmployeeManagementSystem.DTO.VendorRequest;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Service.*;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/admin")
@PreAuthorize("hasRole('ADMIN')")
public class AdminController {

    private final EmployeeService employeeService;
    private final DeliveryService.VendorService vendorService;
    private final LeaveRequestService leaveRequestService;
    private final RestaurantService restaurantService;
    private final SubscriptionService subscriptionService;
    private final LeaveAccrualService accrualService;
    private final SickLeaveResetService resetService;

    public AdminController(
            EmployeeService employeeService,
            DeliveryService.VendorService vendorService,
            LeaveRequestService leaveRequestService,
            RestaurantService restaurantService,
            SubscriptionService subscriptionService,
            LeaveAccrualService accrualService,
            SickLeaveResetService resetService) {
        this.employeeService = employeeService;
        this.vendorService = vendorService;
        this.leaveRequestService = leaveRequestService;
        this.restaurantService = restaurantService;
        this.subscriptionService = subscriptionService;
        this.accrualService = accrualService;
        this.resetService=resetService;
    }

    //
    // EMPLOYEE BRANCH
    //

    /** List every employee in the system. */
    @GetMapping("/employees")
    public ResponseEntity<?> getAllEmployees() {
        return ResponseEntity.ok(employeeService.getAllEmployees());
    }

    /** Create a regular employee. */
    @PostMapping("/employees")
    public ResponseEntity<Employee> createEmployee(@RequestBody EmployeeDTO dto) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(employeeService.createEmployee(dto));
    }

    /** Promote / create a manager. */
    @PostMapping("/employees/manager")
    public ResponseEntity<Employee> createManager(@RequestBody EmployeeDTO dto) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(employeeService.createManager(dto));
    }

    /** Update any employee record. */
```

```
    @PutMapping("/employees/{id}")
    public ResponseEntity<Employee> updateEmployee(
            @PathVariable long id,
            @RequestBody EmployeeDTO dto) {
        return ResponseEntity.ok(employeeService.updateEmployee(id, dto));
    }

    /** Hard-delete an employee. */
    @DeleteMapping("/employees/{id}")
    public ResponseEntity<String> deleteEmployee(@PathVariable long id) {
        employeeService.deleteEmployee(id);
        return ResponseEntity.ok("Employee with id: " + id + " has been deleted.");
    }

    /** Soft-deactivate an employee (sets status to INACTIVE). */
    @PutMapping("/employees/{id}/inactive")
    public ResponseEntity<String> deactivateEmployee(@PathVariable long id) {
        return employeeService.inactivateUser(id);
    }

    //
    // VENDOR BRANCH  (only admin can register vendors)
    //

    /** List every vendor. */
    @GetMapping("/vendors")
    public ResponseEntity<List<VendorDTO>> getAllVendors() {
        return ResponseEntity.ok(vendorService.getAllVendors());
    }

    @PostMapping("/vendors")
    public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(vendorService.registerVendor(request));
    }

    /** Remove a vendor by ID. */
    @DeleteMapping("/vendors/{vendorId}")
    public ResponseEntity<String> deleteVendor(@PathVariable Long vendorId) {
        vendorService.deleteVendorById(vendorId);
        return ResponseEntity.ok("Vendor with id: " + vendorId + " has been removed.");
    }

    //
    // LEAVE MANAGEMENT BRANCH
    //

    /** Fetch all pending leave requests across the organisation. */
    @GetMapping("/leaves/pending")
    public ResponseEntity<?> getPendingLeaves() {
        return ResponseEntity.ok(leaveRequestService.getPendingLeaves());
    }

    /** Approve a leave request. */
    @PutMapping("/leaves/{id}/approve")
    public ResponseEntity<?> approveLeave(@PathVariable Long id) {
        return ResponseEntity.ok(leaveRequestService.approveLeave(id));
    }

    /** Reject a leave request. */
    @PutMapping("/leaves/{id}/reject")
    public ResponseEntity<?> rejectLeave(@PathVariable Long id) {
        return ResponseEntity.ok(leaveRequestService.rejectLeave(id));
    }

    //
    // RESTAURANT MANAGEMENT BRANCH
    //

    /** View all restaurants. */
    @GetMapping("/restaurants")
    public ResponseEntity<?> getAllRestaurants() {
        return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
    }

    //
```

```
    // SUBSCRIPTION MANAGEMENT BRANCH
    //

    /** View all active subscriptions. */
    @GetMapping("/subscriptions")
    public ResponseEntity<?> getAllSubscriptions() {
        return ResponseEntity.ok(subscriptionService.getAllSubscriptions());
    }

    @PostMapping("/leave/accrue")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> triggerAccrual(
            @RequestParam(required = false)
            @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
    ) {
        LocalDate accrualDate = (date != null) ? date : LocalDate.now();
        accrualService.runMonthlyAccrual(accrualDate);
        return ResponseEntity.ok("Accrual complete for " + accrualDate);
    }
    @PostMapping("/reset-sick")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> triggerSickReset(
            @RequestParam(required = false)
            @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
    ) {
        LocalDate resetDate = (date != null) ? date : LocalDate.now();
        resetService.runMonthlyReset(resetDate);
        return ResponseEntity.ok("Sick leave reset complete for " + resetDate);
    }

}
```

10 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java

118 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.AccessTokenRequest;
import com.example.EmployeeManagementSystem.DTO.AuthRequest;
import com.example.EmployeeManagementSystem.DTO.AuthResponse;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
import com.example.EmployeeManagementSystem.Util.JWTUtil;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.time.Instant;

@RestController
@RequestMapping("/Authenticate")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final JWTUtil jwtUtil;
    private final RefreshTokenService refreshTokenService;
    private final RefreshTokenRepository refreshTokenRepository;

    public AuthController(AuthenticationManager authenticationManager, JWTUtil jwtUtil, RefreshTokenService refreshTokenService, RefreshTokenRepository refreshTokenRepository) {
        this.authenticationManager = authenticationManager;
        this.jwtUtil = jwtUtil;
        this.refreshTokenService = refreshTokenService;
        this.refreshTokenRepository = refreshTokenRepository;
    }

    @PostMapping
    public AuthResponse authenticateToken(@RequestBody AuthRequest authRequest) {
        try {
            System.out.println("1. Trying to authenticate user: " + authRequest.getUsername());
            System.out.println("2. Password length: " + authRequest.getPassword().length());

            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(
                            authRequest.getUsername(),
                            authRequest.getPassword()
                    )
            );

            System.out.println("3. Authentication successful! User: " + auth.getName());
            System.out.println("4. Authorities: " + auth.getAuthorities());

            UserDetails user = (UserDetails) auth.getPrincipal();

            String role = user.getAuthorities().stream()
                    .map(authority -> authority.getAuthority())
                    .filter(authority -> authority.startsWith("ROLE_"))
                    .findFirst()
                    .map(authority -> authority.substring("ROLE_".length()))
                    .orElseThrow(() -> new RuntimeException("Role not found"));

            String accessToken = jwtUtil.generateToken(user.getUsername(),role);

            RefreshToken refreshToken;

            if (user instanceof Employee employee) {
                refreshToken = refreshTokenService.createForEmployee(employee);
            } else if (user instanceof Vendor vendor) {
```

```
                refreshToken = refreshTokenService.createForVendor(vendor);
            } else {
                throw new RuntimeException("Unknown user type");
            }

            refreshTokenRepository.save(refreshToken);
            System.out.println("5. Token generated: " + accessToken);

            return new AuthResponse(accessToken, refreshToken.getToken());
        } catch (Exception e) {
            System.out.println("ERROR: " + e.getClass().getSimpleName());
            System.out.println("Message: " + e.getMessage());
            e.printStackTrace();  // This will show full stack trace
            throw new RuntimeException(e);
        }


    }

    @PostMapping("/refresh")
    public AuthResponse refresh(@RequestBody AccessTokenRequest request) {
        RefreshToken storedToken = refreshTokenRepository.findByToken(request.getRefreshToken())
                .orElseThrow(() -> new RuntimeException("Invalid refresh token"));

        if (storedToken.getExpiryTime().isBefore(Instant.now())) {
            refreshTokenRepository.delete(storedToken);
            throw new RuntimeException("Refresh token expired");
        }

         String username;
         String role;

         if (storedToken.getEmployee() != null) {
             username = storedToken.getEmployee().getUsername();
             role=storedToken.getEmployee().getRole().name();
         } else if (storedToken.getVendor() != null) {
             username = storedToken.getVendor().getUsername();
             role=storedToken.getVendor().getRole().name();
         } else {
             throw new RuntimeException("Refresh token has no owner");
         }

         String newAccessToken = jwtUtil.generateToken(username,role);

         return new AuthResponse(newAccessToken, storedToken.getToken());
     }
}
```

11 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java

33 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Entity.Delivery;
import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import com.example.EmployeeManagementSystem.Service.DeliveryService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

 @RestController
 @RequestMapping("/delivery")
 public class DeliveryController {
     private final DeliveryService deliveryService;

     public DeliveryController(DeliveryService deliveryService) {
         this.deliveryService = deliveryService;
     }

     @PutMapping("/{id}/status")
     public ResponseEntity<Delivery> updateDeliveryStatus(
             @PathVariable Long id,
             @RequestParam DeliveryStatus status) {
         return ResponseEntity.ok(deliveryService.updateDeliveryStatus(id, status));
     }

     @GetMapping("/subscription/{subscriptionId}")
     public ResponseEntity<List<Delivery>> getDeliveriesBySubscription(
             @PathVariable Long subscriptionId) {
         return ResponseEntity.ok(deliveryService.getDeliveriesBySubscription(subscriptionId));
     }
 }

```

12 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeviceController.java

71 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
import com.example.EmployeeManagementSystem.Entity.Device;
import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
import com.example.EmployeeManagementSystem.Service.DeviceService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/devices")
public class DeviceController {
    private final DeviceService deviceService;

    public DeviceController(DeviceService deviceService) {
        this.deviceService = deviceService;
    }

    @PostMapping
    @PreAuthorize("hasRole('TECH_VENDOR')")
    public ResponseEntity<Device> addDevice(@RequestBody DeviceDTO deviceDTO,
                                            Authentication authentication) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(deviceService.addDevice(deviceDTO, authentication));
    }

    @GetMapping("/vendor/me")
    @PreAuthorize("hasRole('TECH_VENDOR')")
    public ResponseEntity<List<DeviceResponseDTO>> getMyVendorDevices(Authentication authentication) {
        return ResponseEntity.ok(deviceService.getDevicesForLoggedInVendor(authentication));
    }

    @GetMapping("/my")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN')")
    public ResponseEntity<List<DeviceResponseDTO>> getMyAssignedDevices(Authentication authentication) {
        return ResponseEntity.ok(deviceService.getDevicesForLoggedInEmployee(authentication));
    }

    @GetMapping("/unassigned")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<List<DeviceResponseDTO>> getUnassignedDevices() {
        return ResponseEntity.ok(deviceService.getUnassignedDevices());
    }

    @PostMapping("/{deviceId}/assign")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> assignDevice(@PathVariable Long deviceId,
                                               @RequestBody AssignDeviceRequest request) {
        DeviceAssignment assignment = deviceService.assignDevice(deviceId, request);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body("Device " + assignment.getDevice().getId()
                        + " assigned to employee " + assignment.getAssignedTo().getEmployeeId());
    }

    @GetMapping("/assignments")
    @PreAuthorize("hasRole('TECH_VENDOR')")
    public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfvendor(Authentication authentication){
        return deviceService.getDeviceAssignmentOfvendor(authentication);

    }

}

```

13 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java

59 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.EmployeeAttendanceResponseDTO;
import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Service.EmployeeService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/employee")
public class EmployeeController {
    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping
    @PreAuthorize("hasAnyRole('MANAGER','ADMIN','EMPLOYEE','VENDOR')")
    public ResponseEntity<?> getAllEmployees(){
        return ResponseEntity.ok(employeeService.getAllEmployees());
    }

    @GetMapping("/me")
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
    public ResponseEntity<Employee> myAccount(Authentication authentication){
        return ResponseEntity.ok(employeeService.getAccount(authentication));
    }

    @GetMapping("/attendance")
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE','VENDOR')")
    public ResponseEntity<EmployeeAttendanceResponseDTO> getAttendanceOverview() {
        return ResponseEntity.ok(employeeService.getEmployeeAttendanceOverview());
    }

    @PutMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Employee> updateEmployee(@PathVariable long id,@RequestBody EmployeeDTO employeeDTO){
        return ResponseEntity.ok(employeeService.updateEmployee(id,employeeDTO));
    }

    @DeleteMapping("/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<?> deleteEmployee(@PathVariable long id){
        employeeService.deleteEmployee(id);
        return ResponseEntity.ok("Employee with id: "+id+" is deleted");
    }

    //should be done by manager
    @PutMapping("/inactive")
    @PreAuthorize("hasAnyRole('MANAGER','ADMIN')")
    public ResponseEntity<String> inactivateEmployee(@RequestParam long employeeId) {
        return employeeService.inactivateUser(employeeId);
    }
}

```

14 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Exception.*;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.context.request.WebRequest;
import org.springframework.security.access.AccessDeniedException;

import java.time.Instant;
import java.time.LocalDateTime;
import java.util.LinkedHashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("timestamp", Instant.now());
        body.put("status", status.value());
        body.put("error", status.getReasonPhrase());
        body.put("message", message);
        body.put("path", request.getDescription(false).replace("uri=", ""));
        return body;
    }

    @ExceptionHandler(EmployeeNotFound.class)
    public ResponseEntity<Map<String, Object>> handleEmployeeNotFound(
            EmployeeNotFound ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(InvalidStartDateException.class)
    public ResponseEntity<Map<String, Object>> handleInvalidStartDate(
            InvalidStartDateException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(InvalidEndDateException.class)
    public ResponseEntity<Map<String, Object>> handleInvalidEndDate(
            InvalidEndDateException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(OverlappingLeaveException.class)
    public ResponseEntity<Map<String, Object>> handleOverlappingLeave(
            OverlappingLeaveException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(DuplicateRequestException.class)
    public ResponseEntity<Map<String, Object>> handleDuplicateRequest(
            DuplicateRequestException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(LeaveRequestNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleLeaveNotFound(
            LeaveRequestNotFoundException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                HttpStatus.NOT_FOUND);
    }

```

```
    @ExceptionHandler(InvalidManagerException.class)
    public ResponseEntity<Map<String, Object>> handleInvalidManager(
            InvalidManagerException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
                HttpStatus.FORBIDDEN);
    }

    @ExceptionHandler(SubscriptionAlreadyExists.class)
    public ResponseEntity<Map<String, Object>> handleSubscriptionExists(
            SubscriptionAlreadyExists ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
                HttpStatus.CONFLICT);
    }


    @ExceptionHandler(VendorNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleVendorNotFound(
            VendorNotFoundException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(RestaurantNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleRestaurantNotFound(
            RestaurantNotFoundException ex, WebRequest request) {
        return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
                HttpStatus.NOT_FOUND);
    }

     @ExceptionHandler(UnauthorizedAccessException.class)
     public ResponseEntity<Map<String, Object>> handleUnauthorized(
             UnauthorizedAccessException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
                 HttpStatus.FORBIDDEN);
     }

     @ExceptionHandler(IllegalArgumentException.class)
     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
             IllegalArgumentException ex, WebRequest request) {
         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
                 HttpStatus.BAD_REQUEST);
     }

     @ExceptionHandler(DataIntegrityViolationException.class)
     public ResponseEntity<Map<String, Object>> handleDataIntegrity(
             DataIntegrityViolationException ex, WebRequest request) {
         return new ResponseEntity<>(
                 buildError(HttpStatus.CONFLICT, "Cannot delete: related records exist", request),
                 HttpStatus.CONFLICT);
     }

     @ExceptionHandler(AccessDeniedException.class)
     public ResponseEntity<Map<String, Object>> handleAccessDenied(AccessDeniedException ex) {
         return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(
                 "error",     "Access denied",
                 "message",   ex.getMessage(),
                 "timestamp", LocalDateTime.now().toString()
         ));
     }

}
```

15 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java

```
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Service.LeaveDashboardService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api")
public class LeaveDashboardController {

    private final LeaveDashboardService dashboardService;
    private final EmployeeRepo employeeRepository;

    public LeaveDashboardController(LeaveDashboardService dashboardService,
                                    EmployeeRepo employeeRepository) {
        this.dashboardService   = dashboardService;
        this.employeeRepository = employeeRepository;
    }

    //  GET own dashboard (any authenticated employee)
    // GET /api/employees/me/leave-dashboard
    @GetMapping("/employees/me/leave-dashboard")
    public ResponseEntity<LeaveDashboardResponse> getMyDashboard(
            @AuthenticationPrincipal UserDetails caller) {

        Long callerId = resolveEmployeeId(caller);
        return ResponseEntity.ok(
                dashboardService.getDashboard(callerId, callerId, resolveRole(caller)));
    }

    //  GET any employee's dashboard (manager/admin only)
    // GET /api/employees/5/leave-dashboard
    @GetMapping("/employees/{id}/leave-dashboard")
    public ResponseEntity<LeaveDashboardResponse> getEmployeeDashboard(
            @PathVariable Long id,
            @AuthenticationPrincipal UserDetails caller) {

        Long   callerId   = resolveEmployeeId(caller);
        String callerRole = resolveRole(caller);
        return ResponseEntity.ok(
                dashboardService.getDashboard(id, callerId, callerRole));
    }

    //  GET team dashboard (manager/admin only)
    // GET /api/employees/me/team-dashboard
    @GetMapping("/employees/me/team-dashboard")
    public ResponseEntity<List<LeaveDashboardResponse>> getTeamDashboard(
            @AuthenticationPrincipal UserDetails caller) {

        Long   callerId   = resolveEmployeeId(caller);
        String callerRole = resolveRole(caller);
        return ResponseEntity.ok(
                dashboardService.getTeamDashboard(callerId, callerRole));
    }

    //  PATCH mark warning read
    // PATCH /api/warnings/42/read
    @PatchMapping("/warnings/{warningId}/read")
    public ResponseEntity<Void> markWarningRead(
            @PathVariable Long warningId,
            @AuthenticationPrincipal UserDetails caller) {

        Long   callerId   = resolveEmployeeId(caller);
        String callerRole = resolveRole(caller);
```

```
        dashboardService.markWarningRead(warningId, callerId, callerRole);
        return ResponseEntity.noContent().build();
    }

    //  Resolve caller from Spring Security context
    // Adapt these two methods to however your UserDetails stores
    // the employee ID and role (e.g. a custom UserDetails impl)
    private Long resolveEmployeeId(UserDetails caller) {
        return employeeRepository
                .findByEmail(caller.getUsername())
                .orElseThrow(() -> new IllegalStateException("Authenticated user not found"))
                .getEmployeeId();
    }

    private String resolveRole(UserDetails caller) {
        return caller.getAuthorities().stream()
                .findFirst()
                .map(a -> a.getAuthority().replace("ROLE_", ""))
                .orElse("EMPLOYEE");
    }
}
```

16 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.ActionDTO;
import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("leave_requests")
public class LeaveRequestController {
    private final LeaveRequestService leaveRequestService;

    public LeaveRequestController(LeaveRequestService leaveRequestService) {
        this.leaveRequestService = leaveRequestService;
    }

    @GetMapping
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
    public ResponseEntity<List<LeaveResponseDTO>> getAllLeaveRequest() {
        return ResponseEntity.ok(leaveRequestService.getAllTheLeaveRequest());
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
    public ResponseEntity<?> addLeaveRequest(Authentication authentication,@RequestBody LeaveRequestDTO dto){
        return leaveRequestService.createRequest(authentication,dto);
    }

    @GetMapping("/pending")
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
    public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests() {
        return ResponseEntity.ok(leaveRequestService.getAllThePendingLeaveRequests());
    }

    @PutMapping("/approval")
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
    public ResponseEntity<?> updateLeaveRequestStatus(@RequestBody ActionDTO actionDTO,Authentication authentication){
      return leaveRequestService.updateLeaveRequestStatus(actionDTO,authentication);
    }

    @PutMapping("/cancel/{leaveId}")
    @PreAuthorize("hasAuthority('LEAVE_CANCEL')")
    public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,@PathVariable long leaveId){
      return leaveRequestService.cancelLeaveRequest(authentication, leaveId);
    }

    @GetMapping("/employee")
    @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
    public ResponseEntity<?> getEmployeeLeaves(Authentication authentication) {
        return leaveRequestService.getLeaveRequestsByEmployee(authentication);
    }

}

```

17 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LoginController.java

17 lines

```
package com.example.EmployeeManagementSystem.Controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.Map;

@RestController
 public class LoginController {

     @GetMapping("/login")
     public ResponseEntity<?> login() {
         return ResponseEntity.ok().body(Map.of("message", "Please use POST to /api/auth/login with credentials"));
     }
 }

```

18 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ManagerController.java

66 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.*;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Service.ManagerService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

 import java.util.List;

 @RestController
 @RequestMapping("/manager")
 public class ManagerController {
    private final ManagerService manangerService;

     public ManagerController(ManagerService manangerService) {
         this.manangerService = manangerService;
     }

     @PostMapping("/createAdmin")
     public Employee createAdmin(@RequestBody EmployeeDetails employeeDetails){
         return manangerService.createAdmin(employeeDetails);
     }

     @PostMapping("/employees")
     @PreAuthorize("hasAnyRole('MANAGER')")
     public Employee createEmployee(@RequestBody EmployeeDetails employeeDetails, Authentication authentication){
         return manangerService.createEmp(employeeDetails,authentication);
     }

     @GetMapping("/employees")
     @PreAuthorize("hasRole('MANAGER')")
     public List<Employee> getAllEmployeeByManager(Authentication authentication){
         return manangerService.getAllEmployeeByManager(authentication);
     }

     @GetMapping("/requests")
     @PreAuthorize("hasRole('MANAGER')")
     public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication){
         return manangerService.getAllLeaveRequestByManager(authentication);
     }

     @GetMapping("/Users")
     @PreAuthorize("hasAnyRole('MANAGER','ADMIN')")
     public List<EmployeeDTO> getUsers(){
         return manangerService.getUsers();
     }

     @PutMapping("/promote")
     @PreAuthorize("hasRole('MANAGER')")
     public ResponseEntity<?> promoteUser(Authentication authentication, @RequestBody PromoteRequest promoteRequest){
          return manangerService.promoteUser(authentication,promoteRequest);
     }

     @GetMapping("/pending")
     @PreAuthorize("hasAnyRole('MANAGER')")
     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests(Authentication authentication) {
         return ResponseEntity.ok(manangerService.getAllThePendingLeaveRequests(authentication));
     }



 }

```

19 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OauthController.java

52 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.AuthResponse;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Service.CombinedUserDetailService;
import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
import com.example.EmployeeManagementSystem.Util.JWTUtil;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.*;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.util.LinkedMultiValueMap;
import org.springframework.util.MultiValueMap;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.client.RestTemplate;

import java.io.IOException;
import java.util.Collections;
import java.util.Map;
import java.util.Optional;
import java.util.UUID;

@RestController
@RequestMapping("/auth/google")
public class OauthController {

    @GetMapping("/init")
    public void initOAuth(@RequestParam(defaultValue = "EMPLOYEE") String role,
                          @RequestParam(defaultValue = "Asia/Kolkata") String timezone,
                          HttpServletRequest request,
                          HttpServletResponse response) throws IOException {

        // Now hand off to Spring's built-in OAuth2 flow
        System.out.println("Now hand off to Spring's built-in OAuth2 flow");
        response.sendRedirect("/oauth2/authorization/google");
    }
}

```

20 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java

124 line

```
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Service.RestaurantService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.stream.Collectors;

/**
 * Restaurant management endpoints.
 *
 *  VENDOR-ONLY (write operations)
 * POST   /restaurants                        — Create a restaurant (vendor supplies vendorId in body)
 * PUT    /restaurants/{id}                   — Update restaurant details (vendor ownership verified)
 * PUT    /restaurants/{id}/deactivate        — Soft-delete a restaurant  (vendor ownership verified)
 * GET    /restaurants/vendor/{vendorId}      — List MY restaurants (vendor views their own)
 *
 *  EMPLOYEE / MANAGER (read-only browse)
 * GET    /restaurants                        — Browse all active restaurants
 * GET    /restaurants/{id}                   — Get a specific restaurant
 * GET    /restaurants/by-slot?slot=LUNCH     — Find restaurants that serve a given meal slot
 *
 * Employees use the browse endpoints to pick a restaurant when creating a subscription.
 * Employees and managers cannot create, update, or deactivate restaurants.
 */
@RestController
@RequestMapping("/restaurants")
public class RestaurantController {

    private final RestaurantService restaurantService;

    public RestaurantController(RestaurantService restaurantService) {
        this.restaurantService = restaurantService;
    }

    //  VENDOR-ONLY write operations

    /**
     * Create a new restaurant.
     * Request body must include vendorId — the service validates the vendor exists.
     */
    @PostMapping
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<RestaurantDTO> createRestaurant(@RequestBody RestaurantRequest
request,Authentication authentication) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(restaurantService.createRestaurant(request,authentication));
    }

    /**
     * Update a restaurant's name, address, description, or supported meal slots.
     * vendorId in the request body is used to verify ownership before any change is made.
     */
    @PutMapping("/{id}")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<RestaurantDTO> updateRestaurant(@PathVariable Long id,
                                                          @RequestBody RestaurantRequest request,
                                                          Authentication authentication) {
        return ResponseEntity.ok(restaurantService.updateRestaurant(id, request,authentication));
    }

    /**
     * Soft-deactivate a restaurant. Active subscriptions remain in the DB but
     * new deliveries will stop since the restaurant is inactive.
```

```
     * Pass vendorId as a query param so ownership can be verified.
     */
    @PutMapping("/{id}/deactivate")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<String> deactivateRestaurant(@PathVariable Long id,
                                                       Authentication authentication) {
        restaurantService.deactivateRestaurant(id, authentication);
        return ResponseEntity.ok("Restaurant " + id + " has been deactivated.");
    }

    /**
     * List all active restaurants owned by a specific vendor.
     * Only the vendor themselves should call this to see their portfolio.
     */
    @GetMapping("/vendor")
    @PreAuthorize("hasRole('VENDOR')")
    public ResponseEntity<List<RestaurantDTO>> getRestaurantsByVendor(Authentication authentication) {
        return ResponseEntity.ok(restaurantService.getRestaurantsByVendor(authentication));
    }

    //  EMPLOYEE / MANAGER read-only browse

    /**
     * Browse all active restaurants.
     * Employees use this to discover available restaurants before subscribing.
     */
    @GetMapping
    @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
    public ResponseEntity<List<RestaurantDTO>> getAllActiveRestaurants() {
         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
     }

     /**
      * Get a specific restaurant by id.
      */
     @GetMapping("/{id}")
     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
     public ResponseEntity<RestaurantDTO> getRestaurant(@PathVariable Long id) {
         return ResponseEntity.ok(restaurantService.getRestaurant(id));
     }

     /**
      * Find all active restaurants that serve a specific meal slot.
      * Example: GET /restaurants/by-slot?slot=LUNCH
      * Employees call this when building their subscription to see valid options for each slot.
      */
     @GetMapping("/by-slot")
     @PreAuthorize("hasAnyRole('ADMIN','VENDOR','EMPLOYEE','MANAGER')")
     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByMealSlot(@RequestParam MealSlot slot) {
         return ResponseEntity.ok(restaurantService.getRestaurantsByMealSlot(slot));
     }

}
```

21 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java

50 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Scheduler.LeaveStatusScheduler;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

 /**
  * Controller for Scheduler Operations
  */
 @RestController
 @RequestMapping("/scheduler")
 public class SchedulerController {

     private static final Logger log = LoggerFactory.getLogger(SchedulerController.class);

     private final LeaveStatusScheduler leaveStatusScheduler;

     public SchedulerController(LeaveStatusScheduler leaveStatusScheduler) {
         this.leaveStatusScheduler = leaveStatusScheduler;
     }

     /**
      * Manual trigger for leave scheduler
      */
     @PostMapping("/run")
     public ResponseEntity<?> runScheduler() {
         try {
             leaveStatusScheduler.runManually();

             return ResponseEntity.ok().body(
                     Map.of(
                             "status", "success",
                             "message", "Scheduler executed successfully"
                     )
             );

         } catch (Exception e) {
             return ResponseEntity.internalServerError().body(
                     Map.of(
                             "status", "error",
                             "message", e.getMessage()
                     )
             );
         }
     }
 }
```

22 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.AdminActionDTO;
import com.example.EmployeeManagementSystem.DTO.RepairDTO;
import com.example.EmployeeManagementSystem.DTO.ServiceRequestDTO;
import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
import com.example.EmployeeManagementSystem.Service.ServiceRequestService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
@RequestMapping("/service-requests")
public class ServiceRequestController {
    private final ServiceRequestService serviceRequestService;

    public ServiceRequestController(ServiceRequestService serviceRequestService) {
        this.serviceRequestService = serviceRequestService;
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN')")
    public ResponseEntity<ServiceRequest> createServiceRequest(
            @RequestBody ServiceRequestDTO serviceRequestDTO,
            Authentication authentication) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(serviceRequestService.createServiceRequest(serviceRequestDTO, authentication));
    }

    @GetMapping("/open")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<List<ServiceRequestDTO>> getAllOpenServiceRequests() {
        return ResponseEntity.ok(serviceRequestService.getAllOpenServiceRequests());
    }

    @PutMapping("/admin")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<ServiceRequest> updateServiceRequestByAdmin(
            Authentication authentication,
            @RequestBody AdminActionDTO actionDTO) {
        return ResponseEntity.ok(serviceRequestService.updateServiceRequestByAdmin(authentication, actionDTO));
    }

    @GetMapping("/vendor/me")
    @PreAuthorize("hasRole('TECH_VENDOR')")
    public ResponseEntity<List<ServiceRequest>> getAllServiceRequestByVendor(Authentication authentication) {
        return ResponseEntity.ok(serviceRequestService.getAllServiceRequestByVendor(authentication));
    }

    @PutMapping("/vendor")
    @PreAuthorize("hasRole('TECH_VENDOR')")
    public ResponseEntity<ServiceRequest> updateServiceRequestByVendor(
            Authentication authentication,
            @RequestBody RepairDTO repairDTO) {
        return ResponseEntity.ok(serviceRequestService.updateServiceRequestByVendor(authentication, repairDTO));
    }
}
```

23 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java

9 li

```
package com.example.EmployeeManagementSystem.Controller;

import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContext;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
import org.springframework.security.web.context.SecurityContextRepository;
import org.springframework.web.bind.annotation.*;
import jakarta.servlet.http.Cookie;

import java.util.Map;

@RestController
@RequestMapping("/session")
public class SessionAuthController {

    private final AuthenticationManager authenticationManager;
    private final SecurityContextRepository securityContextRepository = new HttpSessionSecurityContextRepository();

    public SessionAuthController(AuthenticationManager authenticationManager) {
        this.authenticationManager = authenticationManager;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody Map<String, String> body,
                                   HttpServletRequest request,
                                   HttpServletResponse response) {
        String username = body.get("username");
        String password = body.get("password");

        try {
            Authentication auth = authenticationManager.authenticate(
                    new UsernamePasswordAuthenticationToken(username, password)
            );

            // Create and set security context
            SecurityContext context = SecurityContextHolder.createEmptyContext();
            context.setAuthentication(auth);
            SecurityContextHolder.setContext(context);

            // CRITICAL: Save the context to the HTTP session
            HttpSession session = request.getSession(true);
            securityContextRepository.saveContext(context, request, response);

            // Alternative: Manual session attribute (redundant but safe)
            session.setAttribute(HttpSessionSecurityContextRepository.SPRING_SECURITY_CONTEXT_KEY, context);

            return ResponseEntity.ok(Map.of(
                    "message", "Login successful",
                    "username", auth.getName(),
                    "sessionId", session.getId(),
                    "roles", auth.getAuthorities().toString()
            ));

        } catch (Exception e) {
            return ResponseEntity.status(401).body(Map.of("error", "Invalid credentials: " + e.getMessage()));
        }
    }

    @PostMapping("/logout")
    public ResponseEntity<?> logout(HttpSession session,
                                    HttpServletRequest request,
                                    HttpServletResponse response) {
```

```
        // Invalidate Spring Security context
        SecurityContextHolder.clearContext();

        // Invalidate HTTP session (kills OAuth2 session too)
        session.invalidate();

        // Clear the session cookie
        Cookie cookie = new Cookie("JSESSIONID", null);
        cookie.setMaxAge(0);
        cookie.setPath("/");
        response.addCookie(cookie);

        return ResponseEntity.ok("Logged out");
    }

    @GetMapping("/me")
    public ResponseEntity<?> me(Authentication authentication) {
        if (authentication == null || !authentication.isAuthenticated()) {
            return ResponseEntity.status(401).body(Map.of("error", "Not logged in"));
        }
        return ResponseEntity.ok(Map.of(
                "username", authentication.getName(),
                "roles", authentication.getAuthorities().toString(),
                "authenticated", true
        ));
    }
}
```

24 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java

7 li

```
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Service.SubscriptionService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/subscription")
public class SubscriptionController {
    private final SubscriptionService subscriptionService;

    public SubscriptionController(SubscriptionService subscriptionService) {
        this.subscriptionService = subscriptionService;
    }

    //Authentication makes sure that whoever is logged can add subscription and cannot add other subscription
    @PostMapping
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN','VENDOR')")
    public ResponseEntity<?> addSubscription(@RequestBody SubscriptionRequest request,Authentication authentication) {
        subscriptionService.addSubscription(request,authentication);
        return ResponseEntity.status(HttpStatus.CREATED)
                .body("Added new subscription for user: " + authentication.getName());
    }

    @GetMapping
    @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
    public List<SubscriptionDTO> getAllSubscriptions() {
        return subscriptionService.getAllSubscriptions();
    }


    @PutMapping("/{id}")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> updateSubscription(
            @PathVariable Long id,
            @RequestBody SubscriptionRequest request) {
        return ResponseEntity.ok(subscriptionService.updateSubscription(id, request));
    }

    @PutMapping("/{id}/pause")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> pauseSubscription(@PathVariable Long id) {
        return ResponseEntity.ok(subscriptionService.pauseSubscription(id));
    }

    @PutMapping("/{id}/resume")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> resumeSubscription(@PathVariable Long id) {
        return ResponseEntity.ok(subscriptionService.resumeSubscription(id));
    }

    @PutMapping("/{id}/expire")
    @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
    public ResponseEntity<Subscription> expireSubscription(@PathVariable Long id) {
        return ResponseEntity.ok(subscriptionService.expireSubscription(id));
    }

    @GetMapping("/user")
    @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
    public ResponseEntity<List<SubscriptionDTO>> getSubscriptionOfUser(Authentication authentication){
        return ResponseEntity.ok(subscriptionService.getSubscriptionOfUser(authentication));
```

```
    }

}

```

25 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java

101 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Service.TotpService;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/totp")
public class TotpController {

    private final TotpService totpService;
    private final EmployeeRepo employeeRepo;

    public TotpController(TotpService totpService, EmployeeRepo employeeRepo) {
        this.totpService = totpService;
        this.employeeRepo = employeeRepo;
    }

    /**
     * Step 1 — Employee calls this to get their QR code URI.
     * They scan it with Google Authenticator / Authy.
     */
    @PostMapping("/setup")
    public ResponseEntity<?> setup(Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        String secret = totpService.generateSecret();
        employee.setTotpSecret(secret);
        employee.setTotpEnabled(false); // not enabled until verified
        employeeRepo.save(employee);

        String qrUri = totpService.getQrCodeUri(secret, employee.getName());

        return ResponseEntity.ok(Map.of(
                "secret", secret,          // show this as backup code
                "qrCodeUri", qrUri         // embed this in a QR image on frontend
        ));
    }

    /**
     * Step 2 — Employee scans QR, types their first code to confirm setup.
     * Only after this is totpEnabled = true.
     */
    @PostMapping("/verify-setup")
    public ResponseEntity<?> verifySetup(@RequestBody Map<String, String> body,
                                         Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        String code = body.get("code");
        if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
            return ResponseEntity.badRequest().body(Map.of("error", "Invalid code. Try again."));
        }

        employee.setTotpEnabled(true);
        employeeRepo.save(employee);

        return ResponseEntity.ok(Map.of("message", "TOTP 2FA enabled successfully"));
    }

    /**
     * Step 3 — On every login, if totpEnabled=true, call this after Basic Auth succeeds.
     * Returns 200 if code is valid (you can then issue a session or JWT).
     */
    @PostMapping("/validate")
```

```
    public ResponseEntity<?> validate(@RequestBody Map<String, String> body,
                                      Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        if (!employee.isTotpEnabled()) {
            return ResponseEntity.ok(Map.of("message", "TOTP not enabled for this user"));
        }

        String code = body.get("code");
        if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
            return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
        }

        return ResponseEntity.ok(Map.of("message", "TOTP verified", "authenticated", true));
    }

    /** Disable TOTP for the logged-in employee */
    @DeleteMapping("/disable")
    public ResponseEntity<?> disable(Authentication authentication) {
        Employee employee = employeeRepo.findByName(authentication.getName())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        employee.setTotpSecret(null);
        employee.setTotpEnabled(false);
        employeeRepo.save(employee);

        return ResponseEntity.ok(Map.of("message", "TOTP disabled"));
     }
}
```

26 Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java

59 lines

```java
package com.example.EmployeeManagementSystem.Controller;

import com.example.EmployeeManagementSystem.DTO.VendorDTO;
import com.example.EmployeeManagementSystem.DTO.VendorRequest;
import com.example.EmployeeManagementSystem.Service.DeliveryService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/vendors")
public class VendorController {

    private final DeliveryService.VendorService vendorService;

    public VendorController(DeliveryService.VendorService vendorService) {
        this.vendorService = vendorService;
    }

    @PostMapping("/register")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
                .body(vendorService.registerVendor(request));
    }

    @GetMapping
    @PreAuthorize("hasAnyRole('FOOD_VENDOR','ADMIN','TECH_VENDOR')")
    public ResponseEntity<List<VendorDTO>> getAllVendors() {
        return ResponseEntity.ok(vendorService.getAllVendors());
    }

    @GetMapping("/me")
    @PreAuthorize("hasAuthority('VENDOR_WRITE')")
    public ResponseEntity<VendorDTO> getVendor(Authentication authentication) {
        return ResponseEntity.ok(vendorService.getVendor(authentication));
    }


    @PutMapping("/update")
    @PreAuthorize("hasAuthority('VENDOR_WRITE')")
    public ResponseEntity<VendorDTO> updateVendor(
            @RequestBody VendorRequest request,
            Authentication authentication) {
        return ResponseEntity.ok(vendorService.updateVendor(request, authentication));
    }

    @DeleteMapping
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<String> deleteVendor(Authentication authentication) {
        vendorService.deleteVendor(authentication);
        return ResponseEntity.ok(
                "Vendor account for " + authentication.getName() + " has been closed.");
    }
}
```

27 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java

18 lines

```
package com.example.EmployeeManagementSystem.DTO;

public class AccessTokenRequest {
    private String refreshToken;

    public AccessTokenRequest(String refreshToken) {
        this.refreshToken = refreshToken;
    }

     public void setRefreshToken(String refreshToken) {
         this.refreshToken = refreshToken;
     }

     public String getRefreshToken() {
         return refreshToken;
     }
 }

```

```java
package com.example.EmployeeManagementSystem.DTO;


public class ActionDTO {
    private long leaveRequestId;
    private String action;
    private String remarks;

    public long getLeaveRequestId() {
        return leaveRequestId;
     }

     public void setLeaveRequestId(long leaveRequestId) {
        this.leaveRequestId = leaveRequestId;
     }


     public String getAction() {
        return action;
     }

     public void setAction(String action) {
        this.action = action;
     }

     public String getRemarks() {
        return remarks;
     }

     public void setRemarks(String remarks) {
        this.remarks = remarks;
     }
 }

```

29 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminActionDTO.java

34 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;

public class AdminActionDTO {
    private Long serviceRequestId;
    private ServiceRequestStatus status;
    private String adminRemarks;

     public Long getServiceRequestId() {
         return serviceRequestId;
     }

     public void setServiceRequestId(Long serviceRequestId) {
         this.serviceRequestId = serviceRequestId;
     }

     public String getAdminRemarks() {
         return adminRemarks;
     }

     public void setAdminRemarks(String adminRemarks) {
         this.adminRemarks = adminRemarks;
     }

     public ServiceRequestStatus getStatus() {
         return status;
     }

     public void setStatus(ServiceRequestStatus status) {
         this.status = status;
     }
 }

```

30 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AssignDeviceRequest.java

14 lines

```java
package com.example.EmployeeManagementSystem.DTO;

public class AssignDeviceRequest {
    private Long employeeId;

    public Long getEmployeeId() {
        return employeeId;
    }

     public void setEmployeeId(Long employeeId) {
         this.employeeId = employeeId;
     }
 }

```

31 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java

28 lines

```java
package com.example.EmployeeManagementSystem.DTO;


import com.fasterxml.jackson.annotation.JsonProperty;

public class AuthRequest {
    @JsonProperty("username")
    private String Username;
    @JsonProperty("password")
     private String Password;

     public String getPassword() {
         return Password;
     }

     public void setPassword(String password) {
         Password = password;
     }

     public String getUsername() {
         return Username;
     }

     public void setUsername(String username) {
         Username = username;
     }
 }

```

32 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java

28 lines

```java
package com.example.EmployeeManagementSystem.DTO;

public class AuthResponse {
    private String accessToken;
    private String refreshToken;

    public AuthResponse(String accessToken, String token) {
        this.accessToken=accessToken;
        this.refreshToken=token;
     }

     public String getAccessToken() {
         return accessToken;
     }

     public void setAccessToken(String accessToken) {
         this.accessToken = accessToken;
     }

     public String getRefreshToken() {
         return refreshToken;
     }

     public void setRefreshToken(String refreshToken) {
         this.refreshToken = refreshToken;
     }
 }

```

33 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.jav

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
import com.example.EmployeeManagementSystem.Enum.DeviceType;

import java.time.LocalDate;

public class DeviceAssignmentResponseDTO {
    private Long id;
     private Long deviceId;
     private String deviceName;
     private String deviceBrand;
     private DeviceType deviceType;
     private Long employeeId;
     private String employeeName;
     private AssignmentStatus status;
     private LocalDate assignedDate;

     public Long getId() {
         return id;
     }

     public void setId(Long id) {
         this.id = id;
     }

     public String getDeviceName() {
         return deviceName;
     }

     public void setDeviceName(String deviceName) {
         this.deviceName = deviceName;
     }

     public Long getDeviceId() {
         return deviceId;
     }

     public void setDeviceId(Long deviceId) {
         this.deviceId = deviceId;
     }

     public DeviceType getDeviceType() {
         return deviceType;
     }

     public void setDeviceType(DeviceType deviceType) {
         this.deviceType = deviceType;
     }

     public String getDeviceBrand() {
         return deviceBrand;
     }

     public void setDeviceBrand(String deviceBrand) {
         this.deviceBrand = deviceBrand;
     }

     public String getEmployeeName() {
         return employeeName;
     }

     public void setEmployeeName(String employeeName) {
         this.employeeName = employeeName;
     }

     public Long getEmployeeId() {
         return employeeId;
     }

     public void setEmployeeId(Long employeeId) {
```

```
        this.employeeId = employeeId;
    }

    public AssignmentStatus getStatus() {
        return status;
    }

    public void setStatus(AssignmentStatus status) {
        this.status = status;
    }

    public LocalDate getAssignedDate() {
        return assignedDate;
    }

    public void setAssignedDate(LocalDate assignedDate) {
        this.assignedDate = assignedDate;
    }
}

```

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.DeviceType;

import java.time.LocalDate;

public class DeviceDTO {
    private String deviceName;       // e.g. "Dell XPS 15"
    private String brand;
     private DeviceType deviceType;
     private LocalDate warrantyExpiryDate;

     public String getDeviceName() {
         return deviceName;
     }

     public void setDeviceName(String deviceName) {
         this.deviceName = deviceName;
     }

     public String getBrand() {
         return brand;
     }

     public void setBrand(String brand) {
         this.brand = brand;
     }

     public DeviceType getDeviceType() {
         return deviceType;
     }

     public void setDeviceType(DeviceType deviceType) {
         this.deviceType = deviceType;
     }

     public LocalDate getWarrantyExpiryDate() {
         return warrantyExpiryDate;
     }

     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
         this.warrantyExpiryDate = warrantyExpiryDate;
     }
 }

```

35 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceResponseDTO.java

103 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
import com.example.EmployeeManagementSystem.Enum.DeviceType;

import java.time.LocalDate;

 public class DeviceResponseDTO {
     private Long id;
     private String deviceName;
     private String brand;
     private DeviceType deviceType;
     private DeviceStatus deviceStatus;
     private Vendor vendor;
     private Employee assignedEmployee;
     private String assignedEmployeeName;
     private LocalDate assignedDate;

     private LocalDate warrantyExpiryDate;

     public Long getId() {
         return id;
     }

     public void setId(Long id) {
         this.id = id;
     }

     public String getDeviceName() {
         return deviceName;
     }

     public void setDeviceName(String deviceName) {
         this.deviceName = deviceName;
     }

     public String getBrand() {
         return brand;
     }

     public void setBrand(String brand) {
         this.brand = brand;
     }

     public DeviceType getDeviceType() {
         return deviceType;
     }

     public void setDeviceType(DeviceType deviceType) {
         this.deviceType = deviceType;
     }

     public DeviceStatus getDeviceStatus() {
         return deviceStatus;
     }

     public void setDeviceStatus(DeviceStatus deviceStatus) {
         this.deviceStatus = deviceStatus;
     }

     public Vendor getVendor() {
         return vendor;
     }

     public void setVendor(Vendor vendor) {
         this.vendor = vendor;
     }

     public Employee getAssignedEmployee() {
```

```
        return assignedEmployee;
    }

    public void setAssignedEmployee(Employee assignedEmployee) {
        this.assignedEmployee = assignedEmployee;
    }

    public String getAssignedEmployeeName() {
        return assignedEmployeeName;
    }

    public void setAssignedEmployeeName(String assignedEmployeeName) {
        this.assignedEmployeeName = assignedEmployeeName;
    }

    public LocalDate getAssignedDate() {
        return assignedDate;
    }

    public void setAssignedDate(LocalDate assignedDate) {
        this.assignedDate = assignedDate;
    }

    public LocalDate getWarrantyExpiryDate() {
        return warrantyExpiryDate;
    }

    public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
         this.warrantyExpiryDate = warrantyExpiryDate;
     }
}

```

36 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.j

```java
package com.example.EmployeeManagementSystem.DTO;

import java.util.List;

public class EmployeeAttendanceResponseDTO {
    private long workingCount;
    private long onLeaveCount;
    private List<EmployeeStatusDTO> workingEmployees;
    private List<EmployeeStatusDTO> onLeaveEmployees;

     public long getWorkingCount() {
         return workingCount;
     }

     public void setWorkingCount(long workingCount) {
         this.workingCount = workingCount;
     }

     public long getOnLeaveCount() {
         return onLeaveCount;
     }

     public void setOnLeaveCount(long onLeaveCount) {
         this.onLeaveCount = onLeaveCount;
     }

     public List<EmployeeStatusDTO> getWorkingEmployees() {
         return workingEmployees;
     }

     public void setWorkingEmployees(List<EmployeeStatusDTO> workingEmployees) {
         this.workingEmployees = workingEmployees;
     }

     public List<EmployeeStatusDTO> getOnLeaveEmployees() {
         return onLeaveEmployees;
     }

     public void setOnLeaveEmployees(List<EmployeeStatusDTO> onLeaveEmployees) {
         this.onLeaveEmployees = onLeaveEmployees;
     }
 }

```

37 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDetails.java

41 lines

```java
package com.example.EmployeeManagementSystem.DTO;

public class EmployeeDetails {
    private String name;
    private String email;
    private String password;
    private String timezone;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getTimezone() {
        return timezone;
    }

    public void setTimezone(String timezone) {
        this.timezone = timezone;
    }
}

```

38 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java

50 lines

```java
package com.example.EmployeeManagementSystem.DTO;


public class EmployeeDTO {
    private String name;
    private String email;
    private String dept;
    private String password;
    private String timezone;

     public String getTimezone() {
         return timezone;
     }

     public void setTimezone(String timezone) {
         this.timezone = timezone;
     }

     public String getName() {
         return name;
     }

     public void setName(String name) {
         this.name = name;
     }

     public String getEmail() {
         return email;
     }

     public void setEmail(String email) {
         this.email = email;
     }

     public String getDept() {
         return dept;
     }

     public void setDept(String dept) {
         this.dept = dept;
     }

     public String getPassword() {
         return password;
     }

     public void setPassword(String password) {
         this.password = password;
     }
 }
```

39 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeStatusDTO.java

71 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;

public class EmployeeStatusDTO {
    private long employeeId;
    private String name;
    private String email;
     private String dept;
     private Role role;
     private Status status;
     private String timezone;

     public long getEmployeeId() {
         return employeeId;
     }

     public void setEmployeeId(long employeeId) {
         this.employeeId = employeeId;
     }

     public String getName() {
         return name;
     }

     public void setName(String name) {
         this.name = name;
     }

     public String getEmail() {
         return email;
     }

     public void setEmail(String email) {
         this.email = email;
     }

     public String getDept() {
         return dept;
     }

     public void setDept(String dept) {
         this.dept = dept;
     }

     public Role getRole() {
         return role;
     }

     public void setRole(Role role) {
         this.role = role;
     }

     public Status getStatus() {
         return status;
     }

     public void setStatus(Status status) {
         this.status = status;
     }

     public String getTimezone() {
         return timezone;
     }

     public void setTimezone(String timezone) {
         this.timezone = timezone;
     }
 }

```

40 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveDashboardResponse.java

115 line

```java
package com.example.EmployeeManagementSystem.DTO;

import java.math.BigDecimal;
import java.util.List;

public class LeaveDashboardResponse {

    private Long   employeeId;
    private String employeeName;
     private int    year;

     // Individual leave balances
     private LeaveBalanceDto sick;
     private LeaveBalanceDto casual;
     private LeaveBalanceDto maternity;       // null for non-female employees

     // Carry-forward meter (casual + maternity combined)
     private CarryForwardMeterDto carryForwardMeter;

     // Active warnings (unread)
     private List<WarningDto> warnings;

     //  nested DTOs

     public static class LeaveBalanceDto {
         private String     leaveType;
         private BigDecimal opening;
         private BigDecimal accrued;
         private BigDecimal used;
         private BigDecimal available;
         private boolean    carriesForward;

         public LeaveBalanceDto(String leaveType, BigDecimal opening,
                                BigDecimal accrued, BigDecimal used,
                                BigDecimal available, boolean carriesForward) {
             this.leaveType     = leaveType;
             this.opening       = opening;
             this.accrued       = accrued;
             this.used          = used;
             this.available     = available;
             this.carriesForward = carriesForward;
         }

         public String     getLeaveType()      { return leaveType; }
         public BigDecimal getOpening()        { return opening; }
         public BigDecimal getAccrued()        { return accrued; }
         public BigDecimal getUsed()           { return used; }
         public BigDecimal getAvailable()      { return available; }
         public boolean    isCarriesForward()  { return carriesForward; }
     }

     public static class CarryForwardMeterDto {
         private BigDecimal current;     // e.g. 28.00
         private int        cap;         // always 30
         private String     label;       // "28/30"
         private String     status;      // SAFE | WARNING | CRITICAL | CAPPED

         public CarryForwardMeterDto(BigDecimal current, int cap) {
             this.current = current;
             this.cap     = cap;
             this.label   = current.toPlainString() + "/" + cap;
             this.status  = resolveStatus(current, cap);
         }

         private String resolveStatus(BigDecimal current, int cap) {
             int cmp = current.compareTo(BigDecimal.valueOf(cap));
             if (cmp >= 0)                                          return "CAPPED";
             if (current.compareTo(BigDecimal.valueOf(cap - 1)) >= 0) return "CRITICAL";
             if (current.compareTo(BigDecimal.valueOf(cap - 2)) >= 0) return "WARNING";
             return "SAFE";
         }
```

```

        public BigDecimal getCurrent() { return current; }
        public int        getCap()     { return cap; }
        public String     getLabel()   { return label; }
        public String     getStatus()  { return status; }
    }

    public static class WarningDto {
        private Long   id;
        private String type;
        private String message;
        private String date;

        public WarningDto(Long id, String type, String message, String date) {
            this.id = id; this.type = type;
            this.message = message; this.date = date;
        }

        public Long   getId()      { return id; }
        public String getType()    { return type; }
        public String getMessage() { return message; }
        public String getDate()    { return date; }
    }

    //  root getters & setters

    public Long                getEmployeeId()       { return employeeId; }
    public String              getEmployeeName()     { return employeeName; }
     public int                 getYear()             { return year; }
     public LeaveBalanceDto     getSick()             { return sick; }
     public LeaveBalanceDto     getCasual()           { return casual; }
     public LeaveBalanceDto     getMaternity()        { return maternity; }
     public CarryForwardMeterDto getCarryForwardMeter(){ return carryForwardMeter; }
     public List<WarningDto>    getWarnings()         { return warnings; }

     public void setEmployeeId(Long id)                        { this.employeeId = id; }
     public void setEmployeeName(String n)                     { this.employeeName = n; }
     public void setYear(int y)                                { this.year = y; }
     public void setSick(LeaveBalanceDto b)                    { this.sick = b; }
     public void setCasual(LeaveBalanceDto b)                  { this.casual = b; }
     public void setMaternity(LeaveBalanceDto b)               { this.maternity = b; }
     public void setCarryForwardMeter(CarryForwardMeterDto m)  { this.carryForwardMeter = m; }
     public void setWarnings(List<WarningDto> w)               { this.warnings = w; }
}
```

41 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java

48 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.LeaveType;

import java.time.LocalDate;

// DTO used when employee applies for leave.
// Contains only input fields required from employee.
public class LeaveRequestDTO {
    private LeaveType leaveType;
    private LocalDate startDate;
    private LocalDate endDate;
    private String reason;


    public LeaveType getLeaveType() {
        return leaveType;
    }

    public void setLeaveType(LeaveType leaveType) {
        this.leaveType = leaveType;
    }

    public LocalDate getStartDate() {
        return startDate;
    }

    public void setStartDate(LocalDate startDate) {
        this.startDate = startDate;
    }

    public LocalDate getEndDate() {
        return endDate;
    }

    public void setEndDate(LocalDate endDate) {
        this.endDate = endDate;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }
}

```

42 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java

73 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;

import java.time.LocalDate;

public class LeaveResponseDTO {
    private long leaveRequestId;
     private long employeeId;
     private LeaveType leaveType;
     private LocalDate startDate;
     private LocalDate endDate;
     private String reason;
     private LeaveStatus status;

     public long getLeaveRequestId() {
         return leaveRequestId;
     }

     public void setLeaveRequestId(long leaveRequestId) {
         this.leaveRequestId = leaveRequestId;
     }

     public long getEmployeeId() {
         return employeeId;
     }

     public void setEmployeeId(long employeeId) {
         this.employeeId = employeeId;
     }

     public LocalDate getStartDate() {
         return startDate;
     }

     public void setStartDate(LocalDate startDate) {
         this.startDate = startDate;
     }

     public LeaveType getLeaveType() {
         return leaveType;
     }

     public void setLeaveType(LeaveType leaveType) {
         this.leaveType = leaveType;
     }

     public LocalDate getEndDate() {
         return endDate;
     }

     public void setEndDate(LocalDate endDate) {
         this.endDate = endDate;
     }

     public String getReason() {
         return reason;
     }

     public void setReason(String reason) {
         this.reason = reason;
     }

     public LeaveStatus getStatus() {
         return status;
     }

     public void setStatus(LeaveStatus status) {
         this.status = status;
     }
```

```
}

```

43 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;

import java.time.LocalDate;

public class ManagerLeaveResponseDTO {
    private String employeeName;
     private LeaveType leaveType;
     private LocalDate startDate;
     private LocalDate endDate;
     private String reason;
     private LeaveStatus status;

     public String getEmployeeName() {
         return employeeName;
     }

     public void setEmployeeName(String employeeName) {
         this.employeeName = employeeName;
     }

     public LeaveType getLeaveType() {
         return leaveType;
     }

     public void setLeaveType(LeaveType leaveType) {
         this.leaveType = leaveType;
     }

     public LocalDate getStartDate() {
         return startDate;
     }

     public void setStartDate(LocalDate startDate) {
         this.startDate = startDate;
     }

     public LocalDate getEndDate() {
         return endDate;
     }

     public void setEndDate(LocalDate endDate) {
         this.endDate = endDate;
     }

     public String getReason() {
         return reason;
     }

     public void setReason(String reason) {
         this.reason = reason;
     }

     public LeaveStatus getStatus() {
         return status;
     }

     public void setStatus(LeaveStatus status) {
         this.status = status;
     }
 }

```

44 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\PromoteRequest.java

23 lines

```java
package com.example.EmployeeManagementSystem.DTO;

public class PromoteRequest {
    private String email;
    private String password;

    public String getEmail() {
        return email;
    }

     public void setEmail(String email) {
         this.email = email;
     }

     public String getPassword() {
         return password;
     }

     public void setPassword(String password) {
         this.password = password;
     }
 }

```

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;

public class RepairDTO {
    private Long requestId;
    private String resolution;
    private ServiceRequestStatus status;

     public Long getRequestId() {
         return requestId;
     }

     public void setRequestId(Long requestId) {
         this.requestId = requestId;
     }

     public String getResolution() {
         return resolution;
     }

     public void setResolution(String resolution) {
         this.resolution = resolution;
     }

     public ServiceRequestStatus getStatus() {
         return status;
     }

     public void setStatus(ServiceRequestStatus status) {
         this.status = status;
     }
 }

```

46 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java

42 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;

import java.util.Set;

public class RestaurantDTO {
    private Long id;
    private String name;
     private String address;
     private String description;
     private Set<MealSlot> supportedMealSlots;
     private boolean active;
     private Long vendorId;
     private String vendorName;

     public Long getId() { return id; }
     public void setId(Long id) { this.id = id; }

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getAddress() { return address; }
     public void setAddress(String address) { this.address = address; }

     public String getDescription() { return description; }
     public void setDescription(String description) { this.description = description; }

     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
         this.supportedMealSlots = supportedMealSlots;
     }

     public boolean isActive() { return active; }
     public void setActive(boolean active) { this.active = active; }

     public Long getVendorId() { return vendorId; }
     public void setVendorId(Long vendorId) { this.vendorId = vendorId; }

     public String getVendorName() { return vendorName; }
     public void setVendorName(String vendorName) { this.vendorName = vendorName; }
}
```

47 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java

46 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

import java.util.Set;

 public class RestaurantRequest {

     @NotBlank
     private String name;

     @NotBlank
     private String address;

     private String description;

     /**
      * Which meal slots this restaurant serves.
      * Must be at least one of: BREAKFAST, LUNCH, DINNER.
      */
     @NotNull
     @NotEmpty
     private Set<MealSlot> supportedMealSlots;

     /**
      * The vendor ID of the owner making this request.
      * Used to verify ownership on updates/deletes.
      */

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getAddress() { return address; }
     public void setAddress(String address) { this.address = address; }

     public String getDescription() { return description; }
     public void setDescription(String description) { this.description = description; }

     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
         this.supportedMealSlots = supportedMealSlots;
     }
 }
```

48 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestDTO.java

44 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;

public class ServiceRequestDTO {

    private Long deviceId;
    private ServiceRequestType requestType;
    private String issueDescription;
     private boolean urgent;

     public Long getDeviceId() {
         return deviceId;
     }

     public void setDeviceId(Long deviceId) {
         this.deviceId = deviceId;
     }

     public ServiceRequestType getRequestType() {
         return requestType;
     }

     public void setRequestType(ServiceRequestType requestType) {
         this.requestType = requestType;
     }

     public String getIssueDescription() {
         return issueDescription;
     }

     public void setIssueDescription(String issueDescription) {
         this.issueDescription = issueDescription;
     }

     public boolean isUrgent() {
         return urgent;
     }

     public void setUrgent(boolean urgent) {
         this.urgent = urgent;
     }
 }

```

49 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java

103 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;

import java.time.DayOfWeek;
import java.time.ZonedDateTime;

 public class SubscriptionDTO {
     private long subscriptionId;
     private long userId;
     private MealSlot mealSlot;
     private ScheduleType scheduleType;
     private DayOfWeek dayOfWeek;
     private SubscriptionStatus status;
     private ZonedDateTime nextDeliveryTime;

     private Long restaurantId;
     private String restaurantName;
     private String restaurantAddress;

     public void setRestaurantId(Long restaurantId) {
         this.restaurantId = restaurantId;
     }

     public void setRestaurantName(String restaurantName) {
         this.restaurantName = restaurantName;
     }

     public void setRestaurantAddress(String restaurantAddress) {
         this.restaurantAddress = restaurantAddress;
     }

     public Long getRestaurantId() {
         return restaurantId;
     }

     public String getRestaurantName() {
         return restaurantName;
     }

     public String getRestaurantAddress() {
         return restaurantAddress;
     }

     public long getSubscriptionId() {
         return subscriptionId;
     }

     public void setSubscriptionId(long subscriptionId) {
         this.subscriptionId = subscriptionId;
     }

     public long getUserId() {
         return userId;
     }

     public void setUserId(long userId) {
         this.userId = userId;
     }

     public MealSlot getMealSlot() {
         return mealSlot;
     }

     public void setMealSlot(MealSlot mealSlot) {
         this.mealSlot = mealSlot;
     }

     public ScheduleType getScheduleType() {
```

```
        return scheduleType;
    }

    public void setScheduleType(ScheduleType scheduleType) {
        this.scheduleType = scheduleType;
    }

    public DayOfWeek getDayOfWeek() {
        return dayOfWeek;
    }

    public void setDayOfWeek(DayOfWeek dayOfWeek) {
        this.dayOfWeek = dayOfWeek;
    }

    public SubscriptionStatus getStatus() {
        return status;
    }

    public void setStatus(SubscriptionStatus status) {
        this.status = status;
    }

    public ZonedDateTime getNextDeliveryTime() {
        return nextDeliveryTime;
    }

    public void setNextDeliveryTime(ZonedDateTime nextDeliveryTime) {
         this.nextDeliveryTime = nextDeliveryTime;
     }
}

```

50 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java

72 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

import java.time.DayOfWeek;
import java.util.List;

 public class SubscriptionRequest {
     private ScheduleType scheduleType;
     private DayOfWeek dayOfWeek;
     @NotNull
     @NotEmpty
     private List<MealSlot> mealSlots;
     @NotNull
     private List<SlotOrder> slotOrders;
     public static class SlotOrder {
         @NotNull
         private MealSlot mealSlot;

         @NotNull
         private Long restaurantId;

         public MealSlot getMealSlot() {
             return mealSlot;
         }

         public void setMealSlot(MealSlot mealSlot) {
             this.mealSlot = mealSlot;
         }

         public Long getRestaurantId() {
             return restaurantId;
         }

         public void setRestaurantId(Long restaurantId) {
             this.restaurantId = restaurantId;
         }
     }

     public ScheduleType getScheduleType() {
         return scheduleType;
     }

     public void setScheduleType(ScheduleType scheduleType) {
         this.scheduleType = scheduleType;
     }

     public List<MealSlot> getMealSlots() {
         return mealSlots;
     }

     public void setMealSlots(List<MealSlot> mealSlots) {
         this.mealSlots = mealSlots;
     }

     public DayOfWeek getDayOfWeek() {
         return dayOfWeek;
     }

     public void setDayOfWeek(DayOfWeek dayOfWeek) {
         this.dayOfWeek = dayOfWeek;
     }



     public List<SlotOrder> getSlotOrders() { return slotOrders; }
     public void setSlotOrders(List<SlotOrder> slotOrders) { this.slotOrders = slotOrders; }
 }
```

51 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java

26 lines

```java
package com.example.EmployeeManagementSystem.DTO;

import java.time.LocalDate;

public class VendorDTO {
    private Long id;
    private String name;
    private String email;
    private String phone;
     private LocalDate registeredAt;

     public Long getId() { return id; }
     public void setId(Long id) { this.id = id; }

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getEmail() { return email; }
     public void setEmail(String email) { this.email = email; }

     public String getPhone() { return phone; }
     public void setPhone(String phone) { this.phone = phone; }

     public LocalDate getRegisteredAt() { return registeredAt; }
     public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
 }
```

52 Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java

47 lines

```
package com.example.EmployeeManagementSystem.DTO;

import com.example.EmployeeManagementSystem.Enum.Role;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;

public class VendorRequest {

    @NotBlank
     private String name;

     @Email
     @NotBlank
     private String email;

     @NotBlank
     private String phone;

     private String password;

     private Role role;

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getEmail() { return email; }
     public void setEmail(String email) { this.email = email; }

     public String getPhone() { return phone; }
     public void setPhone(String phone) { this.phone = phone; }

     public String getPassword() {
         return password;
     }

     public void setPassword(String password) {
         this.password = password;
     }

     public Role getRole() {
         return role;
     }

     public void setRole(Role role) {
         this.role = role;
     }
 }
```

53 Current: src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
package com.example.EmployeeManagementSystem;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@SpringBootApplication
@EnableScheduling
public class EmployeeManagementSystemApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeManagementSystemApplication.class, args);
    }

 }

```

```java
// src/main/java/com/example/EmployeeManagementSystem/Entity/ApiKey.java
package com.example.EmployeeManagementSystem.Entity;

import jakarta.persistence.*;
import java.time.Instant;
import java.util.UUID;

@Entity
@Table(name = "api_keys")
 public class ApiKey {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String apiKey;

    @Column(nullable = false)
    private String keyName;

    @ManyToOne
    @JoinColumn(name = "employee_id")
    private Employee employee;

    @ManyToOne
    @JoinColumn(name = "vendor_id")
    private Vendor vendor;

    @Column(nullable = false)
    private String permissions; // Comma-separated permissions

    @Column(nullable = false)
    private Instant createdAt;

    private Instant expiresAt;

    private Instant lastUsedAt;

    @Column(nullable = false)
    private boolean active = true;

    @PrePersist
    public void prePersist() {
        this.apiKey = "ems_" + UUID.randomUUID().toString().replace("-", "");
        this.createdAt = Instant.now();
    }

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getApiKey() { return apiKey; }
    public void setApiKey(String apiKey) { this.apiKey = apiKey; }

    public String getKeyName() { return keyName; }
    public void setKeyName(String keyName) { this.keyName = keyName; }

    public Employee getEmployee() { return employee; }
    public void setEmployee(Employee employee) { this.employee = employee; }

    public Vendor getVendor() { return vendor; }
    public void setVendor(Vendor vendor) { this.vendor = vendor; }

    public String getPermissions() { return permissions; }
    public void setPermissions(String permissions) { this.permissions = permissions; }

    public Instant getCreatedAt() { return createdAt; }
    public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }

    public Instant getExpiresAt() { return expiresAt; }
```

```
    public void setExpiresAt(Instant expiresAt) { this.expiresAt = expiresAt; }

    public Instant getLastUsedAt() { return lastUsedAt; }
    public void setLastUsedAt(Instant lastUsedAt) { this.lastUsedAt = lastUsedAt; }

    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }
}
```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import jakarta.persistence.*;

import java.time.Instant;

@Entity
 @Table(name = "delivery", uniqueConstraints = { @UniqueConstraint(name = "uq_subscription_scheduled_time",
         columnNames = {"subscription_id", "scheduledDeliveryTime"})})
 public class Delivery {
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;

     @ManyToOne
     @JoinColumn(name = "subscription_id")
     private Subscription subscription;
     private MealSlot mealSlot;
     private Instant scheduledDeliveryTime;
     private Instant actualDeliveryTime;

     @Enumerated(EnumType.STRING)
     private DeliveryStatus status;

     private Instant createdAt;

     @PrePersist
     public void prePersist() {
         createdAt = Instant.now();
         if (status == null) {
             status = DeliveryStatus.SCHEDULED;
         }
     }

     public MealSlot getMealSlot() {
         return mealSlot;
     }

     public void setMealSlot(MealSlot mealSlot) {
         this.mealSlot = mealSlot;
     }

     public Subscription getSubscription() {
         return subscription;
     }

     public void setSubscription(Subscription subscription) {
         this.subscription = subscription;
     }

     public Long getId() {
         return id;
     }

     public void setId(Long id) {
         this.id = id;
     }

     public Instant getScheduledDeliveryTime() {
         return scheduledDeliveryTime;
     }

     public void setScheduledDeliveryTime(Instant scheduledDeliveryTime) {
         this.scheduledDeliveryTime = scheduledDeliveryTime;
     }

     public Instant getActualDeliveryTime() {
         return actualDeliveryTime;
     }
```

```

    public void setActualDeliveryTime(Instant actualDeliveryTime) {
        this.actualDeliveryTime = actualDeliveryTime;
    }

    public DeliveryStatus getStatus() {
        return status;
    }

    public void setStatus(DeliveryStatus status) {
        this.status = status;
    }

    public Instant getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(Instant createdAt) {
        this.createdAt = createdAt;
    }
}

```

```java
package com.example.EmployeeManagementSystem.Entity;


import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
import com.example.EmployeeManagementSystem.Enum.DeviceType;
import jakarta.persistence.*;

import java.time.LocalDate;
import java.time.LocalDateTime;

 @Entity
 public class Device {
     @Id
     @GeneratedValue(strategy = GenerationType.AUTO)
     private Long id;

     private String deviceName;       // e.g. "Dell XPS 15"
     private String brand;

     @ManyToOne
     @JoinColumn(name = "vendor_id")
     private Vendor techVendor;

     @OneToOne(mappedBy = "device")
     private DeviceAssignment currentAssignment;

     private DeviceStatus deviceStatus;

     private DeviceType deviceType;

     private LocalDateTime createdAt;

     private LocalDate warrantyExpiryDate;

     public Long getId() {
         return id;
     }

     public void setId(Long id) {
         this.id = id;
     }

     public String getDeviceName() {
         return deviceName;
     }

     public void setDeviceName(String deviceName) {
         this.deviceName = deviceName;
     }

     public String getBrand() {
         return brand;
     }

     public void setBrand(String brand) {
         this.brand = brand;
     }

     public Vendor getTechVendor() {
         return techVendor;
     }

     public void setTechVendor(Vendor techVendor) {
         this.techVendor = techVendor;
     }

     public DeviceType getDeviceType() {
         return deviceType;
     }

     public void setDeviceType(DeviceType deviceType) {
```

```
        this.deviceType = deviceType;
    }

    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    public DeviceStatus getDeviceStatus() {
        return deviceStatus;
    }

    public void setDeviceStatus(DeviceStatus deviceStatus) {
        this.deviceStatus = deviceStatus;
    }

    public DeviceAssignment getCurrentAssignment() {
        return currentAssignment;
    }

    public void setCurrentAssignment(DeviceAssignment currentAssignment) {
        this.currentAssignment = currentAssignment;
    }

    public LocalDate getWarrantyExpiryDate() {
         return warrantyExpiryDate;
     }

     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
         this.warrantyExpiryDate = warrantyExpiryDate;
     }
}

```

57 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\DeviceAssignment.java

68 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
import jakarta.persistence.*;

import java.time.LocalDate;

@Entity
public class DeviceAssignment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne
    @JoinColumn(name = "device_id")
    private Device device;

    @ManyToOne
    @JoinColumn(name = "employee_id")
    private Employee assignedTo;

    @Enumerated(EnumType.STRING)
    private AssignmentStatus status;

    private LocalDate assignedDate;

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public Device getDevice() {
        return device;
    }

    public void setDevice(Device device) {
        this.device = device;
    }

    public Employee getAssignedTo() {
        return assignedTo;
    }

    public void setAssignedTo(Employee assignedTo) {
        this.assignedTo = assignedTo;
    }

    public LocalDate getAssignedDate() {
        return assignedDate;
    }

    public void setAssignedDate(LocalDate assignedDate) {
        this.assignedDate = assignedDate;
    }

    public AssignmentStatus getStatus() {
        return status;
    }

    public void setStatus(AssignmentStatus status) {
        this.status = status;
    }
}

```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.Gender;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

import java.time.LocalDate;
import java.util.Collection;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.stream.Collectors;

@Entity
public class Employee implements UserDetails {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long employeeId;
    @Column(nullable = true,unique = false)
    private String name;
    @Column(unique = true, nullable = false)
    private String email;
    @Column(nullable = true)
    private String dept;
    @Enumerated(EnumType.STRING)
    private Status status;
    private String password;

    //  Self-referential — replaces `String manager`
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "manager_id", nullable = true)
    @JsonIgnore // prevent infinite recursion in serialization
    private Employee manager;


    private LocalDate joined_at;
    @Enumerated(EnumType.STRING)
    private Role role;
    @Column(nullable = true)
    private String totpSecret;
    @Column(nullable = false)
    private boolean totpEnabled = false;
    /** IANA timezone id, e.g. "Asia/Kolkata", "America/New_York". Defaults to UTC. */
    @Column(nullable = false)
    private String timezone = "UTC";
    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private Gender gender;

    public String getTimezone() {
        return timezone;
    }

    public void setTimezone(String timezone) {
        this.timezone = timezone;
    }

    public Role getRole() {
        return role;
    }

    public void setRole(Role role) {
        this.role = role;
    }

    public long getEmployeeId() {
```

```
        return employeeId;
    }

    public void setEmployeeId(long employeeId) {
        this.employeeId = employeeId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getDept() {
        return dept;
    }

    public void setDept(String dept) {
         this.dept = dept;
     }

     public Status getStatus() {
         return status;
     }

     public void setStatus(Status status) {
         this.status = status;
     }

     public LocalDate getJoined_at() {
         return joined_at;
     }

     public void setJoined_at(LocalDate joined_at) {
         this.joined_at = joined_at;
     }

     public Gender getGender() { return gender; }
     public void setGender(Gender gender) { this.gender = gender; }

     @PrePersist
     public void initialSetup() {
         this.status = Status.ACTIVE;
         this.joined_at = LocalDate.now();
         if (this.timezone == null || this.timezone.isBlank()) {
             this.timezone = "UTC";
         }
     }

     @Override
     public Collection<? extends GrantedAuthority> getAuthorities() {
         Set<SimpleGrantedAuthority> authorities=new HashSet<>();
         authorities.add(new SimpleGrantedAuthority("ROLE_"+role.name()));
         authorities.addAll(role.getPermissions().stream()
                 .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
                 .collect(Collectors.toSet()));
         return authorities;
     }

     public String getPassword() {
         return password;
     }

     @Override
     public String getUsername() {
         return email;
```

```
    }

    @Override
    public boolean isAccountNonExpired() {
        return UserDetails.super.isAccountNonExpired();
    }

    @Override
    public boolean isAccountNonLocked() {
        return UserDetails.super.isAccountNonLocked();
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return UserDetails.super.isCredentialsNonExpired();
    }

    @Override
    public boolean isEnabled() {
        return UserDetails.super.isEnabled();
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getTotpSecret() {
        return totpSecret;
    }

    public boolean isTotpEnabled() {
        return totpEnabled;
    }

    public void setTotpSecret(String totpSecret) {
        this.totpSecret = totpSecret;
    }

    public void setTotpEnabled(boolean totpEnabled) {
        this.totpEnabled = totpEnabled;
    }

    public Employee getManager() {
        return manager;
    }

    public void setManager(Employee manager) {
        this.manager = manager;
    }

}
```

59 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveEntitlement.java

104 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import jakarta.persistence.*;

import java.math.BigDecimal;

@Entity
@Table(name = "leave_entitlement", uniqueConstraints = @UniqueConstraint(columnNames = {"employee_id", "leave_type_id", "year"}))
public class LeaveEntitlement {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "leave_type_id", nullable = false)
    private LeaveType leaveType;

    @Column(nullable = false)
    private Integer year;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal openingBalance = BigDecimal.ZERO;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal accruedThisYear = BigDecimal.ZERO;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal usedThisYear = BigDecimal.ZERO;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal closingBalance = BigDecimal.ZERO;

    // Convenience: current available = opening + accrued - used
    @Transient
    public BigDecimal getAvailableBalance() {
        return openingBalance.add(accruedThisYear).subtract(usedThisYear);
    }

    // --- getters & setters ---
    public Long getId() {
        return id;
    }

    public Employee getEmployee() {
        return employee;
    }

    public LeaveType getLeaveType() {
        return leaveType;
    }

    public Integer getYear() {
        return year;
    }

    public BigDecimal getOpeningBalance() {
        return openingBalance;
    }

    public BigDecimal getAccruedThisYear() {
        return accruedThisYear;
    }

    public BigDecimal getUsedThisYear() {
        return usedThisYear;
```

```
    }

    public BigDecimal getClosingBalance() {
        return closingBalance;
    }

    public void setEmployee(Employee e) {
        this.employee = e;
    }

    public void setLeaveType(LeaveType lt) {
        this.leaveType = lt;
    }

    public void setYear(Integer y) {
        this.year = y;
    }

    public void setOpeningBalance(BigDecimal b) {
        this.openingBalance = b;
    }

    public void setAccruedThisYear(BigDecimal a) {
        this.accruedThisYear = a;
    }

    public void setUsedThisYear(BigDecimal u) {
        this.usedThisYear = u;
    }

     public void setClosingBalance(BigDecimal c) {
         this.closingBalance = c;
     }
}
```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;
import jakarta.persistence.*;

import java.time.LocalDate;

@Entity
 public class LeaveRequest {
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private long id;
     @Enumerated(EnumType.STRING)
     private LeaveType leaveType;
     private LocalDate startDate;
     private LocalDate endDate;
     private String reason;
     @Enumerated(EnumType.STRING)
     private LeaveStatus status;
     private String Manager;
     private String remarks;
     @ManyToOne
     @JoinColumn(name = "employee_id")
     private Employee employee;


     public long getId() {
         return id;
     }

     public void setId(long id) {
         this.id = id;
     }

     public LeaveType getLeaveType() {
         return leaveType;
     }

     public void setLeaveType(LeaveType leaveType) {
         this.leaveType = leaveType;
     }

     public LocalDate getStartDate() {
         return startDate;
     }

     public void setStartDate(LocalDate startDate) {
         this.startDate = startDate;
     }

     public LocalDate getEndDate() {
         return endDate;
     }

     public void setEndDate(LocalDate endDate) {
         this.endDate = endDate;
     }

     public LeaveStatus getStatus() {
         return status;
     }

     public void setStatus(LeaveStatus status) {
         this.status = status;
     }

     public String getManager() {
         return Manager;
     }

```

```
    public void setManager(String manager) {
        Manager = manager;
    }

    public String getRemarks() {
        return remarks;
    }

    public void setRemarks(String remarks) {
        this.remarks = remarks;
    }

    public Employee getEmployee() {
        return employee;
    }

    public void setEmployee(Employee employee) {
        this.employee = employee;
    }

    public String getReason() {
        return reason;
    }

    public void setReason(String reason) {
        this.reason = reason;
    }

     @PrePersist
     public void init(){
         this.status=LeaveStatus.PENDING;
     }
}

```

61 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveType.java

49 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.Gender;
import jakarta.persistence.*;

@Entity
@Table(name = "leave_type")
public class LeaveType {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false, unique = true, length = 50)
    private String name;                    // SICK, CASUAL, MATERNITY

    @Column(nullable = false)
    private boolean carriesForward;

    @Column(nullable = false)
    private boolean monthlyReset;           // true = sick leave wipes at month end

    @Column(nullable = false)
    private int maxCarryForward;            // 30 for carry-forwardable types

    @Column(nullable = false)
    private boolean genderRestricted;

    @Enumerated(EnumType.STRING)
    @Column(nullable = true)
    private Gender genderRestriction;       // null = open to all

    // --- getters & setters ---
    public Integer getId() { return id; }
    public String getName() { return name; }
    public boolean isCarriesForward() { return carriesForward; }
    public boolean isMonthlyReset() { return monthlyReset; }
    public int getMaxCarryForward() { return maxCarryForward; }
    public boolean isGenderRestricted() { return genderRestricted; }
    public Gender getGenderRestriction() { return genderRestriction; }

    public void setId(Integer id) { this.id = id; }
    public void setName(String name) { this.name = name; }
    public void setCarriesForward(boolean cf) { this.carriesForward = cf; }
    public void setMonthlyReset(boolean mr) { this.monthlyReset = mr; }
    public void setMaxCarryForward(int max) { this.maxCarryForward = max; }
    public void setGenderRestricted(boolean gr) { this.genderRestricted = gr; }
    public void setGenderRestriction(Gender g) { this.genderRestriction = g; }
}
```

62 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveWarning.java

58 lines

```java
package com.example.EmployeeManagementSystem.Entity;


import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDate;

@Entity
@Table(name = "leave_warning")
 public class LeaveWarning {

    public enum WarningType {
        APPROACHING_28,   // 2 months before cap
        APPROACHING_29,   // 1 month before cap
        CAP_REACHED,      // exactly at 30
        DAY_WASTED        // already at 30, day lost
    }

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private WarningType warningType;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String message;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal carryForwardBalance;

    @Column(nullable = false)
    private LocalDate warningDate;

    @Column(nullable = false)
    private boolean isRead = false;

    // --- getters & setters ---
    public Long getId() { return id; }
    public Employee getEmployee() { return employee; }
    public WarningType getWarningType() { return warningType; }
    public String getMessage() { return message; }
    public BigDecimal getCarryForwardBalance() { return carryForwardBalance; }
    public LocalDate getWarningDate() { return warningDate; }
    public boolean isRead() { return isRead; }

    public void setEmployee(Employee e) { this.employee = e; }
    public void setWarningType(WarningType wt) { this.warningType = wt; }
    public void setMessage(String m) { this.message = m; }
    public void setCarryForwardBalance(BigDecimal b) { this.carryForwardBalance = b; }
    public void setWarningDate(LocalDate d) { this.warningDate = d; }
    public void setRead(boolean r) { this.isRead = r; }
}
```

63 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java

62 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import jakarta.persistence.*;

import java.time.Instant;
@Entity
public class RefreshToken {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;

     @Column(nullable = false, unique = true)
     private String token;

     @Column(nullable = false)
     private Instant expiryTime;

     @ManyToOne
     @JoinColumn(name = "employee_id")
     private Employee employee;

     @ManyToOne
     @JoinColumn(name = "vendor_id")
     private Vendor vendor;

     public Long getId() {
         return id;
     }

     public String getToken() {
         return token;
     }

     public void setToken(String token) {
         this.token = token;
     }

     public Instant getExpiryTime() {
         return expiryTime;
     }

     public void setExpiryTime(Instant expiryTime) {
         this.expiryTime = expiryTime;
     }

     public Employee getEmployee() {
         return employee;
     }

     public void setEmployee(Employee employee) {
         this.employee = employee;
     }

     public Vendor getVendor() {
         return vendor;
     }

     public void setVendor(Vendor vendor) {
         this.vendor = vendor;
     }
 }

```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import jakarta.persistence.*;
import java.util.Set;


@Entity
public class Restaurant {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false)
    private String address;

    private String description;

    @ElementCollection(targetClass = MealSlot.class, fetch = FetchType.EAGER)
    @CollectionTable(name = "restaurant_meal_slots",
            joinColumns = @JoinColumn(name = "restaurant_id"))
    @Enumerated(EnumType.STRING)
    @Column(name = "meal_slot")
    private Set<MealSlot> supportedMealSlots;

    private boolean active = true;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "vendor_id", nullable = false)
    private Vendor vendor;

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getAddress() { return address; }
    public void setAddress(String address) { this.address = address; }

    public String getDescription() { return description; }
    public void setDescription(String description) { this.description = description; }

    public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
    public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
        this.supportedMealSlots = supportedMealSlots;
    }

    public boolean isActive() { return active; }
    public void setActive(boolean active) { this.active = active; }

    public Vendor getVendor() { return vendor; }
    public void setVendor(Vendor vendor) { this.vendor = vendor; }
}
```

65 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ServiceRequest.java

141 lines

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
import jakarta.persistence.*;

import java.time.LocalDateTime;

@Entity
 public class ServiceRequest {
     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;

     @ManyToOne
     @JoinColumn(name = "device_id")
     private Device device;

     @ManyToOne
     @JoinColumn(name = "raised_by")
     private Employee raisedBy;       // employee reporting issue

     @ManyToOne
     @JoinColumn(name = "reviewed_by")
     private Employee reviewedBy;     // admin who reviewed

     @Enumerated(EnumType.STRING)
     private ServiceRequestType requestType;

     @Enumerated(EnumType.STRING)
     private ServiceRequestStatus status;

     private String issueDescription; // "screen cracked", "won't boot"
     private String adminRemarks;     // admin notes on decision
     private String resolution;       // what was done finally

     private boolean urgent;          // employee can flag as urgent

     private LocalDateTime raisedAt;
     private LocalDateTime resolvedAt;

     @PrePersist
     void onCreate() { this.raisedAt = LocalDateTime.now(); }

     public Employee getRaisedBy() {
         return raisedBy;
     }

     public void setRaisedBy(Employee raisedBy) {
         this.raisedBy = raisedBy;
     }

     public Device getDevice() {
         return device;
     }

     public void setDevice(Device device) {
         this.device = device;
     }

     public Long getId() {
         return id;
     }

     public void setId(Long id) {
         this.id = id;
     }

     public Employee getReviewedBy() {
         return reviewedBy;
     }
```

```
    public void setReviewedBy(Employee reviewedBy) {
        this.reviewedBy = reviewedBy;
    }

    public ServiceRequestStatus getStatus() {
        return status;
    }

    public void setStatus(ServiceRequestStatus status) {
        this.status = status;
    }

    public ServiceRequestType getRequestType() {
        return requestType;
    }

    public void setRequestType(ServiceRequestType requestType) {
        this.requestType = requestType;
    }

    public String getIssueDescription() {
        return issueDescription;
    }

    public void setIssueDescription(String issueDescription) {
        this.issueDescription = issueDescription;
    }

     public String getAdminRemarks() {
         return adminRemarks;
     }

     public void setAdminRemarks(String adminRemarks) {
         this.adminRemarks = adminRemarks;
     }

     public String getResolution() {
         return resolution;
     }

     public void setResolution(String resolution) {
         this.resolution = resolution;
     }

     public boolean isUrgent() {
         return urgent;
     }

     public void setUrgent(boolean urgent) {
         this.urgent = urgent;
     }

     public LocalDateTime getRaisedAt() {
         return raisedAt;
     }

     public void setRaisedAt(LocalDateTime raisedAt) {
         this.raisedAt = raisedAt;
     }

     public LocalDateTime getResolvedAt() {
         return resolvedAt;
     }

     public void setResolvedAt(LocalDateTime resolvedAt) {
         this.resolvedAt = resolvedAt;
     }
}
```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
import jakarta.persistence.*;

import java.time.DayOfWeek;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.LocalTime;
@Entity
public class Subscription {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    @Enumerated(EnumType.STRING)
    private MealSlot slot;
    private ScheduleType scheduleType;
    private DayOfWeek dayOfWeek;
    @Enumerated(EnumType.STRING)
    private SubscriptionStatus status;
    private Instant nextDeliveryTime;
    private LocalTime created_at;

    @ManyToOne
    @JoinColumn(name="employee_id")
    private Employee employee;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "restaurant_id", nullable = false)
    private Restaurant restaurant;

    @PrePersist
    public void init(){
        this.created_at=LocalTime.now();
    }

    public long getId() {
        return id;
    }

    public void setId(long id) {
        this.id = id;
    }

    public MealSlot getSlot() {
        return slot;
    }

    public void setSlot(MealSlot slot) {
        this.slot = slot;
    }

    public ScheduleType getScheduleType() {
        return scheduleType;
    }

    public void setScheduleType(ScheduleType scheduleType) {
        this.scheduleType = scheduleType;
    }

    public DayOfWeek getDayOfWeek() {
        return dayOfWeek;
    }

    public void setDayOfWeek(DayOfWeek dayOfWeek) {
        this.dayOfWeek = dayOfWeek;
    }

    public SubscriptionStatus getStatus() {
```

```
        return status;
    }

    public void setStatus(SubscriptionStatus status) {
        this.status = status;
    }

    public LocalTime getCreated_at() {
        return created_at;
    }

    public void setCreated_at(LocalTime created_at) {
        this.created_at = created_at;
    }

    public Instant getNextDeliveryTime() {
        return nextDeliveryTime;
    }

    public void setNextDeliveryTime(Instant nextDeliveryTime) {
        this.nextDeliveryTime = nextDeliveryTime;
    }

    public Employee getEmployee() {
        return employee;
    }

    public void setEmployee(Employee employee) {
         this.employee = employee;
     }
     public Restaurant getRestaurant() { return restaurant; }
     public void setRestaurant(Restaurant restaurant) { this.restaurant = restaurant; }
}

```

```java
package com.example.EmployeeManagementSystem.Entity;

import com.example.EmployeeManagementSystem.Enum.Role;
import jakarta.persistence.*;
import jakarta.xml.bind.annotation.XmlAttribute;
import org.jspecify.annotations.Nullable;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;

 import java.time.LocalDate;
 import java.util.Collection;
 import java.util.HashSet;
 import java.util.List;
 import java.util.Set;
 import java.util.stream.Collectors;

 /**
  * Represents a restaurant vendor/owner.
  * Vendors can register and manage their restaurants.
  * They cannot access employee, leave, or subscription APIs.
  */
 @Entity
 @Table(name = "vendor")
 public class Vendor implements UserDetails {

     @Id
     @GeneratedValue(strategy = GenerationType.IDENTITY)
     private Long id;

     @Column(nullable = false)
     private String name;

     @Column(unique = true, nullable = false)
     private String email;

     @Column(nullable = false)
     private String phone;

     @Column(nullable = false)
     private String password;

     @Enumerated(EnumType.STRING)
     private Role role;

     private LocalDate registeredAt;

     @PrePersist
     public void init() {
         if (this.registeredAt == null) {
             this.registeredAt = LocalDate.now();
         }
     }

     public Long getId() { return id; }
     public void setId(Long id) { this.id = id; }

     public String getName() { return name; }
     public void setName(String name) { this.name = name; }

     public String getEmail() { return email; }
     public void setEmail(String email) { this.email = email; }

     public String getPhone() { return phone; }
     public void setPhone(String phone) { this.phone = phone; }

     public LocalDate getRegisteredAt() { return registeredAt; }
     public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }


     @Override
```

```
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Role effectiveRole = role;
        Set<SimpleGrantedAuthority> authorities=new HashSet<>();
        authorities.add(new SimpleGrantedAuthority("ROLE_"+effectiveRole.name()));
        authorities.addAll(effectiveRole.getPermissions().stream()
                .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
                .collect(Collectors.toSet()));
        return authorities;
    }

    @Override
    public @Nullable String getPassword() {
        return password;
    }

    @Override
    public String getUsername() {
        return email;
    }

    @Override
    public boolean isAccountNonExpired() {
        return UserDetails.super.isAccountNonExpired();
    }

    @Override
    public boolean isAccountNonLocked() {
        return UserDetails.super.isAccountNonLocked();
     }

     @Override
     public boolean isCredentialsNonExpired() {
         return UserDetails.super.isCredentialsNonExpired();
     }

     @Override
     public boolean isEnabled() {
         return UserDetails.super.isEnabled();
     }

     public void setPassword(String password) {
         this.password = password;
     }

     public Role getRole() {
         return role;
     }

     public void setRole(Role role) {
         this.role = role;
     }
}
```

68 Current: src\main\java\com\example\EmployeeManagementSystem\Entity\YearEndCarryForwardLog.java

62 lin

```java
package com.example.EmployeeManagementSystem.Entity;


import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "year_end_carry_forward_log")
 public class YearEndCarryForwardLog {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "employee_id", nullable = false)
    private Employee employee;

    @Column(nullable = false)
    private Integer processedYear;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "leave_type_id", nullable = false)
    private LeaveType leaveType;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal closingBalance;      // raw closing before cap

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal cappedBalance;       // after global 30-day cap applied

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal daysWasted = BigDecimal.ZERO;

    @Column(nullable = false, precision = 5, scale = 2)
    private BigDecimal openingNextYear;     // written into next year's entitlement

    @Column(nullable = false)
    private LocalDateTime processedAt = LocalDateTime.now();

    // --- getters & setters ---
    public Long getId() { return id; }
    public Employee getEmployee() { return employee; }
    public Integer getProcessedYear() { return processedYear; }
    public LeaveType getLeaveType() { return leaveType; }
    public BigDecimal getClosingBalance() { return closingBalance; }
    public BigDecimal getCappedBalance() { return cappedBalance; }
    public BigDecimal getDaysWasted() { return daysWasted; }
    public BigDecimal getOpeningNextYear() { return openingNextYear; }
    public LocalDateTime getProcessedAt() { return processedAt; }

    public void setEmployee(Employee e) { this.employee = e; }
    public void setProcessedYear(Integer y) { this.processedYear = y; }
    public void setLeaveType(LeaveType lt) { this.leaveType = lt; }
    public void setClosingBalance(BigDecimal b) { this.closingBalance = b; }
    public void setCappedBalance(BigDecimal b) { this.cappedBalance = b; }
    public void setDaysWasted(BigDecimal d) { this.daysWasted = d; }
    public void setOpeningNextYear(BigDecimal b) { this.openingNextYear = b; }
    public void setProcessedAt(LocalDateTime dt) { this.processedAt = dt; }
}

```

69 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java

8 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum AssignmentStatus {
    ACTIVE,
    RETURNED,
    REVOKED
}

```

70 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java

9 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum DeliveryStatus {
    SCHEDULED,
    IN_PROGRESS,
    DELIVERED,
    MISSED
}

```

71 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java

10 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum DeviceStatus {
    AVAILABLE,
    ASSIGNED,
    UNDER_REPAIR,
    CONDEMNED,
    RETURNED_TO_VENDOR
}

```

Current:
src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceType.java

15 lines

```
package com.example.EmployeeManagementSystem.Enum;

public enum DeviceType {
    LAPTOP,
    DESKTOP,
    PHONE,
    TABLET,
    MONITOR,
    KEYBOARD,
     MOUSE,
     HEADSET,
     OTHER

 }

```

```java
package com.example.EmployeeManagementSystem.Enum;

public enum Gender {
    M, F, O
}
```

74 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java

9 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum LeaveStatus {
    PENDING,
    APPROVED,
    REJECTED,
    CANCELLED
}

```

75 **Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java** 8 lines

```
package com.example.EmployeeManagementSystem.Enum;

public enum LeaveType {
    SICK,
    CASUAL,
    EARNED
}

```

```java
package com.example.EmployeeManagementSystem.Enum;

public enum MealSlot {
    BREAKFAST,
    LUNCH,
    DINNER
}

```

77 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Permission.java

21 lines

```
package com.example.EmployeeManagementSystem.Enum;

import java.util.Set;

public enum Permission {
    LEAVE_READ,
    LEAVE_WRITE,
    LEAVE_UPDATE,
    LEAVE_CANCEL,
     SUBSCRIPTION_WRITE,
     SUBSCRIPTION_READ,
     SUBSCRIPTION_UPDATE,
     VENDOR_UPDATE,
     VENDOR_DELETE,
     RESTAURANTS_ADD,
     RESTAURANTS_READ,
     RESTAURANTS_DELETE,
     PROFILE_READ,
     VENDOR_WRITE;
 }

```

```java
package com.example.EmployeeManagementSystem.Enum;

import java.util.Collection;
import java.util.Set;

import static com.example.EmployeeManagementSystem.Enum.Permission.*;


public enum Role {
    EMPLOYEE(Set.of(LEAVE_WRITE,
           LEAVE_CANCEL,
            SUBSCRIPTION_WRITE,
           SUBSCRIPTION_UPDATE,
            RESTAURANTS_READ,
            SUBSCRIPTION_READ,
            PROFILE_READ)),
    MANAGER(Set.of(LEAVE_UPDATE,
            LEAVE_WRITE,
            SUBSCRIPTION_WRITE,
            RESTAURANTS_READ,
            SUBSCRIPTION_UPDATE,
            LEAVE_READ,
            SUBSCRIPTION_READ,
            PROFILE_READ)),
    FOOD_VENDOR(Set.of(
            VENDOR_WRITE,
            VENDOR_DELETE,
            RESTAURANTS_ADD,
            RESTAURANTS_DELETE,
            RESTAURANTS_READ,
            PROFILE_READ
    )),
    ADMIN(Set.of(
            LEAVE_READ,
            LEAVE_WRITE,
            LEAVE_UPDATE,
            LEAVE_CANCEL,
            RESTAURANTS_READ,
            SUBSCRIPTION_READ,
            SUBSCRIPTION_WRITE,
            SUBSCRIPTION_UPDATE,
            VENDOR_WRITE,
            VENDOR_UPDATE,
            VENDOR_DELETE,
            PROFILE_READ

    )),
    TECH_VENDOR(Set.of(VENDOR_WRITE)),
    USER(Set.of(PROFILE_READ));
    private final Set<Permission> permissions;

    Role(Set<Permission> permissions) {
        this.permissions = permissions;
    }

    public Set<Permission> getPermissions() {
     return permissions;
    }
}

```

79 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java

7 lines

```
package com.example.EmployeeManagementSystem.Enum;

public enum ScheduleType {
    DAILY,
    WEEKLY
}

```

80 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestStatus.java

9 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum ServiceRequestStatus {
    OPEN,
    SENT_FOR_REPAIR,
    REPAIR_DONE,
    REJECTED
}

```

81 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestType.java

10 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum ServiceRequestType {
    REPAIR,
    REPLACEMENT,
    UPGRADE,
    MAINTENANCE,
    RETURN
}

```

```java
package com.example.EmployeeManagementSystem.Enum;

public enum Status {
    ACTIVE,
    INACTIVE,
    ON_LEAVE
}

```

83 Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java

8 lines

```java
package com.example.EmployeeManagementSystem.Enum;

public enum SubscriptionStatus {
    ACTIVE,
    PAUSED,
    EXPIRED
}

```

84 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.jav

```
package com.example.EmployeeManagementSystem.Exception;

public class DuplicateRequestException extends RuntimeException {
    public DuplicateRequestException(String message) {
        super(message);
    }
}

```

85 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java

10 lines

```
package com.example.EmployeeManagementSystem.Exception;

import org.springframework.security.core.userdetails.UsernameNotFoundException;

public class EmployeeNotFound extends UsernameNotFoundException {
    public EmployeeNotFound(String message) {
        super(message);
    }
}

```

86 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class InvalidEndDateException extends RuntimeException{
    public InvalidEndDateException(String message) {
        super(message);
    }
}

```

87 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class InvalidManagerException extends RuntimeException{
    public InvalidManagerException(String message) {
        super(message);
    }
}

```

88 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class InvalidStartDateException extends RuntimeException{
    public InvalidStartDateException(String message) {
        super(message);
    }
}

```

89 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundExcept

```
package com.example.EmployeeManagementSystem.Exception;

public class LeaveRequestNotFoundException extends RuntimeException{
    public LeaveRequestNotFoundException(String message) {
        super(message);
    }
}

```

90 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.jav

```java
package com.example.EmployeeManagementSystem.Exception;

public class OverlappingLeaveException extends RuntimeException{
    public OverlappingLeaveException(String message) {
        super(message);
    }
}

```

91 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException

```
package com.example.EmployeeManagementSystem.Exception;

public class RestaurantNotFoundException extends RuntimeException {
    public RestaurantNotFoundException(String message) {
        super(message);
    }
}

```

92 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class SubscriptionAlreadyExists extends RuntimeException{
    public SubscriptionAlreadyExists(String message) {
        super(message);
    }
}

```

93 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.

```
package com.example.EmployeeManagementSystem.Exception;

public class UnauthorizedAccessException extends RuntimeException {
    public UnauthorizedAccessException(String message) {
        super(message);
    }
}
```

94 Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java

```java
package com.example.EmployeeManagementSystem.Exception;

public class VendorNotFoundException extends RuntimeException {
    public VendorNotFoundException(String message) {
        super(message);
    }
}
```

95 Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java

45 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/Filter/ApiKeyFilter.java
package com.example.EmployeeManagementSystem.Filter;

import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationToken;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;

public class ApiKeyFilter extends OncePerRequestFilter {

    private static final String API_KEY_HEADER = "X-API-Key";

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String apiKey = request.getHeader(API_KEY_HEADER);

        if (apiKey != null && !apiKey.isEmpty()) {
            ApiKeyAuthenticationToken authToken = new ApiKeyAuthenticationToken(apiKey);
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }

        filterChain.doFilter(request, response);
    }

    @Override
    protected boolean shouldNotFilter(HttpServletRequest request) {
        String path = request.getRequestURI();
        return path.startsWith("/Authenticate") ||
                path.startsWith("/session/login") ||
                path.startsWith("/session/logout") ||
                path.startsWith("/oauth2") ||
                path.startsWith("/login/oauth2") ||
                path.startsWith("/swagger-ui") ||
                path.startsWith("/v3/api-docs") ||
                path.startsWith("/api-keys/generate"); // Allow API key generation without API key
    }
}
```

```java
package com.example.EmployeeManagementSystem.Filter;

import com.example.EmployeeManagementSystem.Util.JWTUtil;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    @Autowired
    @Qualifier("combinedUserDetailService")
    private UserDetailsService customUserDetailService;
    @Autowired
    private JWTUtil jwtUtil;


    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        String token=null;
        String username=null;

        String authHeader=request.getHeader("Authorization");
        try {

            if (authHeader != null && authHeader.startsWith("Bearer ")) {
                token = authHeader.substring(7);
                username = jwtUtil.extractUsernameFromToken(token);
            }

        } catch (io.jsonwebtoken.ExpiredJwtException e) {

            response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
            response.setContentType("application/json");

            response.getWriter().write("""
        {
          "error": "TOKEN_EXPIRED",
          "message": "JWT expired. Please login again to get a new token."
        }
        """);

            return;
        }
        if (username!=null && SecurityContextHolder.getContext().getAuthentication()==null){
             UserDetails userDetails=customUserDetailService.loadUserByUsername(username);
             if(userDetails!=null && jwtUtil.validateToken(username,userDetails,token)){
                 UsernamePasswordAuthenticationToken authenticationToken=new UsernamePasswordAuthenticationToken(
                         userDetails,null,userDetails.getAuthorities()
                 );
                 SecurityContextHolder.getContext().setAuthentication(authenticationToken);
             }
        }

    filterChain.doFilter(request,response);


     }
```

```

}

```

97 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java

34 lines

```java
// src/main/java/com/example/EmployeeManagementSystem/Repository/ApiKeyRepository.java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.ApiKey;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.transaction.annotation.Transactional;

 import java.time.Instant;
 import java.util.List;
 import java.util.Optional;

 public interface ApiKeyRepository extends JpaRepository<ApiKey, Long> {

     Optional<ApiKey> findByApiKeyAndActiveTrue(String apiKey);


     List<ApiKey> findByEmployee_EmployeeIdAndActiveTrue(Long employeeId);


     List<ApiKey> findByVendor_IdAndActiveTrue(Long vendorId);

     @Modifying
     @Transactional
     @Query("UPDATE ApiKey a SET a.lastUsedAt = :now WHERE a.apiKey = :apiKey")
     void updateLastUsed(@Param("apiKey") String apiKey, @Param("now") Instant now);

     @Modifying
     @Transactional
     @Query("UPDATE ApiKey a SET a.active = false WHERE a.expiresAt < :now AND a.active = true")
     int expireOldKeys(@Param("now") Instant now);
 }
```

98 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java

23 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Delivery;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

 import java.time.Instant;
 import java.util.List;

 public interface DeliveryRepository extends JpaRepository<Delivery,Long> {
     List<Delivery> findByStatusAndScheduledDeliveryTimeLessThanEqual(
             DeliveryStatus status, Instant time);

     @Query("SELECT d FROM Delivery d WHERE d.subscription = :subscription " +
             "AND d.status != :status ORDER BY d.scheduledDeliveryTime DESC")
     List<Delivery> findRecentDeliveriesBySubscription(
             @Param("subscription") Subscription subscription,
             @Param("status") DeliveryStatus status);
 }

```

99 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java

```java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.util.List;
 import java.util.Optional;

 public interface DeviceAssignmentRepo extends JpaRepository<DeviceAssignment,Long> {
     Optional<DeviceAssignment> findByDeviceIdAndStatus(Long deviceId, AssignmentStatus status);
     List<DeviceAssignment> findByAssignedToEmployeeIdAndStatus(Long employeeId, AssignmentStatus status);

     @Query(value = "SELECT da.* FROM device_assignment da JOIN device d ON da.device_id = d.id WHERE d.vendor_id = :vendorId",nativeQuery = true)
     List<DeviceAssignment> getAllAssignmentsByVendorId(@Param("vendorId") long vendorId);


 }

```

100 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceRepository.java

13 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Device;
import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface DeviceRepository extends JpaRepository<Device,Long> {
     List<Device> findByTechVendorId(Long vendorId);
     List<Device> findByDeviceStatus(DeviceStatus deviceStatus);
 }

```

101 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java

26 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

 import java.util.List;
 import java.util.Optional;

 @Repository
 public interface EmployeeRepo extends JpaRepository<Employee,Long> {
     Optional<Employee> findByEmail(String email);
     Optional<Employee> findByName(String name);
     long countByRole(Role role);

     List<Employee> findByManager(Employee manager);
     List<Employee> findByStatus(Status status);
     List<Employee> findByRole(Role role);
     @Query("SELECT e FROM Employee e WHERE e.manager.employeeId = :managerId")
     List<Employee> findByManagerEmployeeId(@Param("managerId") Long managerId);
 }

```

102 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveEntitlementRepository.

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

import java.math.BigDecimal;
import java.util.List;
 import java.util.Optional;

 public interface LeaveEntitlementRepository extends JpaRepository<LeaveEntitlement, Long> {

     Optional<LeaveEntitlement> findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
             Long employeeId, Integer leaveTypeId, Integer year);

     List<LeaveEntitlement> findByEmployeeEmployeeIdAndYear(Long employeeId, Integer year);

     @Query("""
         SELECT COALESCE(SUM(e.closingBalance), 0)
         FROM LeaveEntitlement e
         WHERE e.employee.employeeId = :empId
           AND e.leaveType.carriesForward = true
           AND e.year IN :years
     """)
     BigDecimal sumCarryForwardBalances(
             @Param("empId") Long employeeId,
             @Param("years") List<Integer> years);

     List<LeaveEntitlement> findByLeaveTypeIdAndYear(Integer leaveTypeId, Integer year);
 }
```

103 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveRequestRepo.java

74 line

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

 import java.time.LocalDate;
 import java.util.List;

 public interface LeaveRequestRepo extends JpaRepository<LeaveRequest, Long> {

     List<LeaveRequest> findByStatus(LeaveStatus status);

     /**
      * Find leave requests by start date and status
      * Used by scheduler to find leaves starting today
      */
     List<LeaveRequest> findByStartDateAndStatus(LocalDate startDate, LeaveStatus status);

     // Fix the checkDuplicate query (fix parameter names)
     @Query(value = "SELECT COUNT(*) " +
             "FROM leave_request l " +
             "WHERE l.employee_id = :employeeId " +
             "AND l.start_date = :startDate " +
             "AND l.end_date = :endDate " +
             "AND l.status=:status",nativeQuery = true)
     long checkDuplicate(@Param("employeeId") long employeeId,
                         @Param("startDate") LocalDate startDate,
                         @Param("endDate") LocalDate endDate,
                         @Param("status") String status);

     // Fix overlapping leave query (was commented in your code)
     @Query(value = "select COUNT(*) " +
             "from leave_request l " +
             "where l.employee_id=:employeeId " +
             "AND l.status IN ('APPROVED','PENDING') " +
             "AND l.start_date <= :endDate " +
             "AND l.end_date >= :startDate",nativeQuery = true)
     long countOverlappingLeave(
             @Param("employeeId") long employeeId,
             @Param("startDate") LocalDate startDate,
             @Param("endDate") LocalDate endDate
     );

     // Add missing methods
     List<LeaveRequest> findByEmployeeOrderByStartDateDesc(Employee employee);


     List<LeaveRequest> findByEndDateAndStatus(LocalDate endDate, LeaveStatus status);

     // Add for leave duration validation
     @Query("SELECT COALESCE(SUM(DATEDIFF(l.endDate, l.startDate) + 1), 0) " +
             "FROM LeaveRequest l WHERE l.employee = :employee " +
             "AND l.leaveType = :leaveType " +
             "AND YEAR(l.startDate) = :year " +
             "AND l.status IN ('APPROVED', 'PENDING')")
     long countDaysByEmployeeAndLeaveTypeAndYear(@Param("employee") Employee employee,
                                                 @Param("leaveType") LeaveType leaveType,
                                                 @Param("year") int year);

     @Query("SELECT lr FROM LeaveRequest lr JOIN FETCH lr.employee WHERE lr.status = :status")
     List<LeaveRequest> findApprovedLeavesWithEmployee(LeaveStatus status);


     List<LeaveRequest> findByEmployee_Manager(Employee manager);

     List<LeaveRequest> findByStatusAndEmployee_Manager(LeaveStatus status, Employee manager);
```

```
}
```

104 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveType;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

 public interface LeaveTypeRepository extends JpaRepository<LeaveType, Integer> {
     Optional<LeaveType> findByName(String name);
     List<LeaveType> findByMonthlyResetTrue();
 }
```

105 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.jav

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
import org.springframework.data.jpa.repository.JpaRepository;

import java.time.LocalDate;
import java.util.List;

public interface LeaveWarningRepository extends JpaRepository<LeaveWarning, Long> {

    // All unread warnings for an employee — shown on dashboard
    List<LeaveWarning> findByEmployeeEmployeeIdAndIsReadFalseOrderByWarningDateDesc(Long employeeId);

    // All warnings for an employee (read + unread) — for history view
    List<LeaveWarning> findByEmployeeEmployeeIdOrderByWarningDateDesc(Long employeeId);

    // Prevent duplicate warnings for same type on same date
    boolean existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(
            Long employeeId, LeaveWarning.WarningType warningType, LocalDate date
    );
}
```

106 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

 public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {

     Optional<RefreshToken> findByToken(String token);

     // Needed so EmployeeService can clean up tokens before deleting an employee
     void deleteByEmployee(Employee employee);

     // Needed so VendorService can clean up tokens before deleting a vendor
     void deleteByVendor(Vendor vendor);
 }
```

107 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java

```java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Restaurant;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {

    /** All active restaurants owned by a vendor. */
    List<Restaurant> findByVendor_EmailAndActiveTrue(String email);

    /** All active restaurants (for employees browsing available options). */
    List<Restaurant> findByActiveTrue();

    /**
     * Active restaurants that support a given meal slot.
     * Employees use this to discover which restaurants they can subscribe to for a given slot.
     */
    @Query("SELECT r FROM Restaurant r JOIN r.supportedMealSlots s " +
            "WHERE r.active = true AND s = :mealSlot")
    List<Restaurant> findActiveByMealSlot(@Param("mealSlot") MealSlot mealSlot);

    /** Check if a restaurant belongs to a vendor (ownership guard). */
    boolean existsByIdAndVendor_Email(Long restaurantId, String email);
}
```

108 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.ja

```
package com.example.EmployeeManagementSystem.Repository;


import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
import jdk.dynalink.linker.LinkerServices;
import org.springframework.data.jpa.repository.JpaRepository;

 import java.util.List;

 public interface ServiceRequestRepository extends JpaRepository<ServiceRequest,Long> {

     List<ServiceRequest> findByStatus(ServiceRequestStatus status);
     List<ServiceRequest> findByDeviceTechVendorAndStatus(
             Vendor vendor,
             ServiceRequestStatus status
     );

 }

```

109 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java

```java
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Subscription;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

import java.time.Instant;
 import java.util.List;

 @Repository
 public interface SubscriptionRepository extends JpaRepository<Subscription,Long> {
     @Query(value = "select count(*) from subscription s where s.employee_id=:employeeId and s.slot=:slot and s.status=:status",nativeQuery = true)
     int checkSubscriptionExists(
             @Param("employeeId") long employeeId,
             @Param("slot") String slot,
             @Param("status") String status
     );

     List<Subscription> findByEmployee_employeeId(long id);

     @Query(value = "SELECT * FROM subscription s WHERE s.status = :status AND s.next_delivery_time IS NOT NULL AND s.next_delivery_time <= :currentTime", nativeQuery = true)
     List<Subscription> findDueSubscriptions(
             @Param("status") String status,
             @Param("currentTime") Instant currentTime  //  was LocalDateTime
     );

     List<Subscription> findByEmployee_name(String name);
     List<Subscription> findByEmployee_Email(String email);
 }

```

110 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java

13 lines

```
package com.example.EmployeeManagementSystem.Repository;

import com.example.EmployeeManagementSystem.Entity.Vendor;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
 public interface VendorRepo extends JpaRepository<Vendor, Long> {
     Optional<Vendor> findByEmail(String email);
     boolean existsByEmail(String email);
 }
```

111 Current: src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRe

```
package com.example.EmployeeManagementSystem.Repository;


import com.example.EmployeeManagementSystem.Entity.YearEndCarryForwardLog;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface YearEndCarryForwardLogRepository
        extends JpaRepository<YearEndCarryForwardLog, Long> {

    List<YearEndCarryForwardLog> findByEmployeeEmployeeIdAndProcessedYear(
            Long employeeId, Integer year
    );

    // Check if year-end has already been run for this year (idempotency guard)
    boolean existsByProcessedYear(Integer year);
}
```

112 Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java

30 lines

```java
package com.example.EmployeeManagementSystem.Scheduler;

import com.example.EmployeeManagementSystem.Service.DeliveryService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

@Component
 public class DeliveryScheduler {
     private static final Logger logger = LoggerFactory.getLogger(DeliveryScheduler.class);
     private final DeliveryService deliveryService;

     public DeliveryScheduler(DeliveryService deliveryService) {
         this.deliveryService = deliveryService;
     }

     // Runs every hour
     @Scheduled(cron = "0 0 * * * *")
     public void processDeliveries() {
         logger.info("Starting delivery processing scheduler...");
         try {
             deliveryService.processDueDeliveries();
             logger.info("Delivery processing completed successfully");
         } catch (Exception e) {
             logger.error("Error processing deliveries: ", e);
         }
     }
 }

```

113 Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveAccrualScheduler.java

```java
package com.example.EmployeeManagementSystem.Scheduler;

import com.example.EmployeeManagementSystem.Service.CarryForwardWarningService;
import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.time.LocalDate;

@Component
public class LeaveAccrualScheduler {

    private static final Logger log = LoggerFactory.getLogger(LeaveAccrualScheduler.class);

    private final LeaveAccrualService accrualService;
    private final SickLeaveResetService resetService;
    private final CarryForwardWarningService warningService;
    private final YearEndCarryForwardService yearEndService;

    public LeaveAccrualScheduler(LeaveAccrualService accrualService,
                                 SickLeaveResetService resetService,
                                 CarryForwardWarningService warningService,
                                 YearEndCarryForwardService yearEndService) {
        this.accrualService = accrualService;
        this.resetService = resetService;
        this.warningService = warningService;
        this.yearEndService = yearEndService;
    }

    // 1st of month — accrue then warn
    @Scheduled(cron = "0 5 0 1 * ?")
    public void runMonthlyAccrual() {
        LocalDate today = LocalDate.now();
        accrualService.runMonthlyAccrual(today);
        warningService.runWarningCheck(today);
    }

    // Last day of month 23:55 — sick leave reset
    @Scheduled(cron = "0 55 23 L * ?")
    public void runSickLeaveReset() {
        resetService.runMonthlyReset(LocalDate.now());
    }

    // Dec 31 at 22:00 — year-end carry-forward cap
    @Scheduled(cron = "0 0 22 31 12 ?")
    public void runYearEnd() {
        int closingYear = LocalDate.now().getYear();
        yearEndService.runYearEnd(closingYear);
    }
}
```

114 Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java

```java
package com.example.EmployeeManagementSystem.Scheduler;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.ZoneId;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@Component
public class LeaveStatusScheduler {

    private static final Logger log = LoggerFactory.getLogger(LeaveStatusScheduler.class);

    private final LeaveRequestRepo leaveRequestRepo;
    private final EmployeeRepo employeeRepo;

    public LeaveStatusScheduler(LeaveRequestRepo leaveRequestRepo,
                                EmployeeRepo employeeRepo) {
        this.leaveRequestRepo = leaveRequestRepo;
        this.employeeRepo = employeeRepo;
    }

    /**
     * Runs every day at midnight UTC
     */
    @Scheduled(cron = "0 0 0 * * *", zone = "UTC")
    @Transactional
    public void updateEmployeeLeaveStatus() {

        log.info("=== Starting Timezone-Aware Leave Scheduler ===");

        List<LeaveRequest> leaves =
                leaveRequestRepo.findApprovedLeavesWithEmployee(LeaveStatus.APPROVED);

        Map<Long, List<LeaveRequest>> employeeLeavesMap = leaves.stream()
                .collect(Collectors.groupingBy(lr -> lr.getEmployee().getEmployeeId()));

        int onLeaveCount = 0;
        int activeCount = 0;

        for (Map.Entry<Long, List<LeaveRequest>> entry : employeeLeavesMap.entrySet()) {

            Employee employee = entry.getValue().get(0).getEmployee();

            ZoneId zone = ZoneId.of(
                    employee.getTimezone() != null ? employee.getTimezone() : "UTC"
            );

            LocalDate today = LocalDate.now(zone);

            boolean isOnLeaveToday = entry.getValue().stream().anyMatch(leave ->
                    !today.isBefore(leave.getStartDate()) &&
                            !today.isAfter(leave.getEndDate())
            );

            //  CASE 1: Should be ON_LEAVE
            if (isOnLeaveToday && employee.getStatus() != Status.ON_LEAVE) {

                employee.setStatus(Status.ON_LEAVE);
```

```
                employeeRepo.save(employee);
                onLeaveCount++;

                log.info("Employee {} ({}) marked ON_LEAVE | TZ: {}",
                        employee.getEmployeeId(),
                        employee.getName(),
                        zone);
            }

            //  CASE 2: Should be ACTIVE
            if (!isOnLeaveToday && employee.getStatus() == Status.ON_LEAVE) {

                employee.setStatus(Status.ACTIVE);
                employeeRepo.save(employee);
                activeCount++;

                log.info("Employee {} ({}) marked ACTIVE | TZ: {}",
                        employee.getEmployeeId(),
                        employee.getName(),
                        zone);
            }
        }

        log.info("=== Scheduler Completed ===");
        log.info("ON_LEAVE updated: {}", onLeaveCount);
        log.info("ACTIVE updated: {}", activeCount);
    }

    /**
     * Manual trigger (for testing)
     */
    public void runManually() {
        log.info("Manual scheduler trigger started...");
        updateEmployeeLeaveStatus();
    }
}
```

115 Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.ja

```java
// src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationProvider.java
package com.example.EmployeeManagementSystem.security;

import com.example.EmployeeManagementSystem.Entity.ApiKey;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collection;
import java.util.List;
import java.util.stream.Collectors;

@Component
public class ApiKeyAuthenticationProvider implements AuthenticationProvider {

    private final ApiKeyRepository apiKeyRepository;

    public ApiKeyAuthenticationProvider(ApiKeyRepository apiKeyRepository) {
        this.apiKeyRepository = apiKeyRepository;
    }

    @Override
    @Transactional
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
        String apiKey = null;

        // Extract API key from the authentication object
        if (authentication instanceof ApiKeyAuthenticationToken) {
            apiKey = ((ApiKeyAuthenticationToken) authentication).getApiKey();
        }

        if (apiKey == null || apiKey.isEmpty()) {
            throw new RuntimeException("Invalid API key");
        }

        ApiKey key = apiKeyRepository.findByApiKeyAndActiveTrue(apiKey)
                .orElseThrow(() -> new RuntimeException("Invalid or inactive API key"));

        // Check expiration
        if (key.getExpiresAt() != null && key.getExpiresAt().isBefore(Instant.now())) {
            throw new RuntimeException("API key has expired");
        }

        // Update last used timestamp
        apiKeyRepository.updateLastUsed(apiKey, Instant.now());

        // Build authorities from permissions
        List<GrantedAuthority> authorities = new ArrayList<>();

        if (key.getPermissions() != null && !key.getPermissions().isEmpty()) {
            authorities.addAll(Arrays.stream(key.getPermissions().split(","))
                    .map(String::trim)
                    .map(SimpleGrantedAuthority::new)
                    .collect(Collectors.toList()));
        }

        Object principal;
        if (key.getEmployee() != null) {
            principal = key.getEmployee();
            authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getEmployee().getRole().name()));
        } else if (key.getVendor() != null) {
```

```
            principal = key.getVendor();
            authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
        } else {
            throw new RuntimeException("No user associated with API key");
        }

        return new ApiKeyAuthenticationToken(principal, apiKey, authorities);
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
    }
}
```

116 Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.jav

```java
// src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationToken.java
package com.example.EmployeeManagementSystem.security;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import java.util.Collection;

public class ApiKeyAuthenticationToken implements Authentication {

    private static final long serialVersionUID = 1L;

    private final Object principal;
    private final Object credentials;
    private final String apiKey;
    private final Collection<? extends GrantedAuthority> authorities;
    private boolean authenticated;

    // Constructor for unauthenticated API key (before validation)
    public ApiKeyAuthenticationToken(String apiKey) {
        this.apiKey = apiKey;
        this.principal = null;
        this.credentials = null;
        this.authorities = null;
        this.authenticated = false;
    }

    // Constructor for authenticated API key (after validation)
    public ApiKeyAuthenticationToken(Object principal, Object credentials,
                                     Collection<? extends GrantedAuthority> authorities) {
        this.apiKey = null;
        this.principal = principal;
        this.credentials = credentials;
        this.authorities = authorities;
        this.authenticated = true;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return authorities;
    }

    @Override
    public Object getCredentials() {
        return credentials;
    }

    @Override
    public Object getDetails() {
        return null;
    }

    @Override
    public Object getPrincipal() {
        return principal;
    }

    @Override
    public boolean isAuthenticated() {
        return authenticated;
    }

    @Override
    public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
        this.authenticated = isAuthenticated;
    }

    @Override
    public String getName() {
        if (principal instanceof org.springframework.security.core.userdetails.UserDetails) {
            return ((org.springframework.security.core.userdetails.UserDetails) principal).getUsername();
        }
```

```
        if (principal instanceof com.example.EmployeeManagementSystem.Entity.Employee) {
            return ((com.example.EmployeeManagementSystem.Entity.Employee) principal).getUsername();
        }
        if (principal instanceof com.example.EmployeeManagementSystem.Entity.Vendor) {
            return ((com.example.EmployeeManagementSystem.Entity.Vendor) principal).getUsername();
        }
        return principal != null ? principal.toString() : apiKey;
    }

    public String getApiKey() {
        return apiKey;
    }
}
```

117 Current: src\main\java\com\example\EmployeeManagementSystem\Service\CarryForwardWarningService.jav

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Service
public class CarryForwardWarningService {

    private static final Logger log = LoggerFactory.getLogger(CarryForwardWarningService.class);

    private static final BigDecimal CAP          = BigDecimal.valueOf(30);
    private static final BigDecimal WARN_AT_29   = BigDecimal.valueOf(29);
    private static final BigDecimal WARN_AT_28   = BigDecimal.valueOf(28);

    private final EmployeeRepo employeeRepository;
    private final LeaveEntitlementRepository entitlementRepository;
    private final LeaveWarningRepository warningRepository;

    public CarryForwardWarningService(EmployeeRepo employeeRepository,
                                      LeaveEntitlementRepository entitlementRepository,
                                      LeaveWarningRepository warningRepository) {
        this.employeeRepository = employeeRepository;
        this.entitlementRepository = entitlementRepository;
        this.warningRepository = warningRepository;
    }

    @Transactional
    public void runWarningCheck(LocalDate checkDate) {
        List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);

        for (Employee employee : activeEmployees) {
            BigDecimal carryForwardTotal = calculateCarryForwardBalance(employee, checkDate);
            evaluateAndWarn(employee, carryForwardTotal, checkDate);
        }

        log.info("Warning check complete for {} — {} employees evaluated",
                checkDate, activeEmployees.size());
    }

    // ----------------------------------------------------------------
    // Sum available balance across all carry-forwardable types
    // for the current year (this is what will carry forward if unused)
    // ----------------------------------------------------------------
    private BigDecimal calculateCarryForwardBalance(Employee employee, LocalDate date) {
        int year = date.getYear();

        List<LeaveEntitlement> entitlements =
                entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(), year);

        return entitlements.stream()
                .filter(e -> e.getLeaveType().isCarriesForward())
                .map(LeaveEntitlement::getAvailableBalance)
                .reduce(BigDecimal.ZERO, BigDecimal::add);
    }

    // ----------------------------------------------------------------
    // Decide which warning to issue based on carry-forward total
    // ----------------------------------------------------------------
```

```
    private void evaluateAndWarn(Employee employee, BigDecimal balance, LocalDate date) {
        int cmp30 = balance.compareTo(CAP);
        int cmp29 = balance.compareTo(WARN_AT_29);
        int cmp28 = balance.compareTo(WARN_AT_28);

        if (cmp30 > 0) {
            // Balance EXCEEDS cap — a day was wasted (cap enforcement happens in Phase 5,
            // but we flag it here so the warning appears immediately after accrual)
            issueWarning(employee, LeaveWarning.WarningType.DAY_WASTED, balance, date,
                    "A carry-forward day was wasted this month because your balance exceeded the 30-day cap.");

        } else if (cmp30 == 0) {
            issueWarning(employee, LeaveWarning.WarningType.CAP_REACHED, balance, date,
                    "Carry-forward cap reached (30 days). Next month's unused leave will be wasted.");

        } else if (cmp29 >= 0) {
            issueWarning(employee, LeaveWarning.WarningType.APPROACHING_29, balance, date,
                    "You have " + balance.toPlainString() + " carry-forward days. "
                            + "1 month until cap. Use leave before it stops accumulating.");

        } else if (cmp28 >= 0) {
            issueWarning(employee, LeaveWarning.WarningType.APPROACHING_28, balance, date,
                    "You have " + balance.toPlainString() + " carry-forward days. "
                            + "2 months until cap (30). Plan to use leave soon.");
        }
        // Below 28 — no warning needed
    }

    // ----------------------------------------------------------------
    // Persist warning — skip if same type already issued today
    // ----------------------------------------------------------------
    private void issueWarning(Employee employee, LeaveWarning.WarningType type,
                              BigDecimal balance, LocalDate date, String message) {

        boolean alreadyIssued = warningRepository
                .existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(employee.getEmployeeId(), type, date);

        if (alreadyIssued) {
            log.debug("Warning {} already issued for employee {} on {}", type, employee.getEmployeeId(), date);
            return;
        }

        LeaveWarning warning = new LeaveWarning();
        warning.setEmployee(employee);
        warning.setWarningType(type);
        warning.setMessage(message);
        warning.setCarryForwardBalance(balance);
        warning.setWarningDate(date);

        warningRepository.save(warning);

        log.info("Warning issued — employee={} type={} balance={} date={}",
                employee.getEmployeeId(), type, balance, date);
    }
}
```

118 Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Optional;

@Service("combinedUserDetailService")
public class CombinedUserDetailService implements UserDetailsService {

    private final VendorRepo vendorRepository;
    private final EmployeeRepo employeeRepository;

    public CombinedUserDetailService(VendorRepo vendorRepository, EmployeeRepo employeeRepository) {
        this.vendorRepository = vendorRepository;
        this.employeeRepository = employeeRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Optional<Employee> employee = employeeRepository.findByEmail(username);
        if (employee.isPresent()) {
            return employee.get(); // must implement UserDetails
        }

        // Then try Vendor
        Optional<Vendor> vendor = vendorRepository.findByEmail(username);
        if (vendor.isPresent()) {
            return vendor.get();
        }

        throw new UsernameNotFoundException("User not found: " + username);
    }
}

```

119 Current: src\main\java\com\example\EmployeeManagementSystem\Service\CustomOAuth2UserService.java

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
import org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
import org.springframework.security.oauth2.core.user.DefaultOAuth2User;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

@Service
public class CustomOAuth2UserService extends DefaultOAuth2UserService {

    private final EmployeeRepo employeeRepo;
    private final VendorRepo vendorRepo;
    private final PasswordEncoder passwordEncoder;

    public CustomOAuth2UserService(EmployeeRepo employeeRepo,
                                   VendorRepo vendorRepo,
                                   PasswordEncoder passwordEncoder) {
        this.employeeRepo = employeeRepo;
        this.vendorRepo = vendorRepo;
        this.passwordEncoder = passwordEncoder;
    }

    @Override
    public OAuth2User loadUser(OAuth2UserRequest userRequest) {
        System.out.println("=== CustomOAuth2UserService.loadUser() called ===");
        OAuth2User oAuth2User = super.loadUser(userRequest);

        String email = oAuth2User.getAttribute("email");
        String name  = oAuth2User.getAttribute("name");
        System.out.println("OAuth email: " + email);

        boolean employeeExists = employeeRepo.findByEmail(email).isPresent();
        boolean vendorExists   = vendorRepo.findByEmail(email).isPresent();
        System.out.println("employeeExists=" + employeeExists + ", vendorExists=" + vendorExists);

        List<GrantedAuthority> authorities = new   ArrayList<>(oAuth2User.getAuthorities());

        if (employeeExists) {
            Employee employee = employeeRepo.findByEmail(email).get();
            authorities.add(new SimpleGrantedAuthority("ROLE_" + employee.getRole().name()));

            // ADD THIS — loads LEAVE_CANCEL, SUBSCRIPTION_READ, etc. into the session
            employee.getRole().getPermissions().forEach(permission ->
                    authorities.add(new SimpleGrantedAuthority(permission.name()))
            );

            System.out.println("Returning existing employee with role: " + employee.getRole());
            return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
        }

        if (vendorExists) {
            // add vendor role if needed
            authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
            System.out.println("Returning existing vendor");
            return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
        }

        // New user
        registerAsUser(email, name);
```

```
        authorities.add(new SimpleGrantedAuthority("ROLE_USER"));
        return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
    }

    private void registerAsUser(String email, String name) {
        Employee e = new Employee();
        e.setEmail(email);
        e.setName(name);
        e.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
        e.setRole(Role.USER);
        e.setTimezone("Asia/Kolkata");
        e.setDept("UNASSIGNED");
        employeeRepo.saveAndFlush(e); // flush immediately so OAuth2SuccessHandler sees the record
        System.out.println("New USER registered and flushed for: " + email);
    }
}
```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.VendorDTO;
import com.example.EmployeeManagementSystem.DTO.VendorRequest;
import com.example.EmployeeManagementSystem.Entity.Delivery;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.DeliveryRepository;
import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.Instant;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class DeliveryService {
    private final SubscriptionRepository subscriptionRepository;
    private final DeliveryRepository deliveryRepository;
    private final DeliveryTimeService deliveryTimeService;
    private static final Logger logger = LoggerFactory.getLogger(DeliveryService.class);

    public DeliveryService(SubscriptionRepository subscriptionRepository,
                           DeliveryRepository deliveryRepository,
                           DeliveryTimeService deliveryTimeService) {
        this.subscriptionRepository = subscriptionRepository;
        this.deliveryRepository = deliveryRepository;
        this.deliveryTimeService = deliveryTimeService;
    }

    // AFTER
    @Transactional
    public void processDueDeliveries() {
        Instant now = Instant.now();
        List<Subscription> dueSubscriptions = subscriptionRepository
                .findDueSubscriptions(SubscriptionStatus.ACTIVE.name(), now);

        for (Subscription subscription : dueSubscriptions) {
            try {
                Delivery delivery = new Delivery();
                delivery.setSubscription(subscription);
                delivery.setMealSlot(subscription.getSlot());
                delivery.setScheduledDeliveryTime(subscription.getNextDeliveryTime());
                delivery.setStatus(DeliveryStatus.IN_PROGRESS);
                deliveryRepository.save(delivery);

                Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
                subscription.setNextDeliveryTime(nextDelivery);
                subscriptionRepository.save(subscription);

            } catch (DataIntegrityViolationException e) {
                // Duplicate delivery already exists — just advance the next delivery time
                logger.warn("Duplicate delivery skipped for subscription ID: {}", subscription.getId());
                Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
                subscription.setNextDeliveryTime(nextDelivery);
                subscriptionRepository.save(subscription);
            }
        }
    }
```

```java

    @Transactional
    public Delivery updateDeliveryStatus(Long deliveryId, DeliveryStatus status) {
        Delivery delivery = deliveryRepository.findById(deliveryId)
                .orElseThrow(() -> new RuntimeException("Delivery not found"));

        delivery.setStatus(status);
        if (status == DeliveryStatus.DELIVERED) {
            delivery.setActualDeliveryTime(Instant.now());
        }

        return deliveryRepository.save(delivery);
    }

    public List<Delivery> getDeliveriesBySubscription(Long subscriptionId) {
        Subscription subscription = subscriptionRepository.findById(subscriptionId)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        return deliveryRepository.findRecentDeliveriesBySubscription(subscription,
DeliveryStatus.SCHEDULED);
    }

    @Service
    public static class VendorService {

        private static final Logger log = LoggerFactory.getLogger(VendorService.class);
        private final VendorRepo vendorRepo;
        private final PasswordEncoder passwordEncoder;

        public VendorService(VendorRepo vendorRepo, PasswordEncoder passwordEncoder) {
            this.vendorRepo = vendorRepo;
            this.passwordEncoder = passwordEncoder;
        }

        public VendorDTO registerVendor(VendorRequest request) {
            if (vendorRepo.existsByEmail(request.getEmail())) {
                throw new IllegalArgumentException(
                        "A vendor with email " + request.getEmail() + " already exists");
            }
            Vendor vendor = new Vendor();
            vendor.setName(request.getName());
            vendor.setEmail(request.getEmail());
            vendor.setPhone(request.getPhone());
            vendor.setRole(request.getRole());
            vendor.setPassword(passwordEncoder.encode(request.getPassword()));
            Vendor saved = vendorRepo.save(vendor);
            log.info("Vendor registered: {} (id={})", saved.getName(), saved.getId());
            return toDTO(saved);
        }

        public VendorDTO getVendor(Authentication authentication) {
            String email= AuthUtil.extractEmail(authentication);
            return toDTO(vendorRepo.findByEmail(email).orElseThrow(
                    ()->new VendorNotFoundException("Cannot get others accoun")
            ));
        }

        public List<VendorDTO> getAllVendors() {
            return vendorRepo.findAll().stream()
                    .map(this::toDTO)
                    .collect(Collectors.toList());
        }

        public VendorDTO updateVendor(VendorRequest request,Authentication authentication) {
            String email= AuthUtil.extractEmail(authentication);
            Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
                    ()->new VendorNotFoundException("Cannot update others account")
            );
            if (request.getName() != null) vendor.setName(request.getName());
            if (request.getPhone() != null) vendor.setPhone(request.getPhone());
            if (request.getPassword() != null)
vendor.setPassword(passwordEncoder.encode(request.getPassword()));
            // Email update not allowed to avoid breaking restaurant ownership references
            return toDTO(vendorRepo.save(vendor));
        }

        public void deleteVendor(Authentication authentication) {
```

```
            String email= AuthUtil.extractEmail(authentication);
            Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
                    ()->new VendorNotFoundException("Cannot delete others account")
            );
            vendorRepo.delete(vendor);
            log.info("Vendor deleted: email={}", email);
        }

        public void deleteVendorById(Long vendorId) {
            Vendor vendor = findOrThrow(vendorId);
            vendorRepo.delete(vendor);
            log.info("Vendor deleted by admin: id={}, email={}", vendor.getId(), vendor.getEmail());
        }

        //  helpers

        private Vendor findOrThrow(Long id) {
            return vendorRepo.findById(id)
                    .orElseThrow(() -> new VendorNotFoundException("Vendor with id " + id + " not found"));
        }

        private VendorDTO toDTO(Vendor vendor) {
            VendorDTO dto = new VendorDTO();
            dto.setId(vendor.getId());
            dto.setName(vendor.getName());
            dto.setEmail(vendor.getEmail());
            dto.setPhone(vendor.getPhone());
            dto.setRegisteredAt(vendor.getRegisteredAt());
            return dto;
        }
    }
}
```

121 Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java

74 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import org.springframework.stereotype.Service;

import java.time.*;

@Service
public class DeliveryTimeService {

    private final MealSlotService mealSlotService;

    public DeliveryTimeService(MealSlotService mealSlotService) {
        this.mealSlotService = mealSlotService;
    }
    public Instant calculateNextDelivery(SubscriptionRequest request, MealSlot slot, Employee employee) {
        ZoneId userZone=ZoneId.of(employee.getTimezone());
        if (request.getScheduleType() == ScheduleType.DAILY) {
            return calculateDailyNextDelivery(slot,userZone);
        } else {
            return calculateWeeklyNextDelivery(slot, request.getDayOfWeek(),userZone);
        }
    }

    private Instant calculateDailyNextDelivery(MealSlot slot, ZoneId userZone) {
        LocalTime slotTime = mealSlotService.getTime(slot);
        ZonedDateTime now = ZonedDateTime.now(userZone);
        ZonedDateTime deliveryTime;

        if (now.toLocalTime().isBefore(slotTime)) {
            deliveryTime=ZonedDateTime.of(now.toLocalDate(),slotTime,userZone);
        } else {
            deliveryTime=ZonedDateTime.of(now.toLocalDate().plusDays(1), slotTime,userZone);
        }
        return deliveryTime.toInstant();
    }

    private Instant calculateWeeklyNextDelivery(MealSlot slot, DayOfWeek dayOfWeek,ZoneId userZone) {
        LocalTime slotTime = mealSlotService.getTime(slot);
        ZonedDateTime now=ZonedDateTime.now(userZone);
        DayOfWeek today = now.getDayOfWeek();

        int daysToAdd = dayOfWeek.getValue() - today.getValue();

        if (daysToAdd == 0) {
            if (now.toLocalTime().isBefore(slotTime)) {
                return ZonedDateTime.of(now.toLocalDate(), slotTime,userZone).toInstant();
            } else {
                return ZonedDateTime.of(now.toLocalDate().plusDays(7), slotTime,userZone).toInstant();
            }
        }
        if (daysToAdd < 0) {
            daysToAdd += 7;
        }

        ZonedDateTime deliveryTime = ZonedDateTime.of(
                now.toLocalDate().plusDays(daysToAdd),slotTime,userZone);
        return deliveryTime.toInstant();
    }

    public Instant getNextDeliveryTime(Subscription subscription) {
        ZoneId userZone=ZoneId.of(subscription.getEmployee().getTimezone());
        if (subscription.getScheduleType() == ScheduleType.DAILY) {
            return calculateDailyNextDelivery(subscription.getSlot(),userZone);
        } else {
            return calculateWeeklyNextDelivery(subscription.getSlot(),
subscription.getDayOfWeek(),userZone);
```

```
        }
    }
}

```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
import com.example.EmployeeManagementSystem.Entity.Device;
import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Set;

@Service
public class DeviceService {
    private final DeviceRepository deviceRepository;
    private final VendorRepo vendorRepo;
    private final DeviceAssignmentRepo deviceAssignmentRepo;
    private final EmployeeRepo employeeRepo;

    public DeviceService(DeviceRepository deviceRepository,
                         VendorRepo vendorRepo,
                         DeviceAssignmentRepo deviceAssignmentRepo,
                         EmployeeRepo employeeRepo) {
        this.deviceRepository = deviceRepository;
        this.vendorRepo = vendorRepo;
        this.deviceAssignmentRepo = deviceAssignmentRepo;
        this.employeeRepo = employeeRepo;
    }

    public Device addDevice(DeviceDTO deviceDTO, Authentication authentication) {
        String email = AuthUtil.extractEmail(authentication);
        Vendor vendor = vendorRepo.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("Vendor not found"));

        Device device = new Device();
        device.setDeviceName(deviceDTO.getDeviceName());
        device.setWarrantyExpiryDate(deviceDTO.getWarrantyExpiryDate());
        device.setBrand(deviceDTO.getBrand());
        device.setDeviceStatus(DeviceStatus.AVAILABLE);
        device.setDeviceType(deviceDTO.getDeviceType());
        device.setTechVendor(vendor);
        device.setCreatedAt(LocalDateTime.now());
        return deviceRepository.save(device);
    }

    public List<DeviceResponseDTO> getDevicesForLoggedInVendor(Authentication authentication) {
        String email = AuthUtil.extractEmail(authentication);
        Vendor vendor = vendorRepo.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("Vendor not found"));

        return deviceRepository.findByTechVendorId(vendor.getId()).stream()
                .map(this::toDeviceResponse)
                .toList();
    }
```

```java

    public List<DeviceResponseDTO> getDevicesForLoggedInEmployee(Authentication authentication) {
        String email = AuthUtil.extractEmail(authentication);
        Employee employee = employeeRepo.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        return deviceAssignmentRepo.findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(), AssignmentStatus.ACTIVE)
                .stream()
                .map(DeviceAssignment::getDevice)
                .map(this::toDeviceResponse)
                .toList();
    }

    public List<DeviceResponseDTO> getUnassignedDevices() {
        return deviceRepository.findByDeviceStatus(DeviceStatus.AVAILABLE).stream()
                .map(this::toDeviceResponse)
                .toList();
    }

    @Transactional
    public DeviceAssignment assignDevice(Long deviceId, AssignDeviceRequest request) {
        if (request.getEmployeeId() == null) {
            throw new IllegalArgumentException("employeeId is required");
        }

        Device device = deviceRepository.findById(deviceId)
                .orElseThrow(() -> new RuntimeException("Device not found"));

        Employee employee = employeeRepo.findById(request.getEmployeeId())
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        if (employee.getStatus() != Status.ACTIVE) {
            throw new IllegalArgumentException("Device can only be assigned to an active user");
        }

        if (!Set.of(Role.EMPLOYEE, Role.MANAGER, Role.ADMIN).contains(employee.getRole())) {
            throw new IllegalArgumentException("Devices can only be assigned to employees, managers, or admins");
        }

        if (device.getDeviceStatus() != DeviceStatus.AVAILABLE) {
            throw new IllegalArgumentException("Only available devices can be assigned");
        }

        deviceAssignmentRepo.findByDeviceIdAndStatus(deviceId, AssignmentStatus.ACTIVE)
                .ifPresent(existingAssignment -> {
                    throw new IllegalArgumentException("Device is already assigned");
                });

        DeviceAssignment assignment = new DeviceAssignment();
        assignment.setDevice(device);
        assignment.setAssignedTo(employee);
        assignment.setStatus(AssignmentStatus.ACTIVE);
        assignment.setAssignedDate(LocalDate.now());

        DeviceAssignment savedAssignment = deviceAssignmentRepo.save(assignment);
        device.setCurrentAssignment(savedAssignment);
        device.setDeviceStatus(DeviceStatus.ASSIGNED);
        deviceRepository.save(device);

        return savedAssignment;
    }



    private DeviceResponseDTO toDeviceResponse(Device device) {
        DeviceResponseDTO response = new DeviceResponseDTO();
        response.setId(device.getId());
        response.setDeviceName(device.getDeviceName());
        response.setBrand(device.getBrand());
        response.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
        response.setDeviceType(device.getDeviceType());
        response.setDeviceStatus(device.getDeviceStatus());

        if (device.getTechVendor() != null) {
```

```
            response.setVendor(device.getTechVendor());
        }

        DeviceAssignment currentAssignment = device.getCurrentAssignment();
        if (currentAssignment != null) {
            response.setAssignedDate(currentAssignment.getAssignedDate());

            Employee assignedEmployee = currentAssignment.getAssignedTo();
            if (assignedEmployee != null) {
                response.setAssignedEmployee(assignedEmployee);
                response.setAssignedEmployeeName(assignedEmployee.getName());
            }
        }

        return response;
    }

    public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfvendor(Authentication authentication) {
     String email=AuthUtil.extractEmail(authentication);
     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
             ()->new VendorNotFoundException("Vendor:"+email+" not found")
     );
     List<DeviceAssignment>
deviceAssignments=deviceAssignmentRepo.getAllAssignmentsByVendorId(vendor.getId());
     return deviceAssignments.stream().map(deviceAssignment -> toAssignmentDTO(deviceAssignment)).toList();
    }

    private DeviceAssignmentResponseDTO toAssignmentDTO(DeviceAssignment a) {
        DeviceAssignmentResponseDTO dto = new DeviceAssignmentResponseDTO();
        dto.setId(a.getId());
        dto.setStatus(a.getStatus());
        dto.setAssignedDate(a.getAssignedDate());

        if (a.getDevice() != null) {
            dto.setDeviceId(a.getDevice().getId());
            dto.setDeviceName(a.getDevice().getDeviceName());
            dto.setDeviceBrand(a.getDevice().getBrand());
            dto.setDeviceType(a.getDevice().getDeviceType());
        }

        if (a.getAssignedTo() != null) {
            dto.setEmployeeId(a.getAssignedTo().getEmployeeId());
            dto.setEmployeeName(a.getAssignedTo().getName()); // adjust to your Employee field
        }

        return dto;
    }
}

```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
import com.example.EmployeeManagementSystem.DTO.EmployeeAttendanceResponseDTO;
import com.example.EmployeeManagementSystem.DTO.EmployeeStatusDTO;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.ZoneId;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class EmployeeService {

    private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);

    private final EmployeeRepo employeeRepo;
    private final LeaveRequestRepo leaveRequestRepo;
    private final PasswordEncoder passwordEncoder;
    private final RefreshTokenRepository refreshTokenRepository; // FIX: added

    public EmployeeService(EmployeeRepo employeeRepo,
                           LeaveRequestRepo leaveRequestRepo,
                           PasswordEncoder passwordEncoder,
                           RefreshTokenRepository refreshTokenRepository) {
        this.employeeRepo = employeeRepo;
        this.leaveRequestRepo = leaveRequestRepo;
        this.passwordEncoder = passwordEncoder;
        this.refreshTokenRepository = refreshTokenRepository; // FIX: injected
    }

    public List<Employee> getAllEmployees() {
        return employeeRepo.findAll();
    }

    public Employee createEmployee(EmployeeDTO employeeDTO) {
        log.info("Creating new employee with email: {}", employeeDTO.getEmail());
        var employee = new Employee();
        if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
        if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
        if (employeeDTO.getDept() != null) employee.setDept(employeeDTO.getDept());
        if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
            employee.setTimezone(employeeDTO.getTimezone());
        } else {
            employee.setTimezone("UTC");
        }
        employee.setRole(Role.EMPLOYEE);
        if (employeeDTO.getPassword() == null) {
            throw new RuntimeException("Password is required");
        }
        employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
        return employeeRepo.save(employee);
    }

    public Employee createManager(EmployeeDTO employeeDTO) {
        log.info("Creating new manager with email: {}", employeeDTO.getEmail());
```

```
        var employee = new Employee();
        if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
        if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
        if (employeeDTO.getDept() != null) employee.setDept(employeeDTO.getDept());
        if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
            employee.setTimezone(employeeDTO.getTimezone());
        } else {
            employee.setTimezone("UTC");
        }
        employee.setRole(Role.MANAGER);
        if (employeeDTO.getPassword() == null) {
            throw new RuntimeException("Password is required");
        }
        employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
        return employeeRepo.save(employee);
    }

    public Employee updateEmployee(long id, EmployeeDTO employee) {
        var updateEmployee = employeeRepo.findById(id).orElseThrow(
                () -> new EmployeeNotFound("Employee with id:" + id + " not found")
        );
        if (employee.getName() != null)
            updateEmployee.setName(employee.getName());
        if (employee.getEmail() != null)
            updateEmployee.setEmail(employee.getEmail());
        if (employee.getDept() != null)
            updateEmployee.setDept(employee.getDept());
        if (employee.getTimezone() != null && isValidTimezone(employee.getTimezone())) {
             updateEmployee.setTimezone(employee.getTimezone());
         }
         return employeeRepo.save(updateEmployee);
     }

     /**
      * FIX: Delete order must be:
      *  1. refresh_token  (FK  employee)
      *  2. leave_request  (FK  employee)
      *  3. employee
      *
      * Previously only leave_request was deleted first, causing a
      * FK constraint failure on refresh_token when deleting the employee.
      */
     @Transactional
     public void deleteEmployee(long id) {
         var employee = employeeRepo.findById(id).orElseThrow(
                 () -> new EmployeeNotFound("Employee with id:" + id + " not found")
         );

         // Step 1: delete refresh tokens for this employee
         refreshTokenRepository.deleteByEmployee(employee);

         // Step 2: delete leave requests for this employee
         List<LeaveRequest> leaveRequests =
                 leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
         leaveRequestRepo.deleteAll(leaveRequests);

         // Step 3: delete the employee
         employeeRepo.delete(employee);
     }

     public ResponseEntity<String> inactivateUser(long employeeId) {
         var employee = employeeRepo.findById(employeeId).orElseThrow(
                 () -> new EmployeeNotFound("Employee with id:" + employeeId + " not found")
         );
         if (employee.getStatus() == Status.INACTIVE) {
             return ResponseEntity.badRequest()
                     .body("Status of the employee with id: " + employeeId + " is already set to inactive");
         }
         employee.setStatus(Status.INACTIVE);
         employeeRepo.save(employee);
         return ResponseEntity.ok("Status of the employee with id: " + employeeId + " is set to inactive");
     }

     /** Validate that the given string is a recognized IANA timezone id. */
     private boolean isValidTimezone(String tz) {
         try {
```

```
            ZoneId.of(tz);
            return true;
        } catch (Exception e) {
            return false;
        }
    }

    public Employee getAccount(Authentication authentication) {
        String email = AuthUtil.extractEmail(authentication);
        Employee employee = employeeRepo.findByEmail(email).orElseThrow(
                () -> new EmployeeNotFound("Employee :" + email + " NOT FOUND")
        );
        return employee;
    }

    public EmployeeAttendanceResponseDTO getEmployeeAttendanceOverview() {
        List<Employee> employees = employeeRepo.findAll();

        List<EmployeeStatusDTO> workingEmployees = employees.stream()
                .filter(e -> e.getStatus() == Status.ACTIVE)
                .map(this::toStatusDTO)
                .collect(Collectors.toList());

        List<EmployeeStatusDTO> onLeaveEmployees = employees.stream()
                .filter(e -> e.getStatus() == Status.ON_LEAVE)
                .map(this::toStatusDTO)
                .collect(Collectors.toList());

        EmployeeAttendanceResponseDTO response = new EmployeeAttendanceResponseDTO();
        response.setWorkingCount(workingEmployees.size());
        response.setOnLeaveCount(onLeaveEmployees.size());
        response.setWorkingEmployees(workingEmployees);
        response.setOnLeaveEmployees(onLeaveEmployees);
        return response;
    }

    private EmployeeStatusDTO toStatusDTO(Employee employee) {
        EmployeeStatusDTO dto = new EmployeeStatusDTO();
        dto.setEmployeeId(employee.getEmployeeId());
        dto.setName(employee.getName());
        dto.setEmail(employee.getEmail());
        dto.setDept(employee.getDept());
        dto.setRole(employee.getRole());
        return dto;
    }
}
```

124 Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveAccrualService.java

123 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveType;
import com.example.EmployeeManagementSystem.Enum.Gender;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Service
public class LeaveAccrualService {

    private static final Logger log = LoggerFactory.getLogger(LeaveAccrualService.class);
    private static final BigDecimal ONE_DAY = BigDecimal.ONE;

    private final EmployeeRepo employeeRepository;
    private final LeaveTypeRepository leaveTypeRepository;
    private final LeaveEntitlementRepository entitlementRepository;

    public LeaveAccrualService(EmployeeRepo employeeRepository,
                               LeaveTypeRepository leaveTypeRepository,
                               LeaveEntitlementRepository entitlementRepository) {
        this.employeeRepository = employeeRepository;
        this.leaveTypeRepository = leaveTypeRepository;
        this.entitlementRepository = entitlementRepository;
    }

    @Transactional
    public void runMonthlyAccrual(LocalDate accrualDate) {
        int year = accrualDate.getYear();

        List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
        List<LeaveType> allLeaveTypes = leaveTypeRepository.findAll();

        log.info("Starting monthly accrual for {} — {} active employees",
                accrualDate, activeEmployees.size());

        for (Employee employee : activeEmployees) {
            for (LeaveType leaveType : allLeaveTypes) {

                if (!isEligible(employee, leaveType, accrualDate)) {
                    continue;
                }

                // Find or create entitlement row for this employee/type/year
                LeaveEntitlement entitlement = entitlementRepository
                        .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
                                employee.getEmployeeId(), leaveType.getId(), year)
                        .orElseGet(() -> createNewEntitlement(employee, leaveType, year));

                // Add 1 day accrual
                entitlement.setAccruedThisYear(
                        entitlement.getAccruedThisYear().add(ONE_DAY)
                );

                entitlementRepository.save(entitlement);
                log.debug("Accrued 1 {} day for employee {} (year {})",
                        leaveType.getName(), employee.getEmployeeId(), year);
            }
        }

```

```
        log.info("Monthly accrual complete for {}", accrualDate);
    }

    // ----------------------------------------------------------------
    // Eligibility check — single place for all rules
    // ----------------------------------------------------------------
    private boolean isEligible(Employee employee, LeaveType leaveType, LocalDate accrualDate) {

        // 1. Gender restriction (Maternity = female only)
        if (leaveType.isGenderRestricted()) {
            Gender restriction = leaveType.getGenderRestriction();
            if (restriction != null && restriction != employee.getGender()) {
                return false;
            }
        }

        // 2. Employee must have joined ON OR BEFORE the accrual date
        //    (no accrual for employees joining after the 1st of this month)
        if (employee.getJoined_at() != null && employee.getJoined_at().isAfter(accrualDate)) {
            return false;
        }

        return true;
    }

    // ----------------------------------------------------------------
    // Creates a fresh entitlement row, carrying forward previous year's
    // closing balance if this is the first row of the year
     // ----------------------------------------------------------------
     private LeaveEntitlement createNewEntitlement(Employee employee,
                                                   LeaveType leaveType,
                                                   int year) {
         LeaveEntitlement entitlement = new LeaveEntitlement();
         entitlement.setEmployee(employee);
         entitlement.setLeaveType(leaveType);
         entitlement.setYear(year);

         // Carry forward last year's closing balance (if the type supports it)
         if (leaveType.isCarriesForward()) {
             BigDecimal lastYearClosing = entitlementRepository
                     .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
                             employee.getEmployeeId(), leaveType.getId(), year - 1)
                     .map(LeaveEntitlement::getClosingBalance)
                     .orElse(BigDecimal.ZERO);

             entitlement.setOpeningBalance(lastYearClosing);
         }

         return entitlementRepository.save(entitlement);
     }
}
```

125 Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveDashboardService.java

1 li

```
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

@Service
public class LeaveDashboardService {

    private final EmployeeRepo employeeRepository;
    private final LeaveEntitlementRepository leaveEntitlementRepository; // was not final
    private final LeaveWarningRepository warningRepository;

    public LeaveDashboardService(EmployeeRepo employeeRepository,
                                 LeaveEntitlementRepository leaveEntitlementRepository, // parameter name fixed
                                 LeaveWarningRepository warningRepository) {
        this.employeeRepository = employeeRepository;
        this.leaveEntitlementRepository = leaveEntitlementRepository; // was "entitlementRepository" — mismatch!
        this.warningRepository = warningRepository;
    }

    @Transactional(readOnly = true)
    public LeaveDashboardResponse getDashboard(Long requestedEmployeeId,
                                               Long callerEmployeeId,
                                               String callerRole) {
        assertAccess(requestedEmployeeId, callerEmployeeId, callerRole);

        Employee employee = employeeRepository.findById(requestedEmployeeId)
                .orElseThrow(() -> new IllegalArgumentException(
                        "Employee not found: " + requestedEmployeeId));

        int year = LocalDate.now().getYear();

        // Fixed: was using undefined "employeeId", and now returns List not Optional
        List<LeaveEntitlement> entitlements = leaveEntitlementRepository
                .findByEmployeeEmployeeIdAndYear(requestedEmployeeId, year);

        List<LeaveWarning> warnings =
                warningRepository.findByEmployeeEmployeeIdAndIsReadFalseOrderByWarningDateDesc(
                        requestedEmployeeId);

        LeaveDashboardResponse.LeaveBalanceDto sick      = buildBalanceDto(entitlements, "SICK");
        LeaveDashboardResponse.LeaveBalanceDto casual    = buildBalanceDto(entitlements, "CASUAL");
        LeaveDashboardResponse.LeaveBalanceDto maternity = buildBalanceDto(entitlements, "MATERNITY");

        BigDecimal carryTotal = sum(
                casual    != null ? casual.getAvailable()    : null,
                maternity != null ? maternity.getAvailable() : null
        );

        LeaveDashboardResponse response = new LeaveDashboardResponse();
        response.setEmployeeId(employee.getEmployeeId());
        response.setEmployeeName(employee.getName());
```

```
        response.setYear(year);
        response.setSick(sick);
        response.setCasual(casual);
        response.setMaternity(maternity);
        response.setCarryForwardMeter(new LeaveDashboardResponse.CarryForwardMeterDto(carryTotal, 30));
        response.setWarnings(warnings.stream()
                .map(w -> new LeaveDashboardResponse.WarningDto(
                        w.getId(),
                        w.getWarningType().name(),
                        w.getMessage(),
                        w.getWarningDate().toString()))
                .toList());

        return response;
    }

    //  Mark warning read

    @Transactional
    public void markWarningRead(Long warningId, Long callerEmployeeId, String callerRole) {
        LeaveWarning warning = warningRepository.findById(warningId)
                .orElseThrow(() -> new IllegalArgumentException("Warning not found: " + warningId));

        assertAccess(warning.getEmployee().getEmployeeId(),callerEmployeeId, callerRole);

        warning.setRead(true);
        warningRepository.save(warning);
    }

    //  Team dashboard (manager/admin)

     @Transactional(readOnly = true)
     public List<LeaveDashboardResponse> getTeamDashboard(Long managerId, String callerRole) {
         if (!"MANAGER".equals(callerRole) && !"ADMIN".equals(callerRole)) {
             throw new AccessDeniedException("Only managers and admins can view team dashboards");
         }

         List<Employee> team = employeeRepository.findByManagerEmployeeId(managerId);
         return team.stream()
                 .map(emp -> getDashboard(emp.getEmployeeId(), managerId, callerRole))
                 .toList();
     }

     //  Helpers

     private LeaveDashboardResponse.LeaveBalanceDto buildBalanceDto(List<LeaveEntitlement> entitlements,
                                                                    String typeName) {
         return entitlements.stream()
                 .filter(e -> e.getLeaveType().getName().equals(typeName))
                 .findFirst()
                 .map(e -> new LeaveDashboardResponse.LeaveBalanceDto(
                         typeName,
                         e.getOpeningBalance(),
                         e.getAccruedThisYear(),
                         e.getUsedThisYear(),
                         e.getAvailableBalance(),
                         e.getLeaveType().isCarriesForward()))
                 .orElse(null);  // null = leave type not applicable for this employee
     }

     private BigDecimal sum(BigDecimal a, BigDecimal b) {
         BigDecimal result = BigDecimal.ZERO;
         if (a != null) result = result.add(a);
         if (b != null) result = result.add(b);
         return result;
     }

     private void assertAccess(Long targetId, Long callerId, String role) {
         boolean isSelf    = targetId.equals(callerId);
         boolean isManager = "MANAGER".equals(role) || "ADMIN".equals(role);
         if (!isSelf && !isManager) {
             throw new AccessDeniedException(
                     "Employee " + callerId + " cannot view dashboard for " + targetId);
         }
     }
}
```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.ActionDTO;
import com.example.EmployeeManagementSystem.DTO.AuthRequest;
import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.LeaveType;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Exception.*;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.ZoneId;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@Service
public class LeaveRequestService {
    private final LeaveRequestRepo leaveRequestRepo;
    private final EmployeeRepo employeeRepo;
    private static final Logger log = LoggerFactory.getLogger(LeaveRequestService.class);
    public LeaveRequestService(LeaveRequestRepo leaveRequestRepo, EmployeeRepo employeeRepo) {
        this.leaveRequestRepo = leaveRequestRepo;
        this.employeeRepo = employeeRepo;
    }

    public List<LeaveResponseDTO> getAllTheLeaveRequest(){
        List<LeaveRequest> requestList=leaveRequestRepo.findAll();
        List<LeaveResponseDTO> dtos=new ArrayList<>();
        for(LeaveRequest request:requestList){
           dtos.add(convertToDTO(request));
        }
        return dtos;
    }


    public ResponseEntity<?> createRequest(Authentication authentication, LeaveRequestDTO requestDTO){
        var leaveRequest=new LeaveRequest();
        String email= AuthUtil.extractEmail(authentication);
        //Validate employee exists and is active
        var employee=employeeRepo.findByEmail(email).orElseThrow(()->
                new EmployeeNotFound("Employee with name: "+email+" is not found")
        );
        if (employee.getStatus() != Status.ACTIVE) {
            return ResponseEntity.badRequest()
                    .body(Map.of("error", "Only active employees can apply for leave"));
        }

        leaveRequest.setEmployee(employee);
        //Checking if the start date is before today
        String timezone= employee.getTimezone();
        ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
                ? timezone
                : "UTC");
        LocalDate today = LocalDate.now(zoneId);
        if (requestDTO.getStartDate().isBefore(today)) {
```

```
            throw new InvalidStartDateException("Start date cannot be before current date");
        }
        //Checking if the end date is before start date
        if (requestDTO.getEndDate().isBefore(requestDTO.getStartDate())) {
            throw new InvalidEndDateException("End date must be equal to or greater than start date");
        }
        //optional
        long daysRequested = ChronoUnit.DAYS.between(requestDTO.getStartDate(), requestDTO.getEndDate()) + 1;
        if (daysRequested > 30) {
            return ResponseEntity.badRequest()
                    .body(Map.of("error", "Leave cannot exceed 30 consecutive days"));
        }
        //optional
        if (requestDTO.getLeaveType() == LeaveType.SICK) {
            long sickDaysUsed = leaveRequestRepo.countDaysByEmployeeAndLeaveTypeAndYear(
                    employee, LeaveType.SICK, today.getYear());

            if (sickDaysUsed + daysRequested > 12) { // Assuming 12 sick days per year
                return ResponseEntity.badRequest()
                        .body(Map.of("error", "Sick leave limit exceeded for this year"));
            }
        }
        long duplicateCount= leaveRequestRepo.checkDuplicate(
                employee.getEmployeeId(),
                requestDTO.getStartDate(),
                requestDTO.getEndDate(),
                LeaveStatus.PENDING.name()
        );
         if(duplicateCount>0){
             throw new DuplicateRequestException("Duplicate leave request");
         }

         //checking if there is any overlapping leave request of that employee
         long count= leaveRequestRepo.countOverlappingLeave(
                 employee.getEmployeeId(),
                 requestDTO.getStartDate(),
                 requestDTO.getEndDate()
         );
          if(count>0){
              throw new OverlappingLeaveException("Overlapping leave exists");
          }
          leaveRequest.setStartDate(requestDTO.getStartDate());
          leaveRequest.setEndDate(requestDTO.getEndDate());

          if(requestDTO.getLeaveType()==null){
              return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                      .body("Leave type is required..");
          }
          leaveRequest.setLeaveType(requestDTO.getLeaveType());
          if(requestDTO.getReason()!=null){
              leaveRequest.setReason(requestDTO.getReason());
          }
          LeaveRequest savedRequest=leaveRequestRepo.save(leaveRequest);
          return ResponseEntity.status(HttpStatus.CREATED).body(convertToDTO(savedRequest));
    }

    //Only for manager
    public List<LeaveResponseDTO> getAllThePendingLeaveRequests(){
        List<LeaveRequest> requestList=leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
        List<LeaveResponseDTO> responseDTOS=new ArrayList<>();
        for(LeaveRequest request:requestList){
            responseDTOS.add(convertToDTO(request));
        }
        return responseDTOS;
    }

    public ResponseEntity<?> updateLeaveRequestStatus(ActionDTO actionDTO,Authentication authentication){
        String email= AuthUtil.extractEmail(authentication);
        Employee manager=employeeRepo.findByEmail(email).orElseThrow(
                ()->new EmployeeNotFound("Manager with name: "+email+" Not Found")
        );
        if (actionDTO.getAction() == null) {
            return ResponseEntity.badRequest().body("Action is required");
        }
        log.info("Updating leave request status for id: {}", actionDTO.getLeaveRequestId());
```

```java

        var leaveRequest=leaveRequestRepo.findById(actionDTO.getLeaveRequestId()).orElseThrow(
                ()-> new LeaveRequestNotFoundException("LeaveRequest with id:"+actionDTO.getLeaveRequestId()+" not found")
        );

        if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
            return ResponseEntity.badRequest()
                    .body("Only PENDING requests can be approved or rejected. Current status: "
                            + leaveRequest.getStatus());
        }

         if(actionDTO.getAction().equalsIgnoreCase("approved")){
             leaveRequest.setStatus(LeaveStatus.APPROVED);
         }
         else if(actionDTO.getAction().equalsIgnoreCase("rejected")){
             leaveRequest.setStatus(LeaveStatus.REJECTED);
         }
         else{
             return ResponseEntity.badRequest().body("Invalid action,Use APPROVED or REJECTED");
         }
         leaveRequest.setManager(manager.getEmail());
         if(actionDTO.getRemarks()!=null){
             leaveRequest.setRemarks(actionDTO.getRemarks());
         }
         return ResponseEntity.ok(leaveRequestRepo.save(leaveRequest));
    }

    public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,long leaveId){
        String email= AuthUtil.extractEmail(authentication);
        var employee=employeeRepo.findByEmail(email).orElseThrow(
                ()->new EmployeeNotFound("Employee with email:"+email+" not found")
        );
        var leaveRequest=leaveRequestRepo.findById(leaveId).orElseThrow(
                ()->new LeaveRequestNotFoundException("Leave request with id:"+leaveId+" is not found")
        );
        if(employee.getEmployeeId()!=leaveRequest.getEmployee().getEmployeeId()){
            return ResponseEntity.badRequest().body("You cannot cancel others Leave request..");
        }
        String timezone= employee.getTimezone();
        ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
                ? timezone
                : "UTC");
        LocalDate today = LocalDate.now(zoneId);
        if(leaveRequest.getStartDate().isEqual(today)||leaveRequest.getStartDate().isBefore(today)){
            return ResponseEntity.badRequest().body("You cannot cancel leave request after the leave has started");
        }
        leaveRequest.setStatus(LeaveStatus.CANCELLED);
        leaveRequestRepo.save(leaveRequest);
        return ResponseEntity.ok("Leave Request with id:"+ leaveId+" Successfully cancelled");
    }

    private LeaveResponseDTO convertToDTO(LeaveRequest request){
        var responseDTO=new LeaveResponseDTO();
        responseDTO.setLeaveRequestId(request.getId());
        responseDTO.setEmployeeId(request.getEmployee().getEmployeeId());
        responseDTO.setLeaveType(request.getLeaveType());
        responseDTO.setStartDate(request.getStartDate());
        responseDTO.setEndDate(request.getEndDate());
        responseDTO.setReason(request.getReason());
        responseDTO.setStatus(request.getStatus());
        return responseDTO;
    }

    public ResponseEntity<?> getLeaveRequestsByEmployee(Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        log.info("Fetching leave requests for employee: {}", email);

        Employee employee = employeeRepo.findByEmail(email)
                .orElseThrow(() -> new EmployeeNotFound("Employee not found: " +email ));

        List<LeaveRequest> requests = leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
        List<LeaveResponseDTO> response = requests.stream()
                .map(this::convertToDTO)
                .collect(Collectors.toList());
```

```

        return ResponseEntity.ok(Map.of(
                "employeeName", email,
                "totalRequests", response.size(),
                "requests", response
        ));
    }
    public List<LeaveResponseDTO> getPendingLeaves() {
        List<LeaveRequest> pendingRequests = leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
        return pendingRequests.stream()
                .map(this::convertToDTO)
                .collect(Collectors.toList());
    }

    /**
     * Approve a leave request by ID (simplified version for AdminController).
     * Returns LeaveResponseDTO directly.
     */
    public LeaveResponseDTO approveLeave(Long id) {
        LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
                .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not found"));

        if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
            throw new IllegalStateException("Only PENDING requests can be approved. Current status: " + leaveRequest.getStatus());
        }

        leaveRequest.setStatus(LeaveStatus.APPROVED);
        leaveRequest.setManager("ADMIN"); // Or get from security context if needed

        LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
        return convertToDTO(saved);
    }

    /**
     * Reject a leave request by ID (simplified version for AdminController).
     * Returns LeaveResponseDTO directly.
     */
    public LeaveResponseDTO rejectLeave(Long id) {
        LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
                .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not found"));

        if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
            throw new IllegalStateException("Only PENDING requests can be rejected. Current status: " + leaveRequest.getStatus());
        }

        leaveRequest.setStatus(LeaveStatus.REJECTED);
        leaveRequest.setManager("ADMIN");

        LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
        return convertToDTO(saved);
    }
}

```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.*;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.jspecify.annotations.Nullable;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class ManagerService {
    private final EmployeeRepo employeeRepo;
    private final PasswordEncoder passwordEncoder;
    private final LeaveRequestRepo leaveRequestRepo;

    public ManagerService(EmployeeRepo employeeRepo, PasswordEncoder passwordEncoder, LeaveRequestRepo leaveRequestRepo) {
        this.employeeRepo = employeeRepo;
        this.passwordEncoder = passwordEncoder;
        this.leaveRequestRepo = leaveRequestRepo;
    }

    public Employee createEmp(EmployeeDetails employeeDetails, Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        Employee manager= employeeRepo.findByEmail(email).orElseThrow(
                ()->new EmployeeNotFound("Manager :"+email+"Not found")
        );
        Employee employee=new Employee();
        employee.setName(employeeDetails.getName());
        employee.setEmail(employeeDetails.getEmail());
        employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
        employee.setDept(manager.getDept());
        employee.setRole(Role.EMPLOYEE);
        employee.setManager(manager);
        employee.setTimezone(employeeDetails.getTimezone());
        return employeeRepo.save(employee);
    }


    public List<Employee> getAllEmployeeByManager(Authentication authentication) {
        String email=AuthUtil.extractEmail(authentication);
        Employee manager=employeeRepo.findByEmail(email).orElseThrow(
                ()->new EmployeeNotFound("Manger:"+email+" not found")
        );
        return employeeRepo.findByManager(manager);
    }

    public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication) {
        String email=AuthUtil.extractEmail(authentication);
       Employee manager=employeeRepo.findByEmail(email).orElseThrow(
               ()->new EmployeeNotFound("Manager with "+email+" not found")
       );
       List<LeaveRequest> requests=leaveRequestRepo.findByEmployee_Manager(manager);
       return requests.stream().map(request->toManagerLeaveResponseDTO(request)).toList();
    }

    public List<EmployeeDTO> getUsers(){
        List<Employee> employeeList=employeeRepo.findByRole(Role.USER);
        return employeeList.stream().map(this::toDTO).toList();
    }
```

```java

    public ResponseEntity<?> promoteUser(Authentication authentication, PromoteRequest promoteRequest){
        Employee user=employeeRepo.findByEmail(promoteRequest.getEmail()).orElseThrow(
                ()->new EmployeeNotFound("Employee:"+promoteRequest.getEmail()+" not found")
        );
        if(user.getRole()!=Role.USER){
            return ResponseEntity.ok("Employee already promoted");
        }
        String managerEmail=AuthUtil.extractEmail(authentication);
        Employee manager=employeeRepo.findByEmail(managerEmail).orElseThrow(
                ()->new EmployeeNotFound("Manager:"+managerEmail+" not found")
        );

        user.setRole(Role.EMPLOYEE);
        user.setManager(manager);
        user.setDept(manager.getDept());
        user.setPassword(passwordEncoder.encode(promoteRequest.getPassword()));
        Employee promotedEmployee=employeeRepo.save(user);
        return ResponseEntity.ok(toDTO(promotedEmployee));
    }

    public EmployeeDTO toDTO(Employee employee){
        EmployeeDTO employeeDTO=new EmployeeDTO();
        employeeDTO.setName(employee.getName());
        employeeDTO.setEmail(employee.getEmail());
        employeeDTO.setDept(employee.getDept());
        employeeDTO.setPassword(employee.getPassword());
        employeeDTO.setTimezone(employee.getTimezone());
        return employeeDTO;
    }

    public Employee createAdmin(EmployeeDetails employeeDetails) {
        Employee employee=new Employee();
        employee.setName(employeeDetails.getName());
        employee.setEmail(employeeDetails.getEmail());
        employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
        employee.setDept("Main");
        employee.setRole(Role.ADMIN);
        employee.setTimezone(employeeDetails.getTimezone());
        return employeeRepo.save(employee);
    }

    public List<LeaveResponseDTO> getAllThePendingLeaveRequests(Authentication authentication) {
        String email=AuthUtil.extractEmail(authentication);
        Employee manager=employeeRepo.findByEmail(email).orElseThrow(
                ()->new EmployeeNotFound("Manager with "+email+" not found")
        );
        List<LeaveRequest>
leaveRequests=leaveRequestRepo.findByStatusAndEmployee_Manager(LeaveStatus.PENDING,manager);
        return leaveRequests.stream()
                .map(this::convertToLeaveResponseDTO)
                .collect(Collectors.toList());
    }

    private LeaveResponseDTO convertToLeaveResponseDTO(LeaveRequest leaveRequest) {
        LeaveResponseDTO dto = new LeaveResponseDTO();
        dto.setLeaveRequestId(leaveRequest.getId());
        dto.setEmployeeId(leaveRequest.getEmployee().getEmployeeId());
        dto.setLeaveType(leaveRequest.getLeaveType());
        dto.setStartDate(leaveRequest.getStartDate());
        dto.setEndDate(leaveRequest.getEndDate());
        dto.setReason(leaveRequest.getReason());
        dto.setStatus(leaveRequest.getStatus());
        return dto;
    }

    private ManagerLeaveResponseDTO toManagerLeaveResponseDTO(LeaveRequest leaveRequest){
        ManagerLeaveResponseDTO managerLeaveResponseDTO=new ManagerLeaveResponseDTO();
        managerLeaveResponseDTO.setEmployeeName(leaveRequest.getEmployee().getName());
        managerLeaveResponseDTO.setLeaveType(leaveRequest.getLeaveType());
        managerLeaveResponseDTO.setStartDate(leaveRequest.getStartDate());
        managerLeaveResponseDTO.setEndDate(leaveRequest.getEndDate());
        managerLeaveResponseDTO.setStatus(leaveRequest.getStatus());
        managerLeaveResponseDTO.setReason(leaveRequest.getReason());
        return managerLeaveResponseDTO;
    }
```

```
}

```

128 Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java

22 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Enum.MealSlot;
import org.springframework.stereotype.Service;

import java.time.LocalTime;
import java.util.Map;

@Service
 public class MealSlotService {

     private final Map<MealSlot, LocalTime> slotLocalTimeMap = Map.of(
             MealSlot.BREAKFAST, LocalTime.of(8, 0),
             MealSlot.LUNCH, LocalTime.of(13, 0),
             MealSlot.DINNER, LocalTime.of(21, 0)
     );

     public LocalTime getTime(MealSlot slot) {
         return slotLocalTimeMap.get(slot);
     }
 }

```

129 Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java

41 lines

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import org.springframework.stereotype.Service;

 import java.time.Instant;
 import java.time.temporal.ChronoUnit;
 import java.util.UUID;

 @Service
 public class RefreshTokenService {
     private final EmployeeRepo employeeRepo;
     private final VendorRepo vendorRepo;

     public RefreshTokenService(RefreshTokenRepository refreshTokenRepository, EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
         this.employeeRepo = employeeRepo;
         this.vendorRepo= vendorRepo;
     }

     public RefreshToken createForEmployee(Employee employee) {
         RefreshToken refreshToken = new RefreshToken();
         refreshToken.setToken(UUID.randomUUID().toString());
         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
         refreshToken.setEmployee(employee);
         return refreshToken;
     }

     public RefreshToken createForVendor(Vendor vendor) {
         RefreshToken refreshToken = new RefreshToken();
         refreshToken.setToken(UUID.randomUUID().toString());
         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
         refreshToken.setVendor(vendor);
         return refreshToken;
     }
 }

```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
import com.example.EmployeeManagementSystem.Entity.Restaurant;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
import com.example.EmployeeManagementSystem.Exception.UnauthorizedAccessException;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.stream.Collectors;

@Service
public class RestaurantService {

    private static final Logger log = LoggerFactory.getLogger(RestaurantService.class);

    private final RestaurantRepository restaurantRepository;
    private final VendorRepo vendorRepo;

    public RestaurantService(RestaurantRepository restaurantRepository, VendorRepo vendorRepo) {
        this.restaurantRepository = restaurantRepository;
        this.vendorRepo = vendorRepo;
    }

    /**
     * Create a new restaurant.
     * Only the owning vendor (matched by vendorId in the request) may do this.
     */
    public RestaurantDTO createRestaurant(RestaurantRequest request,Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        Vendor vendor=vendorRepo.findByEmail(email)
                .orElseThrow(() -> new VendorNotFoundException(
                        "Vendor with email " + email + " not found"));

        Restaurant restaurant = new Restaurant();
        restaurant.setName(request.getName());
        restaurant.setAddress(request.getAddress());
        restaurant.setDescription(request.getDescription());
        restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
        restaurant.setVendor(vendor);
        restaurant.setActive(true);

        Restaurant saved = restaurantRepository.save(restaurant);
        log.info("Restaurant created: {} by vendor {}", saved.getName(), vendor.getId());
        return toDTO(saved);
    }

    /**
     * Update restaurant details.
     * Vendor must own this restaurant — otherwise 403.
     */
    public RestaurantDTO updateRestaurant(Long restaurantId, RestaurantRequest request,Authentication
authentication) {
        Restaurant restaurant = findOrThrow(restaurantId);
        String email= AuthUtil.extractEmail(authentication);
        Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
                ()->new VendorNotFoundException("Vendor Not found")
        );
        assertOwnership(restaurantId, vendor.getEmail());

        if (request.getName() != null) restaurant.setName(request.getName());
```

```
        if (request.getAddress() != null) restaurant.setAddress(request.getAddress());
        if (request.getDescription() != null) restaurant.setDescription(request.getDescription());
        if (request.getSupportedMealSlots() != null && !request.getSupportedMealSlots().isEmpty()) {
            restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
        }
        return toDTO(restaurantRepository.save(restaurant));
    }

    /**
     * Deactivate a restaurant (soft delete).
     * Only the owning vendor may do this.
     * Active subscriptions to this restaurant will no longer receive new deliveries
     * — the Subscription status should be handled separately.
     */
    public void deactivateRestaurant(Long restaurantId, Authentication authentication) {
        Restaurant restaurant = findOrThrow(restaurantId);
        String email= AuthUtil.extractEmail(authentication);
        Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
                ()->new VendorNotFoundException("Vendor Not found")
        );
        assertOwnership(restaurantId, vendor.getEmail());
        restaurant.setActive(false);
        restaurantRepository.save(restaurant);
        log.info("Restaurant deactivated: id={}", restaurantId);
    }

    /**
     * Get all restaurants owned by a vendor.
     * VENDOR-only endpoint.
     */
    public List<RestaurantDTO> getRestaurantsByVendor(Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        vendorRepo.findByEmail(email)
                .orElseThrow(() -> new VendorNotFoundException("Vendor " + email + " not found"));
        return restaurantRepository.findByVendor_EmailAndActiveTrue(email)
                .stream().map(this::toDTO).collect(Collectors.toList());
    }

    /**
     * Get all active restaurants.
     * Available to EMPLOYEE so they can browse and choose.
     */
    public List<RestaurantDTO> getAllActiveRestaurants() {
        return restaurantRepository.findByActiveTrue()
                .stream().map(this::toDTO).collect(Collectors.toList());
    }

    /**
     * Get active restaurants that serve a specific meal slot.
     * Employees call this when subscribing, to see valid restaurant options for a slot.
     */
    public List<RestaurantDTO> getRestaurantsByMealSlot(MealSlot mealSlot) {
        return restaurantRepository.findActiveByMealSlot(mealSlot)
                .stream().map(this::toDTO).collect(Collectors.toList());
    }

    public RestaurantDTO getRestaurant(Long id) {
        return toDTO(findOrThrow(id));
    }

    //  helpers

    private Restaurant findOrThrow(Long id) {
        return restaurantRepository.findById(id)
                .orElseThrow(() -> new RestaurantNotFoundException(
                        "Restaurant with id " + id + " not found"));
    }

    /**
     * Throws UnauthorizedAccessException if the restaurant does not belong to the vendor.
     * This is the core access-control guard for all mutating vendor operations.
     */
    private void assertOwnership(Long restaurantId, String email) {
        if (!restaurantRepository.existsByIdAndVendor_Email(restaurantId, email)) {
            throw new UnauthorizedAccessException(
                    "Vendor " + email + " does not own restaurant " + restaurantId);
```

```
        }
    }

    public RestaurantDTO toDTO(Restaurant r) {
        RestaurantDTO dto = new RestaurantDTO();
        dto.setId(r.getId());
        dto.setName(r.getName());
        dto.setAddress(r.getAddress());
        dto.setDescription(r.getDescription());
        dto.setSupportedMealSlots(r.getSupportedMealSlots());
        dto.setActive(r.isActive());
        dto.setVendorId(r.getVendor().getId());
        dto.setVendorName(r.getVendor().getName());
        return dto;
    }
}
```

131 Current: src\main\java\com\example\EmployeeManagementSystem\Service\ServiceRequestService.java

16
lin

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.AdminActionDTO;
import com.example.EmployeeManagementSystem.DTO.RepairDTO;
import com.example.EmployeeManagementSystem.DTO.ServiceRequestDTO;
import com.example.EmployeeManagementSystem.Entity.Device;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.ServiceRequestRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;
import org.w3c.dom.stylesheets.LinkStyle;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Vector;

@Service
public class ServiceRequestService {
    private final ServiceRequestRepository serviceRequestRepository;
    private final EmployeeRepo employeeRepo;
    private final DeviceRepository deviceRepository;
    private final VendorRepo vendorRepo;

    public ServiceRequestService(ServiceRequestRepository serviceRequestRepository, EmployeeRepo employeeRepo, DeviceRepository deviceRepository, VendorRepo vendorRepo) {
        this.serviceRequestRepository = serviceRequestRepository;
        this.employeeRepo = employeeRepo;
        this.deviceRepository = deviceRepository;
        this.vendorRepo = vendorRepo;
    }

    public ServiceRequest createServiceRequest(ServiceRequestDTO dto, Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        Employee employee = employeeRepo.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("Employee not found"));

        // 2. Fetch device
        Device device = deviceRepository.findById(dto.getDeviceId())
                .orElseThrow(() -> new RuntimeException("Device not found"));

        // 3. Validate device is assigned to employee
        if (device.getCurrentAssignment() == null ||
                !device.getCurrentAssignment().getAssignedTo().equals(employee)) {
            throw new RuntimeException("Device not assigned to this employee");
        }

        // 4. Validate device status
        if (device.getDeviceStatus() != DeviceStatus.ASSIGNED) {
            throw new RuntimeException("Device is not in usable state");
        }

        // 5. Create Service Request
        ServiceRequest request = new ServiceRequest();
        request.setDevice(device);
        request.setRaisedBy(employee);
        request.setRequestType(dto.getRequestType());
        request.setIssueDescription(dto.getIssueDescription());
        request.setUrgent(dto.isUrgent());
        request.setStatus(ServiceRequestStatus.OPEN);
        return serviceRequestRepository.save(request);
    }
```

```java


    public List<ServiceRequestDTO> getAllOpenServiceRequests(){
        List<ServiceRequest> serviceRequestList=serviceRequestRepository.findByStatus(ServiceRequestStatus.OPEN);
        return serviceRequestList.stream().map(serviceRequest -> toServiceRequestDTO(serviceRequest)).toList();

    }

    private ServiceRequestDTO toServiceRequestDTO(ServiceRequest serviceRequest){
        ServiceRequestDTO requestDTO=new ServiceRequestDTO();
        requestDTO.setDeviceId(serviceRequest.getDevice().getId());
        requestDTO.setRequestType(serviceRequest.getRequestType());
        requestDTO.setIssueDescription(serviceRequest.getIssueDescription());
        requestDTO.setUrgent(serviceRequest.isUrgent());
        return requestDTO;
    }

    public ServiceRequest updateServiceRequestByAdmin(Authentication authentication, AdminActionDTO actionDTO){
        ServiceRequest serviceRequest=serviceRequestRepository.findById(actionDTO.getServiceRequestId()).orElseThrow(
                ()->new RuntimeException("Service Request Not Found")
        );
        String email=AuthUtil.extractEmail(authentication);
        Employee admin=employeeRepo.findByEmail(email).orElseThrow(
                ()->new EmployeeNotFound("Admin not found:"+email)
        );
        Device device=serviceRequest.getDevice();
        if(actionDTO.getStatus()==ServiceRequestStatus.SENT_FOR_REPAIR){
            device.setDeviceStatus(DeviceStatus.UNDER_REPAIR);
            deviceRepository.save(device);
        }
        serviceRequest.setReviewedBy(admin);
        serviceRequest.setAdminRemarks(actionDTO.getAdminRemarks());
        serviceRequest.setStatus(actionDTO.getStatus());
        return serviceRequestRepository.save(serviceRequest);
    }

    public List<ServiceRequest> getAllServiceRequestByVendor(Authentication authentication){
        String email=AuthUtil.extractEmail(authentication);
        Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
                ()->new VendorNotFoundException("Vendor not found: "+email)
        );

        return serviceRequestRepository.findByDeviceTechVendorAndStatus(vendor,ServiceRequestStatus.SENT_FOR_REPAIR);
    }

    public ServiceRequest updateServiceRequestByVendor(
            Authentication authentication, RepairDTO repairDTO
            ) {

        // 1. Get vendor
        String email = AuthUtil.extractEmail(authentication);
        Vendor vendor = vendorRepo.findByEmail(email)
                .orElseThrow(() -> new RuntimeException("Vendor not found"));

        // 2. Get request
        ServiceRequest request = serviceRequestRepository.findById(repairDTO.getRequestId())
                .orElseThrow(() -> new RuntimeException("Request not found"));

        // 3. Validate request belongs to vendor
        if (!request.getDevice().getTechVendor().getId().equals(vendor.getId())) {
            throw new RuntimeException("Unauthorized access");
        }

        // 4. Validate status
        if (request.getStatus() != ServiceRequestStatus.SENT_FOR_REPAIR) {
            throw new RuntimeException("Request is not in repair stage");
        }

        Device device = request.getDevice();

        // 5. Update based on repair result
```

```
        if (repairDTO.getStatus()==ServiceRequestStatus.REPAIR_DONE) {
            request.setResolution(repairDTO.getResolution());
            request.setStatus(repairDTO.getStatus());

            device.setDeviceStatus(DeviceStatus.ASSIGNED); // back to employee
        } else {
            request.setResolution(repairDTO.getResolution());
            request.setStatus(ServiceRequestStatus.REPAIR_DONE);

            device.setDeviceStatus(DeviceStatus.CONDEMNED); // not usable
        }

        request.setResolvedAt(LocalDateTime.now());

        deviceRepository.save(device);
        return serviceRequestRepository.save(request);
    }
}
```

132 Current: src\main\java\com\example\EmployeeManagementSystem\Service\SickLeaveResetService.java

65 lin

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveType;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

@Service
public class SickLeaveResetService {

    private static final Logger log = LoggerFactory.getLogger(SickLeaveResetService.class);

    private final LeaveTypeRepository leaveTypeRepository;
    private final LeaveEntitlementRepository entitlementRepository;

    public SickLeaveResetService(LeaveTypeRepository leaveTypeRepository,
                                 LeaveEntitlementRepository entitlementRepository) {
        this.leaveTypeRepository = leaveTypeRepository;
        this.entitlementRepository = entitlementRepository;
    }

    @Transactional
    public void runMonthlyReset(LocalDate resetDate) {
        // resetDate = last day of the month being reset (e.g. 2025-06-30)
        int year = resetDate.getYear();

        // Find all leave types that reset monthly (only SICK in your design)
        List<LeaveType> resetTypes = leaveTypeRepository.findByMonthlyResetTrue();

        for (LeaveType leaveType : resetTypes) {
            List<LeaveEntitlement> entitlements =
                    entitlementRepository.findByLeaveTypeIdAndYear(leaveType.getId(), year);

            for (LeaveEntitlement entitlement : entitlements) {

                BigDecimal available = entitlement.getAvailableBalance();

                if (available.compareTo(BigDecimal.ZERO) > 0) {
                    // Wipe unused days — move accrued back to zero effect
                    // We do this by setting accrued = used (so available = 0)
                    // This preserves the audit trail of what was used
                    entitlement.setAccruedThisYear(entitlement.getUsedThisYear());

                    log.info("Sick leave reset for employee {} — {} unused day(s) expired on {}",
                            entitlement.getEmployee().getEmployeeId(), available, resetDate);
                } else {
                    log.debug("Sick leave reset for employee {} — nothing to expire",
                            entitlement.getEmployee().getEmployeeId());
                }

                entitlementRepository.save(entitlement);
            }
        }

        log.info("Sick leave monthly reset complete for {}", resetDate);
    }
}
```

```java
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Restaurant;
import com.example.EmployeeManagementSystem.Entity.Subscription;
import com.example.EmployeeManagementSystem.Enum.MealSlot;
import com.example.EmployeeManagementSystem.Enum.ScheduleType;
import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
import com.example.EmployeeManagementSystem.Exception.SubscriptionAlreadyExists;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
import com.example.EmployeeManagementSystem.Util.AuthUtil;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Service;

import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.util.ArrayList;
import java.util.List;

@Service
public class SubscriptionService {

    private final SubscriptionRepository subscriptionRepository;
    private final EmployeeRepo employeeRepo;
    private final DeliveryTimeService deliveryTimeService;
    private final RestaurantRepository restaurantRepository;

    public SubscriptionService(SubscriptionRepository subscriptionRepository,
                               EmployeeRepo employeeRepo,
                               DeliveryTimeService deliveryTimeService,
                               RestaurantRepository restaurantRepository) {
        this.subscriptionRepository = subscriptionRepository;
        this.employeeRepo = employeeRepo;
        this.deliveryTimeService = deliveryTimeService;
        this.restaurantRepository = restaurantRepository;
    }

    /**
     * Add subscriptions for an employee.
     *
     * Each SlotOrder specifies a meal slot AND the restaurant for that slot,
     * so an employee can have breakfast from Restaurant A and lunch from Restaurant B.
     *
     * Validations:
     * - Employee must exist and be ACTIVE
     * - Restaurant must exist and be active
     * - Restaurant must support the requested meal slot
     * - No duplicate active subscription for the same employee + slot combination
     */
    public void addSubscription(SubscriptionRequest request,Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        Employee employee = employeeRepo.findByEmail(email)
                .orElseThrow(() -> new EmployeeNotFound(
                        "Employee with id " + email + " not found"));

        if (request.getSlotOrders() == null || request.getSlotOrders().isEmpty()) {
            throw new IllegalArgumentException("slotOrders cannot be null or empty");
        }

        if (request.getScheduleType() == ScheduleType.DAILY) {
            request.setDayOfWeek(null);
        } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
            if (request.getDayOfWeek() == null) {
                throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
            }
```

```
        }

        for (SubscriptionRequest.SlotOrder slotOrder : request.getSlotOrders()) {
            MealSlot slot = slotOrder.getMealSlot();
            Long restaurantId = slotOrder.getRestaurantId();

            // 1. Validate restaurant exists and is active
            Restaurant restaurant = restaurantRepository.findById(restaurantId)
                    .orElseThrow(() -> new RestaurantNotFoundException(
                            "Restaurant with id " + restaurantId + " not found"));
            if (!restaurant.isActive()) {
                throw new IllegalArgumentException(
                        "Restaurant '" + restaurant.getName() + "' is not currently active");
            }

            // 2. Validate the restaurant actually serves this meal slot
            if (!restaurant.getSupportedMealSlots().contains(slot)) {
                throw new IllegalArgumentException(
                        "Restaurant '" + restaurant.getName() + "' does not serve " + slot +
                                ". Supported slots: " + restaurant.getSupportedMealSlots());
            }

            // 3. Check for duplicate active subscription (same employee + slot)
            if (subscriptionRepository.checkSubscriptionExists(
                    employee.getEmployeeId(), slot.name(), SubscriptionStatus.ACTIVE.name()) > 0) {
                throw new SubscriptionAlreadyExists(
                        "Employee " + employee.getEmployeeId() +
                                " already has an active " + slot + " subscription. " +
                                 "Cancel the existing one before subscribing to a new restaurant.");
            }

            // 4. Create subscription
            Subscription subscription = new Subscription();
            subscription.setEmployee(employee);
            subscription.setSlot(slot);
            subscription.setRestaurant(restaurant);
            subscription.setScheduleType(request.getScheduleType());
            subscription.setDayOfWeek(request.getDayOfWeek());
            subscription.setStatus(SubscriptionStatus.ACTIVE);
            subscription.setNextDeliveryTime(
                    deliveryTimeService.calculateNextDelivery(request, slot, employee));

            subscriptionRepository.save(subscription);
        }
    }

    public List<SubscriptionDTO> getAllSubscriptions() {
        List<Subscription> subscriptions = subscriptionRepository.findAll();
        List<SubscriptionDTO> dtoList = new ArrayList<>();
        for (Subscription s : subscriptions) {
            dtoList.add(convertToDTO(s));
        }
        return dtoList;
    }

    public Subscription updateSubscription(Long id, SubscriptionRequest request) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        Employee employee = subscription.getEmployee();

        if (request.getScheduleType() == ScheduleType.DAILY) {
            subscription.setDayOfWeek(null);
        } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
            if (request.getDayOfWeek() == null) {
                throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
            }
            subscription.setDayOfWeek(request.getDayOfWeek());
        }
        subscription.setScheduleType(request.getScheduleType());

        // Update slot & restaurant if a single slotOrder is provided
        if (request.getSlotOrders() != null && !request.getSlotOrders().isEmpty()) {
            if (request.getSlotOrders().size() > 1) {
                throw new IllegalArgumentException(
                        "Update supports one slot at a time. Use separate subscriptions for multiple
slots.");
```

```
            }
            SubscriptionRequest.SlotOrder slotOrder = request.getSlotOrders().get(0);
            MealSlot newSlot = slotOrder.getMealSlot();
            Restaurant restaurant = restaurantRepository.findById(slotOrder.getRestaurantId())
                    .orElseThrow(() -> new RestaurantNotFoundException(
                            "Restaurant " + slotOrder.getRestaurantId() + " not found"));
            if (!restaurant.isActive()) {
                throw new IllegalArgumentException("Restaurant '" + restaurant.getName() + "' is not active");
            }
            if (!restaurant.getSupportedMealSlots().contains(newSlot)) {
                throw new IllegalArgumentException(
                        "Restaurant '" + restaurant.getName() + "' does not serve " + newSlot);
            }
            subscription.setSlot(newSlot);
            subscription.setRestaurant(restaurant);
        }

        subscription.setNextDeliveryTime(
                deliveryTimeService.calculateNextDelivery(request, subscription.getSlot(), employee));

        return subscriptionRepository.save(subscription);
    }

    public Subscription pauseSubscription(Long id) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        subscription.setStatus(SubscriptionStatus.PAUSED);
        subscription.setNextDeliveryTime(null);
        return subscriptionRepository.save(subscription);
    }

    public Subscription resumeSubscription(Long id) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        subscription.setStatus(SubscriptionStatus.ACTIVE);
        subscription.setNextDeliveryTime(deliveryTimeService.getNextDeliveryTime(subscription));
        return subscriptionRepository.save(subscription);
    }

    public Subscription expireSubscription(Long id) {
        Subscription subscription = subscriptionRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Subscription not found"));
        subscription.setStatus(SubscriptionStatus.EXPIRED);
        subscription.setNextDeliveryTime(null);
        return subscriptionRepository.save(subscription);
    }

    public List<SubscriptionDTO> getSubscriptionOfUser(Authentication authentication) {
        String email= AuthUtil.extractEmail(authentication);
        List<Subscription> subscriptions = subscriptionRepository.findByEmployee_Email(email);
        List<SubscriptionDTO> dtoList = new ArrayList<>();
        for (Subscription s : subscriptions) {
            dtoList.add(convertToDTO(s));
        }
        return dtoList;
    }

    //  helpers

    private SubscriptionDTO convertToDTO(Subscription s) {
        SubscriptionDTO dto = new SubscriptionDTO();
        dto.setSubscriptionId(s.getId());
        dto.setUserId(s.getEmployee().getEmployeeId());
        dto.setMealSlot(s.getSlot());
        dto.setScheduleType(s.getScheduleType());
        if (s.getScheduleType() == ScheduleType.WEEKLY) {
            dto.setDayOfWeek(s.getDayOfWeek());
        }
        dto.setStatus(s.getStatus());
        if (s.getNextDeliveryTime() != null) {
            ZoneId userZone = ZoneId.of(s.getEmployee().getTimezone());
            dto.setNextDeliveryTime(s.getNextDeliveryTime().atZone(userZone));
        }
        // Restaurant info
        if (s.getRestaurant() != null) {
```

```
            dto.setRestaurantId(s.getRestaurant().getId());
            dto.setRestaurantName(s.getRestaurant().getName());
            dto.setRestaurantAddress(s.getRestaurant().getAddress());
        }
        return dto;
    }
}
```

134 Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java

40 lines

```java
package com.example.EmployeeManagementSystem.Service;

import dev.samstevens.totp.code.*;
import dev.samstevens.totp.qr.*;
import dev.samstevens.totp.secret.DefaultSecretGenerator;
import dev.samstevens.totp.secret.SecretGenerator;
import dev.samstevens.totp.time.SystemTimeProvider;
import dev.samstevens.totp.time.TimeProvider;
import org.springframework.stereotype.Service;

@Service
public class TotpService {

    private final SecretGenerator secretGenerator = new DefaultSecretGenerator();
    private final TimeProvider timeProvider = new SystemTimeProvider();
    private final CodeGenerator codeGenerator = new DefaultCodeGenerator();
    private final CodeVerifier codeVerifier = new DefaultCodeVerifier(codeGenerator, timeProvider);

    /** Generate a new Base32 secret for a user */
    public String generateSecret() {
        return secretGenerator.generate();
    }

    /** Generate a QR code URI — user scans this with Google Authenticator */
    public String getQrCodeUri(String secret, String username) {
        return String.format(
                "otpauth://totp/EmployeeMS:%s?secret=%s&issuer=EmployeeManagementSystem",
                username, secret
        );
    }

    /** Verify the 6-digit code the user typed */
    public boolean verifyCode(String secret, String code) {
        try {
            return codeVerifier.isValidCode(secret, code);
        } catch (Exception e) {
            return false;
        }
    }
}
```

135 Current: src\main\java\com\example\EmployeeManagementSystem\Service\YearEndCarryForwardService.jav

```
package com.example.EmployeeManagementSystem.Service;

import com.example.EmployeeManagementSystem.Entity.*;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

@Service
public class YearEndCarryForwardService {

    private static final Logger log = LoggerFactory.getLogger(YearEndCarryForwardService.class);
    private static final BigDecimal CAP = BigDecimal.valueOf(30);

    private final EmployeeRepo employeeRepository;
    private final LeaveEntitlementRepository entitlementRepository;
    private final YearEndCarryForwardLogRepository logRepository;
    private final LeaveWarningRepository warningRepository;

    public YearEndCarryForwardService(
            EmployeeRepo employeeRepository,
            LeaveEntitlementRepository entitlementRepository,
            YearEndCarryForwardLogRepository logRepository,
            LeaveWarningRepository warningRepository) {
        this.employeeRepository = employeeRepository;
        this.entitlementRepository = entitlementRepository;
        this.logRepository = logRepository;
        this.warningRepository = warningRepository;
    }

    // ----------------------------------------------------------------
    // Main entry point — called by scheduler on Dec 31 at 22:00
    // ----------------------------------------------------------------
    @Transactional
    public void runYearEnd(int closingYear) {

        // Idempotency guard — never run twice for same year
        if (logRepository.existsByProcessedYear(closingYear)) {
            log.warn("Year-end carry-forward already processed for {}. Skipping.", closingYear);
            return;
        }

        List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
        log.info("Starting year-end carry-forward for {} — {} employees", closingYear,
activeEmployees.size());

        for (Employee employee : activeEmployees) {
            processEmployeeYearEnd(employee, closingYear);
        }

        log.info("Year-end carry-forward complete for {}", closingYear);
    }

    // ----------------------------------------------------------------
    // Per-employee processing
    // ----------------------------------------------------------------
    private void processEmployeeYearEnd(Employee employee, int closingYear) {
        int nextYear = closingYear + 1;

```

```
        // Fetch all entitlements for the closing year
        List<LeaveEntitlement> entitlements =
                entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(),
closingYear);

        // Separate carry-forwardable from non-carry-forwardable
        List<LeaveEntitlement> carryForwardable = entitlements.stream()
                .filter(e -> e.getLeaveType().isCarriesForward())
                .collect(Collectors.toList());

        // Step 1 — compute raw closing balance for each type
        //           closing = opening + accrued - used
        Map<LeaveEntitlement, BigDecimal> rawClosing = carryForwardable.stream()
                .collect(Collectors.toMap(
                        e -> e,
                        e -> e.getOpeningBalance()
                                .add(e.getAccruedThisYear())
                                .subtract(e.getUsedThisYear())
                                .max(BigDecimal.ZERO)   // never negative
                ));

        // Step 2 — sum all carry-forwardable closing balances
        BigDecimal totalRaw = rawClosing.values().stream()
                .reduce(BigDecimal.ZERO, BigDecimal::add);

        // Step 3 — apply cap proportionally if total exceeds 30
        Map<LeaveEntitlement, BigDecimal> cappedClosing = applyCap(rawClosing, totalRaw);

        // Step 4 — persist: update closing balance on current year row,
        //           create/update opening balance on next year row, write log
         for (LeaveEntitlement entitlement : carryForwardable) {
             BigDecimal raw    = rawClosing.get(entitlement);
             BigDecimal capped = cappedClosing.get(entitlement);
             BigDecimal wasted = raw.subtract(capped);

             // Update closing balance on the current year's row
             entitlement.setClosingBalance(capped);
             entitlementRepository.save(entitlement);

             // Write or update next year's opening balance
             LeaveEntitlement nextYearRow = entitlementRepository
                     .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
                             employee.getEmployeeId(), entitlement.getLeaveType().getId(), nextYear)
                     .orElseGet(() -> buildNextYearRow(employee, entitlement.getLeaveType(), nextYear));

             nextYearRow.setOpeningBalance(capped);
             entitlementRepository.save(nextYearRow);

             // Write audit log
             writeLog(employee, closingYear, entitlement.getLeaveType(), raw, capped, wasted);

             // Issue DAY_WASTED warning if any days were lost
             if (wasted.compareTo(BigDecimal.ZERO) > 0) {
                 issueWastedWarning(employee, wasted, capped);
                 log.info("Employee {} lost {} day(s) of {} to cap at year end",
                         employee.getEmployeeId(), wasted, entitlement.getLeaveType().getName());
             }
         }

         // Step 5 — zero out sick leave closing balance (it never carries)
         entitlements.stream()
                 .filter(e -> !e.getLeaveType().isCarriesForward())
                 .forEach(e -> {
                     e.setClosingBalance(BigDecimal.ZERO);
                     entitlementRepository.save(e);
                 });

         log.info("Year-end processed for employee {} — total raw={} capped={}",
                 employee.getEmployeeId(), totalRaw, CAP.min(totalRaw));
     }

     // ----------------------------------------------------------------
     // Cap logic — distribute cap proportionally across leave types
     //
     // Example: casual=20, maternity=16  total=36, over cap by 6
     //   casual  gets: 20/36 * 30 = 16.67
```

```java
    //   maternity gets: 16/36 * 30 = 13.33
    //   total = 30 exactly
    // ----------------------------------------------------------------
    private Map<LeaveEntitlement, BigDecimal> applyCap(
            Map<LeaveEntitlement, BigDecimal> rawClosing,
            BigDecimal totalRaw) {

        if (totalRaw.compareTo(CAP) <= 0) {
            // Under cap — no adjustment needed
            return rawClosing;
        }

        return rawClosing.entrySet().stream()
                .collect(Collectors.toMap(
                        Map.Entry::getKey,
                        entry -> {
                            BigDecimal raw = entry.getValue();
                            // proportion = raw / totalRaw * CAP
                            return raw
                                    .divide(totalRaw, 10, RoundingMode.HALF_UP)
                                    .multiply(CAP)
                                    .setScale(2, RoundingMode.HALF_UP);
                        }
                ));
    }

    // ----------------------------------------------------------------
    // Build a blank next-year entitlement row
    // ----------------------------------------------------------------
    private LeaveEntitlement buildNextYearRow(Employee employee, LeaveType leaveType, int year) {
        LeaveEntitlement row = new LeaveEntitlement();
        row.setEmployee(employee);
        row.setLeaveType(leaveType);
        row.setYear(year);
        row.setOpeningBalance(BigDecimal.ZERO);
        row.setAccruedThisYear(BigDecimal.ZERO);
        row.setUsedThisYear(BigDecimal.ZERO);
        row.setClosingBalance(BigDecimal.ZERO);
        return entitlementRepository.save(row);
    }

    // ----------------------------------------------------------------
    // Write audit log entry
    // ----------------------------------------------------------------
    private void writeLog(Employee employee, int year, LeaveType leaveType,
                          BigDecimal raw, BigDecimal capped, BigDecimal wasted) {
        YearEndCarryForwardLog entry = new YearEndCarryForwardLog();
        entry.setEmployee(employee);
        entry.setProcessedYear(year);
        entry.setLeaveType(leaveType);
        entry.setClosingBalance(raw);
        entry.setCappedBalance(capped);
        entry.setDaysWasted(wasted);
        entry.setOpeningNextYear(capped);
        logRepository.save(entry);
    }

    // ----------------------------------------------------------------
    // Issue a DAY_WASTED warning on the dashboard
    // ----------------------------------------------------------------
    private void issueWastedWarning(Employee employee, BigDecimal wasted, BigDecimal newBalance) {
        LeaveWarning warning = new LeaveWarning();
        warning.setEmployee(employee);
        warning.setWarningType(LeaveWarning.WarningType.DAY_WASTED);
        warning.setCarryForwardBalance(newBalance);
        warning.setWarningDate(LocalDate.now());
        warning.setMessage(
                wasted.toPlainString() + " carry-forward day(s) were lost at year-end " +
                        "because your balance exceeded the 30-day cap. " +
                        "Your opening balance for next year is " + newBalance.toPlainString() + " days."
        );
        warningRepository.save(warning);
    }
}
```

136 Current: src\main\java\com\example\EmployeeManagementSystem\Util\AuthUtil.java

21 lines

```java
package com.example.EmployeeManagementSystem.Util;

import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.stereotype.Component;

@Component
public class AuthUtil {
    public static String extractEmail(Authentication authentication) {
        if (authentication == null) return null;

        // OAuth2 login (session-based) — principal is OAuth2User
        if (authentication.getPrincipal() instanceof OAuth2User oAuth2User) {
            return oAuth2User.getAttribute("email"); // always the actual email
        }

        // JWT or session username/password login — getName() is already the email
        return authentication.getName();
    }
}

```

137 Current: src\main\java\com\example\EmployeeManagementSystem\Util\JWTUtil.java

76 lines

```java
package com.example.EmployeeManagementSystem.Util;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.Vendor;
import com.example.EmployeeManagementSystem.Enum.Role;
import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.Optional;

@Component
public class JWTUtil {

    private final EmployeeRepo employeeRepo;
    private final VendorRepo vendorRepo;

    // FIX 1: Was 1000*60*60 (1 hour) — caused token expiry mid-session.
    // Extended to 24 hours to match the session timeout in application.yaml.
    private final long Expiration_time = 1000L * 60 * 60 * 24;

    @Value("${jwt.secret}")
    private String secret;
    private SecretKey key;

    public JWTUtil(EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
        this.employeeRepo = employeeRepo;
        this.vendorRepo = vendorRepo;
    }

    @PostConstruct
    public void init() {
        key = Keys.hmacShaKeyFor(secret.getBytes());
    }

    public String generateToken(String username,String role) {
        return Jwts.builder()
                .subject(username)
                .claim("role",role)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + Expiration_time))
                .signWith(key)
                .compact();
    }

    public String extractUsernameFromToken(String token) {
        Claims body = extractClaims(token);
        return body.getSubject();
    }

    private Claims extractClaims(String token) {
        return Jwts.parser()
                .verifyWith(key)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    public boolean validateToken(String username, UserDetails userDetails, String token) {
        return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
    }
```

```

    private boolean isTokenExpired(String token) {
        return extractClaims(token).getExpiration().before(new Date());
    }
}
```

138 Current: src\main\java\com\example\EmployeeManagementSystem\Util\OAuth2SuccessHandler.java

85 lines

```java
package com.example.EmployeeManagementSystem.Util;

import com.example.EmployeeManagementSystem.Entity.RefreshToken;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
import com.example.EmployeeManagementSystem.Repository.VendorRepo;
import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.core.Authentication;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.security.web.authentication.SimpleUrlAuthenticationSuccessHandler;
import org.springframework.stereotype.Service;

import java.io.IOException;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;

@Service
public class OAuth2SuccessHandler extends SimpleUrlAuthenticationSuccessHandler{
    private final JWTUtil jwtUtil;
    private final EmployeeRepo employeeRepo;
    private final VendorRepo vendorRepo;
    private final RefreshTokenService refreshTokenService;
    private final RefreshTokenRepository refreshTokenRepository;

    public OAuth2SuccessHandler(JWTUtil jwtUtil,
                                EmployeeRepo employeeRepo,
                                VendorRepo vendorRepo,
                                RefreshTokenService refreshTokenService,
                                RefreshTokenRepository refreshTokenRepository) {
        this.jwtUtil = jwtUtil;
        this.employeeRepo = employeeRepo;
        this.vendorRepo = vendorRepo;
        this.refreshTokenService = refreshTokenService;
        this.refreshTokenRepository = refreshTokenRepository;
    }

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
                                        HttpServletResponse response,
                                        Authentication authentication) throws IOException {

        try{
            OAuth2User oAuth2User = (OAuth2User) authentication.getPrincipal();
            String email = oAuth2User.getAttribute("email");


            // Generate refresh token — check which entity type owns this email
            RefreshToken refreshToken;
            String role;
            var employeeOpt = employeeRepo.findByEmail(email);
            var vendorOpt   = vendorRepo.findByEmail(email);

            if (employeeOpt.isPresent()) {
                role = employeeOpt.get().getRole().name();
                refreshToken = refreshTokenService.createForEmployee(employeeOpt.get());
            } else if (vendorOpt.isPresent()) {
                role = vendorOpt.get().getRole().name();
                refreshToken = refreshTokenService.createForVendor(vendorOpt.get());
            } else {
                response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "User not found post-OAuth");
                return;
            }
            String accessToken = jwtUtil.generateToken(email,role);
            refreshTokenRepository.save(refreshToken);

            // Redirect frontend to a callback page with tokens as query params
            // (or use a fragment # so tokens don't hit server logs)
            String redirectUrl = "http://localhost:8080/google.html"   // your frontend page
```

```
                    + "?accessToken="   + URLEncoder.encode(accessToken,            StandardCharsets.UTF_8)
                    + "&refreshToken="  + URLEncoder.encode(refreshToken.getToken(),
StandardCharsets.UTF_8);

            getRedirectStrategy().sendRedirect(request, response, redirectUrl);
        }
        catch (Exception e) {
            // This will now print the REAL error in your console
            System.err.println("=== OAuth2SuccessHandler FAILED ===");
            e.printStackTrace();
            response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
                    "OAuth2 login failed: " + e.getMessage());
        }
    }
}

```

139 Current: src\main\resources\application.yaml

69 lines

```yaml
spring:
  application:
    name: EmployeeManagementSystem

  datasource :
    url : jdbc:mysql://localhost:3306/employee_management_db
    username : ${USER_NAME}
    password : ${PASSWORD}

  jpa:
    hibernate :
      ddl-auto: update
    show-sql: true

  springdoc:
    api-docs:
      path: /v3/api-docs
      enabled: true
    swagger-ui:
      path: /swagger-ui.html
      enabled: true
      operations-sorter: method
      tags-sorter: alpha
      try-it-out-enabled: true
      filter: true
    show-actuator: true
  mvc:
    static-path-pattern: /**
    session:
      tracking-modes: COOKIE
  web:
    resources:
      static-locations: classpath:/static/
      add-mappings: true
  session:
    timeout: 86400s  # 24 hours in seconds (was 1440m which is incorrect format)
    store-type: none  # Use default cookie-based session

  servlet:
    session:
      cookie:
        http-only: true
        same-site: strict
        secure: false     # set true in production with HTTPS

  security:
    debug: true
    oauth2:
      client:
        registration:
          google:
            client-id: ${client_id}
            client-secret: ${client_secret}
            scope:
              - email
              - profile

logging:
  level:
    org.springframework.security: DEBUG
    com.example.EmployeeManagementSystem: DEBUG
jwt:
  secret: ${JWT_SECRET}
# Add this to your application.yaml
api:
  key:
    enabled: true
    header-name: X-API-Key

```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS - Admin Console</title>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        :root {
            --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
            --panel: #1e2436; --panel2: #252c40;
            --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
            --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
            --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
            --green: #36d399; --green-dim: rgba(54,211,153,0.12);
            --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
            --red: #f87171; --red-dim: rgba(248,113,113,0.12);
            --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
            --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
        }
        body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-height: 1.5; overflow: hidden; }
        .app { display: flex; height: 100vh; }
        .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display: flex; flex-direction: column; overflow-y: auto; }
        .logo { padding: 20px; border-bottom: 1px solid var(--border); }
        .logo h2 { font-size: 20px; color: var(--text); }
        .logo span { color: var(--accent); }
        .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
        .nav-item { padding: 12px 20px; margin: 4px 12px; border-radius: 8px; cursor: pointer; transition: all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; }
        .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
        .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
        .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between; align-items: center; border-bottom: 1px solid var(--border); }
        .page-title { font-size: 20px; font-weight: 600; }
        .user-info { display: flex; align-items: center; gap: 16px; }
        .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; }
        .content { flex: 1; overflow-y: auto; padding: 24px; }
        .view { display: none; }
        .view.active { display: block; }
        .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px; }
        .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--border); }
        .stat-label { font-size: 11px; color: var(--text3); text-transform: uppercase; margin-bottom: 8px; }
        .stat-value { font-size: 28px; font-weight: 600; }
        .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border); padding: 24px; max-width: 600px; }
        .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
        .form-group { display: flex; flex-direction: column; gap: 6px; }
        .form-group.full { grid-column: 1 / -1; }
        label { font-size: 11px; font-weight: 600; color: var(--text3); text-transform: uppercase; }
        input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; }
        input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
        .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border); overflow-x: auto; }
        table { width: 100%; border-collapse: collapse; }
        th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text3); font-size: 11px; font-weight: 600; text-transform: uppercase; }
        td { padding: 12px 16px; border-top: 1px solid var(--border); }
        .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-block; }
        .chip.active, .chip.approved, .chip.delivered { background: var(--green-dim); color: var(--green); }
        .chip.pending, .chip.scheduled { background: var(--amber-dim); color: var(--amber); }
        .chip.inactive, .chip.rejected, .chip.cancelled, .chip.expired { background: var(--red-dim); color: var(--red); }
```

```
        .chip.on_leave { background: var(--violet-dim); color: var(--violet); }
        .chip.employee { background: var(--accent-dim); color: var(--accent); }
        .chip.manager { background: var(--violet-dim); color: var(--violet); }
        .chip.vendor { background: var(--red-dim); color: var(--red); }
        .chip.admin { background: var(--amber-dim); color: var(--amber); }
        .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size: 13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
        .btn-primary { background: var(--accent); color: white; }
        .btn-primary:hover { background: #6ba0ff; }
        .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
        .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid rgba(248,113,113,0.3); }
        .btn-sm { padding: 4px 10px; font-size: 11px; margin: 2px; }
        .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom: 16px; }
        .section-title { font-size: 16px; font-weight: 600; }
        .empty-state { text-align: center; padding: 40px; color: var(--text3); }
        .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background: rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
        .modal-overlay.open { display: flex; }
        .modal { background: var(--panel); border-radius: 12px; padding: 24px; width: 450px; max-width: 90%; }
        .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
        .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
        .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index: 1100; animation: slideIn 0.3s ease; }
        .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid var(--green); }
        .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid var(--red); }
        @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform: translateX(0); opacity: 1; } }
        .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
    </style>
</head>
<body>
<div class="app">
    <!-- Sidebar -->
    <div class="sidebar">
        <div class="logo">
            <h2>E<span>M</span>S</h2>
            <p>Admin Control Center</p>
        </div>
        <div style="flex: 1;">
            <div class="nav-item active" onclick="showView('dashboard', event)">&#9632; Dashboard</div>
            <div class="nav-item" onclick="showView('employees', event)">&#9632; Employees</div>
            <div class="nav-item" onclick="showView('add-employee', event)">&#9632; Add Employee</div>
            <div class="nav-item" onclick="showView('add-manager', event)">&#9632; Add Manager</div>
            <div class="nav-item" onclick="showView('add-vendor', event)">&#9632; Add Vendor</div>
            <div class="nav-item" onclick="showView('apply-leave', event)">&#9632; Apply Leave</div>
            <div class="nav-item" onclick="showView('my-leaves', event)">&#9632; My Leaves</div>
            <div class="nav-item" onclick="showView('pending-leaves', event)">&#9632; Pending Approvals</div>
            <div class="nav-item" onclick="showView('restaurants', event)">&#9632; Restaurants</div>
            <div class="nav-item" onclick="showView('subscriptions', event)">&#9632; Subscriptions</div>
            <div class="nav-item" onclick="showView('add-subscription', event)">&#9632; Add Subscription</div>
            <div class="nav-item" onclick="showView('vendors', event)">&#9632; Vendors</div>
        </div>
    </div>

    <!-- Main Content -->
    <div class="main-content">
        <div class="topbar">
            <div class="page-title" id="pageTitle">Dashboard</div>
            <div class="user-info">
                <span id="username-display"></span>
                <button class="logout-btn" onclick="logout()">Logout</button>
            </div>
        </div>
        <div class="content" id="content">

            <!-- DASHBOARD VIEW -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card"><div class="stat-label">Total Employees</div><div class="stat-value" id="total-employees">--</div></div>
```

```
                    <div class="stat-card"><div class="stat-label">Active</div><div class="stat-value" id="active-employees">--</div></div>
                    <div class="stat-card"><div class="stat-label">On Leave</div><div class="stat-value" id="onleave-employees">--</div></div>
                    <div class="stat-card"><div class="stat-label">Pending Leaves</div><div class="stat-value" id="pending-leaves">--</div></div>
                </div>
                <div class="section-head"><div class="section-title">Recent Leave Requests</div><button class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Status</th></tr></thead><tbody id="recent-leaves"></tbody></table></div>
            </div>

            <!-- EMPLOYEES VIEW -->
            <div id="employees-view" class="view">
                <div class="section-head"><div class="section-title">All Employees</div><button class="btn btn-primary btn-sm" onclick="loadEmployees()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Timezone</th><th>Actions</th></tr></thead><tbody i...
            </div>

            <div id="add-employee-view" class="view">
                <div class="section-title" style="margin-bottom: 20px;">Add Employee</div>
                <div class="form-card">
                    <div class="form-grid">
                        <div class="form-group"><label>Name</label><input type="text" id="employee-name" placeholder="Employee name"></div>
                        <div class="form-group"><label>Email</label><input type="email" id="employee-email" placeholder="employee@company.com"></div>
                        <div class="form-group"><label>Password</label><input type="password" id="employee-password" placeholder="Temporary password"></div>
                        <div class="form-group"><label>Department</label><input type="text" id="employee-dept" placeholder="Department"></div>
                        <div class="form-group full"><label>Timezone</label><input type="text" id="employee-timezone" placeholder="Asia/Kolkata"></div>
                    </div>
                    <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary" onclick="createEmployee()">Create Employee</button><button class="btn btn-secondary" onclick...
                </div>
            </div>

            <div id="add-manager-view" class="view">
                <div class="section-title" style="margin-bottom: 20px;">Add Manager</div>
                <div class="form-card">
                    <div class="form-grid">
                        <div class="form-group"><label>Name</label><input type="text" id="manager-name-create" placeholder="Manager name"></div>
                        <div class="form-group"><label>Email</label><input type="email" id="manager-email-create" placeholder="manager@company.com"></div>
                        <div class="form-group"><label>Password</label><input type="password" id="manager-password-create" placeholder="Temporary password"></div>
                        <div class="form-group"><label>Department</label><input type="text" id="manager-dept-create" placeholder="Department"></div>
                        <div class="form-group full"><label>Timezone</label><input type="text" id="manager-timezone-create" placeholder="Asia/Kolkata"></div>
                    </div>
                    <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary" onclick="createManager()">Create Manager</button><button class="btn btn-secondary" onclick="...
                </div>
            </div>

            <div id="add-vendor-view" class="view">
                <div class="section-title" style="margin-bottom: 20px;">Add Vendor</div>
                <div class="form-card">
                    <div class="form-grid">
                        <div class="form-group"><label>Name</label><input type="text" id="vendor-name-create" placeholder="Vendor name"></div>
                        <div class="form-group"><label>Email</label><input type="email" id="vendor-email-create" placeholder="vendor@company.com"></div>
                        <div class="form-group"><label>Password</label><input type="password" id="vendor-password-create" placeholder="Temporary password"></div>
                        <div class="form-group"><label>Phone</label><input type="text" id="vendor-phone-create" placeholder="+91 90000 00000"></div>
                        <div class="form-group">
```

```
                                <label>Role</label>
                                <select id="vendor-role">
                                    <option value="TECH_VENDOR">tech vendor</option>
                                    <option value="FOOD_VENDOR">Food Vendor</option>
                                </select>
                            </div>
                        </div>
                        <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary" onclick="createVendor()">Create Vendor</button><button class="btn btn-secondary" onclick="cl...
                    </div>
                </div>

                <!-- APPLY LEAVE VIEW -->
                <div id="apply-leave-view" class="view">
                    <div class="section-title" style="margin-bottom: 20px;">Apply for Leave</div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group full"><label>Logged In User</label><input type="email" id="leave-email" placeholder="your.email@company.com" readonly></div>
                            <div class="form-group"><label>Leave Type</label><select id="leave-type"><option value="SICK">Sick Leave</option><option value="CASUAL">Casual Leave</option><option value="E...
                            <div class="form-group"><label>Start Date</label><input type="date" id="leave-start"></div>
                            <div class="form-group"><label>End Date</label><input type="date" id="leave-end"></div>
                            <div class="form-group full"><label>Reason</label><textarea id="leave-reason" rows="3" placeholder="Please provide reason for leave..."></textarea></div>
                        </div>
                        <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary" onclick="applyLeave()">Submit Leave Request</button><button class="btn btn-secondary" onclic...
                    </div>
                </div>

                <!-- MY LEAVES VIEW -->
                <div id="my-leaves-view" class="view">
                    <div class="section-head"><div class="section-title">My Leave Requests</div><button class="btn btn-primary btn-sm" onclick="loadMyLeaves()">Refresh</button></div>
                    <div class="table-wrap"><table><thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End Date</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead><tbody id="my-leaves-...
                </div>

                <!-- PENDING LEAVES VIEW -->
                <div id="pending-leaves-view" class="view">
                    <div class="section-head"><div class="section-title">Pending Leave Approvals</div><button class="btn btn-primary btn-sm" onclick="loadPendingLeaves()">Refresh</button></div>
                    <div class="table-wrap"><table><thead><tr><th>ID</th><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Reason</th><th>Actions</th></tr></thead><tbody id="pending-leaves-li...
                </div>

                <!-- RESTAURANTS VIEW -->
                <div id="restaurants-view" class="view">
                    <div class="section-head"><div class="section-title">Restaurants</div><div><select id="slot-filter" onchange="loadRestaurants()"><option value="">All Slots</option><option value="BR...
                    <div class="table-wrap"><table><thead><tr><th>Name</th><th>Address</th><th>Meal Slots</th><th>Status</th><th>Vendor</th></tr></thead><tbody id="restaurants-list"></tbody></table></div>
                </div>

                <!-- MY SUBSCRIPTIONS VIEW -->
                <div id="subscriptions-view" class="view">
                    <div class="section-head"><div class="section-title">My Meal Subscriptions</div><button class="btn btn-primary btn-sm" onclick="loadSubscriptions()">Refresh</button></div>
                    <div class="table-wrap"><table><thead><tr><th>ID</th><th>Restaurant</th><th>Meal Slot</th><th>Schedule</th><th>Status</th><th>Next Delivery</th><th>Actions</th></tr></thead><tbody i...
                </div>

                <!-- ADD SUBSCRIPTION VIEW -->
                <div id="add-subscription-view" class="view">
                    <div class="section-title" style="margin-bottom: 20px;">Create Meal Subscription</div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group full">
                                <label>Employee Email</label>
                                <input type="email" id="sub-email" placeholder="employee@company.com">
                            </div>
                            <div class="form-group full">
```

```
                            <label>Restaurant</label>
                            <select id="sub-restaurant-select">
                                <option value="">-- Select Restaurant --</option>
                            </select>
                        </div>
                        <div class="form-group full">
                            <label>Meal Slots</label>
                            <div id="meal-slot-checkboxes" style="display:flex; gap:16px; flex-wrap:wrap; margin-top:6px;">
                                <label style="display:flex;align-items:center;gap:6px;text-transform:none;font-size:13px;font-weight:400;color:var(--text);">
                                    <input type="checkbox" value="BREAKFAST" style="width:15px;height:15px;padding:0;background:var(--bg3);"> Breakfast
                                </label>
                                <label style="display:flex;align-items:center;gap:6px;text-transform:none;font-size:13px;font-weight:400;color:var(--text);">
                                    <input type="checkbox" value="LUNCH" style="width:15px;height:15px;padding:0;background:var(--bg3);"> Lunch
                                </label>
                                <label style="display:flex;align-items:center;gap:6px;text-transform:none;font-size:13px;font-weight:400;color:var(--text);">
                                    <input type="checkbox" value="DINNER" style="width:15px;height:15px;padding:0;background:var(--bg3);"> Dinner
                                </label>
                            </div>
                        </div>
                        <div class="form-group">
                            <label>Schedule Type</label>
                            <select id="sub-schedule">
                                <option value="DAILY">Daily</option>
                                <option value="WEEKLY">Weekly</option>
                            </select>
                        </div>
                        <div class="form-group" id="day-of-week-group" style="display:none;">
                            <label>Day of Week</label>
                            <select id="sub-day">
                                <option value="MONDAY">Monday</option>
                                <option value="TUESDAY">Tuesday</option>
                                <option value="WEDNESDAY">Wednesday</option>
                                <option value="THURSDAY">Thursday</option>
                                <option value="FRIDAY">Friday</option>
                                <option value="SATURDAY">Saturday</option>
                                <option value="SUNDAY">Sunday</option>
                            </select>
                        </div>
                    </div>
                    <div class="modal-actions" style="margin-top: 20px;">
                        <button class="btn btn-primary" onclick="addSubscription()">Create Subscription</button>
                        <button class="btn btn-secondary" onclick="clearSubscriptionForm()">Clear</button>
                    </div>
                </div>
            </div>

            <!-- DELIVERIES VIEW -->
            <div id="deliveries-view" class="view">
                <div class="section-head"><div class="section-title">My Deliveries</div><button class="btn btn-primary btn-sm" onclick="loadDeliveries()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Subscription ID</th><th>Meal Slot</th><th>Scheduled Time</th><th>Status</th></tr></thead><tbody id="deliveries-list"></tbody...
            </div>

            <!-- VENDORS VIEW -->
            <div id="vendors-view" class="view">
                <div class="section-head"><div class="section-title">Vendors</div><button class="btn btn-primary btn-sm" onclick="loadVendors()">Refresh</button></div>
                <div class="table-wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Registered</th><th>Actions</th></tr></thead><tbody id="vendors-list"></tbody></tabl...
            </div>

        </div>
    </div>
</div>

<!-- Approve/Reject Modal -->
```

```
<div id="action-modal" class="modal-overlay">
    <div class="modal">
        <div class="modal-title" id="modal-title">Approve Leave Request</div>
        <input type="hidden" id="action-leave-id">
        <input type="hidden" id="action-type">
        <div class="form-group"><label>Reviewed By</label><input type="email" id="manager-email" placeholder="admin@company.com" readonly></div>
        <div class="form-group"><label>Remarks</label><textarea id="action-remarks" rows="3" placeholder="Add remarks..."></textarea></div>
        <div class="modal-actions"><button class="btn btn-primary" id="action-btn" onclick="submitApproval()">Approve</button><button class="btn btn-secondary" onclick="closeModal()">Cancel</button...
    </div>
</div>

<!-- Cancel Leave Modal -->
<div id="cancel-modal" class="modal-overlay">
    <div class="modal">
        <div class="modal-title">Cancel Leave Request</div>
        <input type="hidden" id="cancel-leave-id">
        <div class="form-group"><label>Cancelled By</label><input type="email" id="cancel-email" placeholder="admin@company.com" readonly></div>
        <div class="modal-actions"><button class="btn btn-danger" onclick="confirmCancelLeave()">Cancel Leave</button><button class="btn btn-secondary" onclick="closeCancelModal()">Close</button></...
    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    let currentUser = null;
    const ROLE_DASHBOARDS = {
        EMPLOYEE: '/employee-dashboard.html',
        MANAGER: '/manager-dashboard.html',
        VENDOR: '/vendor-dashboard.html',
        ADMIN: '/dashboard.html'
    };

    (async function initAuth() {
        const token = localStorage.getItem('jwt_token');
        const authType = localStorage.getItem('auth_type');
        if (!token && authType !== 'session') {
            window.location.href = '/login.html';
            return;
        }

        try {
            const response = await fetch(`${API}/employee/me`, {
                headers: getAuthHeader(),
                credentials: 'include'
            });

            if (response.status === 401) {
                window.location.href = '/login.html';
                return;
            }

            if (!response.ok) {
                window.location.href = '/login.html';
                return;
            }

            const employee = await response.json();
            const role = employee.role || 'EMPLOYEE';
            if (role !== 'ADMIN') {
                window.location.href = ROLE_DASHBOARDS[role] || '/employee-dashboard.html';
                return;
            }

            currentUser = employee;
            localStorage.setItem('last_role', role);
            document.getElementById('username-display').textContent = employee.email || employee.name || 'admin';
            document.getElementById('leave-email').value = employee.email || '';
            document.getElementById('manager-email').value = employee.email || '';
            document.getElementById('cancel-email').value = employee.email || '';
            loadDashboard();
```

```
        } catch (e) {
            window.location.href = '/login.html';
        }
    })();

    // ============ FIX 3: AUTH HEADER — session users send no JWT, rely on cookie ============
    // Previously getAuthHeader() always tried to send JWT. Session-logged-in users
    // had no jwt_token, so every API call went out without auth and got 401.
    function getAuthHeader() {
        const token = localStorage.getItem('jwt_token');
        return token ? { 'Authorization': `Bearer ${token}` } : {};
        // Session users rely on credentials:'include' (JSESSIONID cookie) — no header needed.
    }

    async function apiFetch(path, opts = {}) {
        try {
            const response = await fetch(`${API}${path}`, {
                ...opts,
                headers: {
                    'Content-Type': 'application/json',
                    ...getAuthHeader(),
                    ...opts.headers
                },
                credentials: 'include'
            });

            if (response.status === 401) {
                localStorage.removeItem('jwt_token');
                localStorage.removeItem('auth_type');
                showAlert('Session expired. Please login again.', 'error');
                setTimeout(() => window.location.href = '/login.html', 1500);
                throw new Error('Unauthorized');
            }

            if (!response.ok) {
                const error = await response.text();
                throw new Error(error || 'Request failed');
            }

            const text = await response.text();
            if (!text) return null;
            try {
                return JSON.parse(text);  //  try JSON first
            } catch {
                return text;              //  fall back to plain text
            }
        } catch (e) {
            if (!e.message.includes('Unauthorized')) {
                showAlert(e.message, 'error');
            }
            throw e;
        }
    }

    function showAlert(message, type) {
        const alert = document.createElement('div');
        alert.className = `alert alert-${type}`;
        alert.textContent = message;
        document.body.appendChild(alert);
        setTimeout(() => alert.remove(), 3000);
    }

    function showView(viewName, event) {
        document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
        document.getElementById(`${viewName}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
        if (event && event.target) event.target.classList.add('active');
        document.getElementById('pageTitle').textContent = viewName.split('-').map(w =>
            w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
        const loaders = {
            'dashboard': loadDashboard,
            'employees': loadEmployees,
            'add-employee': () => {},
            'add-manager': () => {},
            'add-vendor': () => {},
            'my-leaves': loadMyLeaves,
```

```
            'pending-leaves': loadPendingLeaves,
            'restaurants': loadRestaurants,
            'subscriptions': loadSubscriptions,
            'add-subscription': loadRestaurantDropdown,
            'deliveries': loadDeliveries,
            'vendors': loadVendors
        };
        if (loaders[viewName]) loaders[viewName]();
    }

    // ============ DASHBOARD ============
    async function loadDashboard() {
        try {
            const [employees, leaves] = await Promise.all([
                apiFetch('/employee'),
                apiFetch('/leave_requests')
            ]);
            document.getElementById('total-employees').textContent = employees.length;
            document.getElementById('active-employees').textContent = employees.filter(e => e.status === 'ACTIVE').length;
            document.getElementById('onleave-employees').textContent = employees.filter(e => e.status === 'ON_LEAVE').length;
            document.getElementById('pending-leaves').textContent = leaves.filter(l => l.status === 'PENDING').length;
            const recent = leaves.slice(-5).reverse();
            const tbody = document.getElementById('recent-leaves');
            if (!recent.length) {
                tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
            } else {
                tbody.innerHTML = recent.map(l => `<tr>
                    <td>${l.employeeId || 'N/A'}</td>
                    <td><span class="chip">${l.leaveType || 'N/A'}</span></td>
                    <td>${l.startDate || 'N/A'}</td>
                    <td>${l.endDate || 'N/A'}</td>
                    <td><span class="chip ${l.status?.toLowerCase()}">${l.status || 'N/A'}</span></td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ EMPLOYEES ============
    async function loadEmployees() {
        try {
            const employees = await apiFetch('/employee');
            const tbody = document.getElementById('employees-list');
            if (!employees.length) {
                tbody.innerHTML = '<tr><td colspan="8" class="empty-state">No employees found</td></tr>';
            } else {
                tbody.innerHTML = employees.map(e => `<tr>
                    <td>${e.employeeId}</td>
                    <td>${e.name || 'N/A'}</td>
                    <td>${e.email || 'N/A'}</td>
                    <td>${e.dept || 'N/A'}</td>
                    <td><span class="chip ${e.role?.toLowerCase()}">${e.role || 'N/A'}</span></td>
                    <td><span class="chip ${e.status?.toLowerCase()}">${e.status || 'N/A'}</span></td>
                    <td>${e.timezone || 'N/A'}</td>
                    <td><button class="btn btn-danger btn-sm" onclick="deleteEmployee(${e.employeeId})">Delete</button></td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    async function createEmployee() {
        const body = {
            name: document.getElementById('employee-name').value.trim(),
            email: document.getElementById('employee-email').value.trim(),
            password: document.getElementById('employee-password').value,
            dept: document.getElementById('employee-dept').value.trim(),
            timezone: document.getElementById('employee-timezone').value.trim() || 'UTC'
        };
        if (!body.name || !body.email || !body.password || !body.dept) {
            showAlert('Please fill all employee fields', 'error');
            return;
        }
        try {
```

```
            await apiFetch('/admin/employees', { method: 'POST', body: JSON.stringify(body) });
            showAlert('Employee created successfully!', 'success');
            clearEmployeeForm('employee');
            loadEmployees();
        } catch (e) {
            showAlert('Failed to create employee: ' + e.message, 'error');
        }
    }

    async function createManager() {
        const body = {
            name: document.getElementById('manager-name-create').value.trim(),
            email: document.getElementById('manager-email-create').value.trim(),
            password: document.getElementById('manager-password-create').value,
            dept: document.getElementById('manager-dept-create').value.trim(),
            timezone: document.getElementById('manager-timezone-create').value.trim() || 'UTC'
        };
        if (!body.name || !body.email || !body.password || !body.dept) {
            showAlert('Please fill all manager fields', 'error');
            return;
        }
        try {
            await apiFetch('/admin/employees/manager', { method: 'POST', body: JSON.stringify(body) });
            showAlert('Manager created successfully!', 'success');
            clearEmployeeForm('manager');
            loadEmployees();
        } catch (e) {
            showAlert('Failed to create manager: ' + e.message, 'error');
        }
    }

    function clearEmployeeForm(type) {
        const fields = type === 'manager'
            ? ['manager-name-create', 'manager-email-create', 'manager-password-create', 'manager-dept-create', 'manager-timezone-create']
            : ['employee-name', 'employee-email', 'employee-password', 'employee-dept', 'employee-timezone'];
        fields.forEach(id => document.getElementById(id).value = '');
    }

    async function deleteEmployee(employeeId) {
        if (!confirm(`Delete employee #${employeeId}?`)) return;
        try {
            await apiFetch(`/admin/employee/${employeeId}`, { method: 'DELETE' });
            showAlert('Employee deleted successfully!', 'success');
            loadEmployees();
            loadDashboard();
        } catch (e) {
            showAlert('Failed to delete employee: ' + e.message, 'error');
        }
    }

    // ============ LEAVE REQUESTS ============
    async function applyLeave() {
        const leaveType = document.getElementById('leave-type').value;
        const startDate = document.getElementById('leave-start').value;
        const endDate = document.getElementById('leave-end').value;
        const reason = document.getElementById('leave-reason').value.trim();
        if (!startDate || !endDate) {
            showAlert('Please fill all required fields', 'error');
            return;
        }
        try {
            await apiFetch('/leave_requests', {
                method: 'POST',
                body: JSON.stringify({ leaveType, startDate, endDate, reason })
            });
            showAlert('Leave request submitted successfully!', 'success');
            clearLeaveForm();
            loadMyLeaves();
        } catch (e) { showAlert('Failed to submit: ' + e.message, 'error'); }
    }

    async function loadMyLeaves() {
    try {
        const data = await apiFetch('/leave_requests/employee');
```

```
        const leaves = data.requests || []; //  extract the array from wrapper
        const tbody = document.getElementById('my-leaves-list');
        if (!leaves.length) {
            tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
        } else {
            tbody.innerHTML = leaves.map(l => `<tr>
                <td>${l.leaveRequestId}</td>
                <td><span class="chip">${l.leaveType}</span></td>
                <td>${l.startDate}</td>
                <td>${l.endDate}</td>
                <td><span class="chip ${l.status?.toLowerCase()}">${l.status}</span></td>
                <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-'}</td>
                <td>${l.status === 'PENDING' ? `<button class="btn btn-danger btn-sm" onclick="openCancelModal(${l.leaveRequestId})">Cancel</button>` : '-'}</td>
            </tr>`).join('');
        }
    } catch (e) { console.error(e); }
}

    async function loadPendingLeaves() {
        try {
            const leaves = await apiFetch('/leave_requests/pending');
            const tbody = document.getElementById('pending-leaves-list');
            if (!leaves.length) {
                tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No pending leave requests</td></tr>';
            } else {
                tbody.innerHTML = leaves.map(l => `<tr>
                    <td>${l.leaveRequestId}</td>
                    <td>${l.employeeId}</td>
                    <td><span class="chip">${l.leaveType}</span></td>
                    <td>${l.startDate}</td>
                    <td>${l.endDate}</td>
                    <td style="max-width:150px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-'}</td>
                    <td class="action-row">
                        <button class="btn btn-primary btn-sm" onclick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
                        <button class="btn btn-danger btn-sm" onclick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>
                    </td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    function openActionModal(leaveId, action) {
        document.getElementById('action-leave-id').value = leaveId;
        document.getElementById('action-type').value = action;
        document.getElementById('modal-title').textContent = `${action} Leave Request #${leaveId}`;
        document.getElementById('action-btn').textContent = action;
        document.getElementById('action-btn').className = action === 'APPROVED' ? 'btn btn-primary' : 'btn btn-danger';
        document.getElementById('action-modal').classList.add('open');
    }

    async function submitApproval() {
        const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
        const action = document.getElementById('action-type').value;
        const remarks = document.getElementById('action-remarks').value;
        try {
            await apiFetch('/leave_requests/approval', {
                method: 'PUT',
                body: JSON.stringify({ leaveRequestId, action, remarks })
            });
            showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
            closeModal();
            loadPendingLeaves();
        } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    function openCancelModal(leaveId) {
        document.getElementById('cancel-leave-id').value = leaveId;
        document.getElementById('cancel-modal').classList.add('open');
    }

```

```
    async function confirmCancelLeave() {
        const leaveId = document.getElementById('cancel-leave-id').value;
        try {
            await apiFetch(`/leave_requests/cancel/${leaveId}`, { method: 'PUT' });
            showAlert('Leave cancelled successfully!', 'success');
            closeCancelModal();
            loadMyLeaves();
        } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    function clearLeaveForm() {
        document.getElementById('leave-start').value = '';
        document.getElementById('leave-end').value = '';
        document.getElementById('leave-reason').value = '';
    }

    // ============ RESTAURANTS ============
    async function loadRestaurants() {
        try {
            const slot = document.getElementById('slot-filter').value;
            const path = slot ? `/restaurants/by-slot?slot=${slot}` : '/restaurants';
            const restaurants = await apiFetch(path);
            const tbody = document.getElementById('restaurants-list');
            if (!restaurants.length) {
                tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No restaurants found</td></tr>';
            } else {
                tbody.innerHTML = restaurants.map(r => `<tr>
                    <td><strong>${r.name}</strong></td>
                    <td>${r.address || 'N/A'}</td>
                    <td>${r.supportedMealSlots ? r.supportedMealSlots.map(s => `<span class="chip">${s}</span>`).join(' ') : 'N/A'}</td>
                    <td><span class="chip ${r.active ? 'active' : 'inactive'}">${r.active ? 'ACTIVE' : 'INACTIVE'}</span></td>
                    <td>${r.vendorName || r.vendorId || 'N/A'}</td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ SUBSCRIPTIONS ============

    // Populate restaurant dropdown — called when Add Subscription view loads
    async function loadRestaurantDropdown() {
        try {
            const restaurants = await apiFetch('/restaurants');
            const arr = Array.isArray(restaurants) ? restaurants : [];
            const select = document.getElementById('sub-restaurant-select');
            select.innerHTML = '<option value="">-- Select Restaurant --</option>';
            arr.filter(r => r.active).forEach(r => {
                const opt = document.createElement('option');
                opt.value = r.id || r.restaurantId;
                opt.textContent = r.name;
                // Store supported slots as data attribute for filtering
                opt.dataset.slots = JSON.stringify(r.supportedMealSlots || []);
                select.appendChild(opt);
            });
        } catch (e) { console.error('Failed to load restaurants:', e); }
    }

    // When restaurant changes, only show meal slot checkboxes it supports
    document.getElementById('sub-restaurant-select')?.addEventListener('change', function () {
        const selected = this.options[this.selectedIndex];
        const supported = selected?.dataset?.slots ? JSON.parse(selected.dataset.slots) : [];
        const boxes = document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]');
        boxes.forEach(cb => {
            const wrapper = cb.parentElement;
            if (!supported.length || supported.includes(cb.value)) {
                wrapper.style.display = 'flex';
                wrapper.style.opacity = '1';
                cb.disabled = false;
            } else {
                cb.checked = false;
                cb.disabled = true;
                wrapper.style.opacity = '0.3';
            }
        });
```

```
    });

    async function loadSubscriptions() {
        try {
            const subscriptions = await apiFetch('/subscription/user');
            const arr = Array.isArray(subscriptions) ? subscriptions : [];
            const tbody = document.getElementById('subscriptions-list');
            tbody.innerHTML = arr.length
                ? arr.map(s => `<tr>
                <td>${s.subscriptionId}</td>
                <td>${s.restaurantName || 'N/A'}</td>
                <td><span class="chip">${s.mealSlot}</span></td>
                <td>${s.scheduleType}</td>
                <td><span class="chip ${s.status?.toLowerCase()}">${s.status}</span></td>
                <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleDateString() : 'Not scheduled'}</td>
                <td class="action-row">
                    ${s.status === 'ACTIVE' ? `<button class="btn btn-secondary btn-sm" onclick="pauseSubscription(${s.subscriptionId})">Pause</button>` : ''}
                    ${s.status === 'PAUSED' ? `<button class="btn btn-primary btn-sm" onclick="resumeSubscription(${s.subscriptionId})">Resume</button>` : ''}
                    <button class="btn btn-danger btn-sm" onclick="expireSubscription(${s.subscriptionId})">Expire</button>
                </td>
            </tr>`).join('')
                : '<tr><td colspan="7" class="empty-state">No subscriptions found</td></tr>';
        } catch (e) { console.error(e); }
    }

    async function addSubscription() {
        const email = document.getElementById('sub-email').value.trim();
        const restaurantId = parseInt(document.getElementById('sub-restaurant-select').value);
        const scheduleType = document.getElementById('sub-schedule').value;
        const dayOfWeek = document.getElementById('sub-day').value;

        const selectedSlots = [...document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]:checked')]
            .map(cb => cb.value);

        if (!email) { showAlert('Please enter employee email', 'error'); return; }
        if (!restaurantId) { showAlert('Please select a restaurant', 'error'); return; }
        if (!selectedSlots.length) { showAlert('Please select at least one meal slot', 'error'); return; }

        const body = {
            scheduleType,
            mealSlots: selectedSlots,                           // ["BREAKFAST", "LUNCH"]
            slotOrders: selectedSlots.map(slot => ({            // [{ mealSlot: "BREAKFAST", restaurantId: 1 }, ...]
                mealSlot: slot,
                restaurantId: restaurantId
            }))
        };

        if (scheduleType === 'WEEKLY') body.dayOfWeek = dayOfWeek;

        try {
            await apiFetch('/subscription', { method: 'POST', body: JSON.stringify(body) });
            showAlert('Subscription created successfully!', 'success');
            clearSubscriptionForm();
            loadSubscriptions();
        } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    async function pauseSubscription(id) {
        try { await apiFetch(`/subscription/${id}/pause`, { method: 'PUT' }); showAlert('Subscription paused', 'success'); loadSubscriptions(); }
        catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    async function resumeSubscription(id) {
        try { await apiFetch(`/subscription/${id}/resume`, { method: 'PUT' }); showAlert('Subscription resumed', 'success'); loadSubscriptions(); }
        catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    async function expireSubscription(id) {
```

```
        try { await apiFetch(`/subscription/${id}/expire`, { method: 'PUT' }); showAlert('Subscription expired', 'success'); loadSubscriptions(); }
        catch (e) { showAlert('Failed: ' + e.message, 'error'); }
    }

    function clearSubscriptionForm() {
        document.getElementById('sub-email').value = '';
        document.getElementById('sub-restaurant-select').value = '';
        document.getElementById('sub-schedule').value = 'DAILY';
        document.getElementById('day-of-week-group').style.display = 'none';
        // Uncheck all and re-enable all slots
        document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]').forEach(cb => {
            cb.checked = false;
            cb.disabled = false;
            cb.parentElement.style.opacity = '1';
            cb.parentElement.style.display = 'flex';
        });
    }

    document.getElementById('sub-schedule')?.addEventListener('change', function () {
        document.getElementById('day-of-week-group').style.display = this.value === 'WEEKLY' ? 'block' : 'none';
    });

    // ============ DELIVERIES ============
    async function loadDeliveries() {
        try {
            const subscriptions = await apiFetch('/subscription/user');
            let allDeliveries = [];
            for (const sub of subscriptions) {
                try {
                    const deliveries = await apiFetch(`/delivery/subscription/${sub.subscriptionId}`);
                    allDeliveries.push(...deliveries);
                } catch (e) {}
            }
            const tbody = document.getElementById('deliveries-list');
            if (!allDeliveries.length) {
                tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No deliveries found</td></tr>';
            } else {
                tbody.innerHTML = allDeliveries.map(d => `<tr>
                    <td>${d.id}</td>
                    <td>${d.subscription?.id || 'N/A'}</td>
                    <td><span class="chip">${d.mealSlot || 'N/A'}</span></td>
                    <td>${d.scheduledDeliveryTime ? new Date(d.scheduledDeliveryTime).toLocaleString() : 'N/A'}</td>
                    <td><span class="chip ${d.status?.toLowerCase()}">${d.status || 'SCHEDULED'}</span></td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    // ============ VENDORS ============
    async function loadVendors() {
        try {
            const vendors = await apiFetch('/vendors');
            const tbody = document.getElementById('vendors-list');
            if (!vendors.length) {
                tbody.innerHTML = '<tr><td colspan="6" class="empty-state">No vendors found</td></tr>';
            } else {
                tbody.innerHTML = vendors.map(v => `<tr>
                    <td>${v.id}</td>
                    <td><strong>${v.name}</strong></td>
                    <td>${v.email}</td>
                    <td>${v.phone || 'N/A'}</td>
                    <td>${v.registeredAt || 'N/A'}</td>
                    <td><button class="btn btn-danger btn-sm" onclick="deleteVendor(${v.id})">Delete</button></td>
                </tr>`).join('');
            }
        } catch (e) { console.error(e); }
    }

    async function createVendor() {
        const body = {
            name: document.getElementById('vendor-name-create').value.trim(),
```

```
            email: document.getElementById('vendor-email-create').value.trim(),
            password: document.getElementById('vendor-password-create').value,
            phone: document.getElementById('vendor-phone-create').value.trim(),
            role: document.getElementById('vendor-role').value
        };
        if (!body.name || !body.email || !body.password || !body.phone) {
            showAlert('Please fill all vendor fields', 'error');
            return;
        }
        try {
            await apiFetch('/vendors/register', { method: 'POST', body: JSON.stringify(body) });
            showAlert('Vendor created successfully!', 'success');
            clearVendorForm();
            loadVendors();
        } catch (e) {
            showAlert('Failed to create vendor: ' + e.message, 'error');
        }
    }

    function clearVendorForm() {
        ['vendor-name-create', 'vendor-email-create', 'vendor-password-create', 'vendor-phone-create']
            .forEach(id => document.getElementById(id).value = '');
    }

    async function deleteVendor(vendorId) {
        if (!confirm(`Delete vendor #${vendorId}?`)) return;
        try {
            await apiFetch(`/vendors/${vendorId}`, { method: 'DELETE' });
            showAlert('Vendor deleted successfully!', 'success');
            loadVendors();
        } catch (e) {
            showAlert('Failed to delete vendor: ' + e.message, 'error');
        }
    }

    // ============ MODAL HELPERS ============
    function closeModal() {
        document.getElementById('action-modal').classList.remove('open');
        document.getElementById('action-remarks').value = '';
        document.getElementById('manager-email').value = currentUser?.email || '';
    }

    function closeCancelModal() {
        document.getElementById('cancel-modal').classList.remove('open');
        document.getElementById('cancel-email').value = currentUser?.email || '';
    }

    // ============ FIX 5: LOGOUT — also invalidates server-side session for session users ============
    // Previously logout() only cleared localStorage, leaving the JSESSIONID cookie active.
    function logout(){
        // Clear all localStorage
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('refresh_token');
        localStorage.removeItem('auth_type');
        localStorage.removeItem('last_role');

        // Call server logout to invalidate session
        fetch(`${API}/session/logout`, {
            method: 'POST',
            credentials: 'include'
        }).finally(() => {
            // Force expire the session cookie on client side
            document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
            window.location.href = '/google.html';
        });
    }

    // Set today's date as default for leave form
    document.getElementById('leave-start').valueAsDate = new Date();
    document.getElementById('leave-end').valueAsDate = new Date();

</script>
</body>
</html>
```

141 Current: src\main\resources\static\employee-dashboard.html 721 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Employee Portal</title>
    <link href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
            --bg:#080c10;--bg2:#0d1117;--bg3:#111820;
            --panel:#141c26;--panel2:#1a2332;
            --border:rgba(99,179,237,0.08);--border2:rgba(99,179,237,0.15);
            --text:#d4e6f1;--text2:#7a9bb5;--text3:#4a6d87;
            --accent:#38bdf8;--accent-dim:rgba(56,189,248,0.1);
            --green:#34d399;--green-dim:rgba(52,211,153,0.1);
            --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
            --red:#f87171;--red-dim:rgba(248,113,113,0.1);
            --font:'Sora',sans-serif;--mono:'JetBrains Mono',monospace;
            --sidebar:240px;
        }
        body{font-family:var(--font);background:var(--bg);color:var(--text);font-size:14px;overflow:hidden;height:100vh}
        .app{display:flex;height:100vh}

        /* SIDEBAR */
        .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--border);display:flex;flex-direction:column;position:relative;flex-shrink:0}
        .sidebar::before{content:'';position:absolute;top:0;left:0;right:0;height:1px;background:linear-gradient(90deg,transparent,var(--accent),transparent)}
        .logo{padding:24px 20px;border-bottom:1px solid var(--border)}
        .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
        .logo-icon{width:32px;height:32px;background:linear-gradient(135deg,var(--accent),#0ea5e9);border-radius:8px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size...
        .logo h2{font-size:16px;font-weight:600;color:var(--text);letter-spacing:0.5px}
        .logo p{font-size:11px;color:var(--text3);font-family:var(--mono);margin-top:2px}
        .role-badge{display:inline-flex;align-items:center;gap:5px;padding:4px 10px;background:var(--accent-dim);border:1px solid rgba(56,189,248,0.25);border-radius:20px;font-size:10px;font-weight...
        .role-badge::before{content:'';width:6px;height:6px;background:var(--accent);border-radius:50%;animation:pulse 2s infinite}
        @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
        .nav{flex:1;padding:12px 0;overflow-y:auto}
        .nav-section{padding:4px 20px 8px;font-size:10px;font-weight:600;color:var(--text3);letter-spacing:1.5px;text-transform:uppercase;margin-top:8px}
        .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
        .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
        .nav-item.active{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(56,189,248,0.2)}
        .nav-item .icon{width:20px;text-align:center;font-size:14px;flex-shrink:0}
        .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
        .user-card{display:flex;align-items:center;gap:10px}
        .avatar{width:32px;height:32px;background:linear-gradient(135deg,#0ea5e9,var(--accent));border-radius:50%;display:flex;align-items:center;justify-content:center;font-size:13px;font-weight:6...
        .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
        .user-info-side .email{font-size:10px;color:var(--text3);font-family:var(--mono);white-space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:140px}

        /* MAIN */
        .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
        .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
        .page-title{font-size:16px;font-weight:600;color:var(--text)}
        .topbar-right{display:flex;align-items:center;gap:12px}
        .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
        .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}

        /* CONTENT */
        .content{flex:1;overflow-y:auto;padding:24px}
```

```
58          .view{display:none}
59          .view.active{display:block;animation:fadeIn 0.3s ease}
60          @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
61
62          /* CARDS */
63          .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
64          .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:12px;padding:20px;position:relative;overflow:hidden;transition:border-color 0.2s}
65          .stat-card:hover{border-color:var(--border2)}
66          .stat-card::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
67          .stat-card.blue::after{background:var(--accent)}
68          .stat-card.green::after{background:var(--green)}
69          .stat-card.amber::after{background:var(--amber)}
70          .stat-card.red::after{background:var(--red)}
71          .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;margin-
bottom:10px;font-weight:500}
72          .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
73          .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
74
75          /* SECTION */
76          .section{background:var(--panel);border:1px solid var(--border);border-radius:12px;margin-
bottom:20px;overflow:hidden}
77          .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
78          .section-title{font-size:14px;font-weight:600;color:var(--text)}
79          .section-sub{font-size:12px;color:var(--text3)}
80
81          /* TABLE */
82          .tablewrap{overflow-x:auto}
83          table{width:100%;border-collapse:collapse}
84          th{padding:10px 16px;text-align:left;font-size:11px;color:var(--text3);font-weight:600;letter-
spacing:0.8px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--bord...
85          td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
86          tr:last-child td{border-bottom:none}
87          tr:hover td{background:rgba(56,189,248,0.03);color:var(--text)}
88
89          /* CHIPS */
90          .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600;letter-spacing:0.5px}
91          .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
92          .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
93          .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
94          .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
95          .chip.sick{background:var(--red-dim);color:var(--red)}
96          .chip.casual{background:var(--amber-dim);color:var(--amber)}
97          .chip.earned{background:var(--accent-dim);color:var(--accent)}
98
99          /* FORM */
100          .form-card{padding:24px}
101          .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
102          .form-group{display:flex;flex-direction:column;gap:6px}
103          .form-group.full{grid-column:1/-1}
104          label{font-size:11px;font-weight:600;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
105          input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
106          input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent);background:var(--
bg2)}
107          input::placeholder,textarea::placeholder{color:var(--text3)}
108          select option{background:var(--bg2)}
109
110          /* BUTTONS */
111          .btn{padding:9px 18px;border-radius:8px;border:none;font-family:var(--font);font-size:13px;font-
weight:600;cursor:pointer;transition:all 0.2s}
112          .btn-primary{background:var(--accent);color:#fff}
113          .btn-primary:hover{background:#0ea5e9;transform:translateY(-1px);box-shadow:0 4px 12px
rgba(56,189,248,0.3)}
114          .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
115          .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
116          .btn-sm{padding:6px 12px;font-size:12px}
117          .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
118          .btn-danger:hover{background:var(--red);color:#fff}
119          .form-actions{display:flex;gap:10px;margin-top:20px}
120
121          /* LEAVE BALANCE */
```

```
        .balance-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:12px;padding:20px}
        .balance-item{background:var(--bg3);border:1px solid var(--border);border-radius:10px;padding:16px;text-align:center}
        .balance-days{font-size:32px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .balance-type{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-top:4px}

        /* ALERT */
        .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
        .alert-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.3)}
        .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
        @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}

        .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
        .profile-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px}
        .profile-field{display:flex;flex-direction:column;gap:4px}
        .profile-field .field-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;font-weight:600}
        .profile-field .field-value{font-size:14px;color:var(--text);padding:10px 14px;background:var(--bg3);border-radius:8px;border:1px solid var(--border);font-family:var(--mono)}

        .subscription-card{display:grid;grid-template-columns:repeat(auto-fill,minmax(200px,1fr));gap:12px;padding:20px}
        .sub-item{background:var(--bg3);border:1px solid var(--border);border-radius:10px;padding:16px;transition:border-color 0.2s}
        .sub-item:hover{border-color:var(--border2)}
        .sub-item .meal-name{font-weight:600;color:var(--text);margin-bottom:6px}
        .sub-item .meal-meta{font-size:11px;color:var(--text3)}

        .chip.paused{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
        .chip.expired{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
    </style>
</head>
<body>
<div class="app">
    <!-- SIDEBAR -->
    <div class="sidebar">
        <div class="logo">
            <div class="logo-mark">
                <div class="logo-icon">E</div>
                <h2>EMS Portal</h2>
            </div>
            <p>Employee Management</p>
            <div class="role-badge">Employee</div>
        </div>
        <nav class="nav">
            <div class="nav-section">Overview</div>
            <div class="nav-item active" onclick="showView('dashboard',this)"><span class="icon"></span> Dashboard</div>
            <div class="nav-section">Leave</div>
            <div class="nav-item" onclick="showView('apply-leave',this)"><span class="icon"></span> Apply Leave</div>
            <div class="nav-item" onclick="showView('my-leaves',this)"><span class="icon"></span> My Leave History</div>
            <div class="nav-section">Meals</div>
            <div class="nav-item" onclick="showView('restaurants',this)"><span class="icon"></span> Restaurants</div>
            <div class="nav-item" onclick="showView('subscriptions',this)"><span class="icon"></span> My Subscriptions</div>
            <div class="nav-item" onclick="showView('add-subscription',this)"><span class="icon"></span> Add Subscription</div>
            <div class="nav-section">Account</div>
            <div class="nav-item" onclick="showView('profile',this)"><span class="icon"></span> My Profile</div>
        </nav>
        <div class="sidebar-footer">
            <div class="user-card">
                <div class="avatar" id="avatar-initials">?</div>
                <div class="user-info-side">
                    <div class="name" id="sidebar-name">Loading...</div>
                    <div class="email" id="sidebar-email">—</div>
                </div>
            </div>
        </div>
```

```
    </div>

    <!-- MAIN -->
    <div class="main">
        <div class="topbar">
            <div class="page-title" id="pageTitle">Dashboard</div>
            <div class="topbar-right">
                <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-email"></span>
                <button class="logout-btn" onclick="logout()">Sign Out</button>
            </div>
        </div>
        <div class="content">

            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card blue">
                        <div class="stat-label">Leave Balance</div>
                        <div class="stat-value" id="stat-balance">--</div>
                        <div class="stat-sub">days remaining</div>
                    </div>
                    <div class="stat-card amber">
                        <div class="stat-label">Pending Leaves</div>
                        <div class="stat-value" id="stat-pending">--</div>
                        <div class="stat-sub">awaiting approval</div>
                    </div>
                    <div class="stat-card green">
                        <div class="stat-label">Approved Leaves</div>
                        <div class="stat-value" id="stat-approved">--</div>
                        <div class="stat-sub">this year</div>
                    </div>
                    <div class="stat-card red">
                        <div class="stat-label">Rejected Leaves</div>
                        <div class="stat-value" id="stat-rejected">--</div>
                        <div class="stat-sub">this year</div>
                    </div>
                </div>
                <div class="section">
                    <div class="section-head">
                        <div>
                            <div class="section-title">Recent Leave Requests</div>
                            <div class="section-sub">Your last 5 requests</div>
                        </div>
                        <button class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
                            <tbody id="recent-leaves"><tr><td colspan="5" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">Active Subscriptions</div>
                    </div>
                    <div id="dashboard-subs" class="subscription-card"></div>
                </div>
            </div>

            <!-- APPLY LEAVE -->
            <div id="apply-leave-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">Apply for Leave</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group">
                                <label>Leave Type</label>
                                <select id="leave-type">
                                    <option value="SICK">Sick Leave</option>
                                    <option value="CASUAL">Casual Leave</option>
                                    <option value="EARNED">Earned Leave</option>
```

```
                                </select>
                            </div>
                            <div class="form-group">
                                <label>Start Date</label>
                                <input type="date" id="leave-start">
                            </div>
                            <div class="form-group">
                                <label>End Date</label>
                                <input type="date" id="leave-end">
                            </div>
                            <div class="form-group full">
                                <label>Reason</label>
                                <textarea id="leave-reason" rows="3" placeholder="Please describe the reason for your leave..."></textarea>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
                            <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- MY LEAVES -->
            <div id="my-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">My Leave History</div><div class="section-sub">All your leave requests</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Status</th><th>Action</th></tr></thead>
                            <tbody id="my-leaves-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- RESTAURANTS -->
            <div id="restaurants-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">Available Restaurants</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Name</th><th>Meal Slot</th><th>Status</th><th>Action</th></tr></thead>
                            <tbody id="restaurants-list"><tr><td colspan="4" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- SUBSCRIPTIONS -->
            <div id="subscriptions-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">My Meal Subscriptions</div>
                        <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead>
                            <tr>
                                <th>Restaurant</th>
```

```
                                <th>Meal Slot</th>
                                <th>Schedule</th>
                                <th>Next Delivery</th>
                                <th>Status</th>
                                <th>Action</th>
                            </tr>
                            </thead>
                            <tbody id="subscriptions-list">><tr><td colspan="6" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- ADD SUBSCRIPTION -->
            <div id="add-subscription-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">Subscribe to Meal Plan</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group">
                                <label>Meal Slot</label>
                                <select id="sub-meal-slot">
                                    <option value="BREAKFAST">Breakfast</option>
                                    <option value="LUNCH">Lunch</option>
                                    <option value="DINNER">Dinner</option>
                                </select>
                            </div>
                            <div class="form-group">
                                <label>Restaurant</label>
                                <select id="sub-restaurant-id">
                                    <option value="">Select a restaurant</option>
                                </select>
                            </div>
                            <div class="form-group">
                                <label>Schedule Type</label>
                                <select id="sub-schedule-type">
                                    <option value="DAILY">Daily</option>
                                    <option value="WEEKLY">Weekly</option>
                                </select>
                            </div>
                            <div class="form-group" id="day-of-week-group" style="display:none">
                                <label>Day of Week</label>
                                <select id="sub-day-of-week">
                                    <option value="MONDAY">Monday</option>
                                    <option value="TUESDAY">Tuesday</option>
                                    <option value="WEDNESDAY">Wednesday</option>
                                    <option value="THURSDAY">Thursday</option>
                                    <option value="FRIDAY">Friday</option>
                                    <option value="SATURDAY">Saturday</option>
                                    <option value="SUNDAY">Sunday</option>
                                </select>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="addSubscription()">Subscribe</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- PROFILE -->
            <div id="profile-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">My Profile</div></div>
                    <div class="profile-grid" id="profile-data">
                        <div class="profile-field"><div class="field-label">Name</div><div class="field-value" id="p-name">—</div></div>
                        <div class="profile-field"><div class="field-label">Email</div><div class="field-value" id="p-email">—</div></div>
                        <div class="profile-field"><div class="field-label">Department</div><div class="field-value" id="p-dept">—</div></div>
                        <div class="profile-field"><div class="field-label">Role</div><div class="field-value" id="p-role">—</div></div>
                        <div class="profile-field"><div class="field-label">Status</div><div class="field-value" id="p-status">—</div></div>
```

```
                        <div class="profile-field"><div class="field-label">Timezone</div><div class="field-value" id="p-tz">—</div></div>
                    </div>
                </div>
            </div>

        </div>
    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    let availableRestaurants = [];

    function extractRoleFromToken(token) {
        try {
            const payload = JSON.parse(atob(token.split('.')[1]));
            return payload.role || payload.roles || payload.authorities || '';
        } catch (e) {
            return '';
        }
    }

    // AUTH GUARD
    (function(){
        const token = localStorage.getItem('jwt_token');
        const authType = localStorage.getItem('auth_type');
        if(!token && authType !== 'session') { window.location.href = '/login.html'; return; }
        // Decode JWT to check role
        if(token) {
            try {
                const roles = String(extractRoleFromToken(token));
                if(roles.includes('ROLE_MANAGER')) { window.location.href = '/manager-dashboard.html'; return; }
                if(roles.includes('ROLE_VENDOR')) { window.location.href = '/vendor-dashboard.html'; return; }
                if(roles.includes('ROLE_ADMIN') || roles.includes('ADMIN')) { window.location.href = '/dashboard.html'; return; }
            } catch(e) { console.warn('Could not decode JWT for role check'); }
        }
        initUser();
    })();

    function getAuthHeader(){
        const token = localStorage.getItem('jwt_token');
        return token ? {'Authorization':`Bearer ${token}`} : {};
    }

    async function apiFetch(path, opts={}){
        const token = localStorage.getItem('jwt_token');      //  add this
        const authType = localStorage.getItem('auth_type');   //  add this
        const res = await fetch(`${API}${path}`,{
            ...opts,
            headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
            credentials: (!token && authType === 'session') ? 'include' : 'omit'
        });
    if(res.status===401){
        localStorage.removeItem('jwt_token'); localStorage.removeItem('auth_type');
        showAlert('Session expired. Please login again.','error');
        setTimeout(()=>window.location.href='/login.html',1500);
        throw new Error('Unauthorized');
    }
    if(res.status===403) throw new Error('ACCESS_DENIED');
    if(!res.ok){
        const errText = await res.text();
        throw new Error(errText || 'Request failed');
    }
    const text = await res.text();
    try { return text ? JSON.parse(text) : null; }
    catch { return text; } // plain string response
    }

    async function initUser(){
        try {
            // Try /session/me first, fall back to JWT decode
            let email='', name='';
```

```
            const token = localStorage.getItem('jwt_token');
            if(token){
                try{
                    const p=JSON.parse(atob(token.split('.')[1]));
                    email=p.sub||p.email||'';
                }catch(e){}
            }
            // Try to fetch profile
            try{
                const me = await apiFetch('/employee/me');
                if(me){
                    name=me.name||email; email=me.email||email;
                    document.getElementById('p-name').textContent=me.name||'—';
                    document.getElementById('p-email').textContent=me.email||'—';
                    document.getElementById('p-dept').textContent=me.dept||'—';
                    document.getElementById('p-role').textContent=me.role||'EMPLOYEE';
                    document.getElementById('p-status').textContent=me.status||'—';
                    document.getElementById('p-tz').textContent=me.timezone||'—';
                }
            }catch(e){ console.warn('Could not load profile'); }
            const initials=(name||email||'?').charAt(0).toUpperCase();
            document.getElementById('avatar-initials').textContent=initials;
            document.getElementById('sidebar-name').textContent=name||email||'Employee';
            document.getElementById('sidebar-email').textContent=email;
            document.getElementById('topbar-email').textContent=email;
        }catch(e){console.error(e);}
        loadDashboard();
    }

    function showView(name, el){
        document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
        document.getElementById(`${name}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
        if(el) el.classList.add('active');
        document.getElementById('pageTitle').textContent=name.split('-').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
        const loaders={dashboard:loadDashboard,'my-leaves':loadMyLeaves,'apply-leave':()=>{},'restaurants':loadRestaurants,'subscriptions':loadSubscriptions,'add-subscription':()=>{},'profile':()=>...
        if(loaders[name]) loaders[name]();
        if(name === 'add-subscription') {
            loadRestaurantChoices();
        }
    }

    async function loadDashboard(){
        try{
            const leaves = await apiFetch('/leave_requests/employee');
            const arr = leaves?.requests || leaves || [];
            const pending=arr.filter(l=>l.status==='PENDING').length;
            const approved=arr.filter(l=>l.status==='APPROVED').length;
            const rejected=arr.filter(l=>l.status==='REJECTED').length;
            document.getElementById('stat-pending').textContent=pending;
            document.getElementById('stat-approved').textContent=approved;
            document.getElementById('stat-rejected').textContent=rejected;
            document.getElementById('stat-balance').textContent=Math.max(0,20-approved);
            const recent=arr.slice(-5).reverse();
            document.getElementById('recent-leaves').innerHTML=recent.length
                ? recent.map(l=>`<tr><td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType||'N/A'}</span></td><td>${l.startDate||'—'}</td><td>${l.endDate||'—'}</td><td>${(l.reason...
                : '<tr><td colspan="5" class="empty-state">No leave requests yet</td></tr>';
        }catch(e){document.getElementById('recent-leaves').innerHTML='<tr><td colspan="5" class="empty-state">Could not load leave data</td></tr>';}
        try{
    const subs = await apiFetch('/subscription/user'); //  fix endpoint
    const arr = subs || [];
    document.getElementById('dashboard-subs').innerHTML = arr.length
        ? arr.filter(s => s.status === 'ACTIVE').slice(0,4).map(s => `
            <div class="sub-item">
                <div class="meal-name">${s.restaurantName || 'Restaurant'}</div>
                <div class="meal-meta">
                    ${s.mealSlot || ''} · ${s.scheduleType || ''}
                    ${s.dayOfWeek ? ' · ' + s.dayOfWeek : ''}
                </div>
                <div class="meal-meta" style="margin-top:4px;color:var(--accent)">
```

```
                    Next: ${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleString() : 'N/A'}
                </div>
            </div>`).join('')
        : '<div style="padding:16px;color:var(--text3);font-style:italic">No active subscriptions</div>';
      } catch(e) {
     document.getElementById('dashboard-subs').innerHTML =
        '<div style="padding:16px;color:var(--text3)">Could not load subscriptions</div>';
        }
    }

    async function loadMyLeaves(){
        try{
            const data=await apiFetch('/leave_requests/employee');
            const leaves=data?.requests||data||[];
            document.getElementById('my-leaves-list').innerHTML=leaves.length
                ? leaves.map(l=>`<tr><td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType||'N/A'}</span></td><td>${l.startDate||'—'}</td><td>${l.endDate||'—'}</td><td>${l.numberO...
                : '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
        }catch(e){document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="7" class="empty-state">Could not load leave requests</td></tr>';}
    }

    async function applyLeave(){
        const leaveType=document.getElementById('leave-type').value;
        const startDate=document.getElementById('leave-start').value;
        const endDate=document.getElementById('leave-end').value;
        const reason=document.getElementById('leave-reason').value.trim();
        if(!startDate||!endDate){showAlert('Please fill all required fields','error');return;}
        try{
            await apiFetch('/leave_requests',{method:'POST',body:JSON.stringify({leaveType,startDate,endDate,reason})});
            showAlert('Leave request submitted!','success');
            clearLeaveForm();
            loadDashboard();
            loadMyLeaves();
        }catch(e){showAlert('Failed to submit: '+e.message,'error');}
    }

    async function cancelLeave(id){
    if(!confirm('Cancel this leave request?')) return;
    try{
        await apiFetch(`/leave_requests/cancel/${id}`, {method:'PUT'});
        showAlert('Request cancelled successfully','success');
        loadMyLeaves();
    }
    catch(e){
        if(e.message === 'ACCESS_DENIED') showAlert('You do not have permission to cancel this leave','error');
        else showAlert('Failed to cancel: ' + e.message, 'error');
    }
   }

    function clearLeaveForm(){
        ['leave-start','leave-end','leave-reason'].forEach(id=>{document.getElementById(id).value='';});
    }

    async function loadRestaurants(){
    try{
        const data=await apiFetch('/restaurants');
        availableRestaurants = data || [];
        document.getElementById('restaurants-list').innerHTML=data?.length
            ? data.map(r=>`<tr>
    <td>${r.name||'—'}</td>
    <td>${r.supportedMealSlots?.join(', ') || '—'}</td>
    <td><span class="chip ${r.active ? 'active' : 'rejected'}">${r.active?'Active':'Inactive'}</span></td>
    <td><button class="btn btn-secondary btn-sm"
        onclick='openSubscriptionForRestaurant(${r.restaurantId||r.id}, ${JSON.stringify(r.supportedMealSlots||[])})'>
        Subscribe</button></td>
 </tr>`).join('')
            : '<tr><td colspan="4" class="empty-state">No restaurants available</td></tr>';
    }catch(e){document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-state">Could not load restaurants</td></tr>';}
   }

```

```
    async function loadRestaurantChoices() {
        const select = document.getElementById('sub-restaurant-id');
        try {
            const data = await apiFetch('/restaurants'); //  same as manager
            availableRestaurants = data || [];
            select.innerHTML = availableRestaurants.length
                ? '<option value="">Select a restaurant</option>' +
                availableRestaurants.map(r =>
                    `<option value="${r.restaurantId||r.id}">${r.name}</option>`
                ).join('')
                : '<option value="">No restaurants available</option>';
        } catch(e) {
            select.innerHTML = '<option value="">Could not load restaurants</option>';
        }
    }

   function openSubscriptionForRestaurant(restaurantId, supportedMealSlots) {
    showView('add-subscription', null);
    if (supportedMealSlots?.length) {
        document.getElementById('sub-meal-slot').value = supportedMealSlots[0];
    }
    loadRestaurantChoices().then(() => {
        document.getElementById('sub-restaurant-id').value = String(restaurantId);
    });
   }

    async function loadSubscriptions(){
    try{
        const data = await apiFetch('/subscription/user');
        const arr = data || [];
        document.getElementById('subscriptions-list').innerHTML = arr.length
            ? arr.map(s => `<tr>
                <td>${s.restaurantName || s.restaurantId || '—'}</td>
                <td>${s.mealSlot || '—'}</td>
                <td>${s.scheduleType || '—'}${s.dayOfWeek ? ' · ' + s.dayOfWeek : ''}</td>
                <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleString() : '—'}</td>
                <td><span class="chip ${(s.status||'').toLowerCase()}">${s.status || 'ACTIVE'}</span></td>
                <td><button class="btn btn-danger btn-sm"
onclick="cancelSub(${s.subscriptionId})">Cancel</button></td>
              </tr>`).join('')
            : '<tr><td colspan="6" class="empty-state">No subscriptions found</td></tr>';
    } catch(e) {
        document.getElementById('subscriptions-list').innerHTML =
            '<tr><td colspan="6" class="empty-state">Could not load subscriptions</td></tr>';
    }
}

    document.getElementById('sub-schedule-type').addEventListener('change', function(){
    document.getElementById('day-of-week-group').style.display =
        this.value === 'WEEKLY' ? 'block' : 'none';
    });

   document.getElementById('sub-meal-slot').addEventListener('change', loadRestaurantChoices);

   async function addSubscription(){
    const restaurantId = parseInt(document.getElementById('sub-restaurant-id').value);
    const mealSlot = document.getElementById('sub-meal-slot').value;
    const scheduleType = document.getElementById('sub-schedule-type').value;
    const dayOfWeek = document.getElementById('sub-day-of-week')?.value;

    if(!restaurantId || !mealSlot){
        showAlert('Please fill all fields','error'); return;
    }

    const body = {
        scheduleType,
        mealSlots: [mealSlot],
        slotOrders: [{ mealSlot, restaurantId }]
    };
    if(scheduleType === 'WEEKLY' && dayOfWeek) body.dayOfWeek = dayOfWeek;

    try{
        await apiFetch('/subscription', {method:'POST', body:JSON.stringify(body)});
        showAlert('Subscribed successfully!','success');
        loadSubscriptions();
        loadDashboard();
```

```
    }catch(e){
        showAlert('Subscription failed: '+e.message,'error');
    }
   }

   async function cancelSub(id){
    if(!confirm('Cancel this subscription?')) return;
    try{
        await apiFetch(`/subscription/${id}/expire`, {method:'PUT'});
        showAlert('Subscription cancelled','success');
        loadSubscriptions();
    }
    catch(e){
        showAlert('Failed to cancel: ' + e.message,'error');
    }
}

    function logout(){
        // Clear all localStorage
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('refresh_token');
        localStorage.removeItem('auth_type');
        localStorage.removeItem('last_role');

        // Call server logout to invalidate session
        fetch(`${API}/session/logout`, {
            method: 'POST',
            credentials: 'include'
        }).finally(() => {
            // Force expire the session cookie on client side
            document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
            window.location.href = '/google.html';
        });
    }

    function showAlert(msg,type){
        const el=document.createElement('div');
        el.className=`alert alert-${type}`;el.textContent=msg;
        document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
    }
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Sign In</title>
    <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
            --bg:#070910;
            --panel:#0e1118;
            --panel2:#131821;
            --border:rgba(255,255,255,0.07);
            --border-lit:rgba(255,255,255,0.14);
            --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
            --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
            --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.12);
            --red:#f87171;--red-dim:rgba(248,113,113,0.12);
            --orange:#f59e0b;--orange-dim:rgba(245,158,11,0.12);
            --green:#34d399;--green-dim:rgba(52,211,153,0.12);
            --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
        }

        body{
            font-family:var(--font);
            background:var(--bg);
            color:var(--text);
            min-height:100vh;
            display:flex;
            overflow-y:auto;
        }

        /* ---- BACKGROUND ---- */
        .bg-layer{
            position:fixed;inset:0;z-index:0;
            background:
                    radial-gradient(ellipse 60% 50% at 20% 50%, rgba(79,140,255,0.06) 0%, transparent 70%),
                    radial-gradient(ellipse 40% 60% at 80% 20%, rgba(167,139,250,0.05) 0%, transparent 60%),
                    radial-gradient(ellipse 50% 40% at 70% 80%, rgba(52,211,153,0.04) 0%, transparent 60%);
        }
        .grid-overlay{
            position:fixed;inset:0;z-index:0;
            background-image:
                    linear-gradient(rgba(255,255,255,0.025) 1px, transparent 1px),
                    linear-gradient(90deg, rgba(255,255,255,0.025) 1px, transparent 1px);
            background-size:40px 40px;
            mask-image:radial-gradient(ellipse 80% 80% at center, black 20%, transparent 80%);
        }

        #oauth-loading {
            display: none;
            position: fixed; inset: 0;
            background: linear-gradient(135deg, #0d0f14 0%, #131720 100%);
            z-index: 9999;
            justify-content: center;
            align-items: center;
            flex-direction: column;
            gap: 20px;
        }
        #oauth-loading.visible { display: flex; }
        .spinner {
            width: 48px; height: 48px;
            border: 3px solid #2a2f3f;
            border-top-color: #4f8cff;
            border-radius: 50%;
            animation: spin 0.9s linear infinite;
        }
        @keyframes spin { to { transform: rotate(360deg); } }

```

```
        /* ---- SPLIT LAYOUT ---- */
        .left-panel{
            flex:1;display:flex;flex-direction:column;justify-content:center;
            padding:60px;position:relative;z-index:1;
            animation:fadeLeft 0.8s ease both;
        }
        @keyframes fadeLeft{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
        .brand{margin-bottom:48px}
        .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:20px}
        .brand-icon{
            width:44px;height:44px;background:linear-gradient(135deg,#4f8cff,#6366f1);
            border-radius:12px;display:flex;align-items:center;justify-content:center;
            font-weight:700;font-size:18px;color:#fff;
            box-shadow:0 0 30px rgba(79,140,255,0.35);
            flex-shrink:0;
        }
        .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
        .brand-tagline{font-size:13px;color:var(--text2)}
        .hero-text h1{
            font-size:clamp(36px,4vw,52px);font-weight:700;
            line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;
        }
        .hero-text h1 .hl-blue{color:var(--blue)}
        .hero-text h1 .hl-violet{color:var(--violet)}
        .hero-text h1 .hl-orange{color:var(--orange)}
        .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:380px}
        .feature-pills{display:flex;flex-wrap:wrap;gap:8px;margin-top:32px}
        .pill{
            display:inline-flex;align-items:center;gap:6px;
             padding:6px 14px;border-radius:20px;font-size:12px;font-weight:500;
             border:1px solid var(--border);background:var(--panel);color:var(--text2);
         }
         .pill .dot{width:6px;height:6px;border-radius:50%;flex-shrink:0}
         .pill.blue .dot{background:var(--blue)} .pill.blue{border-color:rgba(79,140,255,0.2);color:rgba(79,140,255,0.8)}
         .pill.violet .dot{background:var(--violet)} .pill.violet{border-color:rgba(167,139,250,0.2);color:rgba(167,139,250,0.8)}
         .pill.orange .dot{background:var(--orange)} .pill.orange{border-color:rgba(245,158,11,0.2);color:rgba(245,158,11,0.8)}

         /* ---- RIGHT PANEL (FORM) ---- */
         .right-panel{
             width:460px;flex-shrink:0;
             display:flex;align-items:flex-start;justify-content:center;
             overflow-y:auto;
             padding:32px 40px;position:relative;z-index:1;
             border-left:1px solid var(--border);
             background:rgba(14,17,24,0.8);
             backdrop-filter:blur(20px);
             animation:fadeRight 0.8s ease both;
         }
         @keyframes fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
         .form-box{width:100%}

         /* ---- ROLE SELECTOR — 3 roles only: Employee | Manager | Admin ---- */
         .role-label{font-size:10px;font-weight:600;color:var(--text3);letter-spacing:1.5px;text-transform:uppercase;font-family:var(--mono);margin-bottom:10px}
         .role-tabs{display:grid;grid-template-columns:repeat(3,1fr);gap:6px;margin-bottom:28px}
         .role-tab{
             padding:10px 8px;border-radius:10px;border:1.5px solid var(--border);
             background:var(--bg);color:var(--text2);font-family:var(--font);
             font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s;
             display:flex;flex-direction:column;align-items:center;gap:4px;
         }
         .role-tab .role-icon{font-size:16px}
         .role-tab:hover{border-color:var(--border-lit);color:var(--text)}
         .role-tab.active-emp{border-color:var(--blue);background:var(--blue-dim);color:var(--blue)}
         .role-tab.active-mgr{border-color:var(--violet);background:var(--violet-dim);color:var(--violet)}
         .role-tab.active-adm{border-color:var(--orange);background:var(--orange-dim);color:var(--orange)}

         /* ---- AUTH TABS ---- */
         .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-bottom:24px;border:1px solid var(--border)}
         .auth-tab{
             flex:1;padding:9px;border:none;background:transparent;
```

```
            color:var(--text2);font-family:var(--font);font-weight:600;
            font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;
        }
        .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}

        /* ---- FORM ELEMENTS ---- */
        .form-group{margin-bottom:16px}
        .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
        .form-input{
            width:100%;padding:11px 14px;
            background:var(--bg);border:1px solid var(--border);
            border-radius:9px;color:var(--text);
            font-family:var(--font);font-size:14px;
            transition:all 0.2s;
        }
        .form-input:focus{outline:none;border-color:var(--blue);box-shadow:0 0 0 3px rgba(79,140,255,0.08)}
        .form-input::placeholder{color:var(--text3)}
        .form-input.focus-violet:focus{border-color:var(--violet);box-shadow:0 0 0 3px rgba(167,139,250,0.08)}
        .form-input.focus-orange:focus{border-color:var(--orange);box-shadow:0 0 0 3px rgba(245,158,11,0.08)}

        /* ---- BUTTONS ---- */
        .btn-login{
            width:100%;padding:12px;border:none;border-radius:9px;
            font-family:var(--font);font-size:14px;font-weight:600;
            cursor:pointer;transition:all 0.2s;margin-top:4px;
            position:relative;overflow:hidden;
        }
        .btn-login.blue{background:var(--blue);color:#fff}
        .btn-login.blue:hover{background:#6ba0ff;box-shadow:0 6px 20px rgba(79,140,255,0.35);transform:translateY(-1px)}
        .btn-login.violet{background:var(--violet);color:#fff}
        .btn-login.violet:hover{background:#b8a0fc;box-shadow:0 6px 20px rgba(167,139,250,0.35);transform:translateY(-1px)}
        .btn-login.orange{background:var(--orange);color:#fff}
        .btn-login.orange:hover{background:#f7b23b;box-shadow:0 6px 20px rgba(245,158,11,0.35);transform:translateY(-1px)}
        .btn-login:active{transform:translateY(0)}
        .btn-login.loading{opacity:0.7;pointer-events:none}

        .divider{display:flex;align-items:center;gap:12px;margin:20px 0;color:var(--text3);font-size:11px;font-family:var(--mono)}
        .divider::before,.divider::after{content:'';flex:1;height:1px;background:var(--border)}

        .btn-google{
            width:100%;padding:11px;background:transparent;
            border:1px solid var(--border-lit);border-radius:9px;
            color:var(--text2);font-family:var(--font);font-size:13px;font-weight:500;
            cursor:pointer;transition:all 0.2s;
            display:flex;align-items:center;justify-content:center;gap:10px;
        }
        .btn-google:hover{background:rgba(255,255,255,0.04);border-color:rgba(255,255,255,0.2);color:var(--text)}

        /* ---- GOOGLE ROLE SELECT ---- */
        .google-role-row{display:flex;gap:8px;margin-bottom:10px;align-items:center}
        .google-role-label{font-size:11px;color:var(--text3);font-family:var(--mono);white-space:nowrap}
        .google-role-select{
            flex:1;padding:8px 10px;background:var(--bg);
            border:1px solid var(--border);border-radius:8px;
            color:var(--text);font-family:var(--font);font-size:13px;cursor:pointer;
        }
        .google-role-select:focus{outline:none;border-color:var(--border-lit)}
        .google-role-select option{background:var(--panel)}

        /* ---- VENDOR LINK ---- */
        .vendor-link-row{
            margin-top:18px;padding-top:16px;
            border-top:1px solid var(--border);
            display:flex;align-items:center;justify-content:space-between;
        }
        .vendor-link-row p{font-size:12px;color:var(--text3);}
        .btn-vendor-link{
            padding:7px 16px;border-radius:8px;
```

```
            border:1px solid rgba(248,113,113,0.3);
            background:var(--red-dim);color:var(--red);
            font-family:var(--font);font-size:12px;font-weight:600;
            cursor:pointer;text-decoration:none;transition:all 0.2s;
            white-space:nowrap;
        }
        .btn-vendor-link:hover{background:rgba(248,113,113,0.2);border-color:var(--red);}

        /* ---- ERROR / STATUS ---- */
        .status-box{
            border-radius:9px;padding:10px 14px;margin-top:14px;
            font-size:13px;font-weight:500;display:none;
            animation:fadeIn 0.3s ease;
        }
        @keyframes fadeIn{from{opacity:0;transform:translateY(-4px)}to{opacity:1;transform:translateY(0)}}
        .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
        .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
        .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid rgba(79,140,255,0.2)}

        /* ---- FOOTER ---- */
        .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
        .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
        .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
        .tz-select{
            flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);
            border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;
        }
        .tz-select:focus{outline:none;border-color:var(--border-lit)}
        .tz-select option{background:var(--panel)}
        .tz-time{font-size:10px;color:var(--blue);font-family:var(--mono)}

        /* ---- RESPONSIVE ---- */
        @media(max-width:900px){
            .left-panel{display:none}
            .right-panel{width:100%;border-left:none;background:var(--bg)}
        }
    </style>
</head>
<body>
<div id="oauth-loading">
    <div class="spinner"></div>
    <div style="text-align: center;">
        <p style="color: #e0e0e0; font-size: 16px; font-weight: 600; margin: 0 0 6px; font-family: 'Outfit', sans-serif;">Signing you in...</p>
        <p style="color: #8b92a8; font-size: 13px; margin: 0; font-family: 'Outfit', sans-serif;">Verifying your Google account</p>
    </div>
</div>
<div class="bg-layer"></div>
<div class="grid-overlay"></div>

<!-- LEFT HERO -->
<div class="left-panel">
    <div class="brand">
        <div class="brand-logo">
            <div class="brand-icon">E</div>
            <div>
                <div class="brand-name">EMS Portal</div>
                <div class="brand-tagline">Employee Management System</div>
            </div>
        </div>
    </div>
    <div class="hero-text">
        <h1>Your workspace,<br><span class="hl-blue">one login</span><br>away.</h1>
        <p>Employees, Managers, and Admins each get their own tailored experience. Sign in to access your personalized dashboard.</p>
    </div>
    <div class="feature-pills">
        <div class="pill blue"><span class="dot"></span>Employee Portal</div>
        <div class="pill violet"><span class="dot"></span>Manager Console</div>
        <div class="pill orange"><span class="dot"></span>Admin Control Center</div>
        <div class="pill blue"><span class="dot"></span>Leave Management</div>
        <div class="pill violet"><span class="dot"></span>Meal Subscriptions</div>
```

```
        <div class="pill orange"><span class="dot"></span>Delivery Tracking</div>
    </div>
</div>

<!-- RIGHT FORM -->
<div class="right-panel">
    <div class="form-box">

        <!-- ROLE SELECTOR: 3 roles only — Vendor removed -->
        <div class="role-label">Sign in as</div>
        <div class="role-tabs">
            <button class="role-tab active-emp" id="role-emp" onclick="selectRole('EMPLOYEE')">
                <span class="role-icon"></span>Employee
            </button>
            <button class="role-tab" id="role-mgr" onclick="selectRole('MANAGER')">
                <span class="role-icon"></span>Manager
            </button>
            <button class="role-tab" id="role-adm" onclick="selectRole('ADMIN')">
                <span class="role-icon"></span>Admin
            </button>
        </div>

        <!-- AUTH METHOD TABS -->
        <div class="auth-tabs">
            <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
            <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
        </div>

        <!-- JWT FORM -->
        <div id="jwt-form">
            <div class="form-group">
                <label class="form-label">Enter your Email</label>
                <input class="form-input" id="jwt-username" type="text" placeholder="your.email@company.com" autocomplete="off">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="jwt-password" type="password" placeholder="••••••••" autocomplete="current-password">
            </div>
            <button class="btn-login blue" id="jwt-btn" onclick="jwtLogin()">Sign In</button>
        </div>

        <!-- SESSION FORM -->
        <div id="session-form" style="display:none">
            <div class="form-group">
                <label class="form-label">Enter your Email</label>
                <input class="form-input" id="session-username" type="text" placeholder="your.email@company.com">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="session-password" type="password" placeholder="••••••••">
            </div>
            <button class="btn-login blue" id="session-btn" onclick="sessionLogin()">Sign In with Session</button>
        </div>

        <!-- OAUTH — Vendor removed from Google role select -->
        <div class="divider">or continue with</div>
        <div class="google-role-row">
            <span class="google-role-label">Sign in as</span>
            <select class="google-role-select" id="oauth-role">
                <option value="EMPLOYEE">Employee</option>
                <option value="MANAGER">Manager</option>
                <option value="ADMIN">Admin</option>
            </select>
        </div>
        <button class="btn-google" onclick="googleLogin()">
            <svg width="18" height="18" viewBox="0 0 24 24">
                <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04 2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
                <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
                <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>
```

```
                <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
            </svg>
            Continue with Google
        </button>

        <div class="status-box" id="status-box"></div>

        <!-- Vendor Login Link -->
        <div class="vendor-link-row">
            <p>Are you a vendor?</p>
            <a class="btn-vendor-link" href="/vendor-login.html">Vendor Login </a>
        </div>

        <!-- FOOTER -->
        <div class="form-footer">
            <div class="tz-row">
                <span class="tz-label">TZ</span>
                <select class="tz-select" id="tz-select" onchange="updateTz()">
                    <option value="Asia/Kolkata">India — IST UTC+5:30</option>
                    <option value="UTC">UTC</option>
                    <option value="America/New_York">New York — EST UTC-5</option>
                    <option value="America/Los_Angeles">Los Angeles — PST UTC-8</option>
                    <option value="Europe/London">London — GMT UTC+0</option>
                    <option value="Europe/Paris">Paris — CET UTC+1</option>
                    <option value="Asia/Tokyo">Tokyo — JST UTC+9</option>
                    <option value="Asia/Dubai">Dubai — GST UTC+4</option>
                    <option value="Australia/Sydney">Sydney — AEDT UTC+11</option>
                    <option value="Asia/Singapore">Singapore — SGT UTC+8</option>
                </select>
                <span class="tz-time" id="tz-time">--:--:--</span>
            </div>
        </div>

    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    const GOOGLE_CLIENT_ID = '525324511556-s9eugd1o761n6cjje2nr60cv8ho6joq2.apps.googleusercontent.com';
    const REDIRECT_URI = 'http://localhost:8080/google.html';

    // Role dashboard mapping — VENDOR removed from this page
    const DASHBOARDS = {
        EMPLOYEE: '/employee-dashboard.html',
        MANAGER:  '/manager-dashboard.html',
        ADMIN:    '/dashboard.html',
        USER:     '/pending.html'
    };

    let currentRole = 'EMPLOYEE';

    function buildAuthOptions(token, authType) {
        const headers = {};
        if (token) headers.Authorization = `Bearer ${token}`;
        return {
            headers,
            credentials: authType === 'session' ? 'include' : 'same-origin'
        };
    }

    async function resolveRoleFromServer(token, authType) {
        const authOptions = buildAuthOptions(token, authType);

        // Check employee/manager/admin endpoint — vendors are not expected here
        const employeeResponse = await fetch(`${API}/employee/me`, authOptions);
        if (employeeResponse.ok) {
            const employee = await employeeResponse.json();
            return employee.role || 'EMPLOYEE';
        }

        throw new Error('Unable to determine authenticated role');
    }

    async function redirectByRole(token, authType = localStorage.getItem('auth_type')) {
    let role;
```

```

    if (token) {
        role = getRoleFromJWT(token);
    } else {
        try {
            const res = await fetch(`${API}/employee/me`, { credentials: 'include' });
            if (res.ok) {
                const emp = await res.json();
                role = emp.role || 'USER';
            } else {
                role = 'USER';
            }
        } catch(e) {
            role = localStorage.getItem('last_role') || 'USER';
        }
    }

    console.log('Final role for redirect:', role);
    localStorage.setItem('last_role', role);

    // Block ALL vendor types from using the main login page
    if (role === 'FOOD_VENDOR' || role === 'TECH_VENDOR') {
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('auth_type');
        localStorage.removeItem('last_role');
        // Reset button state
        const jwtBtn = document.getElementById('jwt-btn');
        const sesBtn = document.getElementById('session-btn');
        if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
        if (sesBtn) { sesBtn.classList.remove('loading'); sesBtn.textContent = 'Sign In with Session'; }
        showStatus('error', 'Vendors must use the Vendor Login page.');
        return;
    }

    const destination = DASHBOARDS[role];
    if (!destination) {
        // Unknown role — don't redirect anywhere, just show error
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('auth_type');
        showStatus('error', 'Unknown role: ' + role + '. Please contact your administrator.');
        const jwtBtn = document.getElementById('jwt-btn');
        if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
        return;
    }

    console.log('Redirecting to:', destination);
    window.location.href = destination;
}

    // ---- Role selector (3 tabs: Employee | Manager | Admin) ----
    function selectRole(role) {
        currentRole = role;
        document.getElementById('role-emp').className = 'role-tab';
        document.getElementById('role-mgr').className = 'role-tab';
        document.getElementById('role-adm').className = 'role-tab';

        const classMap = { EMPLOYEE: 'active-emp', MANAGER: 'active-mgr', ADMIN: 'active-adm' };
        const idMap    = { EMPLOYEE: 'role-emp',   MANAGER: 'role-mgr',   ADMIN: 'role-adm'   };
        document.getElementById(idMap[role]).classList.add(classMap[role]);

        document.getElementById('oauth-role').value = role;

        const colorMap = { EMPLOYEE: 'blue', MANAGER: 'violet', ADMIN: 'orange' };
        const col = colorMap[role];
        document.getElementById('jwt-btn').className     = `btn-login ${col}`;
        document.getElementById('session-btn').className = `btn-login ${col}`;

        document.querySelectorAll('.form-input').forEach(inp => {
            inp.className = (`form-input focus-${col === 'blue' ? '' : col}`).trim();
        });
    }

    // ---- Auth tab switching ----
    function switchTab(type) {
        document.getElementById('jwt-form').style.display     = type === 'jwt'     ? 'block' : 'none';
        document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
```

```
        document.getElementById('tab-jwt').classList.toggle('active',     type === 'jwt');
        document.getElementById('tab-session').classList.toggle('active', type === 'session');
        hideStatus();
    }

    // ---- Status display ----
    function showStatus(type, msg) {
        const box = document.getElementById('status-box');
        box.className     = `status-box ${type}`;
        box.textContent   = msg;
        box.style.display = 'block';
    }
    function hideStatus() {
        document.getElementById('status-box').style.display = 'none';
    }

    // ---- JWT Login ----
    async function jwtLogin() {
        const username = document.getElementById('jwt-username').value.trim();
        const password = document.getElementById('jwt-password').value;
        if (!username || !password) { showStatus('error', 'Please enter your username and password');
return; }

        const btn = document.getElementById('jwt-btn');
        btn.classList.add('loading'); btn.textContent = 'Signing in...';
        showStatus('loading', 'Authenticating...');

        try {
            const res = await fetch(`${API}/Authenticate`, {
                method:  'POST',
                headers: { 'Content-Type': 'application/json' },
                body:    JSON.stringify({ username, password })
            });
            if (res.ok) {
                const data = await res.json();
                localStorage.setItem('jwt_token',     data.accessToken);
                localStorage.setItem('refresh_token', data.refreshToken || '');
                localStorage.setItem('auth_type',     'jwt');
                localStorage.setItem('last_role',     currentRole);
                localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
                showStatus('success', 'Login successful! Redirecting...');
                setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
            } else {
                const err = await res.text();
                showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
                btn.classList.remove('loading');
                btn.textContent = 'Sign In';
            }
        } catch(e) {
            showStatus('error', 'Connection error: ' + e.message);
            btn.classList.remove('loading'); btn.textContent = 'Sign In';
        }
    }

    // ---- Session Login ----
    async function sessionLogin() {
        const username = document.getElementById('session-username').value.trim();
        const password = document.getElementById('session-password').value;
        if (!username || !password) { showStatus('error', 'Please enter your username and password');
return; }

        const btn = document.getElementById('session-btn');
        btn.classList.add('loading'); btn.textContent = 'Signing in...';
        showStatus('loading', 'Authenticating...');

        try {
            const res = await fetch(`${API}/session/login`, {
                method:      'POST',
                headers:     { 'Content-Type': 'application/json' },
                body:        JSON.stringify({ username, password }),
                credentials: 'include'
            });
            if (res.ok) {
                localStorage.setItem('auth_type',     'session');
                localStorage.setItem('last_role',     currentRole);
                localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
```

```
                showStatus('success', 'Login successful! Redirecting...');
                setTimeout(() => redirectByRole(null, 'session'), 600);
            } else {
                const err = await res.json().catch(() => ({}));
                showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
                btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
            }
        } catch(e) {
            showStatus('error', 'Connection error: ' + e.message);
            btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
        }
    }

    // ---- Google OAuth — goes through backend /auth/google/init ----
    function googleLogin() {
        const role     = document.getElementById('oauth-role').value;
        const timezone = document.getElementById('tz-select').value;
        window.location.href =
            `/auth/google/init` +
            `?role=${encodeURIComponent(role)}` +
            `&timezone=${encodeURIComponent(timezone)}`;
    }

   function getRoleFromJWT(token) {
    try {
        const payload = JSON.parse(atob(token.split('.')[1]));
        console.log('JWT payload:', payload);
        const role = payload.role || 'USER';
        // Strip ROLE_ prefix if accidentally present
        return typeof role === 'string' ? role.replace(/^ROLE_/, '') : 'USER';
    } catch (e) {
        console.error('Failed to parse JWT:', e);
        return 'USER';
    }
  }

    // ---- Spring redirects back here with tokens as query params ----
    async function handleOAuthCallback() {
    const params       = new URLSearchParams(window.location.search);
    const accessToken  = params.get('accessToken');
    const refreshToken = params.get('refreshToken');

    if (!accessToken) return;

    window.history.replaceState({}, document.title, window.location.pathname);

    localStorage.setItem('jwt_token',     accessToken);
    localStorage.setItem('refresh_token', refreshToken || '');
    localStorage.setItem('auth_type',     'jwt');

    const role = getRoleFromJWT(accessToken);
    console.log('Authenticated role:', role);
    localStorage.setItem('last_role', role);

    showStatus('success', 'Signed in! Redirecting...');
    setTimeout(() => {
        window.location.href = DASHBOARDS[role] || '/google.html';
    }, 800);
}

    // ---- Timezone ----
    function updateTz() {
        localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
        refreshTzClock();
    }
    function refreshTzClock() {
        const tz = document.getElementById('tz-select').value;
        document.getElementById('tz-time').textContent =
            new Date().toLocaleTimeString('en-GB', { timeZone: tz, hour12: false });
    }

    // ---- Enter key support ----
    document.addEventListener('keydown', e => {
        if (e.key !== 'Enter') return;
        if (document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
        else sessionLogin();
```

```
    });

    // ---- Init ----
    window.onload = () => {
        const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
        document.getElementById('tz-select').value = savedTz;
        refreshTzClock();
        setInterval(refreshTzClock, 1000);

        const params      = new URLSearchParams(window.location.search);
        const accessToken = params.get('accessToken');
        if (accessToken) {
            document.getElementById('oauth-loading').classList.add('visible');
            document.querySelector('.right-panel').style.display = 'none';
            document.querySelector('.left-panel').style.display  = 'none';
            handleOAuthCallback();
        }
    };
</script>
</body>
</html>
```

143 Current: src\main\resources\static\login.html

10 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title</title>
</head>
<body>
<h1>hi</h1>
</body>
 </html>
```

144 Current: src\main\resources\static\manager-dashboard.html 976 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Manager Console</title>
    <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700&family=IBM+Plex+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
            --bg:#0a0b0f;--bg2:#0f1117;--bg3:#14171f;
            --panel:#171c28;--panel2:#1d2336;
            --border:rgba(167,139,250,0.08);--border2:rgba(167,139,250,0.18);
            --text:#e8e0f5;--text2:#8b82a8;--text3:#504d68;
            --accent:#a78bfa;--accent-dim:rgba(167,139,250,0.1);
            --green:#34d399;--green-dim:rgba(52,211,153,0.1);
            --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
            --red:#f87171;--red-dim:rgba(248,113,113,0.1);
            --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
            --font:'Plus Jakarta Sans',sans-serif;--mono:'IBM Plex Mono',monospace;
            --sidebar:250px;
        }
        body{font-family:var(--font);background:var(--bg);color:var(--text);font-size:14px;overflow:hidden;height:100vh}
        .app{display:flex;height:100vh}

        /* SIDEBAR */
        .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--border);display:flex;flex-direction:column;flex-shrink:0;position:relative;overflow:hidden}
        .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:200px;background:linear-gradient(to top,rgba(167,139,250,0.04),transparent);pointer-events:none}
        .logo{padding:22px 20px;border-bottom:1px solid var(--border);position:relative}
        .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
        .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,var(--accent),#7c3aed);border-radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
        .logo h2{font-size:16px;font-weight:700;color:var(--text)}
        .logo p{font-size:11px;color:var(--text3);font-family:var(--mono)}
        .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:linear-gradient(90deg,rgba(167,139,250,0.15),rgba(124,58,237,0.08));border:1px solid rgba(167,139,250,...
        .role-dot{width:6px;height:6px;background:var(--accent);border-radius:50%;animation:glow 2s infinite}
        @keyframes glow{0%,100%{box-shadow:0 0 4px var(--accent)}50%{box-shadow:0 0 10px var(--accent),0 0 20px rgba(167,139,250,0.4)}}
        .nav{flex:1;padding:10px 0;overflow-y:auto}
        .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-spacing:2px;text-transform:uppercase}
        .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
        .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
        .nav-item.active{background:linear-gradient(90deg,var(--accent-dim),rgba(124,58,237,0.06));color:var(--accent);border:1px solid rgba(167,139,250,0.2)}
        .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:10px;font-weight:700;padding:2px 7px;border-radius:10px;min-width:18px;text-align:center}
        .nav-badge.hidden{display:none}
        .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
        .user-card{display:flex;align-items:center;gap:10px}
        .avatar{width:34px;height:34px;background:linear-gradient(135deg,#7c3aed,var(--accent));border-radius:50%;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
        .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
        .user-info-side .role-text{font-size:10px;color:var(--accent);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px}

        /* MAIN */
        .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
        .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
        .page-title{font-size:16px;font-weight:700;color:var(--text)}
        .topbar-right{display:flex;align-items:center;gap:12px}
        .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
```

```
.logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
.content{flex:1;overflow-y:auto;padding:24px}
.view{display:none}
.view.active{display:block;animation:fadeIn 0.3s ease}
@keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}

/* STATS */
.stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-bottom:24px}
.stat-card{background:var(--panel);border:1px solid var(--border);border-radius:14px;padding:20px;position:relative;overflow:hidden;transition:all 0.2s;cursor:default}
.stat-card:hover{border-color:var(--border2);transform:translateY(-1px)}
.stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:3px}
.stat-card.purple::before{background:linear-gradient(90deg,#7c3aed,var(--accent))}
.stat-card.green::before{background:var(--green)}
.stat-card.amber::before{background:var(--amber)}
.stat-card.blue::before{background:var(--blue)}
.stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-bottom:10px;font-weight:600}
.stat-value{font-size:30px;font-weight:700;color:var(--text);font-family:var(--mono);line-height:1}
.stat-sub{font-size:11px;color:var(--text3);margin-top:6px}

/* SECTION */
.section{background:var(--panel);border:1px solid var(--border);border-radius:14px;margin-bottom:20px;overflow:hidden}
.section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center}
.section-title{font-size:14px;font-weight:700;color:var(--text)}
.section-sub{font-size:12px;color:var(--text3);margin-top:2px}

/* TABLE */
.tablewrap{overflow-x:auto}
table{width:100%;border-collapse:collapse}
th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border)}
td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
tr:last-child td{border-bottom:none}
tr:hover td{background:rgba(167,139,250,0.03);color:var(--text)}

/* CHIPS */
.chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-size:11px;font-weight:600}
.chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
.chip.approved{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
.chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
.chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
.chip.employee{background:var(--blue-dim);color:var(--blue)}
.chip.manager{background:var(--accent-dim);color:var(--accent)}

/* BUTTONS */
.btn{padding:8px 16px;border-radius:8px;border:none;font-family:var(--font);font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s}
.btn-primary{background:var(--accent);color:#fff}
.btn-primary:hover{background:#7c3aed;box-shadow:0 4px 12px rgba(167,139,250,0.3)}
.btn-approve{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
.btn-approve:hover{background:var(--green);color:#fff}
.btn-reject{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
.btn-reject:hover{background:var(--red);color:#fff}
.btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
.btn-secondary:hover{border-color:var(--border2);color:var(--text)}
.btn-sm{padding:5px 12px;font-size:11px}
.action-row{display:flex;gap:6px}

/* FORM */
.form-card{padding:24px}
.form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
.form-group{display:flex;flex-direction:column;gap:6px}
.form-group.full{grid-column:1/-1}
label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px}
input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
input::placeholder{color:var(--text3)}
select option{background:var(--bg2)}
.form-actions{display:flex;gap:10px;margin-top:20px}
```

```
122
123         /* ALERT */
124         .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
125         .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
126         .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
127         @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
128         .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
129
130         /* GRID DASHBOARD */
131         .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px;margin-bottom:0}
132
133         /* SEARCH */
134         .search-bar{display:flex;align-items:center;gap:10px;margin-bottom:16px}
135         .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:9px 14px;color:var(--text);font-family:var(--font);font-size:13px}
136         .search-input:focus{outline:none;border-color:var(--accent)}
137         .search-input::placeholder{color:var(--text3)}
138     </style>
139 </head>
140 <body>
141 <div class="app">
142     <!-- SIDEBAR -->
143     <div class="sidebar">
144         <div class="logo">
145             <div class="logo-row">
146                 <div class="logo-icon">M</div>
147                 <h2>EMS Console</h2>
148             </div>
149             <p>Management Portal</p>
150             <div class="role-badge"><span class="role-dot"></span>Manager</div>
151         </div>
152         <nav class="nav">
153             <div class="nav-section">Overview</div>
154             <div class="nav-item active" onclick="showView('dashboard',this)">
155                 <span></span> Dashboard
156             </div>
157             <div class="nav-section">Team</div>
158             <div class="nav-item" onclick="showView('employees',this)"><span></span> All Employees</div>
159             <div class="nav-item" onclick="showView('add-employee',this)"><span></span> Add Employee</div>
160             <div class="nav-item" onclick="showView('promote-users',this)"><span></span> Promote Users
<span class="nav-badge" id="users-badge">0</span></div>
161             <div class="nav-section">Leave Management</div>
162             <div class="nav-item" onclick="showView('pending-leaves',this)">
163                 <span></span> Pending Approvals
164                 <span class="nav-badge" id="pending-badge">0</span>
165             </div>
166             <div class="nav-item" onclick="showView('all-leaves',this)"><span></span> All Leave
Requests</div>
167             <div class="nav-section">Personal</div>
168             <div class="nav-item" onclick="showView('apply-leave',this)"><span></span> Apply Leave</div>
169             <div class="nav-item" onclick="showView('my-leaves',this)"><span></span> My Leaves</div>
170             <div class="nav-item" onclick="showView('my-subscriptions',this)"><span></span> My
Subscriptions</div>
171             <div class="nav-item" onclick="showView('add-subscription',this)"><span></span> Subscribe to
Meal</div>
172             <div class="nav-item" onclick="showView('restaurants',this)"><span></span> Restaurants</div>
173         </nav>
174         <div class="sidebar-footer">
175             <div class="user-card">
176                 <div class="avatar" id="avatar-initials">M</div>
177                 <div class="user-info-side">
178                     <div class="name" id="sidebar-name">Manager</div>
179                     <div class="role-text">Manager</div>
180                 </div>
181             </div>
182         </div>
183     </div>
184
185     <!-- MAIN -->
186     <div class="main">
187         <div class="topbar">
188             <div class="page-title" id="pageTitle">Dashboard</div>
189             <div class="topbar-right">
190                 <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
```

```
email"></span>
                <button class="logout-btn" onclick="logout()">Sign Out</button>
            </div>
        </div>
        <div class="content">

            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card purple">
                        <div class="stat-label">Total Employees</div>
                        <div class="stat-value" id="stat-total">--</div>
                        <div class="stat-sub">in your organization</div>
                    </div>
                    <div class="stat-card green">
                        <div class="stat-label">Active Now</div>
                        <div class="stat-value" id="stat-active">--</div>
                        <div class="stat-sub">currently working</div>
                    </div>
                    <div class="stat-card amber">
                        <div class="stat-label">Pending Approvals</div>
                        <div class="stat-value" id="stat-pending">--</div>
                        <div class="stat-sub">need your action</div>
                    </div>
                    <div class="stat-card blue">
                        <div class="stat-label">On Leave Today</div>
                        <div class="stat-value" id="stat-on-leave">--</div>
                        <div class="stat-sub">employees absent</div>
                    </div>
                </div>
                <div class="dash-grid">
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title"> Needs Attention</div><div class="section-
sub">Pending leave requests</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('pending-
leaves',null)">View All </button>
                        </div>
                        <div class="tablewrap">
                            <table>

<thead><tr><th>Employee</th><th>Type</th><th>Dates</th><th>Action</th></tr></thead>
                                <tbody id="dash-pending"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title"> Team Overview</div><div class="section-
sub">Recent employees</div></div>
                            <button class="btn btn-secondary btn-sm"
onclick="showView('employees',null)">View All </button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Name</th><th>Dept</th><th>Status</th></tr></thead>
                                <tbody id="dash-employees"><tr><td colspan="3" class="empty-
state">Loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                </div>
            </div>

            <!-- ALL EMPLOYEES -->
            <div id="employees-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">All Employees</div><div class="section-sub">Manage
your team</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadEmployees()">
Refresh</button>
                    </div>
                    <div style="padding:16px 20px;border-bottom:1px solid var(--border)">
                        <div class="search-bar">
```

```
                            <input class="search-input" id="emp-search" placeholder="Search by name, email or department..." oninput="filterEmployees()">
                        </div>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Actions</th></tr></thead>
                            <tbody id="employees-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div   >
            </div>

            <!-- ADD EMPLOYEE -->
            <div id="add-employee-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">Add New Employee</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group"><label>Full Name</label><input id="emp-name" placeholder="John Doe"></div>
                            <div class="form-group"><label>Email</label><input type="email" id="emp-email" placeholder="john@company.com"></div>
                            <div class="form-group"><label>Password</label><input type="password" id="emp-pass" placeholder="Temporary password"></div>
                            <div class="form-group"><label>Timezone</label>
                                <select id="emp-timezone">
                                    <option value="UTC">UTC</option>
                                    <option value="Asia/Kolkata">Asia/Kolkata (IST)</option>
                                    <option value="America/New_York">America/New_York (EST)</option>
                                    <option value="America/Los_Angeles">America/Los_Angeles (PST)</option>
                                    <option value="Europe/London">Europe/London (GMT)</option>
                                    <option value="Asia/Singapore">Asia/Singapore (SGT)</option>
                                </select>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="addEmployee()">Add Employee</button>
                            <button class="btn btn-secondary" onclick="clearEmpForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- PROMOTE USERS -->
            <div id="promote-users-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title"> Promote Users</div><div class="section-sub">Onboard pending user registrations to your department</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadUsers()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Name</th><th>Email</th><th>Timezone</th><th>Temp Password</th><th>Action</th></tr></thead>
                            <tbody id="users-list"><tr><td colspan="5" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- PENDING LEAVE APPROVALS -->
            <div id="pending-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title"> Pending Leave Approvals</div><div class="section-sub">Review and take action</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">Refresh</button>
                    </div>
                    <div class="tablewrap">
```

```
                        <table>
<thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
                            <tbody id="pending-leaves-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>



            <!-- ALL LEAVES -->
            <div id="all-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">All Leave Requests</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
                            <tbody id="all-leaves-list"><tr><td colspan="7" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- APPLY LEAVE -->
            <div id="apply-leave-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div>
                            <div class="section-title"> Apply for Leave</div>
                            <div class="section-sub">Submit a new leave request</div>
                        </div>
                    </div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group">
                                <label>Leave Type</label>
                                <select id="leave-type">
                                    <option value="SICK">Sick Leave</option>
                                    <option value="CASUAL">Casual Leave</option>
                                </select>
                            </div>
                            <div class="form-group">
                                <label>Reason</label>
                                <input id="leave-reason" placeholder="Brief reason for leave">
                            </div>
                            <div class="form-group">
                                <label>Start Date</label>
                                <input type="date" id="leave-start">
                            </div>
                            <div class="form-group">
                                <label>End Date</label>
                                <input type="date" id="leave-end">
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
                            <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- MY LEAVES -->
            <div id="my-leaves-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div class="section-title">My Leave Requests</div>
```

```
                        <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>

<thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Status</th></tr></thead>
                            <tbody id="my-leaves-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- MY SUBSCRIPTIONS -->
            <div id="my-subscriptions-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title"> My Subscriptions</div><div class="section-
sub">Your active meal subscriptions</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadMySubscriptions()">
Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>

<thead><tr><th>Slot</th><th>Restaurant</th><th>Address</th><th>Schedule</th><th>Day</th><th>Status</th><th>Next
Delivery</th></tr></thead>
                            <tbody id="my-subscriptions-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- ADD SUBSCRIPTION -->
            <div id="add-subscription-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title"> Subscribe to Meal</div><div class="section-
sub">Add a new meal slot subscription</div></div>
                    </div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group">
                                <label>Schedule Type</label>
                                <select id="sub-schedule" onchange="toggleDayOfWeek()">
                                    <option value="DAILY">Daily</option>
                                    <option value="WEEKLY">Weekly</option>
                                </select>
                            </div>
                            <div class="form-group" id="day-of-week-group" style="display:none">
                                <label>Day of Week</label>
                                <select id="sub-day">
                                    <option value="MONDAY">Monday</option>
                                    <option value="TUESDAY">Tuesday</option>
                                    <option value="WEDNESDAY">Wednesday</option>
                                    <option value="THURSDAY">Thursday</option>
                                    <option value="FRIDAY">Friday</option>
                                    <option value="SATURDAY">Saturday</option>
                                    <option value="SUNDAY">Sunday</option>
                                </select>
                            </div>
                        </div>


                        <!-- Slot Orders -->
                        <div style="margin-top:20px">
                            <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:12px">
                                <label style="font-size:11px;font-weight:700;color:var(--text3);letter-
spacing:0.8px;text-transform:uppercase">Meal Slots & Restaurants</label>
                                <button class="btn btn-secondary btn-sm" onclick="addSlotRow()"> Add
Slot</button>
                            </div>
                            <div id="slot-rows"></div>
                        </div>
```

```
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="submitSubscription()">Subscribe</button>
                            <button class="btn btn-secondary" onclick="clearSubscriptionForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>
            <!-- RESTAURANTS -->
            <div id="restaurants-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title"> Restaurants</div><div class="section-sub">All available restaurants</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>ID</th><th>Name</th><th>Address</th><th>Status</th></tr></thead>
                            <tbody id="restaurants-list"><tr><td colspan="4" class="empty-state">Loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

        </div>
    </div>
</div>

<script>
    const API='http://localhost:8080';
    let allEmployeesCache=[];

    (function(){
        const token=localStorage.getItem('jwt_token');
        const authType=localStorage.getItem('auth_type');
        if(!token&&authType!=='session'){window.location.href='/login.html';return;}
        if(token){
            try{
                const p=JSON.parse(atob(token.split('.')[1]));
                const rawRole = p.role || p.roles || p.authorities || '';
                const normalizedRoles = Array.isArray(rawRole)
                    ? rawRole.map(v => String(v).replace(/^ROLE_/, ''))
                    : [String(rawRole).replace(/^ROLE_/, '')];
                if(normalizedRoles.includes('MANAGER')) { initUser(); return; }
                if(normalizedRoles.includes('EMPLOYEE')){window.location.href='/employee-dashboard.html';return;}
                if(normalizedRoles.includes('VENDOR')){window.location.href='/vendor-dashboard.html';return;}
            }catch(e){}
        }
        initUser();
    })();

    function getAuthHeader(){
        const t=localStorage.getItem('jwt_token');
        return t?{'Authorization':`Bearer ${t}`}:{};
    }
    async function apiFetch(path,opts={}){
        const res=await fetch(`${API}${path}`,{...opts,headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},credentials:'include'});
if(res.status===401){localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');showAlert('Session expired','error');setTimeout(()=>window.location.href='/login.html',1500);t...
        const text=await res.text();return text?JSON.parse(text):null;
    }

    async function initUser(){
        const token=localStorage.getItem('jwt_token');
        let email='';
        if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
        document.getElementById('avatar-initials').textContent=(email||'M').charAt(0).toUpperCase();
```

```
        document.getElementById('sidebar-name').textContent=email||'Manager';
        document.getElementById('topbar-email').textContent=email;
        loadDashboard();
        loadUsers();
    }

    function showView(name,el){
        document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
        document.getElementById(`${name}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
        if(el)el.classList.add('active');
        document.getElementById('pageTitle').textContent=name.split('-').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
       const loaders={dashboard:loadDashboard,employees:loadEmployees,'pending-leaves':loadPendingLeaves,'all-leaves':loadAllLeaves,'my-leaves':loadMyLeaves,'add-employee':()=>{},'promote-users':lo...
        if(loaders[name])loaders[name]();
    }

    async function loadDashboard(){
    try{
        const [emps, leaves] = await Promise.all([
            apiFetch('/manager/employees'),
            apiFetch('/manager/pending')
        ]);

        // Stats
        document.getElementById('stat-total').textContent = emps?.length || 0;
        document.getElementById('stat-active').textContent = emps?.filter(e => e.status === 'ACTIVE').length || 0;
        document.getElementById('stat-on-leave').textContent = emps?.filter(e => e.status === 'ON_LEAVE').length || 0;
        const pending = (leaves || []).filter(l => l.status === 'PENDING');
        document.getElementById('stat-pending').textContent = pending.length;
        document.getElementById('pending-badge').textContent = pending.length;
        document.getElementById('pending-badge').classList.toggle('hidden', pending.length === 0);

        // Build employeeId  name lookup from employees list
        const empMap = {};
        (emps || []).forEach(e => { empMap[e.employeeId] = e.name || '—'; });

        // Dash pending
        document.getElementById('dash-pending').innerHTML = pending.length
            ? pending.slice(0, 5).map(l => {
                const leaveId = l.leaveRequestId || 0;
                const empName = empMap[l.employeeId] || `#${l.employeeId}`;
                return `<tr>
                    <td>${empName}</td>
                    <td><span class="chip pending">${l.leaveType}</span></td>
                    <td>${l.startDate}  ${l.endDate}</td>
                    <td><div class="action-row">
                        <button class="btn btn-approve btn-sm" onclick="approveLeave(${leaveId})"></button>
                        <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})"></button>
                    </div></td>
                </tr>`;
            }).join('')
            : '<tr><td colspan="4" class="empty-state">No pending requests </td></tr>';

        // Dash employees
        document.getElementById('dash-employees').innerHTML = emps?.length
            ? emps.slice(0, 6).map(e => `<tr>
                <td>${e.name || '—'}</td>
                <td>${e.dept || '—'}</td>
                <td><span class="chip ${(e.status || '').toLowerCase()}">${e.status || '—'}</span></td>
            </tr>`).join('')
            : '<tr><td colspan="3" class="empty-state">No employees</td></tr>';

    } catch(e) { console.error(e); }
}

    async function loadEmployees(){
        try{
            const emps=await apiFetch('/manager/employees');
            allEmployeesCache=emps||[];
            renderEmployees(allEmployeesCache);
        }catch(e){document.getElementById('employees-list').innerHTML='<tr><td colspan="7" class="empty-
```

```
state">Could not load employees</td></tr>';}
    }

    function renderEmployees(emps){
        document.getElementById('employees-list').innerHTML=emps.length
            ? emps.map(e=>`<tr><td style="font-family:var(--mono);font-size:12px">#${e.employeeId}</td><td>${e.name||'—'}</td><td style="font-family:var(--mono);font-size:12px">${e.email||'—'}</td>...
            : '<tr><td colspan="7" class="empty-state">No employees found</td></tr>';
    }

    function filterEmployees(){
        const q=document.getElementById('emp-search').value.toLowerCase();
renderEmployees(allEmployeesCache.filter(e=>(e.name||'').toLowerCase().includes(q)||(e.email||'').toLowerCase().includes(q)||(e.dept||'').toLowerCase().includes(q)));
    }

    async function addEmployee(){
    const name     = document.getElementById('emp-name').value.trim();
    const email    = document.getElementById('emp-email').value.trim();
    const password = document.getElementById('emp-pass').value;
    const timezone = document.getElementById('emp-timezone').value;
    if(!name || !email || !password){
        showAlert('Please fill all required fields','error');
        return;
    }
    try{
        await apiFetch('/manager/employees', {
            method: 'POST',
            body: JSON.stringify({ name, email, password, timezone })
        });
        showAlert('Employee added successfully!','success');
        clearEmpForm();
        loadEmployees();
    }catch(e){ showAlert('Failed: '+e.message,'error'); }
    }

   function clearEmpForm(){
    ['emp-name','emp-email','emp-pass'].forEach(id=>document.getElementById(id).value='');
    document.getElementById('emp-timezone').value='UTC';
   }
    async function deleteEmployee(id){
        if(!confirm('Remove this employee?'))return;
        try{await apiFetch(`/employee/${id}`,{method:'DELETE'});showAlert('Employee removed','success');loadEmployees();}
        catch(e){showAlert('Failed to remove employee','error');}
    }

    async function loadPendingLeaves(){
    try{
        const leaves = await apiFetch('/manager/pending');
        console.log('Leave sample:', leaves?.[0]); // remove after confirming
        const pending=(leaves||[]).filter(l=>l.status==='PENDING');
        document.getElementById('pending-leaves-list').innerHTML=pending.length
            ? pending.map(l=>{
                // Try all possible ID field names your DTO might use
                const leaveId = l.leaveRequestId || l.leaveId || l.id || 0;
                return `<tr>
                    <td>${l.employeeName||l.employeeId||'—'}</td>
                    <td><span class="chip pending">${l.leaveType}</span></td>
                    <td>${l.startDate}</td>
                    <td>${l.endDate}</td>
                    <td>${l.numberOfDays||'—'}</td>
                    <td>${(l.reason||'—').substring(0,40)}</td>
                    <td><div class="action-row">
                        <button class="btn btn-approve btn-sm" onclick="approveLeave(${leaveId})">Approve</button>
                        <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})">Reject</button>
                    </div></td>
                </tr>`;
            }).join('')
            : '<tr><td colspan="7" class="empty-state">No pending approvals </td></tr>';
    }catch(e){
        document.getElementById('pending-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
```

```
state">Could not load</td></tr>';
    }
}
    async function approveLeave(id) {
    console.log('Approving leaveRequestId:', id); // verify this is not 0
    try {
        const res = await fetch(`${API}/leave_requests/approval`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
            credentials: 'include',
            body: JSON.stringify({
                leaveRequestId: id,   //  matches ActionDTO exactly
                action: 'APPROVED',   //  check if your service expects uppercase or lowercase
                remarks: ''
            })
        });
        const text = await res.text();
        if (res.ok) {
            showAlert('Leave approved ', 'success');
            loadPendingLeaves(); loadDashboard(); loadAllLeaves();
        } else {
            showAlert('Error: ' + text, 'error');
        }
    } catch(e) { showAlert(e.message, 'error'); }
}

async function rejectLeave(id) {
    console.log('Rejecting leaveRequestId:', id); // verify this is not 0
    try {
        const res = await fetch(`${API}/leave_requests/approval`, {
            method: 'PUT',
            headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
            credentials: 'include',
            body: JSON.stringify({
                leaveRequestId: id,   //  matches ActionDTO exactly
                action: 'REJECTED',   //  check if your service expects uppercase or lowercase
                remarks: ''
            })
        });
        const text = await res.text();
        if (res.ok) {
            showAlert('Leave rejected', 'success');
            loadPendingLeaves(); loadDashboard(); loadAllLeaves();
        } else {
            showAlert('Error: ' + text, 'error');
        }
    } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
   }

    async function loadAllLeaves(){
        try{
            const leaves=await apiFetch('/manager/requests');
            document.getElementById('all-leaves-list').innerHTML=(leaves||[]).length
                ? (leaves).map(l=>`<tr>
    <td>${l.employeeName||'—'}</td>
    <td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType||'—'}</span></td>
    <td>${l.startDate||'—'}</td>
    <td>${l.endDate||'—'}</td>
    <td>${(l.reason||'—').substring(0,50)}</td>
    <td><span class="chip ${(l.status||'').toLowerCase()}">${l.status||'—'}</span></td>
    </tr>`)
    : '<tr><td colspan="7" class="empty-state">No leave requests</td></tr>';
        }catch(e){document.getElementById('all-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';}
    }

    async function applyLeave() {
    const leaveType  = document.getElementById('leave-type').value;
    const reason     = document.getElementById('leave-reason').value.trim();
    const startDate  = document.getElementById('leave-start').value;
    const endDate    = document.getElementById('leave-end').value;

    if (!reason || !startDate || !endDate) {
        showAlert('Please fill all fields', 'error');
        return;
    }
```

```
    if (endDate < startDate) {
        showAlert('End date cannot be before start date', 'error');
        return;
    }

    try {
        const res = await fetch(`${API}/leave_requests`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
            credentials: 'include',
            body: JSON.stringify({ leaveType, reason, startDate, endDate })
        });

        if (res.ok) {
            showAlert('Leave request submitted successfully!', 'success');
            clearLeaveForm();
            loadMyLeaves();
        } else {
            const text = await res.text();
            showAlert('Failed: ' + text, 'error');
        }
    } catch(e) {
        showAlert('Error: ' + e.message, 'error');
    }
}

function clearLeaveForm() {
    document.getElementById('leave-type').value  = 'SICK';
    document.getElementById('leave-reason').value = '';
    document.getElementById('leave-start').value  = '';
    document.getElementById('leave-end').value    = '';
}

    async function loadMyLeaves(){
        try{
            const data=await apiFetch('/leave_requests/employee');
            const leaves=data?.requests||data||[];
            document.getElementById('my-leaves-list').innerHTML=leaves.length
                ? leaves.map(l=>`<tr><td><span class="chip ${(l.leaveType||'').toLowerCase()}">${l.leaveType}</span></td><td>${l.startDate}</td><td>${l.endDate}</td><td>${l.numberOfDays||'—'}</td><...
                : '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
        }catch(e){document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="5" class="empty-state">Could not load</td></tr>';}
    }

    async function loadUsers(){
    try{
        const users = await apiFetch('/manager/Users');
        const badge = document.getElementById('users-badge');
        if(badge){ badge.textContent = (users||[]).length; badge.classList.toggle('hidden',(users||[]).length===0); }
        document.getElementById('users-list').innerHTML = (users||[]).length
            ? users.map(u=>`<tr>
                <td>${u.name||'—'}</td>
                <td style="font-family:var(--mono);font-size:12px">${u.email||'—'}</td>
                <td style="font-family:var(--mono);font-size:12px">${u.timezone||'—'}</td>
                <td><input type="password" placeholder="Set password" id="pwd-${CSS.escape(u.email)}"
                    style="padding:6px 10px;font-size:12px;width:160px"></td>
                <td><button class="btn btn-approve btn-sm" onclick="promoteUser('${u.email}')">Promote</button></td>
            </tr>`).join('')
            : '<tr><td colspan="5" class="empty-state">No pending user registrations</td></tr>';
    }catch(e){
        document.getElementById('users-list').innerHTML='<tr><td colspan="5" class="empty-state">Could not load</td></tr>';
    }
}

async function promoteUser(email){
    const pwdInput = document.getElementById('pwd-'+CSS.escape(email));
    const password = pwdInput?.value?.trim();
    if(!password){ showAlert('Please set a password before promoting','error'); return; }
    try{
        const res = await fetch(`${API}/manager/promote`,{
            method:'PUT',
```

```
            headers:{'Content-Type':'application/json',...getAuthHeader()},
            credentials:'include',
            body: JSON.stringify({ email, password })
        });
        const text = await res.text();
        if(res.ok){
            showAlert(`${email} promoted to Employee `,'success');
            loadUsers();
        } else {
            showAlert('Error: '+text,'error');
        }
    }catch(e){ showAlert('Failed: '+e.message,'error'); }
}
    async function loadRestaurants(){
        try{
            const data = await apiFetch('/restaurants');
            document.getElementById('restaurants-list').innerHTML = (data||[]).length
                ? data.map(r=>`<tr>
                <td style="font-family:var(--mono);font-size:12px">#${r.restaurantId||r.id}</td>
                <td>${r.name||'—'}</td>
                <td style="font-size:12px;color:var(--text3)">${r.address||'—'}</td>
                <td><span class="chip active">${r.status||'ACTIVE'}</span></td>
            </tr>`).join('')
                : '<tr><td colspan="4" class="empty-state">No restaurants found</td></tr>';
        }catch(e){
            document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-state">Could not load</td></tr>';
        }
    }

//  SUBSCRIPTIONS

async function loadMySubscriptions(){
    try{
        const subs = await apiFetch('/subscription/user');
        document.getElementById('my-subscriptions-list').innerHTML = (subs||[]).length
            ? subs.map(s=>`<tr>
                <td><span class="chip pending">${s.mealSlot||'—'}</span></td>
                <td>${s.restaurantName||'—'}</td>
                <td style="font-size:12px;color:var(--text3)">${s.restaurantAddress||'—'}</td>
                <td>${s.scheduleType||'—'}</td>
                <td>${s.dayOfWeek||'—'}</td>
                <td><span class="chip ${(s.status||'').toLowerCase()}">${s.status||'—'}</span></td>
                <td style="font-family:var(--mono);font-size:11px">${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleString() : '—'}</td>
            </tr>`).join('')
            : '<tr><td colspan="7" class="empty-state">No subscriptions yet</td></tr>';
    }catch(e){
        document.getElementById('my-subscriptions-list').innerHTML='<tr><td colspan="7" class="empty-state">Could not load</td></tr>';
    }
}

let slotRowCount = 0;
    async function addSlotRow(){
        slotRowCount++;
        const id = slotRowCount;
        const div = document.createElement('div');
        div.id = `slot-row-${id}`;
        div.style.cssText = 'display:grid;grid-template-columns:1fr 1fr auto;gap:12px;margin-bottom:10px;align-items:end';

        // Fetch restaurants for the dropdown
        let options = '<option value="">Loading...</option>';
        try{
            const data = await apiFetch('/restaurants');
            options = data.map(r => `<option value="${r.restaurantId||r.id}">${r.name}</option>`).join('');
        }catch(e){ options = '<option value="">Could not load</option>'; }

        div.innerHTML = `
        <div class="form-group" style="margin:0">
            <label>Meal Slot</label>
            <select id="slot-meal-${id}">
                <option value="BREAKFAST">Breakfast</option>
                <option value="LUNCH">Lunch</option>
                <option value="DINNER">Dinner</option>
```

```
            </select>
        </div>
        <div class="form-group" style="margin:0">
            <label>Restaurant</label>
            <select id="slot-restaurant-${id}">${options}</select>
        </div>
        <button class="btn btn-reject btn-sm" onclick="removeSlotRow(${id})" style="margin-bottom:1px"></button>
    `;
        document.getElementById('slot-rows').appendChild(div);
    }

function removeSlotRow(id){
    document.getElementById(`slot-row-${id}`)?.remove();
}

function toggleDayOfWeek(){
    const isWeekly = document.getElementById('sub-schedule').value === 'WEEKLY';
    document.getElementById('day-of-week-group').style.display = isWeekly ? 'flex' : 'none';
}

    async function submitSubscription(){
        const scheduleType = document.getElementById('sub-schedule').value;
        const dayOfWeek    = scheduleType === 'WEEKLY' ? document.getElementById('sub-day').value : null;

        const slotOrders = [];
        let valid = true;

        document.querySelectorAll('[id^="slot-row-"]').forEach(row => {
            const idNum        = row.id.split('-')[2];
            const mealSlot     = document.getElementById(`slot-meal-${idNum}`)?.value;
            const restaurantVal = document.getElementById(`slot-restaurant-${idNum}`)?.value;

            if(!mealSlot || !restaurantVal){ valid = false; return; }

            const restaurantId = parseInt(restaurantVal);
            if(isNaN(restaurantId)){ valid = false; return; }

            slotOrders.push({ mealSlot, restaurantId });
        });

        if(!valid || slotOrders.length === 0){
            showAlert('Please add at least one valid slot','error');
            return;
        }

        // Derive mealSlots from slotOrders to satisfy @NotEmpty
        const mealSlots = slotOrders.map(s => s.mealSlot);

        try{
            const res = await fetch(`${API}/subscription`, {
                method: 'POST',
                headers: {'Content-Type':'application/json', ...getAuthHeader()},
                credentials: 'include',
                body: JSON.stringify({ scheduleType, dayOfWeek, mealSlots, slotOrders })
            });
            const text = await res.text();
            if(res.ok){
                showAlert('Subscription added successfully! ','success');
                clearSubscriptionForm();
                loadMySubscriptions();
            } else {
                showAlert('Error: '+text,'error');
            }
        }catch(e){ showAlert('Failed: '+e.message,'error'); }
    }

function clearSubscriptionForm(){
    document.getElementById('sub-schedule').value = 'DAILY';
    document.getElementById('day-of-week-group').style.display = 'none';
    document.getElementById('slot-rows').innerHTML = '';
    slotRowCount = 0;
}

    function logout(){
        // Clear all localStorage
```

```
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('refresh_token');
        localStorage.removeItem('auth_type');
        localStorage.removeItem('last_role');

        // Call server logout to invalidate session
        fetch(`${API}/session/logout`, {
            method: 'POST',
            credentials: 'include'
        }).finally(() => {
            // Force expire the session cookie on client side
            document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
            window.location.href = '/google.html';
        });
    }

    function showAlert(msg,type){
        const el=document.createElement('div');
        el.className=`alert alert-${type}`;el.textContent=msg;
        document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
    }
</script>
</body>
</html>
```

145 Current: src\main\resources\static\pending.html 76 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Account Pending</title>
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body {
            font-family: 'Outfit', sans-serif;
            background: #070910;
            color: #dde4f0;
            min-height: 100vh;
            display: flex;
            align-items: center;
            justify-content: center;
        }
        .card {
            background: #0e1118;
            border: 1px solid rgba(255,255,255,0.07);
            border-radius: 16px;
            padding: 48px;
            max-width: 420px;
            text-align: center;
        }
        .icon { font-size: 48px; margin-bottom: 20px; }
        h1 { font-size: 22px; font-weight: 700; margin-bottom: 12px; }
        p  { font-size: 14px; color: #6b7591; line-height: 1.6; margin-bottom: 24px; }
        .email {
            background: #131821;
            border: 1px solid rgba(79,140,255,0.2);
            border-radius: 8px;
            padding: 10px 16px;
            font-size: 13px;
            color: #4f8cff;
            margin-bottom: 28px;
            font-family: monospace;
        }
        .btn {
            padding: 10px 24px;
            background: transparent;
            border: 1px solid rgba(255,255,255,0.1);
            border-radius: 8px;
            color: #6b7591;
            font-size: 13px;
            cursor: pointer;
        }
        .btn:hover { border-color: rgba(255,255,255,0.2); color: #dde4f0; }
    </style>
</head>
<body>
<div class="card">
    <div class="icon"></div>
    <h1>Account Pending Approval</h1>
    <p>You've successfully signed in with Google. Your account is currently awaiting approval from an admin or manager.</p>
    <div class="email" id="user-email">Loading...</div>
    <p>Once approved, you'll have full access to the portal. Please contact your manager or admin.</p>
    <button class="btn" onclick="logout()">Sign out</button>
</div>

<script>
    // Show the user their email from JWT
    const token = localStorage.getItem('jwt_token');
    if (token) {
        try {
            const payload = JSON.parse(atob(token.split('.')[1]));
            document.getElementById('user-email').textContent = payload.sub || 'Unknown';
        } catch(e) {}
    }

    function logout() {
        localStorage.clear();
```

```
        window.location.href = '/google.html';
    }
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TechVendor Hub</title>
    <link href="https://fonts.googleapis.com/css2?family=Syne:wght@400;500;600;700;800&family=DM+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        :root{
          --bg:#080b10;--bg2:#0c1018;--bg3:#111620;--bg4:#161d2a;
          --panel:#131a26;--panel2:#1a2235;
          --border:rgba(0,210,255,0.08);--border2:rgba(0,210,255,0.18);--border3:rgba(0,210,255,0.35);
          --text:#d4e8f5;--text2:#6b8aaa;--text3:#3a5270;
          --cyan:#00d2ff;--cyan-dim:rgba(0,210,255,0.08);--cyan-glow:rgba(0,210,255,0.15);
          --amber:#f59e0b;--amber-dim:rgba(245,158,11,0.1);
          --green:#10d48e;--green-dim:rgba(16,212,142,0.1);
          --red:#f05454;--red-dim:rgba(240,84,84,0.1);
          --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.1);
          --orange:#fb923c;--orange-dim:rgba(251,146,60,0.1);
          --font:'Syne',sans-serif;--mono:'DM Mono',monospace;
          --sidebar:260px;
        }
        body{font-family:var(--font);background:var(--bg);color:var(--text);font-size:14px;overflow:hidden;height:100vh}
        .app{display:flex;height:100vh}

        /* SCANLINE OVERLAY */
        body::before{content:'';position:fixed;inset:0;background:repeating-linear-gradient(0deg,transparent,transparent 2px,rgba(0,0,0,0.03) 2px,rgba(0,0,0,0.03) 4px);pointer-events:none;z-index:9...

        /* SIDEBAR */
        .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--border);display:flex;flex-direction:column;flex-shrink:0;overflow:hidden;position:relative}
        .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:300px;background:linear-gradient(to top,rgba(0,210,255,0.03),transparent);pointer-events:none}

        .logo{padding:24px 20px 20px;border-bottom:1px solid var(--border)}
        .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
        .logo-hex{width:36px;height:36px;background:linear-gradient(135deg,#00d2ff,#0066ff);clip-path:polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0% 25%);display:flex;align-items:center;justif...
        .logo-name{font-size:17px;font-weight:800;color:var(--text);letter-spacing:-0.5px}
        .logo-sub{font-size:10px;color:var(--text3);font-family:var(--mono);letter-spacing:1px;text-transform:uppercase}
        .vendor-badge{display:inline-flex;align-items:center;gap:6px;margin-top:10px;padding:5px 12px;background:var(--cyan-dim);border:1px solid var(--border2);border-radius:4px;font-size:10px;fon...
        .badge-dot{width:5px;height:5px;background:var(--cyan);border-radius:50%;animation:pulse 2s infinite}
        @keyframes pulse{0%,100%{opacity:1;box-shadow:0 0 0 0 var(--cyan-glow)}50%{opacity:0.7;box-shadow:0 0 0 4px transparent}}

        .nav{flex:1;padding:12px 0;overflow-y:auto}
        .nav::-webkit-scrollbar{width:3px}
        .nav::-webkit-scrollbar-thumb{background:var(--border2);border-radius:2px}
        .nav-section{padding:14px 20px 5px;font-size:9px;font-weight:700;color:var(--text3);letter-spacing:2.5px;text-transform:uppercase;font-family:var(--mono)}
        .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-radius:6px;cursor:pointer;transition:all 0.15s;color:var(--text2);font-size:13px;font-weight:500;p...
        .nav-item:hover{background:var(--cyan-dim);color:var(--cyan)}
        .nav-item.active{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
        .nav-item.active::before{content:'';position:absolute;left:-8px;top:50%;transform:translateY(-50%);width:3px;height:60%;background:var(--cyan);border-radius:0 2px 2px 0}
        .nav-icon{width:16px;height:16px;opacity:0.7;flex-shrink:0}
        .nav-item.active .nav-icon,.nav-item:hover .nav-icon{opacity:1}
        .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:9px;font-weight:700;padding:2px 6px;border-radius:3px;font-family:var(--mono)}
        .nav-badge.amber{background:var(--amber)}

        .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
```

```
.user-row{display:flex;align-items:center;gap:10px}
.avatar{width:34px;height:34px;background:linear-gradient(135deg,#00d2ff22,#0066ff22);border:1px solid var(--border2);border-radius:6px;display:flex;align-items:center;justify-content:cente...
.user-name{font-size:12px;font-weight:600;color:var(--text)}
.user-role{font-size:10px;color:var(--cyan);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px}

/* MAIN */
.main{flex:1;display:flex;flex-direction:column;overflow:hidden}
.topbar{background:var(--bg2);padding:0 24px;height:56px;display:flex;justify-content:space-between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
.page-title{font-size:15px;font-weight:700;color:var(--text);display:flex;align-items:center;gap:8px}
.page-title-prefix{color:var(--cyan);font-family:var(--mono);font-size:12px;font-weight:400}
.topbar-right{display:flex;align-items:center;gap:10px}
.topbar-email{font-size:11px;color:var(--text3);font-family:var(--mono)}
.signout-btn{padding:6px 14px;background:transparent;border:1px solid var(--border2);border-radius:5px;color:var(--text2);font-family:var(--font);font-size:11px;font-weight:600;cursor:point...
.signout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}

.content{flex:1;overflow-y:auto;padding:24px}
.content::-webkit-scrollbar{width:4px}
.content::-webkit-scrollbar-thumb{background:var(--border2)}
.view{display:none}
.view.active{display:block;animation:viewIn 0.25s ease}
@keyframes viewIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}

/* STATS */
.stats-row{display:grid;grid-template-columns:repeat(4,1fr);gap:14px;margin-bottom:22px}
.stat{background:var(--panel);border:1px solid var(--border);border-radius:8px;padding:18px 20px;position:relative;overflow:hidden;cursor:default;transition:border-color 0.2s}
.stat:hover{border-color:var(--border2)}
.stat::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
.stat.c{--c:var(--cyan)} .stat.g{--c:var(--green)} .stat.a{--c:var(--amber)} .stat.r{--c:var(--red)} .stat.v{--c:var(--violet)} .stat.o{--c:var(--orange)}
.stat::after{background:var(--c)}
.stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;font-weight:600;margin-bottom:8px;font-family:var(--mono)}
.stat-val{font-size:28px;font-weight:800;color:var(--text);line-height:1;font-family:var(--mono)}
.stat-sub{font-size:11px;color:var(--text3);margin-top:5px}
.stat-icon{position:absolute;right:16px;top:50%;transform:translateY(-50%);font-size:24px;opacity:0.12}

/* CARDS / SECTIONS */
.card{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-bottom:18px;overflow:hidden}
.card-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center}
.card-title{font-size:13px;font-weight:700;color:var(--text);letter-spacing:0.2px}
.card-sub{font-size:11px;color:var(--text3);margin-top:2px;font-family:var(--mono)}
.card-body{padding:20px}
.dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:18px}

/* TABLE */
.tablewrap{overflow-x:auto}
table{width:100%;border-collapse:collapse}
th{padding:9px 16px;text-align:left;font-size:9px;color:var(--text3);font-weight:700;letter-spacing:1.5px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:12px;color:var(--text2);font-family:var(--mono)}
tr:last-child td{border-bottom:none}
tr:hover td{background:rgba(0,210,255,0.02);color:var(--text)}
td.main-cell{font-family:var(--font);font-size:13px;color:var(--text);font-weight:500}

/* STATUS CHIPS */
.chip{display:inline-flex;align-items:center;gap:5px;padding:3px 9px;border-radius:3px;font-size:10px;font-weight:700;letter-spacing:0.8px;text-transform:uppercase;font-family:var(--mono)}
.chip::before{content:'';width:4px;height:4px;border-radius:50%;flex-shrink:0}
.chip.available{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)} .chip.available::before{background:var(--green)}
.chip.assigned{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)} .chip.assigned::before{background:var(--cyan)}
.chip.under_repair{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)} .chip.under_repair::before{background:var(--amber)}
.chip.condemned{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)} .chip.condemned::before{background:var(--red)}
```

```
115         .chip.returned_to_vendor{background:var(--violet-dim);color:var(--violet);border:1px solid
rgba(167,139,250,0.2)} .chip.returned_to_vendor::before{background:var(--violet)}
116         .chip.open{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
.chip.open::before{background:var(--amber)}
117         .chip.acknowledged{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.acknowledged::before{background:var(--cyan)}
118         .chip.sent_for_repair{background:var(--orange-dim);color:var(--orange);border:1px solid
rgba(251,146,60,0.2)} .chip.sent_for_repair::before{background:var(--orange)}
119         .chip.repair_done{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)} .chip.repair_done::before{background:var(--green)}
120         .chip.resolved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)} .chip.resolved::before{background:var(--green)}
121         .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
.chip.rejected::before{background:var(--red)}
122         .chip.sent{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
.chip.sent::before{background:var(--amber)}
123         .chip.in_progress{background:var(--orange-dim);color:var(--orange);border:1px solid
rgba(251,146,60,0.2)} .chip.in_progress::before{background:var(--orange)}
124         .chip.fixed{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
.chip.fixed::before{background:var(--green)}
125         .chip.irreparable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
.chip.irreparable::before{background:var(--red)}
126         .chip.laptop{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.laptop::before{background:var(--cyan)}
127         .chip.phone{background:var(--violet-dim);color:var(--violet);border:1px solid
rgba(167,139,250,0.2)} .chip.phone::before{background:var(--violet)}
128         .chip.tablet{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
.chip.tablet::before{background:var(--green)}
129         .chip.monitor,.chip.other,.chip.desktop,.chip.keyboard,.chip.mouse,.chip.headset{background:var(--
amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)} .chip.monitor::before,...
130         .chip.active_assign{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)} .chip.active_assign::before{background:var(--green)}
131         .chip.returned{background:var(--text3);color:var(--text2);border:1px solid var(--border)}
.chip.returned::before{background:var(--text2)}
132
133         /* BUTTONS */
134         .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--font);font-size:12px;font-
weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.3px}
135         .btn-primary{background:var(--cyan);color:#080b10}
136         .btn-primary:hover{background:#33dbff;box-shadow:0 0 20px var(--cyan-glow)}
137         .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border2)}
138         .btn-ghost:hover{border-color:var(--border3);color:var(--text)}
139         .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
140         .btn-danger:hover{background:var(--red);color:#fff}
141         .btn-warn{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
142         .btn-warn:hover{background:var(--amber);color:#080b10}
143         .btn-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
144         .btn-success:hover{background:var(--green);color:#080b10}
145         .btn-sm{padding:5px 10px;font-size:11px}
146         .btn-xs{padding:3px 8px;font-size:10px}
147         .action-row{display:flex;gap:6px;flex-wrap:wrap}
148
149         /* FORM */
150         .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
151         .form-group{display:flex;flex-direction:column;gap:6px}
152         .form-group.full{grid-column:1/-1}
153         .form-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:1.5px;font-family:var(--mono)}
154         .form-input{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:10px
13px;color:var(--text);font-family:var(--mono);font-size:12px;transition:border-color 0.2s;wi...
155         .form-input:focus{outline:none;border-color:var(--cyan);box-shadow:0 0 0 3px var(--cyan-dim)}
156         .form-input::placeholder{color:var(--text3)}
157         select.form-input option{background:var(--bg2)}
158         .form-actions{display:flex;gap:10px;margin-top:18px}
159
160         /* EMPTY */
161         .empty{text-align:center;padding:40px 20px;color:var(--text3);font-family:var(--mono);font-
size:12px}
162         .empty-icon{font-size:32px;margin-bottom:10px;opacity:0.4}
163
164         /* ALERT */
165         .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:6px;z-
index:2000;animation:alertIn 0.25s ease;font-size:12px;font-weight:600;font-family:var(--mono);display:flex;a...
166         .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.3)}
167         .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.3)}
```

```
        @keyframes alertIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}

        /* SEARCH */
        .search-row{display:flex;gap:10px;padding:14px 16px;border-bottom:1px solid var(--border)}
        .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:8px 13px;color:var(--text);font-family:var(--mono);font-size:12px}
        .search-input:focus{outline:none;border-color:var(--cyan)}
        .search-input::placeholder{color:var(--text3)}

        /* REPAIR TIMELINE */
        .timeline{padding:20px;display:flex;flex-direction:column;gap:0}
        .tl-item{display:flex;gap:14px;padding-bottom:20px;position:relative}
        .tl-item:last-child{padding-bottom:0}
        .tl-item:not(:last-child)::after{content:'';position:absolute;left:14px;top:28px;bottom:0;width:1px;background:var(--border)}
        .tl-dot{width:28px;height:28px;border-radius:50%;border:2px solid var(--border2);background:var(--bg3);display:flex;align-items:center;justify-content:center;flex-shrink:0;font-size:11px;z-...
        .tl-dot.cyan{border-color:var(--cyan);background:var(--cyan-dim)}
        .tl-dot.amber{border-color:var(--amber);background:var(--amber-dim)}
        .tl-dot.green{border-color:var(--green);background:var(--green-dim)}
        .tl-dot.red{border-color:var(--red);background:var(--red-dim)}
        .tl-content{flex:1;padding-top:4px}
        .tl-title{font-size:12px;font-weight:600;color:var(--text);margin-bottom:2px}
        .tl-meta{font-size:10px;color:var(--text3);font-family:var(--mono)}

        /* DEVICE DETAIL MODAL */
        .modal-overlay{display:none;position:fixed;inset:0;background:rgba(8,11,16,0.85);z-index:1500;align-items:center;justify-content:center;backdrop-filter:blur(4px)}
        .modal-overlay.open{display:flex}
        .modal{background:var(--panel2);border:1px solid var(--border2);border-radius:12px;width:560px;max-width:95%;max-height:85vh;overflow-y:auto;animation:modalIn 0.2s ease}
        @keyframes modalIn{from{opacity:0;transform:scale(0.96)}to{opacity:1;transform:scale(1)}}
        .modal-head{padding:20px 24px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center}
        .modal-title{font-size:16px;font-weight:700;color:var(--text)}
        .modal-close{background:none;border:none;color:var(--text3);font-size:20px;cursor:pointer;padding:0 4px;transition:color 0.2s}
        .modal-close:hover{color:var(--text)}
        .modal-body{padding:24px}
        .detail-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-bottom:20px}
        .detail-item{display:flex;flex-direction:column;gap:4px}
        .detail-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-spacing:1.5px;font-family:var(--mono)}
        .detail-value{font-size:13px;color:var(--text);font-family:var(--mono)}
    </style>
</head>
<body>
<div class="app">

    <!-- SIDEBAR -->
    <div class="sidebar">
        <div class="logo">
            <div class="logo-mark">
                <div class="logo-hex">TV</div>
                <div>
                    <div class="logo-name">TechVendor</div>
                    <div class="logo-sub">Asset Hub</div>
                </div>
            </div>
            <div class="vendor-badge"><span class="badge-dot"></span>Tech Vendor</div>
        </div>

        <nav class="nav">
            <div class="nav-section">Overview</div>
            <div class="nav-item active" onclick="showView('dashboard',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="1" width="6" height="6" rx="1" stroke="currentColor" stroke-width="1.5"/><rect x="9" y="1" width="6" height="6" ...
                Dashboard
            </div>

            <div class="nav-section">Devices</div>
            <div class="nav-item" onclick="showView('my-devices',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="3" width="14" height="10" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M5 13v2M11 13v2M3 15h10" str...
                My Devices
            </div>
```

```
            <div class="nav-item" onclick="showView('add-device',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="8" r="7" stroke="currentColor" stroke-width="1.5"/><path d="M8 5v6M5 8h6" stroke="currentColor" stroke-width...
                Add Device
            </div>

            <div class="nav-section">Service</div>
            <div class="nav-item" onclick="showView('service-requests',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><path d="M8 1l1.5 4.5H14l-3.5 2.5 1.5 4.5L8 10l-4 2.5 1.5-4.5L2 5.5h4.5L8 1z" stroke="currentColor" stroke-width="1.5" stroke-l...
                Service Requests
                <span class="nav-badge amber" id="sr-badge" style="display:none">0</span>
            </div>
            <div class="nav-item" onclick="showView('repair-logs',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><path d="M13 3l-6 6-3-3" stroke="currentColor" stroke-width="1.5" stroke-linecap="round" stroke-linejoin="round"/><path d="M2 1...
                Repair Logs
            </div>

            <div class="nav-section">Assignments</div>
            <div class="nav-item" onclick="showView('assignments',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="6" cy="5" r="3" stroke="currentColor" stroke-width="1.5"/><path d="M1 14c0-3 2-5 5-5" stroke="currentColor" stroke-...
                Assignments
            </div>

            <div class="nav-section">Account</div>
            <div class="nav-item" onclick="showView('profile',this)">
                <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="5" r="3" stroke="currentColor" stroke-width="1.5"/><path d="M2 14c0-3.3 2.7-6 6-6s6 2.7 6 6" stroke="current...
                My Profile
            </div>
        </nav>

        <div class="sidebar-footer">
            <div class="user-row">
                <div class="avatar" id="sidebar-avatar">TV</div>
                <div>
                    <div class="user-name" id="sidebar-name">TechVendor</div>
                    <div class="user-role">Tech Vendor</div>
                </div>
            </div>
        </div>
    </div>

    <!-- MAIN -->
    <div class="main">
        <div class="topbar">
            <div class="page-title">
                <span class="page-title-prefix">// </span>
                <span id="pageTitle">dashboard</span>
            </div>
            <div class="topbar-right">
                <span class="topbar-email" id="topbar-email"></span>
                <button class="signout-btn" onclick="logout()">Sign Out</button>
            </div>
        </div>

        <div class="content">

            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-row">
                    <div class="stat c">
                        <div class="stat-label">Total Devices</div>
                        <div class="stat-val" id="stat-total">--</div>
                        <div class="stat-sub">in catalog</div>
                        <div class="stat-icon"></div>
                    </div>
                    <div class="stat g">
                        <div class="stat-label">Available</div>
                        <div class="stat-val" id="stat-avail">--</div>
                        <div class="stat-sub">ready to assign</div>
                        <div class="stat-icon"></div>
                    </div>
                    <div class="stat a">
```

```
                        <div class="stat-label">Under Repair</div>
                        <div class="stat-val" id="stat-repair">--</div>
                        <div class="stat-sub">in service</div>
                        <div class="stat-icon"></div>
                    </div>
                    <div class="stat r">
                        <div class="stat-label">Open Requests</div>
                        <div class="stat-val" id="stat-open">--</div>
                        <div class="stat-sub">need attention</div>
                        <div class="stat-icon">!</div>
                    </div>
                </div>

                <div class="dash-grid">
                    <div class="card">
                        <div class="card-head">
                            <div>
                                <div class="card-title">Recent Service Requests</div>
                                <div class="card-sub">employee-reported issues</div>
                            </div>
                            <button class="btn btn-ghost btn-sm" onclick="showView('service-requests',null)">View all </button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Device</th><th>Issue</th><th>Status</th></tr></thead>
                                <tbody id="dash-sr"></tbody>
                            </table>
                        </div>
                    </div>

                    <div class="card">
                        <div class="card-head">
                            <div>
                                <div class="card-title">Active Repairs</div>
                                <div class="card-sub">currently being serviced</div>
                            </div>
                            <button class="btn btn-ghost btn-sm" onclick="showView('repair-logs',null)">View all </button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Device</th><th>Sent</th><th>Status</th></tr></thead>
                                <tbody id="dash-repairs"></tbody>
                            </table>
                        </div>
                    </div>
                </div>

                <div class="card">
                    <div class="card-head">
                        <div class="card-title">Device Status Overview</div>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Device</th><th>Type</th><th>Status</th><th>Assigned To</th></tr></thead>
                            <tbody id="dash-devices"></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- MY DEVICES -->
            <div id="my-devices-view" class="view">
                <div class="card">
                    <div class="card-head">
                        <div>
                            <div class="card-title">My Device Catalog</div>
                            <div class="card-sub">all devices supplied by you</div>
                        </div>
                        <div class="action-row">
                            <select class="form-input btn-sm" id="device-filter" onchange="filterDevices()" style="width:140px;padding:5px 10px;font-size:11px">
                                <option value="">All Status</option>
                                <option value="AVAILABLE">Available</option>
```

```
                                <option value="ASSIGNED">Assigned</option>
                                <option value="UNDER_REPAIR">Under Repair</option>
                                <option value="CONDEMNED">Condemned</option>
                                <option value="RETURNED_TO_VENDOR">Returned</option>
                            </select>
                            <button class="btn btn-ghost btn-sm" onclick="loadMyDevices()">Refresh</button>
                        </div>
                    </div>
                    <div class="search-row">
                        <input class="search-input" id="device-search" placeholder="Search by name, serial, brand..." oninput="filterDevices()">
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>ID</th><th>Device Name</th><th>Brand</th><th>Type</th><th>Warranty</th><th>Status</th><th>Actions</th></tr></thead>
                            <tbody id="devices-list"></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- ADD DEVICE -->
            <div id="add-device-view" class="view">
                <div class="card" style="max-width:700px">
                    <div class="card-head">
                        <div>
                            <div class="card-title">Add New Device</div>
                            <div class="card-sub">register a device to your catalog</div>
                        </div>
                    </div>
                    <div class="card-body">
                        <div class="form-grid">
                            <div class="form-group">
                                <label class="form-label">Device Name</label>
                                <input class="form-input" id="d-name" placeholder="Dell XPS 15 9520">
                            </div>
                            <div class="form-group">
                                <label class="form-label">Brand</label>
                                <input class="form-input" id="d-brand" placeholder="Dell">
                            </div>
                            <div class="form-group">
                                <label class="form-label">Device Type</label>
                                <select class="form-input" id="d-type">
                                    <option value="LAPTOP">Laptop</option>
                                    <option value="DESKTOP">Desktop</option>
                                    <option value="PHONE">Phone</option>
                                    <option value="TABLET">Tablet</option>
                                    <option value="MONITOR">Monitor</option>
                                    <option value="KEYBOARD">Keyboard</option>
                                    <option value="MOUSE">Mouse</option>
                                    <option value="HEADSET">Headset</option>
                                    <option value="OTHER">Other</option>
                                </select>
                            </div>
                            <div class="form-group">
                                <label class="form-label">Warranty Expiry</label>
                                <input class="form-input" type="date" id="d-warranty">
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="addDevice()">Register Device</button>
                            <button class="btn btn-ghost" onclick="clearDeviceForm()">Clear</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- SERVICE REQUESTS -->
            <div id="service-requests-view" class="view">
                <div class="card">
                    <div class="card-head">
                        <div>
                            <div class="card-title">Service Requests</div>
                            <div class="card-sub">issues reported by employees for your devices</div>
```

```
                        </div>
                        <div class="action-row">
                            <select class="form-input btn-sm" id="sr-filter" onchange="filterServiceRequests()" style="width:140px;padding:5px 10px;font-size:11px">
                                <option value="">All Status</option>
                                <option value="OPEN">Open</option>
                                <option value="ACKNOWLEDGED">Acknowledged</option>
                                <option value="SENT_FOR_REPAIR">Sent for Repair</option>
                                <option value="REPAIR_DONE">Repair Done</option>
                                <option value="RESOLVED">Resolved</option>
                                <option value="REJECTED">Rejected</option>
                            </select>
                            <button class="btn btn-ghost btn-sm" onclick="loadServiceRequests()">Refresh</button>
                        </div>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>ID</th><th>Device</th><th>Issue Type</th><th>Description</th><th>Urgent</th><th>Status</th><th>Raised</th><th>Actions</th></tr></thead>
                            <tbody id="sr-list"></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- REPAIR LOGS -->
            <div id="repair-logs-view" class="view">
                <div class="card">
                    <div class="card-head">
                        <div>
                            <div class="card-title">Repair Logs</div>
                            <div class="card-sub">devices sent to your service center</div>
                        </div>
                        <button class="btn btn-ghost btn-sm" onclick="loadRepairLogs()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
<thead><tr><th>ID</th><th>Device</th><th>Issue</th><th>Technician</th><th>Sent</th><th>Expected Return</th><th>Cost</th><th>Status</th><th>Actions</th></tr></thead>
                            <tbody id="repair-list"></tbody>
                        </table>
                    </div>
                </div>

                <!-- UPDATE REPAIR STATUS FORM -->
                <div class="card" id="repair-update-form" style="max-width:600px;display:none">
                    <div class="card-head">
                        <div class="card-title">Update Repair Status</div>
                    </div>
                    <div class="card-body">
                        <input type="hidden" id="repair-id">
                        <div class="form-grid">
                            <div class="form-group"><label class="form-label">Repair Status</label>
                                <select class="form-input" id="repair-status-input">
                                    <option value="SENT">Sent</option>
                                    <option value="IN_PROGRESS">In Progress</option>
                                    <option value="FIXED">Fixed</option>
                                    <option value="IRREPARABLE">Irreparable</option>
                                </select>
                            </div>
                            <div class="form-group"><label class="form-label">Technician Name</label><input class="form-input" id="repair-tech" placeholder="Technician name"></div>
                            <div class="form-group"><label class="form-label">Repair Cost ()</label><input class="form-input" type="number" id="repair-cost" placeholder="0.00"></div>
                            <div class="form-group"><label class="form-label">Invoice Number</label><input class="form-input" id="repair-invoice" placeholder="INV-2024-XXXX"></div>
                            <div class="form-group"><label class="form-label">Actual Return Date</label><input class="form-input" type="date" id="repair-return-date"></div>
                            <div class="form-group"><label class="form-label">Fixed Successfully</label>
                                <select class="form-input" id="repair-fixed">
                                    <option value="true">Yes</option>
                                    <option value="false">No</option>
                                </select>
                            </div>
```

```
                            <div class="form-group full"><label class="form-label">Repair Notes</label><input class="form-input" id="repair-notes" placeholder="What was repaired / replaced..."></div>
                            <div class="form-group full"><label class="form-label">Post Repair Action</label>
                                <select class="form-input" id="repair-action">
                                    <option value="reassigned">Reassigned to employee</option>
                                    <option value="returned to stock">Returned to stock (Available)</option>
                                    <option value="condemned">Condemned (beyond repair)</option>
                                    <option value="returned to vendor">Returned to vendor</option>
                                </select>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="updateRepair()">Update Repair</button>
                            <button class="btn btn-ghost" onclick="document.getElementById('repair-update-form').style.display='none'">Cancel</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- ASSIGNMENTS -->
            <div id="assignments-view" class="view">
                <div class="card">
                    <div class="card-head">
                        <div>
                            <div class="card-title">Device Assignments</div>
                            <div class="card-sub">who has your devices</div>
                        </div>
                        <button class="btn btn-ghost btn-sm" onclick="loadAssignments()"> Refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead>
                            <tr>
                                <th>Device</th>
                                <th>Type</th>
                                <th>Employee</th>
                                <th>Assigned Date</th>
                                <th>Status</th>
                            </tr>
                            </thead>
                            <tbody id="assignments-list"></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- PROFILE -->
            <div id="profile-view" class="view">
                <div class="card" style="max-width:600px">
                    <div class="card-head">
                        <div class="card-title">My Profile</div>
                    </div>
                    <div class="card-body">
                        <div style="display:flex;align-items:center;gap:16px;margin-bottom:24px;padding-bottom:20px;border-bottom:1px solid var(--border)">
                            <div style="width:56px;height:56px;background:var(--cyan-dim);border:2px solid var(--border2);border-radius:10px;display:flex;align-items:center;justify-content:center;f...
                            <div>
                                <div style="font-size:18px;font-weight:700;color:var(--text)" id="profile-name">--</div>
                                <div style="font-size:11px;color:var(--cyan);font-family:var(--mono);margin-top:3px">TECH_VENDOR</div>
                            </div>
                        </div>
                        <div class="detail-grid">
                            <div class="detail-item"><div class="detail-label">Email</div><div class="detail-value" id="profile-email">--</div></div>
                            <div class="detail-item"><div class="detail-label">Phone</div><div class="detail-value" id="profile-phone">--</div></div>
                            <div class="detail-item"><div class="detail-label">Registered</div><div class="detail-value" id="profile-reg">--</div></div>
                            <div class="detail-item"><div class="detail-label">Status</div><div class="detail-value"><span class="chip active_assign">ACTIVE</span></div></div>
```

```
                        </div>
                    </div>
                </div>
            </div>

        </div>
    </div>
</div>

<!-- DEVICE DETAIL MODAL -->
<div class="modal-overlay" id="device-modal">
    <div class="modal">
        <div class="modal-head">
            <div class="modal-title" id="modal-device-name">Device Details</div>
            <button class="modal-close" onclick="closeModal()"></button>
        </div>
        <div class="modal-body">
            <div class="detail-grid" id="modal-detail-grid"></div>
            <div style="margin-top:16px">
                <div class="card-title" style="margin-bottom:12px;font-size:12px;color:var(--text3);font-family:var(--mono);text-transform:uppercase;letter-spacing:1px">Repair History</div>
                <div id="modal-repair-history"></div>
            </div>
        </div>
    </div>
</div>

<script>
    const API = 'http://localhost:8080';
    let devicesCache = [], srCache = [], repairsCache = [], assignmentsCache = [];

    //  AUTH
    (function(){
      const token = localStorage.getItem('jwt_token');
      if(!token){ window.location.href='/login.html'; return; }
      try{
        const p = JSON.parse(atob(token.split('.')[1]));
        const role = (p.role||'').replace(/^ROLE_/,'');
        if(role !== 'TECH_VENDOR'){ window.location.href='/google.html'; return; }
        const email = p.sub||p.email||'TechVendor';
        document.getElementById('topbar-email').textContent = email;
        document.getElementById('sidebar-name').textContent = email.split('@')[0];
        document.getElementById('sidebar-avatar').textContent = email.charAt(0).toUpperCase();
        document.getElementById('profile-avatar').textContent = email.charAt(0).toUpperCase();
        document.getElementById('profile-email').textContent = email;
      }catch(e){ window.location.href='/login.html'; }
      initDashboard();
    })();

    function getAuthHeader(){
      const t = localStorage.getItem('jwt_token');
      return t ? {'Authorization': `Bearer ${t}`} : {};
    }

    async function apiFetch(path, opts={}){
      const res = await fetch(`${API}${path}`,{
        ...opts,
        headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
        credentials:'include'
      });
      if(res.status===401){
        localStorage.removeItem('jwt_token');
        window.location.href='/login.html';
        throw new Error('Unauthorized');
      }
      const text = await res.text();
      if(!res.ok) throw new Error(text || 'Request failed');
      return text ? JSON.parse(text) : null;
    }

    //  NAV
    function showView(name, el){
      document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
      document.getElementById(`${name}-view`).classList.add('active');
      document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
      if(el) el.classList.add('active');
```

```
      document.getElementById('pageTitle').textContent = name.replace(/-/g,' ');
      const loaders = {
        'dashboard': loadDashboard,
        'my-devices': loadMyDevices,
        'service-requests': loadServiceRequests,
        'repair-logs': loadRepairLogs,
        'assignments': loadAssignments,
        'profile': loadProfile,
        'add-device': ()=>{}
      };
      if(loaders[name]) loaders[name]();
    }

    //  INIT
    async function initDashboard(){
      await Promise.all([loadMyDevices(), loadServiceRequests(), loadRepairLogs(), loadAssignments()]);
      loadDashboard();
    }

    //  DASHBOARD
    function loadDashboard(){
      document.getElementById('stat-total').textContent = devicesCache.length || 0;
      document.getElementById('stat-avail').textContent =
devicesCache.filter(d=>d.deviceStatus===='AVAILABLE').length || 0;
      document.getElementById('stat-repair').textContent =
devicesCache.filter(d=>d.deviceStatus===='UNDER_REPAIR').length || 0;
      const openSR = srCache.filter(s=>s.status===='OPEN'||s.status===='ACKNOWLEDGED');
      document.getElementById('stat-open').textContent = openSR.length || 0;

      // update badge
      const badge = document.getElementById('sr-badge');
      if(openSR.length){ badge.textContent=openSR.length; badge.style.display=''; }
      else badge.style.display='none';

      // dash SR table
      document.getElementById('dash-sr').innerHTML = srCache.slice(0,5).map(s=>`
        <tr><td class="main-cell">${s.deviceName||'Device #'+s.deviceId}</td>
        <td>${(s.requestType||'').toLowerCase()}</td>
        <td>${chip(s.status)}</td></tr>`).join('') ||
        `<tr><td colspan="3" class="empty">No service requests</td></tr>`;

      // dash repairs
      document.getElementById('dash-repairs').innerHTML =
repairsCache.filter(r=>r.repairStatus!=='FIXED'&&r.repairStatus!=='IRREPARABLE').slice(0,5).map(r=>`
        <tr><td class="main-cell">${r.deviceName||'Device #'+r.deviceId}</td>
        <td>${r.sentDate||'—'}</td>
        <td>${chip(r.repairStatus)}</td></tr>`).join('') ||
        `<tr><td colspan="3" class="empty">No active repairs</td></tr>`;

      // dash devices
      document.getElementById('dash-devices').innerHTML = devicesCache.slice(0,8).map(d=>`
        <tr><td class="main-cell">${d.deviceName||'—'}</td>
        <td>${chip(d.deviceType)}</td>
        <td>${chip(d.deviceStatus)}</td>
       <td>${d.assignedEmployeeName||'—'}</td></tr>`).join('') ||
        `<tr><td colspan="5" class="empty">No devices</td></tr>`;
    }

    //  DEVICES
    async function loadMyDevices(){
  try{
    const data = await apiFetch('/devices/vendor/me');
    devicesCache = data || [];
    renderDevices(devicesCache);
  }catch(e){ renderDevices([]); }
}

    function renderDevices(list){
  document.getElementById('devices-list').innerHTML = list.length
    ? list.map(d=>`<tr>
        <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${d.id}</td>
        <td class="main-cell">${d.deviceName||'—'}</td>
        <td>${d.brand||'—'}</td>
        <td>${chip(d.deviceType)}</td>
        <td style="font-size:11px;font-family:var(--mono)">
          ${d.warrantyExpiryDate
```

```
            ? (new Date(d.warrantyExpiryDate) > new Date()
                ? `<span style="color:var(--green)"> ${d.warrantyExpiryDate}</span>`
                : `<span style="color:var(--red)"> ${d.warrantyExpiryDate}</span>`)
            : '<span style="color:var(--text3)">—</span>'}
        </td>
        <td>${chip(d.deviceStatus)}</td>
        <td><div class="action-row">
          <button class="btn btn-ghost btn-xs" onclick="viewDevice(${d.id})">Details</button>
        </div></td>
      </tr>`).join('')
    : `<tr><td colspan="7"><div class="empty"><div class="empty-icon"></div>No devices registered yet</div></td></tr>`;
}

    function filterDevices(){
      const q = document.getElementById('device-search').value.toLowerCase();
      const f = document.getElementById('device-filter').value;
      renderDevices(devicesCache.filter(d=>{
        const matchQ = !q || (d.deviceName||'').toLowerCase().includes(q)||(d.serialNumber||'').toLowerCase().includes(q)||(d.brand||'').toLowerCase().includes(q);
        const matchF = !f || d.deviceStatus === f;
        return matchQ && matchF;
      }));
    }

    async function addDevice(){
  const body = {
    deviceName: document.getElementById('d-name').value.trim(),
    brand: document.getElementById('d-brand').value.trim(),
    deviceType: document.getElementById('d-type').value,
    warrantyExpiryDate: document.getElementById('d-warranty').value || null,
  };
  if(!body.deviceName || !body.brand || !body.deviceType){
    showAlert('Device name, brand and type are required','error'); return;
  }
  try{
    await apiFetch('/devices', {method:'POST', body:JSON.stringify(body)});
    showAlert('Device registered successfully!','success');
    clearDeviceForm();
    loadMyDevices();
  }catch(e){ showAlert('Failed: '+e.message,'error'); }
}


   function clearDeviceForm(){
  ['d-name','d-brand','d-warranty'].forEach(id=>document.getElementById(id).value='');
  document.getElementById('d-type').value='LAPTOP';
}

    //  SERVICE REQUESTS
    async function loadServiceRequests(){
      try{
        const data = await apiFetch('/tech-vendor/service-requests');
        srCache = data || [];
        renderServiceRequests(srCache);
      }catch(e){ renderServiceRequests([]); }
    }

    function renderServiceRequests(list){
      document.getElementById('sr-list').innerHTML = list.length
        ? list.map(s=>`<tr>
            <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${s.id}</td>
            <td class="main-cell">${s.deviceName||'Device #'+s.deviceId}</td>
            <td>${(s.requestType||'').toLowerCase().replace('_',' ')}</td>
            <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap">${s.issueDescription||'—'}</td>
            <td>${s.urgent ? '<span style="color:var(--red);font-family:var(--mono);font-size:10px;font-weight:700">URGENT</span>' : '<span style="color:var(--text3);font-family:var(--mono);font-si...
            <td>${chip(s.status)}</td>
            <td style="font-size:10px;font-family:var(--mono);color:var(--text3)">${s.raisedAt ? s.raisedAt.split('T')[0] : '—'}</td>
            <td><div class="action-row">
              ${s.status==='OPEN' ? `<button class="btn btn-success btn-xs" onclick="acknowledgeRequest(${s.id})">Acknowledge</button>` : ''}
              ${s.status==='ACKNOWLEDGED' ? `<button class="btn btn-warn btn-xs"
```

```
onclick="sendForRepair(${s.id})">Send for Repair</button>` : ''}
                ${s.status=='REPAIR_DONE' ? `<button class="btn btn-success btn-xs" onclick="resolveRequest(${s.id})">Resolve</button>` : ''}
              </div></td>
            </tr>`).join('')
          : `<tr><td colspan="8"><div class="empty"><div class="empty-icon"></div>No service requests</div></td></tr>`;
      }

      function filterServiceRequests(){
        const f = document.getElementById('sr-filter').value;
        renderServiceRequests(!f ? srCache : srCache.filter(s=>s.status===f));
      }

      async function acknowledgeRequest(id){
        try{
          await apiFetch(`/tech-vendor/service-requests/${id}/acknowledge`, {method:'PUT'});
          showAlert('Request acknowledged','success');
          loadServiceRequests(); loadDashboard();
        }catch(e){ showAlert('Failed: '+e.message,'error'); }
      }

      async function sendForRepair(id){
        try{
          await apiFetch(`/tech-vendor/service-requests/${id}/send-repair`, {method:'PUT'});
          showAlert('Device sent for repair','success');
          loadServiceRequests(); loadRepairLogs(); loadMyDevices(); loadDashboard();
        }catch(e){ showAlert('Failed: '+e.message,'error'); }
      }

      async function resolveRequest(id){
        try{
          await apiFetch(`/tech-vendor/service-requests/${id}/resolve`, {method:'PUT'});
          showAlert('Request resolved','success');
          loadServiceRequests(); loadDashboard();
        }catch(e){ showAlert('Failed: '+e.message,'error'); }
      }

      //  REPAIR LOGS
      async function loadRepairLogs(){
        try{
          const data = await apiFetch('/tech-vendor/repair-logs');
          repairsCache = data || [];
          renderRepairLogs(repairsCache);
        }catch(e){ renderRepairLogs([]); }
      }

      function renderRepairLogs(list){
        document.getElementById('repair-list').innerHTML = list.length
          ? list.map(r=>`<tr>
              <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${r.id}</td>
              <td class="main-cell">${r.deviceName||'Device #'+r.deviceId}</td>
              <td style="max-width:140px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap">${r.issueReported||'—'}</td>
              <td>${r.technicianName||'—'}</td>
              <td style="font-size:11px;font-family:var(--mono)">${r.sentDate||'—'}</td>
              <td style="font-size:11px;font-family:var(--mono)">${r.expectedReturnDate||'—'}</td>
              <td style="font-family:var(--mono)">${r.repairCost ? ''+r.repairCost : '—'}</td>
              <td>${chip(r.repairStatus)}</td>
              <td><button class="btn btn-ghost btn-xs" onclick="openRepairUpdate(${r.id})">Update</button></td>
            </tr>`).join('')
          : `<tr><td colspan="9"><div class="empty"><div class="empty-icon"></div>No repair logs</div></td></tr>`;
      }

      function openRepairUpdate(id){
        document.getElementById('repair-id').value = id;
        const form = document.getElementById('repair-update-form');
        form.style.display = 'block';
        form.scrollIntoView({behavior:'smooth'});
      }

      async function updateRepair(){
        const id = document.getElementById('repair-id').value;
        const body = {
```

```
        repairStatus: document.getElementById('repair-status-input').value,
        technicianName: document.getElementById('repair-tech').value.trim(),
        repairCost: parseFloat(document.getElementById('repair-cost').value)||null,
        invoiceNumber: document.getElementById('repair-invoice').value.trim(),
        actualReturnDate: document.getElementById('repair-return-date').value||null,
        fixedSuccessfully: document.getElementById('repair-fixed').value==='true',
        repairNotes: document.getElementById('repair-notes').value.trim(),
        postRepairAction: document.getElementById('repair-action').value
      };
      try{
        await apiFetch(`/tech-vendor/repair-logs/${id}`, {method:'PUT', body:JSON.stringify(body)});
        showAlert('Repair log updated!','success');
        document.getElementById('repair-update-form').style.display='none';
        loadRepairLogs(); loadMyDevices(); loadDashboard();
      }catch(e){ showAlert('Failed: '+e.message,'error'); }
    }

    //  ASSIGNMENTS
    async function loadAssignments(){
    try{
    const data = await apiFetch('/devices/assignments'); // confirm your actual endpoint path
    assignmentsCache = data || [];
    document.getElementById('assignments-list').innerHTML = assignmentsCache.length
      ? assignmentsCache.map(a=>`<tr>
          <td class="main-cell">${a.deviceName||'—'}</td>
          <td>${chip(a.deviceType)}</td>
          <td>${a.employeeName||'—'}</td>
          <td style="font-size:11px;font-family:var(--mono)">${a.assignedDate||'—'}</td>
          <td>${chip(a.status===='ACTIVE'?'active_assign':a.status?.toLowerCase())}</td>
        </tr>`).join('')
      : `<tr><td colspan="5"><div class="empty"><div class="empty-icon"></div>No assignments found</div></td></tr>`;
    }catch(e){}
   }

    //  PROFILE
    async function loadProfile(){
  try{
    const data = await apiFetch('/vendors/me');  // fix endpoint
    document.getElementById('profile-name').textContent = data.name || '--';
    document.getElementById('profile-email').textContent = data.email || '--';
    document.getElementById('profile-phone').textContent = data.phone || '--';
    document.getElementById('profile-reg').textContent = data.registeredAt || '--';
    document.getElementById('profile-avatar').textContent = (data.name||'T').charAt(0).toUpperCase();
    document.getElementById('sidebar-name').textContent = data.name || 'TechVendor';
  }catch(e){}
}

    //  DEVICE DETAIL MODAL
    async function viewDevice(id){
  const d = devicesCache.find(x=>x.id===id);
  if(!d) return;
  document.getElementById('modal-device-name').textContent = d.deviceName;

  // Derive warranty status from expiry date
  const today = new Date();
  const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
  const warrantyDisplay = warrantyExpiry
    ? (warrantyExpiry > today
        ? `<span style="color:var(--green);font-family:var(--mono)"> Valid until ${d.warrantyExpiryDate}</span>`
        : `<span style="color:var(--red);font-family:var(--mono)"> Expired ${d.warrantyExpiryDate}</span>`)
    : '<span style="color:var(--text3);font-family:var(--mono)">—</span>';

  document.getElementById('modal-detail-grid').innerHTML = `
    <div class="detail-item"><div class="detail-label">Brand</div><div class="detail-value">${d.brand||'—'}</div></div>
    <div class="detail-item"><div class="detail-label">Type</div><div class="detail-value">${chip(d.deviceType)}</div></div>
    <div class="detail-item"><div class="detail-label">Status</div><div class="detail-value">${chip(d.deviceStatus)}</div></div>
    <div class="detail-item"><div class="detail-label">Warranty</div><div class="detail-value">${warrantyDisplay}</div></div>
    <div class="detail-item"><div class="detail-label">Assigned To</div><div class="detail-value">${d.assignedEmployeeName||'<span style="color:var(--text3)">Unassigned</span>'}</div></div>
    <div class="detail-item"><div class="detail-label">Added On</div><div class="detail-
```

```
value">${d.createdAt ? d.createdAt.split('T')[0] : '—'}</div></div>
    `;

    const repairs = repairsCache.filter(r=>r.deviceId===id);
    document.getElementById('modal-repair-history').innerHTML = repairs.length
      ? `<div class="timeline">${repairs.map(r=>`
          <div class="tl-item">
            <div class="tl-dot
${r.repairStatus===='FIXED'?'green':r.repairStatus===='IRREPARABLE'?'red':r.repairStatus===='IN_PROGRESS'?'amber':'
cyan'}"></div>
            <div class="tl-content">
              <div class="tl-title">${r.issueReported||'Repair'}</div>
              <div class="tl-meta">${chip(r.repairStatus)} · ${r.sentDate||'—'}
${r.actualReturnDate||'pending'} · ${r.repairCost ? ''+r.repairCost : 'cost pending'}</div>
            </div>
          </div>`).join('')}</div>`
      : `<div class="empty" style="padding:16px">No repair history</div>`;

    document.getElementById('device-modal').classList.add('open');
  }

      function closeModal(){
        document.getElementById('device-modal').classList.remove('open');
      }

      //  HELPERS
      function chip(val){
        if(!val) return '—';
        const cls = val.toLowerCase().replace(/ /g,'_');
        return `<span class="chip ${cls}">${val.replace(/_/g,' ')}</span>`;
      }

      function showAlert(msg, type){
        const el = document.createElement('div');
        el.className = `alert alert-${type}`;
        el.innerHTML = `<span>${type===='success'?'':''}</span>${msg}`;
        document.body.appendChild(el);
        setTimeout(()=>el.remove(), 3000);
      }

      function logout(){
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('auth_type');

fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>window.location.href='/vendor-
login.html');
      }
  </script>
  </body>
  </html>
```

147 Current: src\main\resources\static\vendor-dashboard.html 529 lines

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Vendor Hub</title>
    <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&family=Space+Mono:wght@400;700&d
isplay=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
         :root{
             --bg:#08100a;--bg2:#0c160e;--bg3:#101c12;
             --panel:#131f15;--panel2:#182219;
             --border:rgba(52,211,153,0.08);--border2:rgba(52,211,153,0.18);
             --text:#d4edd9;--text2:#6b9474;--text3:#3d5e44;
             --accent:#34d399;--accent-dim:rgba(52,211,153,0.1);
             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
             --font:'DM Sans',sans-serif;--mono:'Space Mono',monospace;
             --sidebar:245px;
         }
         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
         .app{display:flex;height:100vh}

         /* SIDEBAR */
         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative}
         .sidebar::before{content:'';position:absolute;left:0;top:0;bottom:0;width:2px;background:linear-
gradient(to bottom,transparent,var(--accent),transparent)}
         .logo{padding:22px 20px;border-bottom:1px solid var(--border)}
         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
         .logo p{font-size:10px;color:var(--text3);font-family:var(--mono)}
         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:var(--accent-
dim);border:1px solid rgba(52,211,153,0.2);border-radius:4px;font-size:10px;font-weight:7...
         .nav{flex:1;padding:10px 0;overflow-y:auto}
         .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase;font-family:var(--mono)}
         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:6px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
         .nav-item.active{background:var(--accent-dim);color:var(--accent);border-left:2px solid var(--
accent);margin-left:6px}
         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
         .user-card{display:flex;align-items:center;gap:10px}
         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:6px;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
         .user-info-side .role-text{font-size:9px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:1px}

         /* MAIN */
         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
         .page-title{font-size:16px;font-weight:700;color:var(--text);font-family:var(--mono)}
         .topbar-right{display:flex;align-items:center;gap:12px}
         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:4px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;transition:all...
         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
         .content{flex:1;overflow-y:auto;padding:24px}
         .view{display:none}
         .view.active{display:block;animation:fadeIn 0.25s ease}
         @keyframes fadeIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}

         /* STATS */
         .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:14px;margin-
```

```
bottom:24px}
        .stat-card{background:var(--panel);border:1px solid var(--border);border-radius:10px;padding:18px;position:relative;transition:border-color 0.2s}
        .stat-card:hover{border-color:var(--border2)}
        .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:2px;border-radius:10px 10px 0 0}
        .stat-card.green::before{background:var(--accent)}
        .stat-card.amber::before{background:var(--amber)}
        .stat-card.blue::before{background:var(--blue)}
        .stat-card.red::before{background:var(--red)}
        .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1.2px;margin-bottom:10px;font-weight:600;font-family:var(--mono)}
        .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}

        /* SECTION */
        .section{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-bottom:18px;overflow:hidden}
        .section-head{padding:14px 20px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center}
        .section-title{font-size:13px;font-weight:700;color:var(--text);font-family:var(--mono)}
        .section-sub{font-size:11px;color:var(--text3);margin-top:2px}

        /* TABLE */
        .tablewrap{overflow-x:auto}
        table{width:100%;border-collapse:collapse}
        th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
        td{padding:11px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
        tr:last-child td{border-bottom:none}
        tr:hover td{background:rgba(52,211,153,0.03);color:var(--text)}

        /* CHIPS */
        .chip{display:inline-flex;align-items:center;padding:3px 8px;border-radius:4px;font-size:10px;font-weight:700;font-family:var(--mono);letter-spacing:0.5px}
        .chip.active{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(52,211,153,0.2)}
        .chip.inactive{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
        .chip.pending{background:var(--amber-dim);color:var(--amber)}
        .chip.delivered{background:var(--accent-dim);color:var(--accent)}
        .chip.processing{background:var(--blue-dim);color:var(--blue)}
        .chip.breakfast{background:var(--amber-dim);color:var(--amber)}
        .chip.lunch{background:var(--accent-dim);color:var(--accent)}
        .chip.dinner{background:var(--blue-dim);color:var(--blue)}

        /* BUTTONS */
        .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--mono);font-size:11px;font-weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.5px;text-transform:uppercase}
        .btn-primary{background:var(--accent);color:#08100a}
        .btn-primary:hover{background:#22c77a;box-shadow:0 4px 12px rgba(52,211,153,0.3)}
        .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
         .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
         .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
         .btn-danger:hover{background:var(--red);color:#fff}
         .btn-warning{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
         .btn-warning:hover{background:var(--amber);color:#000}
         .btn-sm{padding:5px 10px;font-size:10px}
         .action-row{display:flex;gap:6px}

         /* FORM */
         .form-card{padding:22px}
         .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
         .form-group{display:flex;flex-direction:column;gap:5px}
         .form-group.full{grid-column:1/-1}
         label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;font-family:var(--mono)}
         input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:9px 12px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-colo...
         input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
         input::placeholder,textarea::placeholder{color:var(--text3)}
         select option{background:var(--bg2)}
         .form-actions{display:flex;gap:10px;margin-top:18px}

         /* ALERT */
         .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:8px;z-index:1100;animation:slideIn 0.3s ease;font-size:12px;font-weight:700;font-family:var(--mono)}
         .alert-success{background:var(--accent-dim);color:var(--accent);border:1px solid
```

```
rgba(52,211,153,0.3)}
        .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
        @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
        .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:28px;font-family:var(--mono);font-size:12px}

        /* RESTAURANT CARDS */
        .restaurant-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(240px,1fr));gap:14px;padding:20px}
        .restaurant-card{background:var(--bg3);border:1px solid var(--border);border-radius:10px;padding:18px;transition:all 0.2s;position:relative;overflow:hidden}
        .restaurant-card::before{content:'';position:absolute;top:0;left:0;bottom:0;width:3px;background:var(--accent)}
        .restaurant-card:hover{border-color:var(--border2);transform:translateX(2px)}
        .restaurant-card.inactive::before{background:var(--red)}
        .rest-name{font-weight:700;color:var(--text);margin-bottom:6px;font-size:14px}
        .rest-meta{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:12px}
        .rest-actions{display:flex;gap:8px}
    </style>
</head>
<body>
<div class="app">
    <!-- SIDEBAR -->
    <div class="sidebar">
        <div class="logo">
            <div class="logo-row">
                <div class="logo-icon">V</div>
                <h2>Vendor Hub</h2>
            </div>
            <p>EMS Vendor Portal</p>
            <div class="role-badge">// VENDOR</div>
        </div>
        <nav class="nav">
            <div class="nav-section">// overview</div>
            <div class="nav-item active" onclick="showView('dashboard',this)"><span></span> Dashboard</div>
            <div class="nav-section">// restaurants</div>
            <div class="nav-item" onclick="showView('my-restaurants',this)"><span></span> My Restaurants</div>
            <div class="nav-item" onclick="showView('add-restaurant',this)"><span></span> Add Restaurant</div>
            <div class="nav-section">// operations</div>
            <div class="nav-item" onclick="showView('subscriptions',this)"><span></span> Subscriptions</div>
            <div class="nav-item" onclick="showView('deliveries',this)"><span></span> Deliveries</div>
            <div class="nav-section">// account</div>
            <div class="nav-item" onclick="showView('profile',this)"><span></span> My Profile</div>
        </nav>
        <div class="sidebar-footer">
            <div class="user-card">
                <div class="avatar" id="avatar-initials">V</div>
                <div class="user-info-side">
                    <div class="name" id="sidebar-name">Vendor</div>
                    <div class="role-text">// vendor</div>
                </div>
            </div>
        </div>
    </div>

    <!-- MAIN -->
    <div class="main">
        <div class="topbar">
            <div class="page-title" id="pageTitle">// dashboard</div>
            <div class="topbar-right">
                <span style="font-size:11px;color:var(--text3);font-family:var(--mono)" id="topbar-email"></span>
                <button class="logout-btn" onclick="logout()">SIGN OUT</button>
            </div>
        </div>
        <div class="content">

            <!-- DASHBOARD -->
            <div id="dashboard-view" class="view active">
                <div class="stats-grid">
                    <div class="stat-card green">
                        <div class="stat-label">Restaurants</div>
                        <div class="stat-value" id="stat-restaurants">--</div>
```

```
                        <div class="stat-sub">active listings</div>
                    </div>
                    <div class="stat-card amber">
                        <div class="stat-label">Subscriptions</div>
                        <div class="stat-value" id="stat-subs">--</div>
                        <div class="stat-sub">active customers</div>
                    </div>
                    <div class="stat-card blue">
                        <div class="stat-label">Deliveries</div>
                        <div class="stat-value" id="stat-deliveries">--</div>
                        <div class="stat-sub">total today</div>
                    </div>
                    <div class="stat-card red">
                        <div class="stat-label">Pending</div>
                        <div class="stat-value" id="stat-pending">--</div>
                        <div class="stat-sub">deliveries pending</div>
                    </div>
                </div>
                <div style="display:grid;grid-template-columns:1fr 1fr;gap:18px">
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title">// my_restaurants</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('my-restaurants',null)">view </button>
                        </div>
                        <div id="dash-restaurants" style="padding:4px 0"></div>
                    </div>
                    <div class="section">
                        <div class="section-head">
                            <div><div class="section-title">// recent_deliveries</div></div>
                            <button class="btn btn-secondary btn-sm" onclick="showView('deliveries',null)">view </button>
                        </div>
                        <div class="tablewrap">
                            <table>
                                <thead><tr><th>Meal</th><th>Slot</th><th>Status</th></tr></thead>
                                <tbody id="dash-deliveries"><tr><td colspan="3" class="empty-state">loading...</td></tr></tbody>
                            </table>
                        </div>
                    </div>
                </div>
            </div>

            <!-- MY RESTAURANTS -->
            <div id="my-restaurants-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">// my_restaurants</div><div class="section-sub">Manage your listings</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadMyRestaurants()">refresh</button>
                    </div>
                    <div class="restaurant-grid" id="restaurants-grid">
                        <div class="empty-state">loading...</div>
                    </div>
                </div>
            </div>

            <!-- ADD RESTAURANT -->
            <div id="add-restaurant-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">// add_restaurant</div></div>
                    <div class="form-card">
                        <div class="form-grid">
                            <div class="form-group full">
                                <label>Restaurant Name</label>
                                <input id="rest-name" placeholder="My Restaurant Name">
                            </div>

                            <div class="form-group full">
                                <label>Address</label>
                                <input id="rest-address" placeholder="Street, area, city">
                            </div>

                            <div class="form-group full">
```

```
                                <label>Meal Slots</label>
                                <div style="display:flex;gap:14px;flex-wrap:wrap;padding:10px 12px;background:var(--bg3);border:1px solid var(--border);border-radius:6px">
                                    <label style="display:flex;align-items:center;gap:8px;color:var(--text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
                                        <input type="checkbox" class="rest-slot-check" value="BREAKFAST" style="width:auto"> Breakfast
                                    </label>
                                    <label style="display:flex;align-items:center;gap:8px;color:var(--text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
                                        <input type="checkbox" class="rest-slot-check" value="LUNCH" style="width:auto"> Lunch
                                    </label>
                                    <label style="display:flex;align-items:center;gap:8px;color:var(--text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
                                        <input type="checkbox" class="rest-slot-check" value="DINNER" style="width:auto"> Dinner
                                    </label>
                                </div>
                            </div>

                            <div class="form-group full">
                                <label>Description (optional)</label>
                                <textarea id="rest-desc" rows="2" placeholder="Brief description of your restaurant..."></textarea>
                            </div>
                        </div>
                        <div class="form-actions">
                            <button class="btn btn-primary" onclick="addRestaurant()">CREATE RESTAURANT</button>
                            <button class="btn btn-secondary" onclick="clearRestForm()">CLEAR</button>
                        </div>
                    </div>
                </div>
            </div>

            <!-- SUBSCRIPTIONS -->
            <div id="subscriptions-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">// subscriptions</div><div class="section-sub">Customers subscribed to your restaurants</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>Employee</th><th>Restaurant</th><th>Meal Slot</th><th>From</th><th>To</th><th>Status</th></tr></thead>
                            <tbody id="subscriptions-list">><tr><td colspan="6" class="empty-state">loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- DELIVERIES -->
            <div id="deliveries-view" class="view">
                <div class="section">
                    <div class="section-head">
                        <div><div class="section-title">// deliveries</div></div>
                        <button class="btn btn-secondary btn-sm" onclick="loadDeliveries()">refresh</button>
                    </div>
                    <div class="tablewrap">
                        <table>
                            <thead><tr><th>ID</th><th>Restaurant</th><th>Meal Slot</th><th>Date</th><th>Status</th><th>Action</th></tr></thead>
                            <tbody id="deliveries-list">><tr><td colspan="6" class="empty-state">loading...</td></tr></tbody>
                        </table>
                    </div>
                </div>
            </div>

            <!-- PROFILE -->
```

```
            <div id="profile-view" class="view">
                <div class="section">
                    <div class="section-head"><div class="section-title">// my_profile</div></div>
                    <div style="padding:24px;display:grid;grid-template-columns:1fr 1fr;gap:16px" id="vendor-profile">
                        <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">NAME</div><div style="font-family...
                        <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">EMAIL</div><div style="font-famil...
                        <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">VENDOR ID</div><div style="font-f...
                        <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">ROLE</div><div style="font-family...
                    </div>
                </div>
            </div>

        </div>
    </div>
</div>

<script>
    const API='http://localhost:8080';
    let vendorId=null;

    (function(){
        const token=localStorage.getItem('jwt_token');
        const authType=localStorage.getItem('auth_type');
        if(!token&&authType!=='session'){window.location.href='/login.html';return;}
        if(token){
            try{
                const p=JSON.parse(atob(token.split('.')[1]));
                const roles=p.roles||p.authorities||'';
                if(roles.includes('ROLE_VENDOR')) { initUser(); return; }
                if(roles.includes('ROLE_EMPLOYEE')){window.location.href='/employee-dashboard.html';return;}
                if(roles.includes('ROLE_MANAGER')){window.location.href='/manager-dashboard.html';return;}
                if(roles.includes('ROLE_ADMIN')){window.location.href='/dashboard.html';return;}
            }catch(e){}
        }
        initUser();
    })();

    function getAuthHeader(){const t=localStorage.getItem('jwt_token');return t?{'Authorization':`Bearer ${t}`}:{};}
    async function apiFetch(path,opts={}){
        const res=await fetch(`${API}${path}`,{...opts,headers:{'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},credentials:'include'});
if(res.status===401){localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');showAlert('SESSION EXPIRED','error');setTimeout(()=>window.location.href='/login.html',1500);t...
        const text=await res.text();return text?JSON.parse(text):null;
    }

    async function initUser(){
        const token=localStorage.getItem('jwt_token');
        let email='';
        if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
        document.getElementById('avatar-initials').textContent=(email||'V').charAt(0).toUpperCase();
        document.getElementById('sidebar-name').textContent=(email.split('@')[0]||'Vendor');
        document.getElementById('topbar-email').textContent=email;
        document.getElementById('p-email').textContent=email;

        try{
            const me = await apiFetch('/vendors/me');
            if(me){
                vendorId = me.id || me.vendorId || null;
                document.getElementById('p-name').textContent = me.name || '—';
                document.getElementById('p-id').textContent = vendorId || '—';
            }
        }catch(e){
            console.error('Could not load vendor profile', e);
        }

        loadDashboard();
    }
```

```

    function showView(name,el){
        document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
        document.getElementById(`${name}-view`).classList.add('active');
        document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
        if(el)el.classList.add('active');
        document.getElementById('pageTitle').textContent='// '+name.replace(/-/g,'_');
        const loaders={dashboard:loadDashboard,'my-restaurants':loadMyRestaurants,'add-restaurant':()=>{},'subscriptions':loadSubscriptions,'deliveries':loadDeliveries,'profile':()=>{}};
        if(loaders[name])loaders[name]();
    }

    async function loadDashboard(){
        try{
            const [rests,subs,deliveries]=await Promise.all([
                apiFetch('/restaurants/vendor').catch(()=>[]),
                apiFetch('/subscriptions').catch(()=>[]),
                apiFetch('/deliveries').catch(()=>[])
            ]);

            const restArr=rests||[];
            const subArr=subs||[];
            const delArr=deliveries||[];

            document.getElementById('stat-restaurants').textContent=restArr.filter(r=>r.active!==false).length;
            document.getElementById('stat-subs').textContent=subArr.length;
            document.getElementById('stat-deliveries').textContent=delArr.length;
            document.getElementById('stat-pending').textContent=delArr.filter(d=>d.status==='PENDING').length;

            document.getElementById('dash-restaurants').innerHTML=restArr.length
                ? restArr.slice(0,4).map(r=>`<div style="padding:12px 20px;border-bottom:1px solid var(--border);display:flex;justify-content:space-between;align-items:center"><div><div style="font...
                : '<div class="empty-state">no restaurants yet</div>';

            document.getElementById('dash-deliveries').innerHTML=delArr.length
                ? delArr.slice(0,5).map(d=>`<tr><td>${d.restaurantName||d.restaurantId||'—'}</td><td><span class="chip ${(d.mealSlot||'').toLowerCase()}">${d.mealSlot||'—'}</span></td><td><span cla...
                : '<tr><td colspan="3" class="empty-state">no deliveries</td></tr>';
        }catch(e){console.error(e);}
    }


    async function loadMyRestaurants(){
        try{
            const rests=await apiFetch('/restaurants/vendor');
            const arr=rests||[];
            document.getElementById('restaurants-grid').innerHTML=arr.length
                ? arr.map(r=>`<div class="restaurant-card ${r.active===false?'inactive':''}">
      <div class="rest-name">${r.name||'—'}</div>
      <div class="rest-meta">${r.supportedMealSlots?.join(', ')||'—'} · ID: ${r.id||r.restaurantId||'—'}</div>
      <div style="font-size:12px;color:var(--text2);margin-bottom:8px">${r.address||'—'}</div>
      <div style="font-size:11px;color:var(--text3);margin-bottom:12px">${r.description||''}</div>
      <div class="rest-actions">
        ${r.active!==false
                    ? `<button class="btn btn-warning btn-sm" onclick="deactivateRestaurant(${r.id||r.restaurantId})">DEACTIVATE</button>`
                    : `<span style="font-size:10px;color:var(--red);font-family:var(--mono)">INACTIVE</span>`}
      </div>
    </div>`).join('')
                : '<div class="empty-state">no restaurants found. create one!</div>';
        }catch(e){
            document.getElementById('restaurants-grid').innerHTML='<div class="empty-state">could not load restaurants</div>';
        }
    }


    async function addRestaurant(){
        const name=document.getElementById('rest-name').value.trim();
        const address=document.getElementById('rest-address').value.trim();
        const description=document.getElementById('rest-desc').value.trim();
        const supportedMealSlots=Array.from(document.querySelectorAll('.rest-slot-
```

```
check:checked')).map(el=>el.value);

        if(!name || !address || supportedMealSlots.length===0){
            showAlert('NAME, ADDRESS AND AT LEAST ONE MEAL SLOT REQUIRED','error');
            return;
        }

        try{
            await apiFetch('/restaurants',{
                method:'POST',
                body:JSON.stringify({name,address,description,supportedMealSlots})
            });
            showAlert('RESTAURANT CREATED!','success');
            clearRestForm();
            loadMyRestaurants();
            loadDashboard();
        }catch(e){
            showAlert('FAILED: '+e.message,'error');
        }
    }


    function clearRestForm(){
        ['rest-name','rest-address','rest-desc'].forEach(id=>document.getElementById(id).value='');
        document.querySelectorAll('.rest-slot-check').forEach(el=>el.checked=false);
    }

    async function loadSubscriptions(){
        try{
            const data=await apiFetch('/subscriptions');
            document.getElementById('subscriptions-list').innerHTML=(data||[]).length
                ? data.map(s=>`<tr><td style="font-family:var(--mono);font-size:12px">${s.employeeEmail||s.employeeId||'—'}</td><td>${s.restaurantName||s.restaurantId||'—'}</td><td><span class="chi...
                : '<tr><td colspan="6" class="empty-state">no subscriptions found</td></tr>';
        }catch(e){document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6" class="empty-state">could not load subscriptions</td></tr>';}
    }

    async function loadDeliveries(){
        try{
            const data=await apiFetch('/deliveries');
            document.getElementById('deliveries-list').innerHTML=(data||[]).length
                ? data.map(d=>`<tr><td style="font-family:var(--mono);font-size:11px">#${d.id||'—'}</td><td>${d.restaurantName||d.restaurantId||'—'}</td><td><span class="chip ${(d.mealSlot||'').toL...
                : '<tr><td colspan="6" class="empty-state">no deliveries found</td></tr>';
        }catch(e){document.getElementById('deliveries-list').innerHTML='<tr><td colspan="6" class="empty-state">could not load deliveries</td></tr>';}
    }

    async function markDelivered(id){
        try{await apiFetch(`/deliveries/${id}/delivered`,{method:'PUT'});showAlert('MARKED AS DELIVERED','success');loadDeliveries();}
        catch(e){showAlert('FAILED','error');}
    }

    function logout(){
        // Clear all localStorage
        localStorage.removeItem('jwt_token');
        localStorage.removeItem('refresh_token');
        localStorage.removeItem('auth_type');
        localStorage.removeItem('last_role');

        // Call server logout to invalidate session
        fetch(`${API}/session/logout`, {
            method: 'POST',
            credentials: 'include'
        }).finally(() => {
            // Force expire the session cookie on client side
            document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
            window.location.href = '/google.html';
        });
    }

    function showAlert(msg,type){
```

```
        const el=document.createElement('div');
        el.className=`alert alert-${type}`;el.textContent=msg;
        document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
    }
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>EMS — Vendor Login</title>
    <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
         :root{
             --bg:#070910;
             --panel:#0e1118;
             --panel2:#131821;
             --border:rgba(255,255,255,0.07);
             --border-lit:rgba(255,255,255,0.14);
             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
             --amber:#f9c74f;--amber-dim:rgba(249,199,79,0.12);
             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
         }
         body{font-family:var(--font);background:var(--bg);color:var(--text);min-height:100vh;display:flex;overflow:auto;}

         .bg-layer{position:fixed;inset:0;z-index:0;
             background:
                     radial-gradient(ellipse 60% 50% at 20% 50%, rgba(248,113,113,0.06) 0%,transparent 70%),
                     radial-gradient(ellipse 40% 60% at 80% 20%, rgba(249,199,79,0.05) 0%,transparent 60%),
                     radial-gradient(ellipse 50% 40% at 70% 80%, rgba(248,113,113,0.04) 0%,transparent 60%);
         }
         .grid-overlay{position:fixed;inset:0;z-index:0;background-image:linear-gradient(rgba(255,255,255,0.02) 1px,transparent 1px),linear-gradient(90deg,rgba(255,255,255,0.02) 1px,transparent 1px)...

         /* LEFT */
         .left-panel{flex:1;display:flex;flex-direction:column;justify-content:center;padding:60px;position:relative;z-index:1;animation:fadeLeft 0.8s ease both;}
         @keyframes fadeLeft{from{opacity:0;transform:translateX(-24px)}to{opacity:1;transform:translateX(0)}}
         .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:32px}
         .brand-icon{width:44px;height:44px;background:linear-gradient(135deg,#f87171,#fb923c);border-radius:12px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:18p...
         .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
         .brand-tagline{font-size:13px;color:var(--text2)}
         .hero-text h1{font-size:clamp(36px,4vw,50px);font-weight:700;line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;}
         .hl-red{color:var(--red)}
         .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:360px}

         .info-cards{display:flex;flex-direction:column;gap:12px;margin-top:40px;max-width:360px;}
         .info-card{background:var(--panel);border:1px solid var(--border);border-radius:10px;padding:14px 18px;display:flex;align-items:flex-start;gap:12px;}
         .info-card-icon{font-size:18px;flex-shrink:0;margin-top:1px}
         .info-card-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px}
         .info-card-text span{font-size:12px;color:var(--text2)}

         .back-link{display:inline-flex;align-items:center;gap:6px;font-size:12px;color:var(--text3);text-decoration:none;margin-top:36px;transition:color 0.2s;}
         .back-link:hover{color:var(--text2)}

         /* RIGHT */
         .right-panel{width:460px;flex-shrink:0;display:flex;align-items:flex-start;overflow-y:auto;justify-content:center;padding:32px 40px;position:relative;z-index:1;border-left:1px solid var(--b...
         @keyframes fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
         .form-box{width:100%}

         .vendor-badge{display:inline-flex;align-items:center;gap:8px;background:var(--red-dim);border:1px solid rgba(248,113,113,0.25);border-radius:20px;padding:6px 14px;font-size:12px;font-weight...
```

```
        .vendor-badge .dot{width:7px;height:7px;border-radius:50%;background:var(--red);animation:pulse 1.4s infinite;}
        @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}

        .form-title{font-size:22px;font-weight:700;color:var(--text);margin-bottom:6px;}
        .form-subtitle{font-size:13px;color:var(--text2);margin-bottom:28px;}

        /* AUTH TABS */
        .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-bottom:24px;border:1px solid var(--border)}
        .auth-tab{flex:1;padding:9px;border:none;background:transparent;color:var(--text2);font-family:var(--font);font-weight:600;font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;}
        .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}

        /* FORM ELEMENTS */
        .form-group{margin-bottom:16px}
        .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
        .form-input{width:100%;padding:11px 14px;background:var(--bg);border:1px solid var(--border);border-radius:9px;color:var(--text);font-family:var(--font);font-size:14px;transition:all 0.2s;}
        .form-input:focus{outline:none;border-color:var(--red);box-shadow:0 0 0 3px rgba(248,113,113,0.08)}
        .form-input::placeholder{color:var(--text3)}

        /* BUTTONS */
        .btn-login{width:100%;padding:12px;border:none;border-radius:9px;font-family:var(--font);font-size:14px;font-weight:600;cursor:pointer;transition:all 0.2s;margin-top:4px;background:var(--re...
        .btn-login:hover{background:#f98a8a;box-shadow:0 6px 20px rgba(248,113,113,0.35);transform:translateY(-1px)}
        .btn-login:active{transform:translateY(0)}
        .btn-login.loading{opacity:0.7;pointer-events:none}

        /* STATUS */
        .status-box{border-radius:9px;padding:10px 14px;margin-top:14px;font-size:13px;font-weight:500;display:none;animation:fadeIn 0.3s ease;}
        @keyframes fadeIn{from{opacity:0;transform:translateY(4px)}to{opacity:1;transform:translateY(0)}}
        .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
        .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
        .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid rgba(79,140,255,0.2)}

        .notice{background:rgba(249,199,79,0.06);border:1px solid rgba(249,199,79,0.15);border-radius:9px;padding:12px 14px;margin-top:20px;font-size:12px;color:var(--text2);line-height:1.5;}
        .notice strong{color:var(--amber)}

        /* FOOTER */
        .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
        .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
        .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
        .tz-select{flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;}
        .tz-select:focus{outline:none;border-color:var(--border-lit)}
        .tz-time{font-size:10px;color:var(--red);font-family:var(--mono)}

        @media(max-width:900px){
            .left-panel{display:none}
            .right-panel{width:100%;border-left:none;background:var(--bg)}
        }
    </style>
</head>
<body>
<div class="bg-layer"></div>
<div class="grid-overlay"></div>

<!-- LEFT -->
<div class="left-panel">
    <div class="brand-logo">
        <div class="brand-icon">V</div>
        <div>
            <div class="brand-name">Vendor Hub</div>
            <div class="brand-tagline">Employee Management System</div>
        </div>
    </div>
    <div class="hero-text">
        <h1>Manage your<br><span class="hl-red">vendor account</span><br>securely.</h1>
        <p>Access your deliveries, restaurants, and subscriptions. Your account is created and managed by
```

```
the system admin.</p>
    </div>
    <div class="info-cards">
        <div class="info-card">
            <span class="info-card-icon"></span>
            <div class="info-card-text">
                <strong>Delivery Tracking</strong>
                <span>Monitor your active and scheduled meal deliveries in real-time.</span>
            </div>
        </div>
        <div class="info-card">
            <span class="info-card-icon"></span>
            <div class="info-card-text">
                <strong>Restaurant Management</strong>
                <span>View and update restaurant details and meal offerings.</span>
            </div>
        </div>
        <div class="info-card">
            <span class="info-card-icon"></span>
            <div class="info-card-text">
                <strong>Account by Admin Only</strong>
                <span>Vendor accounts are created by the system admin. Contact your admin if you don't have
credentials.</span>
            </div>
        </div>
    </div>
    <a class="back-link" href="/google.html"> Back to main login</a>
</div>

<!-- RIGHT FORM -->
<div class="right-panel">
    <div class="form-box">

        <div class="vendor-badge"><span class="dot"></span>Vendor Portal</div>
        <div class="form-title">Vendor Sign In</div>
        <div class="form-subtitle">Enter the credentials provided by your system admin.</div>

        <!-- AUTH TABS -->
        <div class="auth-tabs">
            <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
            <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
        </div>

        <!-- JWT FORM -->
        <div id="jwt-form">
            <div class="form-group">
                <label class="form-label">Email / Username</label>
                <input class="form-input" id="jwt-username" type="text" placeholder="vendor@company.com"
autocomplete="off">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="jwt-password" type="password" placeholder="••••••••"
autocomplete="current-password">
            </div>
            <button class="btn-login" id="jwt-btn" onclick="jwtLogin()">Sign In as Vendor</button>
        </div>

        <!-- SESSION FORM -->
        <div id="session-form" style="display:none">
            <div class="form-group">
                <label class="form-label">Email / Username</label>
                <input class="form-input" id="session-username" type="text"
placeholder="vendor@company.com">
            </div>
            <div class="form-group">
                <label class="form-label">Password</label>
                <input class="form-input" id="session-password" type="password" placeholder="••••••••">
            </div>
            <button class="btn-login" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
        </div>

        <div class="status-box" id="status-box"></div>

        <div class="notice">
```

```
            <strong>Don't have an account?</strong> Vendor accounts are created exclusively by the system admin. Please contact your administrator to set up access.
        </div>

        <!-- FOOTER -->
        <div class="form-footer">
            <div class="tz-row">
                <span class="tz-label">TZ</span>
                <select class="tz-select" id="tz-select" onchange="updateTz()">
                    <option value="Asia/Kolkata">India — IST UTC+5:30</option>
                    <option value="UTC">UTC</option>
                    <option value="America/New_York">New York — EST UTC-5</option>
                    <option value="America/Los_Angeles">Los Angeles — PST UTC-8</option>
                    <option value="Europe/London">London — GMT UTC+0</option>
                    <option value="Europe/Paris">Paris — CET UTC+1</option>
                    <option value="Asia/Tokyo">Tokyo — JST UTC+9</option>
                    <option value="Asia/Dubai">Dubai — GST UTC+4</option>
                    <option value="Australia/Sydney">Sydney — AEDT UTC+11</option>
                    <option value="Asia/Singapore">Singapore — SGT UTC+8</option>
                </select>
                <span class="tz-time" id="tz-time">--:--:--</span>
            </div>
        </div>

    </div>
</div>

<script>
    const API = 'http://localhost:8080';

 const DASHBOARDS = {
     FOOD_VENDOR: '/vendor-dashboard.html',
     TECH_VENDOR: '/tech-vendor-dashboard.html'
 };

 //  ALREADY LOGGED IN?
 (function(){
     const token = localStorage.getItem('jwt_token');
     if(!token) return;
     try{
         const role = getRoleFromJWT(token);
         if(DASHBOARDS[role]) window.location.href = DASHBOARDS[role];
     }catch(e){}
 })();

 //  HELPERS
 function getRoleFromJWT(token){
     const payload = JSON.parse(atob(token.split('.')[1]));
     const raw = payload.role || payload.roles || payload.authorities || payload.scope || '';
     const role = Array.isArray(raw) ? raw[0] : raw;
     return String(role).replace(/^ROLE_/, '').toUpperCase().trim();
 }

 function showStatus(type, msg){
     const box = document.getElementById('status-box');
     box.className     = `status-box ${type}`;
     box.textContent   = msg;
     box.style.display = 'block';
 }
 function hideStatus(){
     document.getElementById('status-box').style.display = 'none';
 }

 function switchTab(type){
     document.getElementById('jwt-form').style.display     = type === 'jwt'     ? 'block' : 'none';
     document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
     document.getElementById('tab-jwt').classList.toggle('active',     type === 'jwt');
     document.getElementById('tab-session').classList.toggle('active', type === 'session');
     hideStatus();
 }

 function redirectByRole(role){
     const dest = DASHBOARDS[role];
     if(!dest){
         showStatus('error', 'This portal is for vendors only. Please use the main login page.');
         return false;
```

```
    }
    showStatus('success', `Login successful! Redirecting to ${role === 'TECH_VENDOR' ? 'Tech Vendor' : 'Food Vendor'} dashboard...`);
    setTimeout(() => window.location.href = dest, 700);
    return true;
}

//  JWT LOGIN
async function jwtLogin(){
    const username = document.getElementById('jwt-username').value.trim();
    const password = document.getElementById('jwt-password').value;
    if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }

    const btn = document.getElementById('jwt-btn');
    btn.classList.add('loading'); btn.textContent = 'Signing in...';
    showStatus('loading', 'Authenticating...');

    try{
        const res = await fetch(`${API}/Authenticate`, {
            method:  'POST',
            headers: {'Content-Type': 'application/json'},
            body:    JSON.stringify({username, password})
        });

        if(res.ok){
            const data  = await res.json();
            const token = data.accessToken || data.token;
            const role  = getRoleFromJWT(token);

            if(!DASHBOARDS[role]){
                showStatus('error', 'This portal is for vendors only. Please use the main login page.');
                btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
                return;
            }

            localStorage.setItem('jwt_token',     token);
            localStorage.setItem('refresh_token', data.refreshToken || '');
            localStorage.setItem('auth_type',     'jwt');
            localStorage.setItem('last_role',     role);
            localStorage.setItem('user_timezone', document.getElementById('tz-select').value);

            btn.textContent = 'Signed In ';
            redirectByRole(role);

        }else{
            const err = await res.text();
            showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
            btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
        }
    }catch(e){
        showStatus('error', 'Connection error: ' + e.message);
        btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
    }
}

//  SESSION LOGIN
async function sessionLogin(){
    const username = document.getElementById('session-username').value.trim();
    const password = document.getElementById('session-password').value;
    if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }

    const btn = document.getElementById('session-btn');
    btn.classList.add('loading'); btn.textContent = 'Signing in...';
    showStatus('loading', 'Authenticating...');

    try{
        const res = await fetch(`${API}/session/login`, {
            method:      'POST',
            headers:     {'Content-Type': 'application/json'},
            body:        JSON.stringify({username, password}),
            credentials: 'include'
        });

        if(res.ok){
            // Session login — fetch /employee/me or /vendor/me to get role
            try{
```

```
                const meRes = await fetch(`${API}/vendor/me`, {credentials: 'include'});
                if(meRes.ok){
                    const me   = await meRes.json();
                    const role = String(me.role || '').replace(/^ROLE_/, '').toUpperCase();
                    localStorage.setItem('auth_type',     'session');
                    localStorage.setItem('last_role',     role);
                    localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
                    btn.textContent = 'Signed In ';
                    redirectByRole(role);
                    return;
                }
            }catch(e){}

            // Fallback if /vendor/me not available
            showStatus('error', 'Could not determine vendor type. Please use JWT login.');
            btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';

        }else{
            const err = await res.json().catch(()=>({}));
            showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
            btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
        }
    }catch(e){
        showStatus('error', 'Connection error: ' + e.message);
        btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
    }
}

//  TIMEZONE
function updateTz(){
    localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
    refreshTzClock();
}
function refreshTzClock(){
    const tz = document.getElementById('tz-select').value;
    document.getElementById('tz-time').textContent =
        new Date().toLocaleTimeString('en-GB', {timeZone: tz, hour12: false});
}

document.addEventListener('keydown', e => {
    if(e.key !== 'Enter') return;
    if(document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
    else sessionLogin();
});

window.onload = () => {
    const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
    document.getElementById('tz-select').value = savedTz;
    refreshTzClock();
    setInterval(refreshTzClock, 1000);
};
</script>
</body>
</html>
```

149 Current: src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
package com.example.EmployeeManagementSystem;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class EmployeeManagementSystemApplicationTests {

    @Test
     void contextLoads() {
     }

 }

```

150 Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveAccrualServiceTest.java

1
lin

```
package com.example.EmployeeManagementSystem.service;


import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveType;
import com.example.EmployeeManagementSystem.Enum.Gender;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.*;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

class LeaveAccrualServiceTest {

    @Mock
    EmployeeRepo employeeRepo;
    @Mock
    LeaveTypeRepository leaveTypeRepo;
    @Mock
    LeaveEntitlementRepository entitlementRepo;

    @InjectMocks
    LeaveAccrualService service;

    @BeforeEach void setup() { MockitoAnnotations.openMocks(this); }

    @Test
    void maleEmployee_doesNotAccrueMaternity() {
        Employee emp = employee(Gender.M);
        LeaveType maternity = leaveType("MATERNITY", true, Gender.F);

        when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
        when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));

        service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));

        verify(entitlementRepo, never()).save(any());
    }

    @Test
    void femaleEmployee_accruesMaternity() {
        Employee emp = employee(Gender.F);
        LeaveType maternity = leaveType("MATERNITY", true, Gender.F);

        when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
        when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
        when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), any()))
                .thenReturn(Optional.empty());

        // Simulate new entitlement creation
        when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));

        ArgumentCaptor<LeaveEntitlement> captor = ArgumentCaptor.forClass(LeaveEntitlement.class);
        verify(entitlementRepo, atLeastOnce()).save(captor.capture());

        LeaveEntitlement saved = captor.getAllValues().stream()
```

```
                .filter(e -> e.getAccruedThisYear().compareTo(BigDecimal.ONE) == 0)
                .findFirst().orElseThrow();

        assertThat(saved.getAccruedThisYear()).isEqualByComparingTo("1");
    }

    @Test
    void employeeJoiningAfterAccrualDate_isSkipped() {
        Employee emp = employee(Gender.M);
        emp.setJoined_at(LocalDate.of(2025, 6, 15)); // joins mid-month
        LeaveType casual = leaveType("CASUAL", false, null);

        when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
        when(leaveTypeRepo.findAll()).thenReturn(List.of(casual));

        service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));

        verify(entitlementRepo, never()).save(any());
    }

    // --- helpers ---
    private Employee employee(Gender gender) {
        Employee e = new Employee();
        e.setEmployeeId(1L);
        e.setGender(gender);
        e.setStatus(Status.ACTIVE);
        e.setJoined_at(LocalDate.of(2023, 1, 1));
        return e;
    }

    private LeaveType leaveType(String name, boolean genderRestricted, Gender restriction) {
        LeaveType lt = new LeaveType();
        lt.setId(1);
        lt.setName(name);
        lt.setGenderRestricted(genderRestricted);
        lt.setGenderRestriction(restriction);
        lt.setCarriesForward(!name.equals("SICK"));
        lt.setMonthlyReset(name.equals("SICK"));
        return lt;
    }
}
```

151 Current: src\test\java\com\example\EmployeeManagementSystem\service\SickLeaveResetServiceTest.java

```java
package com.example.EmployeeManagementSystem.service;

import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.*;
import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
import com.example.EmployeeManagementSystem.Entity.LeaveType;
import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.Employee;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.Mockito.*;

class SickLeaveResetServiceTest {

    @Mock
    LeaveTypeRepository leaveTypeRepo;
    @Mock
    LeaveEntitlementRepository entitlementRepo;
    @InjectMocks SickLeaveResetService service;

    @BeforeEach void setup() { MockitoAnnotations.openMocks(this); }

    @Test
    void unusedSickDays_areZeroedAtMonthEnd() {
        LeaveType sick = sickLeaveType();
        LeaveEntitlement entitlement = entitlement(sick, BigDecimal.valueOf(3), BigDecimal.ONE);
        // accrued=3, used=1  available=2 (should be wiped)

        when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
        when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
                .thenReturn(List.of(entitlement));
        when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        service.runMonthlyReset(LocalDate.of(2025, 6, 30));

        // After reset: accrued should equal used  available = 0
        assertThat(entitlement.getAccruedThisYear())
                .isEqualByComparingTo(entitlement.getUsedThisYear());
        assertThat(entitlement.getAvailableBalance())
                .isEqualByComparingTo(BigDecimal.ZERO);
    }

    @Test
    void fullyUsedSickLeave_noChangeNeeded() {
        LeaveType sick = sickLeaveType();
        LeaveEntitlement entitlement = entitlement(sick, BigDecimal.ONE, BigDecimal.ONE);
        // accrued=1, used=1  available=0, nothing to wipe

        when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
        when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
                .thenReturn(List.of(entitlement));
        when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        service.runMonthlyReset(LocalDate.of(2025, 6, 30));

        // accrued unchanged
        assertThat(entitlement.getAccruedThisYear()).isEqualByComparingTo("1");
    }

    // --- helpers ---
    private LeaveType sickLeaveType() {
        LeaveType lt = new LeaveType();
        lt.setId(1);
        lt.setName("SICK");
        lt.setMonthlyReset(true);
```

```
        lt.setCarriesForward(false);
        return lt;
    }

    private LeaveEntitlement entitlement(LeaveType type,
                                         BigDecimal accrued,
                                         BigDecimal used) {
        Employee emp = new Employee();
        emp.setEmployeeId(1L);

        LeaveEntitlement e = new LeaveEntitlement();
        e.setEmployee(emp);
        e.setLeaveType(type);
        e.setYear(2025);
        e.setOpeningBalance(BigDecimal.ZERO);
        e.setAccruedThisYear(accrued);
        e.setUsedThisYear(used);
        e.setClosingBalance(BigDecimal.ZERO);
        return e;
    }
}

```

152 Current: src\test\java\com\example\EmployeeManagementSystem\service\YearEndCarryForwardServiceTest

```java
package com.example.EmployeeManagementSystem.service;

import com.example.EmployeeManagementSystem.Entity.Employee;
import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
import com.example.EmployeeManagementSystem.Entity.LeaveType;
import com.example.EmployeeManagementSystem.Enum.Status;
import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;

import static org.mockito.Mockito.when;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.atLeastOnce;

import static org.assertj.core.api.Assertions.assertThat;
class YearEndCarryForwardServiceTest {

    @Mock
    EmployeeRepo employeeRepo;
    @Mock
    LeaveEntitlementRepository entitlementRepo;
    @Mock
    YearEndCarryForwardLogRepository logRepo;
    @Mock
    LeaveWarningRepository warningRepo;
    @InjectMocks
    YearEndCarryForwardService service;

    @BeforeEach
    void setup() { MockitoAnnotations.openMocks(this); }

    @Test
    void whenTotalUnderCap_noCapApplied() {
        Employee emp = employee();
        LeaveType casual = carryForwardType("CASUAL");

        // casual: opening=0, accrued=8, used=2  closing=6
        LeaveEntitlement ent = entitlement(emp, casual, 0, 8, 2);

        when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
        when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
        when(entitlementRepo.findByEmployeeEmployeeIdAndYear(emp.getEmployeeId(), 2025))
                .thenReturn(List.of(ent));
        when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
                .thenReturn(Optional.empty());
        when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
        when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        service.runYearEnd(2025);

        // closing balance = 6, no cap needed, nothing wasted
        assertThat(ent.getClosingBalance()).isEqualByComparingTo("6");
        verify(warningRepo, never()).save(any());
    }
```

```java

    @Test
    void whenTotalExceedsCap_capAppliedProportionally() {
        Employee emp = employee();
        LeaveType casual    = carryForwardType("CASUAL");
        LeaveType maternity = carryForwardType("MATERNITY");

        // casual: closing=20, maternity: closing=16  total=36, over cap by 6
        LeaveEntitlement casualEnt    = entitlement(emp, casual,    0, 20, 0);
        LeaveEntitlement maternityEnt = entitlement(emp, maternity, 0, 16, 0);

        when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
        when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
        when(entitlementRepo.findByEmployeeEmployeeIdAndYear(emp.getEmployeeId(), 2025))
                .thenReturn(List.of(casualEnt, maternityEnt));
        when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
                .thenReturn(Optional.empty());
        when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
        when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

        service.runYearEnd(2025);

        // casual: 20/36 * 30 = 16.67, maternity: 16/36 * 30 = 13.33, total = 30
        BigDecimal casualCapped    = casualEnt.getClosingBalance();
        BigDecimal maternityCapped = maternityEnt.getClosingBalance();
        BigDecimal total           = casualCapped.add(maternityCapped);

        assertThat(total).isEqualByComparingTo("30.00");
         verify(warningRepo, atLeastOnce()).save(any()); // wasted warning issued
    }

    @Test
    void idempotencyGuard_preventsDoubleRun() {
        when(logRepo.existsByProcessedYear(2025)).thenReturn(true);

        service.runYearEnd(2025);

        verify(employeeRepo, never()).findByStatus(any());
    }

    // --- helpers ---
    private Employee employee() {
        Employee e = new Employee(); e.setEmployeeId(1L); return e;
    }

    private LeaveType carryForwardType(String name) {
        LeaveType lt = new LeaveType();
        lt.setId(name.equals("CASUAL") ? 2 : 3);
        lt.setName(name);
        lt.setCarriesForward(true);
        lt.setMonthlyReset(false);
        return lt;
    }

    private LeaveEntitlement entitlement(Employee emp, LeaveType type,
                                         int opening, int accrued, int used) {
        LeaveEntitlement e = new LeaveEntitlement();
        e.setEmployee(emp); e.setLeaveType(type); e.setYear(2025);
        e.setOpeningBalance(BigDecimal.valueOf(opening));
        e.setAccruedThisYear(BigDecimal.valueOf(accrued));
        e.setUsedThisYear(BigDecimal.valueOf(used));
        e.setClosingBalance(BigDecimal.ZERO);
        return e;
    }
}

```